DRAFT

Battle-Tested Data Pipelines

The step ELT forgot

Patterns for extraction, syntactic transformation, and loading

Alonso Burón Ardiles

Working draft for editorial review

Final cover and diagrams pending design

Battle-Tested Data Pipelines
The step ELT forgot – patterns for extraction, syntactic transformation, and loading

First edition, 2026.

Typeset with Typst. Diagrams by the author.

To the woman I'll spend my life with – whose patience would put every saint to shame, whose light kept me whole through the late nights, and whose love carried this book when I couldn't.

Thank you for enduring the nights I spent glued to my screen muttering about pipelines, for listening to mad ramblings about watermarks and cursors, and for believing in this project when I wasn't sure anyone would read it.

This book exists because you gave me the space, the confidence, and the food to write it.

I love you.

CONTENTS

Why This Book 7
Domain Model 8

Part I: Foundations & Source Archetypes 10
1.1 The EL Myth 11
1.2 What Is Syntactic Transformation 12
1.3 Transactional Sources 14
1.4 Columnar Destinations 18
1.5 The Lies Sources Tell 24
1.6 Hard Rules, Soft Rules 32
1.7 Corridors 35
1.8 Purity vs. Freshness 39
1.9 Idempotency 42

Part II: Full Replace Patterns 45
2.1 Full Scan Strategies 46
2.2 Partition Swap 50
2.3 Staging Swap 54
2.4 Scoped Full Replace 59
2.5 Rolling Window Replace 63
2.6 Sparse Table Extraction 66
2.7 Activity-Driven Extraction 69
2.8 Hash-Based Change Detection 71
2.9 Partial Column Loading 75

Part III: Incremental Extraction Patterns 78
3.1 Timestamp Extraction Foundations 79
3.2 Cursor-Based Timestamp Extraction 81
3.3 Stateless Window Extraction 83
3.4 Cursor from Another Table 86
3.5 Sequential ID Cursor 88
3.6 Hard Delete Detection 91
3.7 Open/Closed Documents 94
3.8 Detail Without Timestamp 98
3.9 Late-Arriving Data 100
3.10 Create vs Update Separation 103

Part IV: Load Strategies 107
4.1 Full Replace Load 108
4.2 Append-Only Load 111
4.3 Merge / Upsert 115
4.4 Append and Materialize 121
4.5 Hybrid Append-Merge 126
4.6 Reliable Loads 128

Part V: The Syntactic Transformation Playbook 132
5.1 Metadata Column Injection 133
5.2 Synthetic Keys 137
5.3 Type Casting and Normalization 140
5.4 Null Handling 145
5.5 Timezone Handling 147

5.6 Charset and Encoding 151
5.7 Nested Data and JSON 154

Part VI: Operating the Pipeline 157
6.1 Monitoring and Observability 158
6.2 The Health Table 162
6.3 Cost Monitoring 168
6.4 SLA Management 172
6.5 Alerting and Notifications 176
6.6 Scheduling and Dependencies 180
6.7 Source System Etiquette 184
6.8 Tiered Freshness 188
6.9 Data Contracts 191
6.10 Extraction Status Gates 196
6.11 Backfill Strategies 199
6.12 Partial Failure Recovery 203
6.13 Duplicate Detection 206
6.14 Reconciliation Patterns 210
6.15 Recovery from Corruption 213

Part VII: Serving the Destination 216
7.1 Don't Pre-Aggregate 217
7.2 Partitioning, Clustering, and Pruning 219
7.3 Pre-Built Views 223
7.4 Query Patterns for Analysts 227
7.5 Cost Optimization by Engine 231
7.6 Point-in-Time from Events 235
7.7 Schema Naming Conventions 240

Part VIII: Appendix 245
8.1 SQL Dialect Reference 246
8.2 Decision Flowchart 257
8.3 Glossary 259
8.4 Domain Model Quick Reference 261
8.5 Orchestrators 264
8.6 Extractors and Loaders 268
8.7 Destinations 274

What This Book Covers

This book is about the space between source and destination – the step that ELT skips over and ETL buries inside a monolith. Specifically:

What we cover	What we don't
Extracting data from transactional systems	Building dashboards or reports
Syntactic transformation: types, nulls, timezones, encodings	Business logic / KPI definitions
Loading into columnar or transactional destinations	Silver/gold layer transformations
Incremental strategies, full replace, and the huge gray middle	Orchestrator-specific tutorials (Airflow, dbt, etc.)
Failure recovery, idempotency, reconciliation	Data modeling / star schemas
Protecting destination costs from bad queries	ML pipelines
Batch extraction patterns	CDC / real-time streaming / event-driven

> **Two corridors, different tradeoffs**
>
> Every pattern in this book plays out differently depending on where the data is going. I call these **corridors**: Transactional -> Columnar (e.g. SQL Server -> BigQuery) and Transactional -> Transactional (e.g. PostgreSQL -> PostgreSQL). Same pattern, different trade-offs. I show both.

> **Tool-agnostic patterns, opinionated appendix**
>
> The patterns in this book use generic orchestrator language – "your orchestrator," "a scheduled job," "a downstream dependency" – because they work regardless of whether you run Dagster, Airflow, Prefect, or cron. The same applies to extractors, loaders, and destination engines. Specific tool recommendations, feature comparisons, and my opinionated picks live in the Appendix (8.3 – Glossary through 8.7 – Destinations).

Why This Book

One Tuesday morning I woke up to a wall of email alerts: a row-count mismatch explosion across dozens of pipelines, and I spent the rest of the day fixing them by hand – re-running extractions and rescanning for deleted rows against source. By the time I was done, I sat back and realized two things. First, the system I had built was complex enough that in a couple of months I wouldn't be able to recreate it without having it written down somewhere. Second, when I went looking for references to categorize what I had built into an existing framework, none went deep enough. There were no strategy books for what I was doing.

So I started writing down the patterns I used as little Python scripts, just for future reference. As I organized them, they split naturally into extraction patterns and loading patterns – and then I found myself with a collection of small but critical scripts that didn't belong in either category. Type casting, naming conventions, timezone fixes: not business logic or aggregation (what ETL calls a Transformation), and not something you can ignore the way ELT treats extraction and loading as contiguous. That leftover

pile is the frame for this whole book: a pipeline does two kinds of transformation. Syntactic transformation changes how data is represented without altering its meaning – casting types, injecting metadata, normalizing identifiers, handling timezones and nulls, synthesizing keys. Semantic transformation changes what the data means – aggregations, business rules, historical modeling – and belongs downstream in the serving layer (Part VII). In my world that semantic half was the analysts' job; mine was to land the data as close to source quality as possible, somewhere they could actually reach it.

Who This Is For

This book is for the engineer who inherited a pile of pipelines that work most of the time – the ones with no monitoring, no validation, and a cloud bill that spikes because someone scheduled a full refresh every 30 minutes on a table that didn't need it.

It's for the first-time data engineer who's building their first pipeline and assumes everything should be incremental and it should just work. (It won't. This book explains why, and what to do about it.)

It's for the senior who has been doing this for years and needs a framework to teach it to their team, or to finally name the patterns they've been applying by instinct.

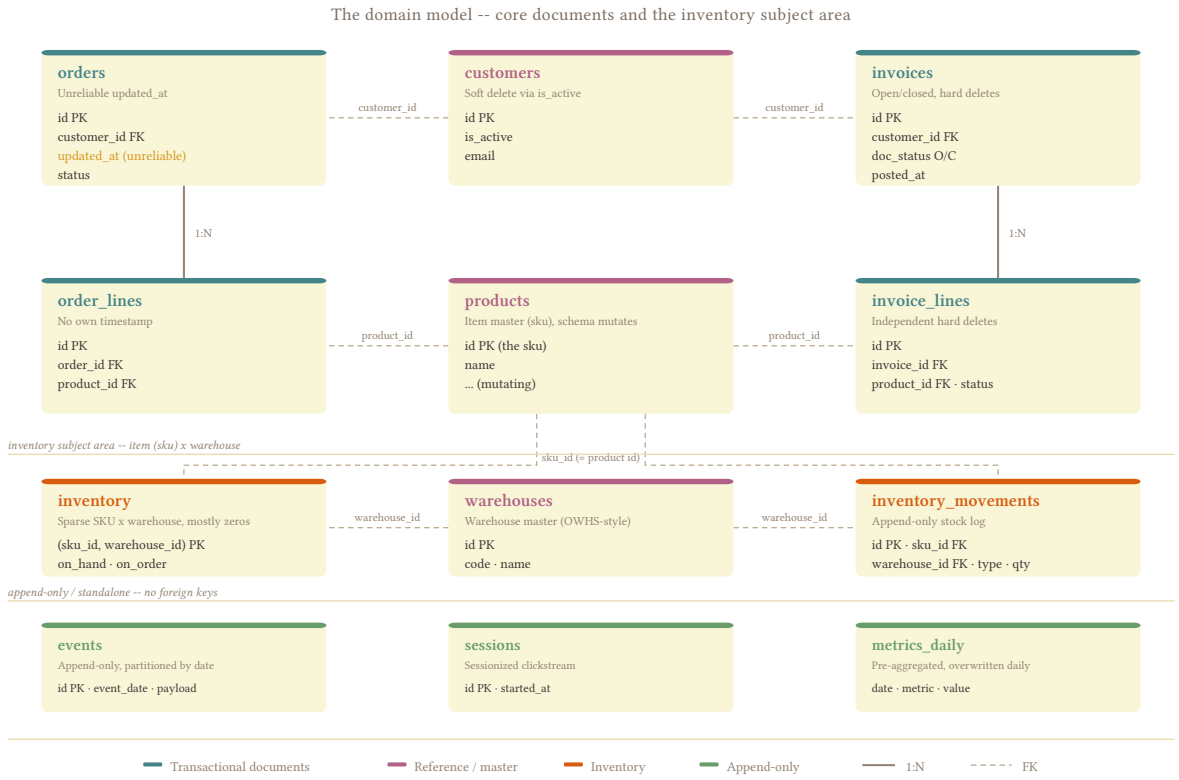
What you get is a pattern language: the decisions, tradeoffs, and failure modes that repeat across every pipeline regardless of stack, and how to monitor, surface, and fix them – not a tutorial that sets up your orchestrator for you.

Domain Model

Every SQL example in this book uses the same fictional schema. Same tables, columns, and quirks; so you can focus on the pattern per source.

The tables were chosen because each one represents a distinct extraction challenge.

Schema



The three standalone tables – `events`, `sessions`, and `metrics_daily` – have no foreign keys into the schema above. `inventory` and `inventory_movements` form the inventory subject area: each row keys on `(sku_id, warehouse_id)`, referencing `products` (the item master – the sku is the product) and `warehouses`. They represent different source archetypes.

The Tables and Why They're Here

Table	Pipeline challenge
`orders`	Has `updated_at` but it's unreliable – trigger only fires on UPDATE, not INSERT. The canonical example of a broken cursor.
`order_lines`	Detail table with no timestamp of its own. Must borrow the header's cursor for incremental extraction.
`customers`	Soft-delete via `is_active`. The flag works for normal application flows; back-office scripts bypass it.
`products`	Schema mutates – new columns appear after deploys, `category` became `product_category` once. The schema drift case.
`invoices`	Open/closed document pattern. Open invoices get hard-deleted regularly. The hard delete case.
`invoice_lines`	Has its own `status` per line, hard-deleted independently – not just cascade from the header. Complicates both delete detection and cursor borrowing.
`events`	Append-only, partitioned by date. The simplest extraction pattern. Nothing is ever updated or deleted.
`sessions`	Sessionized clickstream. Late-arriving events mean sessions can close hours after they open, creating the late-arriving data problem.
`metrics_daily`	Pre-aggregated daily metrics, overwritten on recompute. Partition-level replace is the natural fit.
`inventory`	Sparse cross-product of SKU x Warehouse. Most rows are zeros. Filtering zeros loses information – a zero row and a missing row look identical in the destination.
`inventory_movements`	Append-only log of all stock changes: sales, adjustments, transfers, write-offs. The activity signal for activity-driven extraction. Covers changes that never flow through `order_lines`.

The Soft Rules Baked In

The domain model is designed so that every "always true" business rule is, in fact, a soft rule. None of them have a constraint enforcing them:

- `orders` – "Always has at least one line." Until a UI bug creates an empty order.
- `orders` – "Status goes `pending` → `confirmed` → `shipped`." Until support resets one manually.
- `order_lines` – "Quantities are always positive." Until a return is entered as `-1`.
- `invoices` – "Only open invoices get deleted." Until a year-end cleanup script runs.
- `invoice_lines` – "Line status always matches the header." Until one line is disputed.
- `customers` – "Emails are unique." Until the same customer registers twice and nobody added the unique index.
- `inventory` – "`on_hand` is always >= 0." Until a write-off creates a negative balance.
- `inventory_movements` – "Every stock change creates a movement." Until a bulk import script updates `inventory` directly without logging movements.

When a pattern depends on one of these holding – or breaking – it will say so explicitly. See 1.6 – Hard Rules, Soft Rules.

1.1 The EL Myth

> **One-liner:** Pure EL doesn't exist. The moment data crosses between systems, some syntactic transformation is unavoidable.

1.1.1 Syntactic vs Semantic

You've heard of ETL. It's the standard for a reason: it's most useful when the Business layer and the Data layer are handled by the same person. This is hugely common among analysts, who do the vast majority of data consumption. But handling the intricacies of how to query a database without blowing it up with full table scans? That's a skill most of them don't have.

This is one of the reasons most companies, once they reach a certain size, choose to use an OLAP database for analysis while their ERP and internal apps keep using OLTP for ingestion.

> **OLAP vs OLTP**
>
> OLAP (analytical databases like BigQuery, Snowflake, and ClickHouse) stores data in a columnar way, optimized for full-column `SUM()`s and aggregations. OLTP (transactional databases like PostgreSQL, MySQL, and SQL Server) stores data row by row, optimized for inserts and transactional operations.

The ELT framework (Extract, **Load**, Transform) came as a byproduct of this. The pitch: "let's Extract and Load the data raw into our OLAP, then Transform it there." A valid way of thinking – which sadly forgets how fundamentally different OLTP and OLAP handle things, and how incompatible all SQL dialects really are. I can't simply copy a `DATETIME2` from SQL Server into BigQuery and expect it to behave. I have to cast, handle timezones, normalize dates, inject metadata, and of course – most of the time I want to update incrementally, which (believe me) can increase complexity ten-fold.

1.1.2 The Reality

Pure EL doesn't exist. The moment you move data between systems, something has to give. Types need casting, nulls need handling, timestamps need timezones. This is **syntactic transformation**, and it's unavoidable.

So the pipeline covers two distinct jobs: extraction, syntactic transformation, and loading. The syntactic layer handles type casting, null handling, timezone normalization, metadata injection, key synthesis – everything the data needs to land correctly on the other side. Anything that changes what the data *means* belongs downstream, in the serving layer.

1.1.3 What About the T?

When analysts aggregate, pivot, or build dashboards, they're doing semantic transformation – reshaping the data to serve a business question. That belongs in the serving layer, and Part VII covers it. There's still

a chapter in this book for helping them out, because left unsupervised, an analyst will `SELECT *` on a 3TB events table in Snowflake and then ask you why the bill spiked. I cover how to protect them (and your invoice) in 7.4 – Query Patterns for Analysts.

1.2 What Is Syntactic Transformation

> **One-liner:** Syntactic transformation is everything the data needs to survive the crossing. If it changes what the data means, it belongs in the serving layer.

1.2.1 The Line Between Syntactic and Semantic

To know what should be done by you, you must answer the following question: does this operation change what the data *means*, or does it just make it land correctly?

- Casting a `DATETIME2` to `TIMESTAMP`? Syntactic.
- Replacing `NULL` with `''` because BigQuery handles them differently in `GROUP BY`? Syntactic.
- Converting `BIT` to `BOOLEAN`? Syntactic.

None of these change the business meaning of the data. They just make it survive the crossing. The counter examples do:

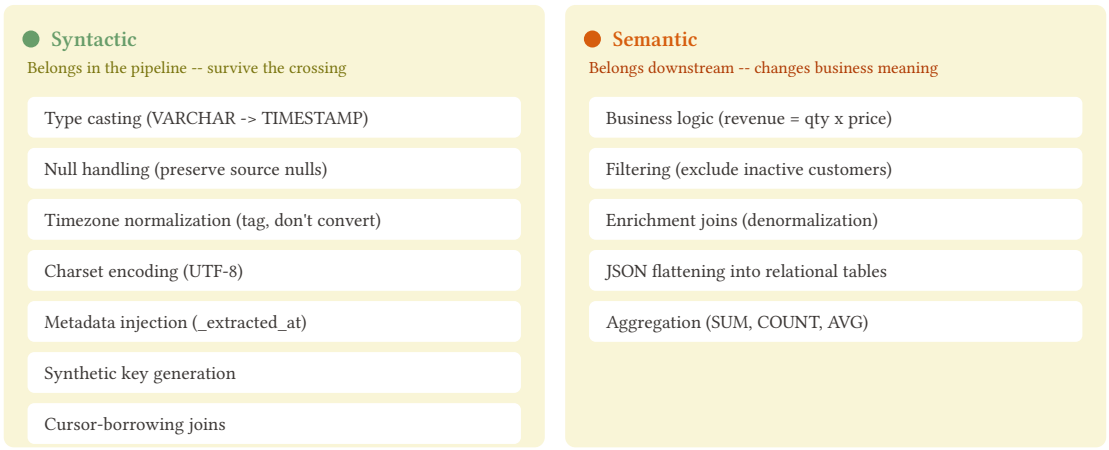
- Calculating `revenue = qty * price`? Semantic – you're creating a derived fact.
- Filtering out inactive customers? Semantic – you're applying a business rule.
- Joining `orders` with `customers` to denormalize a name? Semantic – you're adding meaning that wasn't in the original row.

These belong in the serving layer.

But here's where it gets interesting. `order_lines` has no `updated_at`. If you want to extract incrementally, you *need* to join with `orders` to borrow its timestamp as your cursor. That join doesn't add business meaning – it adds extraction metadata. You're not enriching `order_lines` with order data; you're giving yourself a `_cursor_at` so you know what to pull. That's syntactic.

> **The join test**
>
> If the join adds a column the business cares about, it's semantic transformation. If it adds a column only the pipeline cares about (`_cursor_at`, `_header_updated_at`), it's syntactic.

Syntactic vs semantic transformation -- one or the other during the crossing



1.2.2 The Syntactic Checklist

These are the syntactic operations that belong in the pipeline. Each one gets its own chapter in Part IV, but here's the overview so you know what you're signing up for.

Type casting. Every engine has its own type system, and they don't agree on anything. SQL Server's `DATETIME2` has nanosecond precision; BigQuery's `TIMESTAMP` has microseconds. PostgreSQL's `NUMERIC(18,6)` is exact; BigQuery's `FLOAT64` is not. You will lose precision if you don't map these explicitly. See 5.3 – Type Casting and Normalization.

Null handling. `NULL`, empty string, `0`, `'N/A'` – sources use all of them, and they're not the same thing. The rule here: reflect the source as-is. If the source has NULL, land NULL. Don't COALESCE to a default value at extraction – that's a business decision that belongs downstream. See 5.4 – Null Handling.

Timezone normalization. Source says `2026-03-15 14:30:00` – in what timezone? If the column is `DATETIME2` or `TIMESTAMP WITHOUT TIME ZONE`, you're looking at a naive timestamp. The rule: TZ stays TZ, naive stays naive. Don't convert naive to UTC unless you're certain of the source timezone – guessing wrong silently shifts every row. Know what you're landing and document the assumption. See 5.5 – Timezone Handling.

Charset and encoding. Latin-1 source, UTF-8 destination. Most of the time you won't notice, until a customer name has an `ñ` or an `ü` and your load silently replaces it with `?` or fails entirely. This is especially common with older ERP systems and legacy OLTP sources (SAP, AS/400, Oracle SQL). See 5.6 – Charset and Encoding.

Metadata injection. Every row you land could carry `_extracted_at`, `_batch_id`, and ideally a `_source_hash`. These columns don't exist in the source. You add them during extraction so you can debug, reconcile, and reprocess later. Without them, when something goes wrong (and it will), you have no way to know which batch brought the bad data. But you have to weigh its benefits against the additional processing and eventual delays it could bring. See 5.1 – Metadata Column Injection.

Metadata injection has a cost

At scale, hashing every row for `_source_hash` adds compute on the source or in your pipeline. At 800 tables, this can add 20 minutes to an already long extraction window. Evaluate per table: high-value mutable tables earn the overhead; stable config tables usually don't.

Analysts can use your metadata

"When was this data pulled into our warehouse?" is a valid question, and `_extracted_at` answers exactly that. Be precise though: `_extracted_at` is when *your pipeline* pulled the row, not when the row was last modified in the source. That's `updated_at` (if it exists). A row updated 3 days ago and extracted today has `_extracted_at = today`. Don't let anyone confuse the two.

Key synthesis. The source table has no primary key. Or it has a composite key that's 5 columns wide. Or worse, it has an `id` that gets recycled when rows are deleted. You need something stable to MERGE on, and if the source doesn't give you one, you build it: hash the business key columns into a `_source_hash` or synthesize one. See 5.2 – Synthetic Keys.

Boolean and decimal precision. SQL Server `BIT`, MySQL `TINYINT(1)`, SAP B1 `'Y'/'N'` (or `'S'/'N'` depending on install language), PostgreSQL `BOOLEAN` – every source has its own way of representing booleans. Similarly, `NUMERIC(18,6)` in PostgreSQL is exact while `FLOAT64` in BigQuery is not, and the rounding errors accumulate across millions of rows. Both of these are type casting concerns covered in 5.3 – Type Casting and Normalization.

Nested data / JSON. The source has a `details` column that's a JSON blob. Land it as-is – `STRING`, `JSONB`, `VARIANT`, whatever the destination's native JSON type is. Flattening JSON into normalized tables is semantic transformation – it restructures the data into a shape the source didn't have. If a consumer can't query JSON, build a flattening view downstream. See 5.7 – Nested Data and JSON.

1.3 Transactional Sources

> **One-liner:** Row-oriented, mutable, ACID. The terrain you're extracting from most of the time.

1.3.1 What Makes a Source Transactional

These databases were built to handle one row at a time, fast. INSERT a customer, UPDATE an order status, DELETE a cancelled invoice. They store data row by row, keep it consistent through transactions and locks, and index everything for quick point lookups.

This is great for the applications that sit on top of them. It's less great for you, because what you need to do – pull thousands or millions of rows in bulk – is the opposite of what they were optimized for. You're running full table scans on a system designed for `WHERE id = 42`.

The key thing to internalize: these sources are **mutable**. A row you extracted yesterday might look different today. Or it might be gone. The database won't send you a notification about it. You have to go looking.

1.3.2 The Engines

You'll run into a handful of these in the wild. They all speak SQL, but they all speak it differently.

PostgreSQL. The open source workhorse. Rich type system, `updated_at` triggers are common but never guaranteed. Some teams add them religiously, some don't bother. Watch out for TOAST compression on large text/JSON columns – it can make extraction slower than you'd expect because the data isn't stored inline with the row.

MySQL. Everywhere. If you've worked with a web application's database, you've probably worked with MySQL. The classic trap here is `utf8` vs `utf8mb4`: MySQL's `utf8` is actually 3-byte UTF-8, which means it can't store emoji or certain CJK characters. `utf8mb4` is real UTF-8. If you're extracting text columns coded as `utf8`, you might be getting truncated data and not know it. Also, InnoDB's `DATETIME` has no timezone information at all.

SQL Server. The enterprise standard. You'll see it behind most .NET applications and a lot of corporate ERPs. `DATETIME2` gives you up to 100-nanosecond precision, which sounds great until you try to land it in BigQuery's microsecond `TIMESTAMP` and realize you're truncating. Licensing and access are the real pain: getting read access to a production SQL Server often involves procurement, security reviews, VPN configs, and a DBA who has 47 other priorities before your extraction project.

SAP HANA. Column-oriented under the hood, but the applications on top of it (SAP B1, S/4HANA) treat it transactionally. Proprietary SQL dialect with its own quirks. Limited tooling for extraction compared to the others. Often sits behind thousands of auto-generated tables you're not supposed to query directly – and in the case of S/4HANA, legally might not be allowed to. If you're extracting from SAP, you're already in a special kind of hell and you know it.

Engine	Timezone trap	Encoding trap	Key gotcha
PostgreSQL	`TIMESTAMP` vs `TIMESTAMPTZ`	Usually UTF-8, but check	TOAST on large columns
MySQL	`DATETIME` has no TZ at all	`utf8` != real UTF-8	`utf8mb4` migration state
SQL Server	`DATETIME2` nanosecond precision	Latin-1 legacy common	Access/licensing friction
SAP HANA	Varies by SAP module	Depends on client codepage	Legally restricted access to some tables

1.3.3 What They All Share (That Matters for the Pipeline)

Despite the differences, the fundamentals are the same from an extraction perspective:

They all support `SELECT ... WHERE ...` for pulling data incrementally. That's your primary tool. Every batch extraction pattern in this book starts with a query against the source.

They all have *some* mechanism for detecting changes – `updated_at` columns, triggers, row versions – but none of them make it easy or consistent. Every engine does it differently, every application uses it differently (or doesn't use it at all), and you can't count on any of it being reliable until you've verified it yourself.

They all have schemas that mutate. Columns get added, types get changed, tables get renamed. The application team ships a release, and suddenly your `products` table has 3 new columns you've never seen.

```
-- last week your extraction query was:
SELECT product_id, name, price, category FROM products;

-- after Friday's deploy:
SELECT product_id, name, price, category FROM products;
-- ERROR: column "category" renamed to "product_category"
-- also: new columns "weight_kg", "is_hazardous", "supplier_id"
```

Your pipeline needs to handle this gracefully or it *will* break on a Friday night.

> **SELECT * is valid for extraction**
>
> Contrary to what every SQL best practices guide tells you, `SELECT \*` is a good default when you're cloning a table. You're cloning, not building a report. New column added? It lands automatically. Type changed? Your type dictionary handles it. But renames and deletes **must** fail your pipeline. If `category` becomes `product_category`, your destination still has `category` receiving no new data, and that's unacceptable. Schema relaxing means: always allow additions, handle type changes, never silently accept renames or deletions. More on this in 1.5 – The Lies Sources Tell.

And they all have data quality issues that the application layer "handles" but the database doesn't enforce. These are the soft rules – the things a stakeholder tells you are "always" true, but the schema doesn't guarantee.

1.3.4 What Will Bite You

Locks during extraction. Your `SELECT` could be reading millions of rows while some poor bastard is trying to create a new invoice. In SQL Server you can use `NOLOCK` to avoid blocking, but now you're reading dirty data – rows mid-transaction, half-updated. To be clear, I **don't** recommend using NOLOCK.

NOLOCK (dirty reads, no blocking)

```
-- engine: sqlserver
SELECT order_id, status, updated_at
FROM orders WITH (NOLOCK)
WHERE updated_at >= @last_extraction
```

Default (clean reads, blocks writers)

```
-- engine: sqlserver
SELECT order_id, status, updated_at
FROM orders
WHERE updated_at >= @last_extraction
-- writers wait until this finishes
```

Use a read replica? Now you have replication lag and you might miss rows that were committed seconds before your query ran. There's no free lunch here; you pick your trade-off and document it. (Also, do you really trust the people making the replica at source?)

No reliable `updated_at`. Some tables don't have one. Some have one that only fires on UPDATE, not INSERT. Some have one that the application sets manually and sometimes forgets. It may work 99.9% of the time, that still gets you chewed out when it doesn't.

```
-- engine: postgresql
-- what you expect:
SELECT order_id, created_at, updated_at FROM orders WHERE order_id IN (1001, 1002);
```

order_id	created_at	updated_at
1001	2026-01-15 09:00:00	2026-02-20 14:30:00
1002	2026-03-01 11:00:00	NULL

Order 1002 was just created. The trigger only fires on UPDATE, so `updated_at` is NULL. Your pipeline with `WHERE updated_at >= @last_run` will never see it.

You'll build your pipeline trusting `updated_at`, and three months later discover that 2% of rows were silently missed because the column wasn't being maintained. See 3.10 – Create vs Update Separation.

Hard deletes. The row was there yesterday. Today it's gone. The source won't tell you.

```
-- yesterday's extraction got 4 invoices:
SELECT invoice_id FROM invoices;
```

invoice_id
5001
5002
5003
5004

```
-- today's extraction gets 3:
SELECT invoice_id FROM invoices;
```

invoice_id
5001
5002
5004

Where's 5003? Deleted. No tombstone, no audit log, no `deleted_at` flag. Your destination still has it. Now your data says there are 4 open invoices when there are 3. Detecting hard deletes in batch extraction is one of the hardest problems in this book. See 3.6 – Hard Delete Detection.

Connection limits and DBA etiquette. You're a guest on someone else's production system. Open too many connections, run queries during peak hours, or full-scan their biggest table while the month-end close is running, and the DBA will shut you down. Rightfully.

```
-- your "quick extraction" at 10am:
SELECT * FROM order_lines
WHERE updated_at >= '2026-01-01'
-- 12 million rows, no index on updated_at
-- full table scan, 4 minutes, 100% CPU
-- meanwhile 200 users can't save orders
```

You need to know the source system's capacity, its busy hours, and its tolerance for your workload. See 6.7 – Source System Etiquette.

Encoding traps. The `customers` table has a `name` column. It's `VARCHAR(100)` in Latin-1. You didn't know it was Latin-1 because nobody told you and the metadata just says `VARCHAR`.

```
-- engine: sqlserver
-- what the source has:
SELECT customer_id, name FROM customers WHERE customer_id = 42;
```

customer_id	name
42	José Muñoz

```
-- what lands in BigQuery after a naive load:
SELECT customer_id, name FROM customers WHERE customer_id = 42;
```

customer_id	name
42	Jos? Mu?oz

Every ñ, ü, é silently replaced with `?`. Or worse, the load fails entirely and you don't know why until you dig into the byte encoding. Especially common with older ERP systems and legacy OLTP sources. See 5.6 – Charset and Encoding.

1.4 Columnar Destinations

> **One-liner:** Append-optimized, partitioned, cost-per-query. The terrain when you're loading into an analytical engine.

1.4.1 What Makes a Destination Columnar

These engines store data by column. Every value in `event_date` is packed together, compressed alongside every other `event_date` in the table. Scanning one column across a billion rows is fast. Aggregation – `SUM`, `COUNT`, `GROUP BY` – is what they were built for. The consumers sitting on top of them are dashboards, reports, ML pipelines, analysts running ad-hoc queries.

Three properties define the landing zone:

Append-cheap, mutate-expensive. Inserting new rows is fast and the engines optimize for it. Updating or deleting existing rows is a different story. BigQuery rewrites an entire partition on UPDATE. ClickHouse runs mutations as async background jobs with no completion guarantee. Snowflake handles it better than most, but every MERGE still costs warehouse time. Your load strategy should minimize mutations. Append first, deduplicate or materialize downstream.

Partitioned by default. Almost every table worth its storage is partitioned, usually by date. Partition pruning controls both query performance and cost for everyone downstream. If you load data without aligning to the partition scheme, every consumer pays the price on every query. The load engineer decides the partition strategy. Get it wrong and you're taxing every analyst and dashboard for the lifetime of the table.

Cost lives in the query. A transactional database costs you connections and CPU. A columnar destination costs you bytes scanned (BigQuery) or seconds of compute time (Snowflake, Redshift). A poorly partitioned table or a missing cluster key means every `SELECT` is more expensive than it needs to be. Your loading decisions have a direct, ongoing cost impact on everyone who reads from your tables.

1.4.2 The Engines

1.4.2.1 BigQuery

Serverless, slot-based. No cluster to manage, no warehouse to size. You load data and Google handles the rest. The pricing model is per-byte-scanned on queries, which means your table design directly affects what consumers pay.

Loading mechanics: `bq load` from cloud storage, streaming inserts, or `LOAD DATA` SQL. Parquet and JSONL are the most common formats, however AVRO is preferred. Streaming inserts are fast but newly streamed rows may be briefly invisible to `EXPORT DATA` and table copies – typically minutes, in rare cases up to 90 minutes – and streamed rows can't be modified or deleted until they flush.

DML concurrency is the constraint. BigQuery removed the daily per-table DML limit, but mutating statements (`UPDATE`, `DELETE`, `MERGE`) run at most 2 concurrently per table, with up to 20 queued. Stack too many rapid merges and the queue fills – additional statements fail outright. Prefer append + deduplicate over in-place mutation.

Partitioning is mandatory for cost control. A table without a partition scheme forces a full scan on every query. `require_partition_filter = true` protects consumers from themselves – it rejects any query that doesn't include the partition column in the `WHERE` clause.

```
-- engine: bigquery
-- table definition with partition, cluster, and cost protection
CREATE TABLE `project.dataset.events` (
  event_id STRING,
  event_type STRING,
  event_date DATE,
  payload JSON
)
PARTITION BY event_date
CLUSTER BY event_type
OPTIONS (require_partition_filter = true);
```

> **JSON columns and Parquet don't mix**
>
> BigQuery **cannot** load JSON columns from Parquet files – the job fails permanently. If your source data has JSON or semi-structured fields, load as JSONL. Or strip the JSON columns from the Parquet and load them separately. There's no workaround on the Parquet path.

1.4.2.2 Snowflake

Warehouse-based compute. You pay for the time the warehouse runs, regardless of bytes scanned. More predictable for budgeting, harder to attribute per-query.

Loading goes through stages: internal (Snowflake-managed) or external (S3, GCS, Azure). `COPY INTO` is the bulk loader and it's fast. Snowpipe automates continuous loading from stage to table.

Snowflake's `VARIANT` type handles semi-structured data natively. JSON, Avro, nested Parquet – it all lands in `VARIANT` and you query it with `:` path notation. But there's a catch: loading JSON from Parquet files converts `VARIANT` to a string. You need `PARSE_JSON` on the other side to get it back into a queryable structure.

Some loaders implement full replace via `CREATE TABLE ... CLONE` from staging – a metadata-only operation, fast – but permissions don't carry over on Snowflake. Ensure `FUTURE GRANTS` are set or your consumers lose access after every replace.

`PRIMARY KEY` and `UNIQUE` constraints exist in Snowflake's DDL but they're **not enforced**. They're metadata hints. If your pipeline relies on the destination rejecting duplicates, Snowflake won't help you. Deduplication is your problem.

```
-- engine: snowflake
-- bulk load from stage
COPY INTO events
FROM @my_stage/events/
FILE_FORMAT = (TYPE = 'PARQUET')
MATCH_BY_COLUMN_NAME = CASE_INSENSITIVE;
```

1.4.2.3 ClickHouse

Built for speed on append-only analytical workloads. The MergeTree engine family is the backbone – data lands in parts, and background merges compact them over time.

Fastest engine for raw insert throughput, and the only major columnar engine that gives you an explicit choice on **when** to pay the deduplication cost. `ReplacingMergeTree` (RMT) accepts duplicate keys on insert and collapses them by `ORDER BY` key during merges – with an optional version column to control which row wins. For incremental extract-and-load workloads this is a brilliant fit: every load is a pure `INSERT`, free of MERGE cost, and the dedup bill is either paid lazily by the background merge scheduler or shifted to read time. The trade-off is honest: until a merge runs, duplicates coexist in the parts, so reads that need single-version-per-key correctness have to ask for it – either with `SELECT ... FINAL` (per-query, partition-selective and parallelized since 23.3) or with an `argMax`/window-function dedup view, the same pattern 4.4 – Append and Materialize uses elsewhere. Avoid blanket `OPTIMIZE TABLE ... FINAL` – ClickHouse's own docs warn against it on large tables (single-merge, CPU/IO-heavy); use partition-scoped `OPTIMIZE PARTITION ... FINAL` on a schedule if you need eager compaction.

Mutations (`ALTER TABLE ... UPDATE`, `ALTER TABLE ... DELETE`) are async. You fire the statement and it returns immediately. The actual work happens whenever the merge scheduler gets to it. Consumers querying between the mutation request and the merge see pre-mutation data. RMT sidesteps this entirely – you don't issue updates or deletes, you insert the new version and let the engine collapse.

For this kind of workload, ClickHouse rewards leaning into the merge model. Append every batch into a `ReplacingMergeTree`, build materialized views for the latest state, and choose your dedup tactic per table – scheduled `OPTIMIZE` on hot tables that get read often, `FINAL` on cold tables where read overhead is acceptable, or a mix. What's expensive on other engines (frequent incremental updates) becomes a sequence of cheap appends here. The pain only comes from fighting RMT with row-level `UPDATE`/`DELETE`, which the engine schedules async and offers no completion guarantee for.

```
-- engine: clickhouse
-- table definition with merge-based deduplication
CREATE TABLE events (
  event_id String,
  event_type String,
  event_date Date,
  payload String
)
ENGINE = ReplacingMergeTree()
PARTITION BY toYYYYMM(event_date)
ORDER BY (event_date, event_id);
```

1.4.2.4 Redshift

PostgreSQL dialect, columnar storage underneath. Sort keys and dist keys determine how data is physically laid out and how queries perform. Get them right and range queries fly. Get them wrong and every query shuffles data across all nodes.

Loading goes through `COPY` from S3. This is the fast path. Row-by-row `INSERT` is painfully slow compared to `COPY` – orders of magnitude slower on any meaningful volume. If your pipeline isn't using `COPY`, fix that first.

Redshift runs automatic VACUUM DELETE in the background, so routine cleanup is handled. But if your pipeline does heavy bulk deletes (hard delete detection, merge patterns with large DELETE+INSERT batches), automatic VACUUM may not keep up – monitor `SVV_TABLE_INFO` for unsorted rows and dead-row bloat, and schedule manual VACUUM during off-peak if needed.

Sort keys and dist keys can be changed via `ALTER TABLE` without a full rebuild, but the operation rewrites data in the background and can take hours on large tables – plan them carefully at creation rather than treating them as easily adjustable. Column additions are cheap. Type changes require recreating the table. A `VARCHAR(100)` that should have been `VARCHAR(500)` means a full table rebuild later. Plan your types carefully on initial load.

1.4.3 Type Mapping: Where Syntactic Transformation Happens

When data crosses from a transactional source to a columnar destination, types don't translate cleanly. This is where most of the syntactic work lives, and every engine has its own version of the problem.

Timestamps. PostgreSQL's `TIMESTAMP WITHOUT TIME ZONE` landing in BigQuery becomes `TIMESTAMP`, which is always UTC. If the source stored local times without timezone info, BigQuery now treats them as

UTC and every downstream consumer gets the wrong time. Snowflake distinguishes `TIMESTAMP_TZ` from `TIMESTAMP_NTZ`, so you have a choice – but you have to make it explicitly. See 5.5 – Timezone Handling.

Source Type	BigQuery	Snowflake	ClickHouse	Redshift
`TIMESTAMP` (naive)	`TIMESTAMP` (forced UTC)	`TIMESTAMP_NTZ`	`DateTime`	`TIMESTAMP`
`TIMESTAMPTZ`	`TIMESTAMP`	`TIMESTAMP_TZ`	`DateTime64` with tz	`TIMESTAMPTZ`
`DATETIME2(7)` (SQL Server)	Truncated to microseconds	`TIMESTAMP_NTZ(9)`	`DateTime64(7)`	Truncated to microseconds

BigQuery has no naive datetime

Every `TIMESTAMP` in BigQuery is timezone-aware. If your source has naive timestamps, BigQuery treats them as UTC. If they were actually in `America/Santiago` or `Europe/Berlin`, every value is wrong from the moment it lands. You must handle the timezone during load.

Decimals. `NUMERIC(18,6)` in PostgreSQL has fixed precision. BigQuery's `NUMERIC` supports up to `NUMERIC(38,9)`. Snowflake offers `NUMBER(p,s)` with configurable precision, or `DECFLOAT` for unbound decimals – but DECFLOAT only works with JSONL and CSV loads. Parquet doesn't support it. If your loader converts to `FLOAT64` anywhere in the pipeline, you lose precision. Financial data loaded as floating point is a bug waiting to surface. See 5.3 – Type Casting and Normalization.

JSON and nested data. JSON columns from a transactional source need special handling at every destination:

Destination	JSON Handling	Gotcha
BigQuery	`JSON` type (native)	Cannot load from Parquet. Use JSONL.
Snowflake	`VARIANT` type	Parquet loads produce string, needs `PARSE_JSON`
ClickHouse	`JSON` (native since v25.3) or `String`	Native `JSON` type is recent – legacy tables use `String` with `JSONExtract*`
Redshift	`SUPER` type	Semi-structured queries use PartiQL syntax

See 5.7 – Nested Data and JSON for detailed engine mappings.

1.4.4 Load Formats

Parquet is the fastest and most type-safe bulk load format. But BigQuery's JSON column type is not supported when loading from Parquet – use JSONL or Avro instead. Parquet also produces strings for Snowflake's VARIANT and doesn't support DECFLOAT (Snowflake). Avro is BigQuery's preferred format – it handles JSON natively, preserves schema, and sidesteps the Parquet-JSON limitation entirely. Snowflake and ClickHouse also support Avro loads, though Snowflake still lands complex types into VARIANT. Redshift supports Avro through `COPY` but has a 4 MB max block size limit – Avro blocks larger than that fail the job. JSONL is slower but handles everything on every engine. CSV loses type information entirely.

Format	Speed	Type Safety	JSON Support	Gotcha
Parquet	Fast	High (schema embedded)	BigQuery: JSON columns unsupported, use JSONL/ Avro. Snowflake: string.	Decimal edge cases per engine
Avro	Fast	High (schema embedded)	BigQuery: native. Snowflake: VARIANT.	Redshift: 4 MB max block size. ClickHouse needs schema registry or embedded schema.
JSONL	Medium	Medium (inferred)	Full support everywhere	Slower on large volumes
CSV	Varies	Low (everything is string)	Must quote/escape	Type inference can surprise you. Compressed CSV can't be split for parallel load.

Default format recommendation

BigQuery → Avro. Every other columnar destination with no JSON columns → Parquet (including Redshift, where `COPY` handles Parquet natively with automatic parallel splitting). When you hit edge cases (JSON columns on BigQuery, Snowflake DECFLOAT, Redshift Avro block size limit) → JSONL. **Avoid CSV like the plague** unless your destination gives you no other choice.

1.4.5 Loading Strategies

Your write disposition – replace, append, merge – determines cost, complexity, and correctness. The full spectrum is in Part IV (4.1 – Full Replace Load through 4.5 – Hybrid Append-Merge). When in doubt, full replace: it resets state, eliminates drift, and makes every run idempotent by construction. Earn the complexity of incremental only when the table justifies it.

1.4.6 Schema Evolution

New columns will appear in the source. Your loader needs a schema policy: `evolve` (add new columns automatically) or `freeze` (reject the load). Per-engine behavior for `ALTER TABLE ADD COLUMN`, type widening, and the gotchas of each engine are covered in 8.1 – SQL Dialect Reference. Schema policies as enforceable contracts are in 6.9 – Data Contracts.

Naming convention lock-in

Identifier normalization at load time is a syntactic operation with permanent consequences. If your loader converts `OrderID` to `order_id` (snake_case), that's the column name forever. Changing the convention later re-normalizes already-normalized identifiers, causing failures. Choose your naming convention on day one and never look back.

1.4.7 What Will Bite You

DML concurrency (BigQuery). BigQuery removed the daily per-table DML limit, but mutating statements run at most 2 concurrently per table with up to 20 queued. Flood it with rapid-fire merges and the queue fills – additional statements fail outright. Batch your loads and buffer in a staging table to keep the queue clear.

Partition misalignment. On BigQuery, every DML statement (`MERGE`, `UPDATE`, `DELETE`) rewrites the entire partition it touches – not just the affected rows. If your batch contains data spread across 30 dates, that's 30 full partition rewrites per load run. Costs multiply fast. Keep your load batches as aligned to partition boundaries as possible: one run = one partition. If you can't control the source data spread, stage everything first and then execute one DML statement per partition.

Unique constraints that aren't. Snowflake and BigQuery both accept `PRIMARY KEY` and `UNIQUE` in DDL but neither enforces them – they're optimizer hints, not guarantees. ClickHouse has no unique constraint mechanism outside of `ReplacingMergeTree` (which deduplicates eventually, during merges). If your pipeline assumes the destination rejects duplicates, it won't. Deduplication is your responsibility, always. See 6.13 – Duplicate Detection.

Partition limits. BigQuery caps date-partitioned tables at 10,000 partitions – and a single job (load or query) can only touch 4,000 of them. Partition by day on a table running for 27+ years and you hit the first wall. Try to MERGE or load a batch spanning more than 4,000 dates and BigQuery rejects the job outright. The fix is partitioning by month or year instead of day, but that's a table rebuild. ClickHouse has a different version of this: each INSERT creates a new part, and MergeTree can only merge so fast. Flood it with small inserts and you trigger "too many parts" errors, which throttle further writes until the merge scheduler catches up. Batch your inserts; never insert row by row.

1.5 The Lies Sources Tell

> **One-liner:** A catalog of things you'll be told are true about the source, and why your pipeline can't trust any of them.

Every source system comes with a set of assumptions handed to you by the team that owns it. Some are true. Most are "true until they're not." This chapter is about the ones that will eventually fail – and what to do when they do.

The lies aren't usually malicious. They're the product of developers who built the application for a use case that didn't include data extraction, stakeholders who describe what *should be true* instead of **what is**, and systems that were designed when the data volume was a hundredth of what it is today. Your pipeline has to survive all of them.

The lies sources tell: what they claim vs what actually happens

CLAIM	REALITY
✓ *"The schema is stable."* documented, agreed, signed off	✗ until a column appears next quarter. products gains discontinued_at, no migration notice
✓ *"updated_at is reliable."* trigger updates it on every write	✗ until a bulk update bypasses the trigger. ops script touches 50k rows, cursor misses them all
✓ *"Primary keys are unique and stable."* email serves as the customer key	✗ until a customer registers twice with the same email. no unique index, merge upsert silently collapses two people
✓ *"Deletes don't happen, we use soft deletes."* is_active = false is the policy	✗ until support hard-deletes a draft invoice. row vanishes from source, destination keeps a ghost
✓ *"Timestamps have timezones."* everything is in UTC, the docs say so	✗ until you find naive TIMESTAMP storing local time. BigQuery treats it as UTC, every value is off by hours
✓ *"The data is clean."*	✗ until you find -1 quantities, NULL FKs, 5 spellings of "active".

1.5.1 “The schema is stable”

The most common lie, and the one with the longest tail. Columns get added because a developer needed a new field. Columns get renamed because a product manager decided `cust_flg` should be `is_customer`. Columns get deleted because someone thought “nobody uses this.” None of these come with a heads-up to the data team.

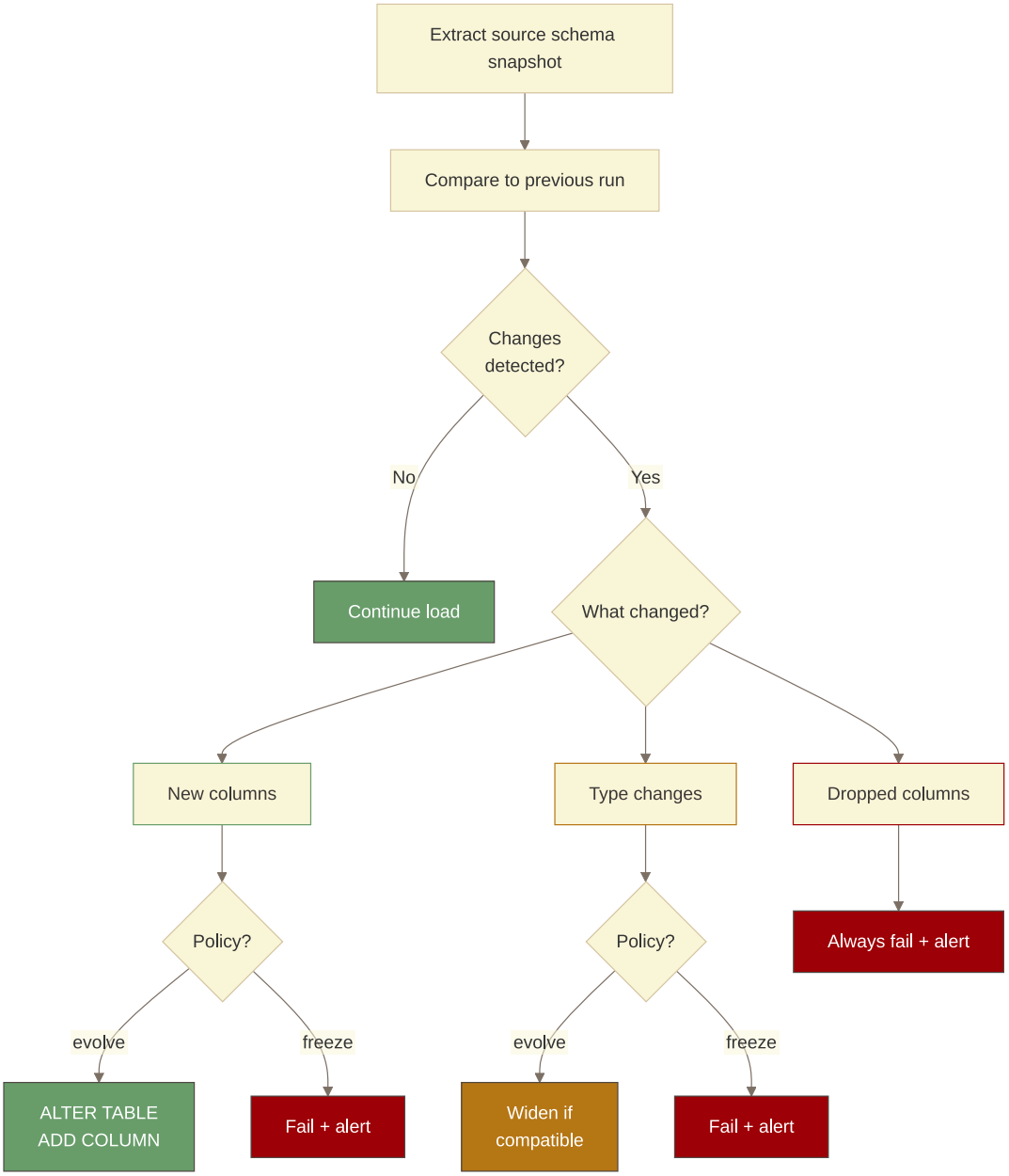
Two principles for surviving schema instability:

Use `SELECT *` at extraction. You’re cloning, not reporting. If the source adds a column, your extraction should pick it up automatically. A `SELECT id, name, email, ...` query is a time bomb – the moment the source adds a column you’re not listing, it silently disappears from your destination.

The detection should happen at the source, before you touch the destination. On each run, snapshot the source schema from `information_schema` and diff it against the snapshot from the previous run (Cached locally or queried on demand, depending on the frequency of update):

```
-- source: transactional
-- Pull current schema snapshot before extraction
SELECT
    column_name,
    data_type,
    ordinal_position,
    is_nullable
FROM information_schema.columns
WHERE table_schema = 'public'
  AND table_name = 'customers'
ORDER BY ordinal_position;
```

Your pipeline should store this result after every successful run. On the next run, compare new vs stored: columns present last run but absent now = deletion or rename – fail immediately, before any data moves. Columns absent last run but present now = addition – add to the destination and continue.



What the loader does in response to that diff is your schema policy (`evolve` or `freeze`) – covered in 1.4 – Columnar Destinations. When it fails, alert it – see 6.5 – Alerting and Notifications.

Schema auto-detection isn't free

BigQuery's `LOAD DATA` with schema auto-detection and Snowflake's `VARIANT` both absorb new columns. But auto-detection can mistype a column (a column with only `"1"` and `"0"` values gets inferred as `BOOL`). And absorbing new columns silently means you won't notice when the column name changed and you now have two columns – one dead and one alive – both tracking the same concept.

See 6.9 – Data Contracts for schema contract enforcement patterns.

1.5.2 "`updated_at` is reliable"

`updated_at` is the most common source of cursor for incremental extraction. It's also the most commonly broken one.

Only fires on UPDATE, not INSERT. An `ON UPDATE` trigger or application code that only sets `updated_at` when a row is modified. New rows arrive with `updated_at = NULL`. Your incremental extraction filters `WHERE updated_at > :last_run` and misses every new row forever.

```
-- source: transactional
-- Orders with no updated_at -- these will never be picked up by a cursor query
SELECT COUNT(*) AS missing_cursor
FROM orders
WHERE updated_at IS NULL;
```

If this returns anything above zero, your incremental extraction is already incomplete.

Set by the application, not the database. Application code that does `UPDATE orders SET ..., updated_at = NOW() WHERE order_id = :order_id`. A direct SQL edit from a back-office script, a database migration, or a developer with `psql` open doesn't go through the application layer. Those rows don't get a new `updated_at`. Your pipeline never sees them change.

> **From production: A closed-month correction with no updated_at change**
>
> A client told us a large correction they'd made to an old, already-closed month wasn't showing up downstream. We pulled the rows they named and found their `updated_at` values unchanged from months earlier, even though the data behind them was clearly different. That single check surfaced the real problem: their corrections weren't going through anything that touched `updated_at`, so the database never recorded that the rows had changed and the cursor kept doing its job perfectly against a signal nobody was maintaining. See 3.1.4 – The Periodic Full Replace for how we ended up loading that table.

The index isn't there. `updated_at` exists but nobody put an index on it. Your incremental query runs a full table scan on every execution. For a table with 50M rows, that's a multi-second query just to find the 200 rows that changed. On a transactional system under concurrent load, that scan will get you a complaint (or a ban) from the DBA.

> **Verify before you commit**
>
> Before building an incremental extraction on a cursor, run three checks: (1) query `WHERE updated_at IS NULL` – if it returns rows, you need a fallback; (2) run `EXPLAIN` on your cursor query – confirm it hits the index; (3) if you can – create a row, wait a minute, then update it and check that `updated_at` changed both times. If any check fails, treat this as an unreliable cursor and plan accordingly.

See 3.10 – Create vs Update Separation for the pattern when `updated_at` only fires on update, and 4.6 – Reliable Loads for fallback strategies.

1.5.3 “Primary keys are unique and stable”

Three different lies bundled into one: that the PK uniquely identifies a row, that the PK stays the same across the row’s lifetime, and that every table even has a PK.

The business doesn’t understand unicity. This is the most common one. You ask “what’s the primary key?” and get “order_id.” You build your merge on `order_id`. A week later you start seeing duplicates or losing data. Turns out the table stores one row per `(order_id, line_number)` – the person you asked thinks about orders, not about how the table is structured. Or it’s `(product_id, warehouse_id)` for inventory, but they only ever query their own warehouse so they genuinely never noticed. The people closest to the application rarely think in terms of relational keys. Never trust a verbal description of the PK. Run the duplicate check yourself on the actual data.

```sql
-- source: transactional
-- Verify that the claimed PK actually produces unique rows
SELECT order_id, COUNT(*) AS occurrences
FROM orders
GROUP BY order_id
HAVING COUNT(*) > 1
ORDER BY occurrences DESC
LIMIT 20;
```

If it returns rows, go back and ask which *combination* of columns is actually unique. Then run the check again on that combination.

The recycled PK. Delete a row, insert a new one, and many systems reuse the integer ID. If your destination has the old row and you receive a new row with the same ID, you have a collision – did the old row change, or is this a completely new entity? In most pipelines you’ll upsert it as an update. The old entity’s history is gone (which may be exactly what you want, to be fair).

The key whose semantics changed. A table that used to have one row per `order_id` gets a `tenant_id` column in a multi-tenant migration. Now uniqueness requires `(tenant_id, order_id)`. The column names didn’t change – the rule for what makes a row unique did. Your pipeline is still merging on `order_id` alone and quietly colliding across tenants.

> **Nullable columns in merge keys**
>
> A merge key column that allows NULLs is a silent bug. In SQL, `NULL != NULL` – two rows where the key column is NULL won’t match on a JOIN or MERGE. Your upsert might skip them and insert duplicates instead of updating. Check nullability on every column you plan to use as a merge key before you build on it. See 5.2 – Synthetic Keys.

No PK at all. Some tables were created without a primary key: reporting tables, view-like tables, tables built by BI teams – or, god help you, by someone in Finance with direct database access. You discover this at extraction time when the key you thought was there has repeated values, or worse, when you find actual duplicates in your destination database.

Run this before you commit to an extraction strategy. If the duplicate check returns rows on a column that was supposed to be unique, your merge key is broken.

DDL says PK exists -- does the engine enforce it?

On transactional sources, a `PRIMARY KEY` constraint is enforced – the engine rejects duplicates. The risk is a DBA who drops it temporarily for a bulk load and forgets to recreate it. `information_schema.table_constraints` tells you what the DDL says. A query for duplicates tells you what the data does. Check both before trusting a PK as your merge key.

See 5.2 – Synthetic Keys for building stable merge keys when the source can't be trusted.

1.5.4 "Deletes don't happen" / "We use soft deletes"

Every system that "never does hard deletes" does hard deletes. The application layer soft-deletes. The back-office script does a real `DELETE`. The developer debugging a data issue deletes the bad rows directly. The scheduled cleanup job runs every night and deletes pending records older than 90 days, and nobody told the data team.

Soft deletes that aren't consistently applied. The `is_active = false` flag works for normal application flows. It doesn't work for the script that directly manipulates `customers` to merge duplicate accounts. Those old accounts just disappear. If your extraction only checks `updated_at`, you'll never see them go.

The "only open invoices get deleted" rule. Classic soft rule from the domain model. Posted invoices are supposedly immutable. Then someone runs a year-end cleanup and deletes a batch of incorrectly posted invoices. Your pipeline has them as closed invoices in the destination forever. The discrepancy surfaces at audit time, not at load time; and it's your fault for not noticing.

```
-- source: transactional
-- Detect hard deletes by comparing yesterday's extracted IDs to today's source
-- Run on the source before extracting
SELECT COUNT(*) AS rows_in_source_today
FROM invoices;
```

```
-- destination: columnar
-- Compare to yesterday's destination count for the same scope
SELECT COUNT(DISTINCT invoice_id) AS rows_in_destination_yesterday
FROM stg_invoices
WHERE DATE(_extracted_at) = DATE_SUB(CURRENT_DATE(), INTERVAL 1 DAY);
```

A drop in the source count with no matching deletes in the destination means hard deletes happened. The only reliable way to detect them is a full count comparison or a full ID set comparison between yesterday's destination and today's source. Incremental extraction on `updated_at` is blind to deletions by design – deleted rows have no `updated_at` because they no longer exist.

Soft delete flags have their own problems

`is_active`, `deleted_at`, `status = 'deleted'` – they all require that every code path removing a record goes through the application layer and sets the flag. Back-office scripts, direct DB access, bulk operations, and third-party integrations often don't. The flag is only as reliable as every write path that touches the table.

See 3.6 – Hard Delete Detection for detection and propagation patterns.

1.5.5 "Timestamps have timezones"

The source database uses `TIMESTAMP WITHOUT TIME ZONE` (PostgreSQL) or `DATETIME` (MySQL, SQL Server). The application "knows" it's UTC. The column doesn't say so. Your pipeline reads `2026-03-06 14:00:00`, assumes UTC, and writes it to BigQuery as `2026-03-06 14:00:00 UTC`.

Except the application was deployed in Santiago, Chile. The value was stored as local time. That's `2026-03-06 17:00:00 UTC`. Every timestamp in your destination is off by 3 hours. The business has been making decisions on wrong data for two years. Nobody noticed because the relative order of events was correct and the absolute times were never spot-checked.

The daylight saving version is worse: the offset changes twice a year, so the error is inconsistent. Some rows are off by 3 hours, others by 4, depending on when they were written.

```
-- source: transactional
-- PostgreSQL: check whether the column has timezone info
SELECT column_name, data_type, datetime_precision
FROM information_schema.columns
WHERE table_name = 'orders'
  AND data_type IN ('timestamp without time zone', 'timestamp with time zone');
```

column_name	data_type	datetime_precision
created_at	timestamp without time zone	6
updated_at	timestamp without time zone	6

`timestamp without time zone` means the database is storing local time with no context. You need to know the application's intended timezone before you can handle the timezone correctly. There's no way to infer it from the data.

BigQuery makes this unforgiving

BigQuery has no naive timestamp type. Every `TIMESTAMP` is UTC. Load a naive timestamp and BigQuery silently treats it as UTC. If it was stored in local time, every value is wrong – and there's no way to fix it after the fact without knowing the original timezone and reloading.

See 5.5 – Timezone Handling for the full timezone-handling playbook.

1.5.6 "The data is clean"

The most optimistic lie. Data is only clean in demos, for everything else, you have to negotiate.

Orphaned foreign keys. `order_lines` rows pointing to an `order_id` that no longer exists. This happens after hard deletes (orders deleted, lines left behind), after migrations (data moved between systems with FK constraints disabled), or after application bugs. Your pipeline loads `order_lines` and the JOIN to `orders` returns NULL. Downstream reports show revenue lines with no associated order.

Duplicate "unique" values. The `customers` table is keyed on email in the application layer – but there's no unique index in the database. Two registrations with the same email land as two rows. Your destination has two customer records with the same email. Every report that uses email as a join key gets doubled.

Constraint violations that went unnoticed. Negative quantities in `order_lines`. Statuses that skipped steps (`pending` directly to `shipped`). NULL values in columns that are "never null." These pass through extraction without errors because your pipeline only checks what's there – it doesn't validate against what should be there.

```sql
-- source: transactional
-- Check orphaned order_lines
SELECT COUNT(*) AS orphaned_lines
FROM order_lines ol
LEFT JOIN orders o ON ol.order_id = o.order_id
WHERE o.order_id IS NULL;
```

```sql
-- source: transactional
-- Check duplicate customer emails
SELECT email, COUNT(*) AS occurrences
FROM customers
GROUP BY email
HAVING COUNT(*) > 1
ORDER BY occurrences DESC
LIMIT 10;
```

Run these as pre-extraction quality checks. They won't always block your pipeline – sometimes you load the dirty data and let downstream handle it – but you need to know the contamination level before you decide.

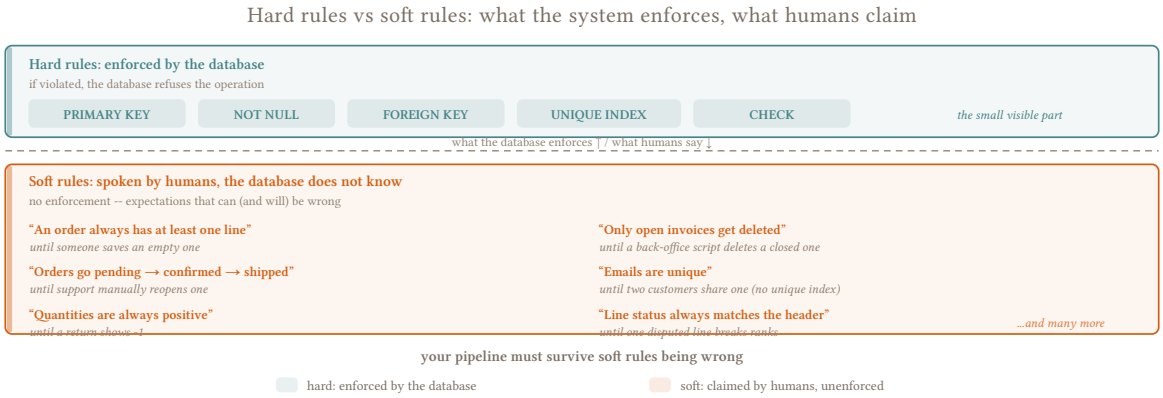
> **The pipeline clones, it doesn't clean**
>
> Your job is to faithfully clone the source to the destination, not to fix the application's data quality problems. An orphaned foreign key in the source should land as an orphaned foreign key in the destination. If you silently drop those rows, downstream teams are missing data and they don't know it. Load it, flag it, let the business decide what to fix.

This is where 1.6 – Hard Rules, Soft Rules becomes critical. Constraints the database doesn't enforce are soft rules. Your pipeline must survive them being wrong – but it should also surface when they are wrong, so someone can fix the root cause.

1.6 Hard Rules, Soft Rules

> **One-liner:** If the database enforces it, it's hard. If a stakeholder told you it's always true, it's soft – and your pipeline must survive it being wrong.

Every source system comes with two layers of truth: what the database actually enforces, and what the business believes is true. These are not the same thing. Your pipeline has to know the difference, because one is a guarantee and the other is a hope (and a pain).



1.6.1 The Distinction

A **hard rule** is enforced by the system. A foreign key constraint, a unique index, a NOT NULL column, a CHECK constraint. The database rejects violations at write time – the bad data never lands. You can build on hard rules unconditionally. If `order_id` has a unique index, your merge key is solid. If `customer_id` is a NOT NULL foreign key into `customers`, every order line has a customer. The system guarantees it.

A **soft rule** is a business expectation with no enforcement behind it. "Quantities are always positive." "Orders go from pending to confirmed to shipped." "Only open invoices get deleted." These are descriptions of how the application is *supposed* to work, told to you by people who have never seen them violated – because violations happen in prod, on the weekend, through a back-office script nobody documented.

The danger with soft rules is that they feel like hard rules. They hold for months. Your pipeline depends on them and nothing breaks. Then one day a developer runs a bulk update that bypasses the application layer, or support manually resets a status, or someone writes a cleanup script and forgets a WHERE clause. The soft rule breaks, and your pipeline either crashes, silently corrupts data, or both.

1.6.2 How to Tell Them Apart

Query the source before you build:

```
-- source: transactional
-- PostgreSQL: enumerate what the database actually enforces on orders
SELECT
    tc.constraint_type,
    tc.constraint_name,
    kcu.column_name
FROM information_schema.table_constraints tc
JOIN information_schema.key_column_usage kcu
    ON tc.constraint_name = kcu.constraint_name
WHERE tc.table_name = 'orders'
ORDER BY tc.constraint_type, kcu.column_name;
```

constraint_type	constraint_name	column_name
PRIMARY KEY	orders_pkey	order_id
FOREIGN KEY	orders_customer_id_fkey	customer_id
NOT NULL	(column constraint)	created_at

What's on this list is hard. Everything else – every verbal description, every README, every data dictionary entry – is soft until proven otherwise. The tell is simple: "this column is always X" with no corresponding constraint in the output above is a soft rule.

> **Treat data dictionaries as soft**
>
> A data dictionary that describes expected values (`status` is one of `pending`, `confirmed`, `shipped`) is documentation, not enforcement. Unless there's a CHECK constraint or an enum type backing it, the column will accept anything the application sends. Validate against the data, not against the dictionary.

1.6.3 The Soft Rules in the Domain Model

These are all real-world patterns disguised as fictional tables. Every one of them will eventually be violated:

Table	Soft Rule	How It Breaks
`orders`	"Always has at least one line"	Empty order saved by a UI bug, or created programmatically before lines are added
`orders`	"Status goes `pending` → `confirmed` → `shipped`"	Support team manually resets a status; migration script backdates records
`order_lines`	"Quantities are always positive"	Return entered as `-1`; bulk correction script uses negative values
`invoices`	"Only open invoices get deleted"	Year-end cleanup script deletes incorrectly posted invoices
`invoice_lines`	"Line status always matches header status"	One line disputed while the rest are approved; partial cancellation
`customers`	"Emails are unique"	Duplicate registration; customer service merges accounts manually

None of these have a constraint in the DDL. All of them are described as "always true" by the people who built the system.

1.6.4 What to Do When a Soft Rule Breaks

Load the data. Don't drop rows. A row that violates a soft rule is still a row that exists in the source. If you drop it, downstream teams are missing data and they don't know why. Dirty data is visible and fixable. Missing data is invisible and dangerous.

Surface the violation. Log it. Flag affected rows with a metadata column so consumers can filter or investigate:

```
-- source: columnar
-- Flag order_lines with negative quantities at load time
SELECT
    *,
    CASE WHEN quantity < 0 THEN TRUE ELSE FALSE END AS _flag_negative_quantity
FROM stg_order_lines;
```

Don't fix it in the pipeline. Coalescing a negative quantity to zero, skipping orders with no lines, or normalizing a status that skipped a step are all transformations that change business data. That belongs downstream, in the hands of whoever owns the business logic. Your job is to clone faithfully and report honestly.

> **Silent correction is the worst outcome**
>
> Fixing soft rule violations in the pipeline hides the root cause. The source system keeps producing bad data, the pipeline keeps silently correcting it, and nobody ever fixes the application. Six months later, someone queries the source directly and finds data that doesn't match the destination. Now you have a trust problem and an archaeology project.

See 6.9 – Data Contracts for how to formalize soft rule monitoring into data contracts with alerting.

1.6.5 Soft Rules and Load Strategy

This is where soft rules cause the most damage in practice: incremental extraction.

The most common failure mode in incremental loading is building a cursor on `updated_at` when `updated_at` is a soft rule. The assumption is that **every** write to a row bumps `updated_at`. It's "always" true. Until it isn't.

Open orders that don't move. An order sitting in `pending` for three weeks never gets touched by the application. Its `updated_at` doesn't change. Your incremental extraction ignores it. Then a bulk migration script updates the `customer_id` on a set of orders and doesn't touch `updated_at`. Your pipeline never sees the change.

Header timestamps without line propagation. An invoice header gets `updated_at` bumped when it's confirmed. The individual `invoice_lines` have no timestamp of their own and no trigger to update when the header changes. Your incremental extraction on `invoice_lines` misses every status change that came through the header.

Bulk scripts that bypass the ORM. A price update runs directly against the `products` table via a SQL script. The ORM's `before_update` hook – which sets `updated_at` – never fires. Your pipeline sees no changes. The prices in the destination are stale.

The mitigations all involve some form of lookback – accepting that `updated_at` isn't perfectly reliable and building in a safety net:

- Reprocess all rows with `updated_at` in the last n days on every run, not just since the last checkpoint
- Reprocess all open/pending records unconditionally, regardless of timestamp – their status can change without bumping `updated_at`
- Run a scoped replace (partition swap) of the current year once a week/day to catch anything the cursor missed

Full replace sidesteps all of this. A table that gets fully replaced every run doesn't care whether `updated_at` is reliable – the whole thing comes fresh. This is another reason to default to full replace and earn incremental complexity only when the table is genuinely too large or too slow to reload. See 1.8 – Purity vs. Freshness.

See 3.2 – Cursor-Based Timestamp Extraction for lookback window patterns, and 4.6 – Reliable Loads for building incrementals that survive unreliable cursors.

1.7 Corridors

> **One-liner:** Same pattern, different trade-offs. Where the data goes changes how you implement everything.

A corridor is the combination of source type and destination type. The extraction pattern looks the same – query the source, apply syntactic transforms, load – but the implementation decisions change completely depending on which corridor you're in. Get this wrong early and you'll have to redo a lot of work on the

pipeline to make it useful, or worse, bleed a ton of money on a system that's mission critical and hard to alter.

1.7.1 The Two Corridors

Transactional → Columnar. PostgreSQL to BigQuery. SQL Server to Snowflake. MySQL to ClickHouse. This is the primary corridor this book focuses on and the one you'll work in most often. The source is mutable, row-oriented, ACID. The destination is append-optimized, columnar, cost-per-query. The gap between them is wide: everything from type systems to cost models to mutation semantics is different.

Transactional → Transactional. PostgreSQL to PostgreSQL. MySQL to SQL Server. An ERP database to a reporting replica. Same class of engine on both ends, often the same dialect. The gap is narrower but not zero – still have schema drift, cursor problems, and hard deletes. And you now have a destination that actually enforces FK constraints, rejects duplicates, and supports cheap `UPDATE`/`DELETE`. That changes your load strategy significantly.

This book doesn't cover Columnar → Columnar or Columnar → Transactional. The first is rare and usually handled by the analytical platform itself (BigQuery cross-region replication, Snowflake data sharing). The second is unusual enough to be its own project.

Corridors -- the source/destination pairing changes every decision

Transactional → Columnar e.g. PostgreSQL → BigQuery		Transactional → Transactional e.g. PostgreSQL → PostgreSQL	
Type system gap	Wide	Type system gap	Narrow
Mutation cost	Expensive	Mutation cost	Cheap
Constraints	None enforced	Constraints	PK, FK, UNIQUE
Cost model	Per bytes/slots	Cost model	Fixed infra
Default load	Append, partition swap	Default load	MERGE / upsert
Deduplication	Your problem	Deduplication	Engine enforces

> **Corridor effect on load strategies**
>
> In T→T, MERGE is cheap and native – the load strategy spectrum compresses. In T→C, MERGE is the costliest DML and the full spread matters.

1.7.2 What Changes at the Crossing

The corridor determines your constraints. Getting this wrong means building a pipeline with the wrong mental model from the start.

Mutation cost. In a transactional destination, `UPDATE` and `DELETE` are cheap, indexed, and ACID. You can MERGE freely. In a columnar destination, mutations are expensive – BigQuery rewrites entire partitions, ClickHouse schedules async jobs, Snowflake burns warehouse time. Your load strategy in T→C should minimize mutations. In T→T, you can lean on `INSERT ON CONFLICT` or `MERGE` without the same cost anxiety.

Constraint enforcement at the destination. A transactional destination can enforce `PRIMARY KEY`, `UNIQUE`, and `FOREIGN KEY`. If you try to load a duplicate, the database rejects it. You can rely on this as a safety net. A columnar destination won't. BigQuery and Snowflake accept PK/UNIQUE in DDL but don't enforce them. ClickHouse has no unique constraint outside of `ReplacingMergeTree`'s eventual deduplication. In T⟶C, deduplication is always your problem. In T⟶T, you can configure the destination to enforce it.

Cost model. Transactional destinations cost CPU and IO – roughly proportional to the rows you touch. Columnar destinations cost bytes scanned (BigQuery) or compute time (Snowflake, Redshift). A badly written syntactic step in BigQuery that forces a full table scan on every run doesn't just waste time – it charges you for it, repeatedly, for the lifetime of the pipeline.

Type system gap. The wider the gap between source and destination type systems, the more syntactic work the pipeline has to do. T⟶T with the same engine (PostgreSQL ⟶ PostgreSQL) has almost no type gap. T⟶C (PostgreSQL ⟶ BigQuery) means navigating timezone coercion, decimal precision, JSON handling, and format compatibility. See 1.4 – Columnar Destinations for the full type mapping.

1.7.3 Where to Process

Every syntactic operation – a CAST, a NULL coalesce, a hash key – runs *somewhere*. The question is where, and what it costs you.

There are four execution points:

Source. The source database does the work inside the extraction query. JOINs for cursor borrowing, CAST for type normalization, CONCAT for synthetic keys.

```
-- source: transactional
-- engine: postgresql
-- Syntactic step at the source: cast types, synthesize key, inject metadata in the
extraction query
SELECT
    order_id::TEXT || '-' || line_num::TEXT AS _source_key,
    order_id,
    line_num,
    quantity::NUMERIC(10,2) AS quantity,
    NOW() AT TIME ZONE 'UTC' AS _extracted_at
FROM order_lines
WHERE updated_at >= :last_run;
```

Free if the source can handle it and you're running at 2am. Dangerous if you're on a busy production ERP at 10am – you're adding work to someone else's production database, and the DBA will find you.

Orchestrator / middleware. Python, Spark, a cloud function. You do the transformation in code between extraction and load. You control it fully, but you're adding infrastructure, memory, and an extra data hop that you pay for. Justified when the source can't express it in SQL, or when volume makes centralized processing necessary.

Staging area. Land raw in a staging table or dataset on the destination, then transform there before writing to the final table. Common pattern in BigQuery (stage raw ⟶ merge to final). Keeps the extraction fast and simple, but all processing costs are on the destination's meter.

Destination (directly). Transform inside the final load query. No staging, no intermediate hop. Works well when the transform is simple and the destination query engine is cheap. In T⟶T this is often the

right call. In T⟶C, a complex transform in the load SQL can scan more data than necessary and inflate costs.

> **Push work to whoever's idle**
>
> Source at 2am? Let it do the work. Production ERP at 10am? Extract raw, process downstream. "Cheapest" isn't just infrastructure cost – it factors in system load and how much the source team will hate you. In T⟶C, prefer doing the syntactic work at the source or orchestrator to avoid expensive destination compute. In T⟶T, the destination is often the cheapest place to process.

1.7.4 Transactional → Columnar

This is the harder corridor. The full details of the destination are in 1.4 – Columnar Destinations, but the strategic implications for the crossing:

Append by default. Mutations are expensive. Your instinct to MERGE every changed row is the wrong default. Append raw, deduplicate or materialize downstream. Reserve MERGE for cases where append genuinely doesn't work.

Partition alignment is your responsibility. The destination has no FK constraints, no row-level locks, no automatic partition management. You decide how data is physically laid out. Load in partition-aligned batches or you're paying for your own mess on every downstream query.

Type casting happens at the crossing. Naive timestamps, decimal precision, JSON columns – all of these need explicit handling before data lands. The destination won't reject bad types gracefully; it'll silently coerce them or fail the job. See 5.3 – Type Casting and Normalization.

The cost of mistakes compounds. A wrong partition strategy, a missing cluster key, an unnecessary full-table scan in your load logic – these are costs every downstream query pays for the life of the pipeline.

1.7.5 Transactional → Transactional

The narrower corridor. Same class of engine on both ends, but don't let that make you complacent.

You can use the destination's constraints. Configure `PRIMARY KEY` and `UNIQUE` on the destination and let the database enforce them. A duplicate load attempt gets rejected at the database level instead of silently creating bad data. This is a genuine advantage over T⟶C.

`INSERT ON CONFLICT` / `MERGE` is cheap. Unlike columnar engines, a transactional destination handles upserts efficiently. You can run them frequently without cost anxiety. This changes your load strategy – you can afford to be more aggressive with incremental merges.

Dialect differences still bite. The source and destination might both be "SQL" but speak it differently. PostgreSQL's `ON CONFLICT DO UPDATE` is not MySQL's `ON DUPLICATE KEY UPDATE` is not SQL Server's `MERGE`. Function names, string handling, date arithmetic, identifier quoting – all of these differ. See 8.1 – SQL Dialect Reference for the full comparison.

You still have all the source problems. Hard deletes, unreliable `updated_at`, soft rules, schema drift – none of these go away because the destination is also transactional. You still need to detect deletes, handle cursor failures, and survive schema changes. The destination being "easy" doesn't mean the source got simpler.

Patterns apply to both corridors

Where the implementation differs, chapters note it explicitly under "By Corridor." When nothing is called out, assume the pattern applies to both.

1.8 Purity vs. Freshness

One-liner: The fundamental tradeoff in batch replication: perfectly stable data requires full replaces at low frequency. Fresher data requires incremental complexity. The right answer depends on the table, the consumer, and the SLA.

Every pipeline decision – how to extract, how often to run, how to load – is a position on this tradeoff. Most pipelines take a position without realizing it, and then you inherit someone else's unexamined defaults six months later when something breaks.

1.8.1 The Two Ends

Purity means the destination is an exact clone of the source at a given point in time. No drift, no missed rows, no accumulated damage from soft rule violations or unreliable cursors. A full replace achieves this – every run resets the world. You pull everything, you replace everything, the destination matches the source exactly as of the extraction timestamp.

Freshness means how recently the destination reflects the source. A table refreshed every 15 minutes is fresh. A table refreshed nightly is stale by mid-morning.

The tension between them is structural. Full replace maximizes purity but caps freshness – you can only refresh as often as a full scan completes. Incremental maximizes freshness but trades purity – missed rows, unreliable cursors, and accumulated drift are inherent to the approach. Every incremental pipeline carries a purity debt that grows until the next full reset corrects it.

Neither extreme is universally correct. The right point on the spectrum depends on the table, the consumer, and what "fresh enough" actually means.

1.8.2 Why Full Replace Is the Purity Champion

A full replace has properties that incremental extraction fundamentally can't match:

It's stateless and idempotent. Run it twice, same result. No cursor state persisting between runs, no checkpoint files to manage, no accumulated decisions from prior executions. If something goes wrong, rerun – the destination will be correct.

It catches everything. Hard deletes, retroactive corrections, soft rule violations, schema drift, rows that were missed by a prior incremental – a full replace picks up all of it by reading the current state of every row directly, without relying on the source to signal changes.

It has no drift accumulation. An incremental pipeline that misses a row today still has that wrong row tomorrow, and the day after. A full replace that runs tonight corrects everything that was wrong since the last full replace.

The cost is the freshness ceiling. If a full scan of `orders` takes 3 hours, the freshest you can be with a pure full replace strategy is 3 hours behind – and that's assuming the scan starts the moment the last one ends. For most tables and most businesses, this is completely acceptable. For a handful, it isn't.

1.8.3 Why Incremental Carries a Purity Debt

Incremental extraction is a performance optimization – a necessary one when the table is too large to scan completely within the schedule window, but an optimization nonetheless, with all the fragility that implies.

The cost is real and often underestimated:

Cursor reliability is a soft rule. The assumption that every write to a row bumps `updated_at` is an expectation, not an enforcement. Bulk scripts bypass it. ORM hooks miss it. Back-office tools don't know it exists. Every row that changes without bumping the cursor is a row your pipeline will never see update. See 1.6 – Hard Rules, Soft Rules.

Hard deletes are invisible. A deleted row leaves no trace for a cursor-based extraction to find. You need a separate delete detection mechanism – a full ID comparison, a count reconciliation, a tombstone table – which adds complexity and its own failure modes. 3.6 – Hard Delete Detection

High frequency has a monetary cost. 288 extractions per day (every 5 minutes) means 288 load jobs, 288 sets of DML operations on the destination, 288 opportunities for partial failures. On BigQuery, that's 288 jobs counting against your DML quota. On Snowflake, that's warehouse time burning through the day.

Drift accumulates silently. A missed row today is still wrong tomorrow. An incremental that has been running for 6 months with a slightly unreliable cursor has 6 months of accumulated drift that nobody has quantified. The destination looks correct – it has data – but it doesn't match the source.

1.8.4 Classifying a Table

How you classify the table determines everything that follows. Work through these in order:

1. What does the business actually need? "Real time" almost never means real time. It means "faster than it is now." Press for a concrete number. "I need it every 15 minutes" is different from "I need it with no more than a 15-minute delay" – and both are different from "I need it when I click refresh." Giving consumers a way to trigger an on-demand extraction can reduce scheduled frequency dramatically, cutting cost without sacrificing the freshness they actually use.

2. How long does a full scan take? Measure it. Don't estimate. A 500k-row table on a production ERP at 2am might scan in 4 minutes. The same table at 10am might take 40. If the scan fits comfortably inside your schedule window, full replace is the answer and the conversation is over.

3. Does it have hard deletes? If yes and the table is small enough for a full replace, use full replace – it handles deletes automatically. If it's too large: you need incremental plus a separate delete detection strategy, which is significant added complexity.

4. Does the source rewrite history? Retroactive corrections are incompatible with incremental. A pricing table where last quarter's prices get adjusted, an ERP where journal entries get reversed and reposted – a cursor on `updated_at` misses these entirely. Full replace is the only safe option, regardless of table size.

5. Is the cursor reliable? Verify it before committing. Query for NULL `updated_at` values. Run EXPLAIN on the cursor query and confirm it hits an index. Create a row and update it and confirm the timestamp changes both times. If any check fails, the cursor is a soft rule and your incremental will accumulate drift.

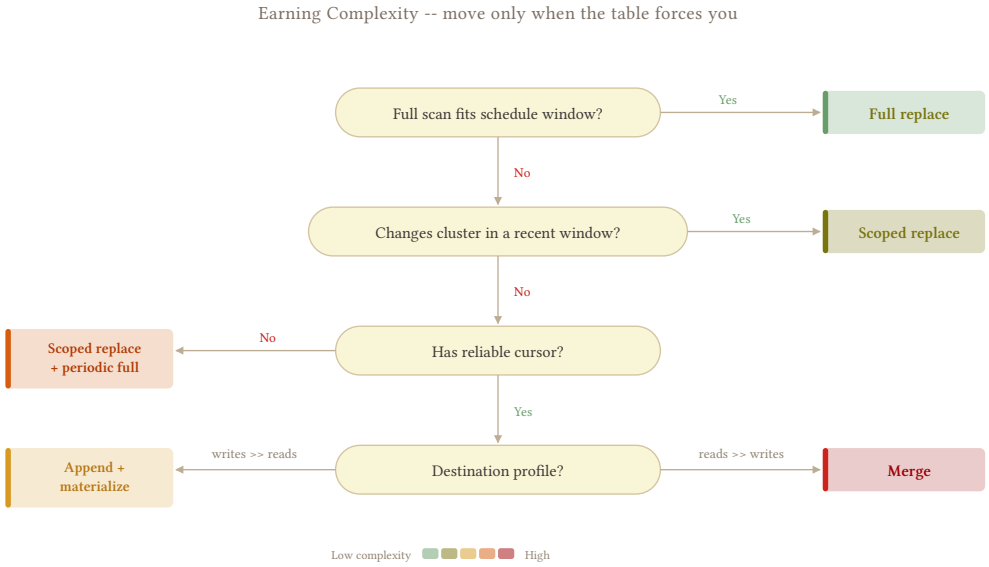


Table type	Full scan fits window?	Reliable cursor?	Recommendation
Dimension / config (`products`)	Yes	–	Full replace every run
Pre-aggregated (`metrics_daily`)	Yes	–	Partition-level replace
Append-only (`events`)	No	Yes (immutable)	Append
Large mutable (`orders`)	No	Yes	Scoped replace (if changes cluster); else A+M or merge by read/write profile
Large mutable, hard deletes (`invoices`)	No	Yes	Scoped replace + delete detection + periodic full
Large mutable, unreliable cursor	No	No	Scoped replace + periodic full

1.8.5 The Hybrid: Periodic Full + Intraday Incremental

For tables where you need both purity and freshness – mutable, large, sub-daily SLA – the hybrid is the answer. Run a full or scoped replace nightly to reset purity. Run incremental extractions intraday to deliver freshness.

> **The incremental doesn't need perfection**
>
> It doesn't need to catch hard deletes. It doesn't need to handle retroactive corrections. It doesn't need a lookback window. The nightly full replace will correct everything the incremental missed. Design the intraday incremental to be fast and simple – a tight cursor window, no delete detection, no complexity – because it's not the source of truth. The full replace is.

This also means the incremental's failure mode is manageable. If it misses a run, the data is stale by one interval until the next incremental or the nightly full. If it accumulates drift, the nightly full resets it. The incremental is a freshness layer on top of a reliable foundation.

See 3.2 – Cursor-Based Timestamp Extraction for how to design the intraday incremental to coexist cleanly with the periodic full.

1.8.6 The SLA Conversation

Have this conversation before you build anything.

"We need the data in real time" is a starting position, not a requirement. When someone says "real time," they usually mean "faster than it is now." The real question is: what decision does this data inform, and how stale can it be before that decision is wrong? A sales dashboard reviewed at 9am every morning is not harmed by a nightly full replace that completes at 6am. A fraud detection system that needs to act on transactions within minutes is a genuinely different animal – and also not a batch problem.

Most business SLAs, when pressed to a concrete number, land somewhere between 30 minutes and daily. And most of the time the actual need is "no more than X delay" rather than "updated every X minutes" – the distinction matters because on-demand extraction (a sensor or manual trigger in your orchestrator) can satisfy the former without the cost of continuous scheduling.

When the SLA genuinely requires sub-hourly continuous refresh: accept it, build for it, and document the purity cost explicitly. The business is choosing freshness over purity. They should understand that the destination may drift, that hard deletes won't be reflected immediately, and that the pipeline complexity – and the infrastructure cost – is higher as a result. That's a valid business decision. Make sure it's a conscious one.

> **Start with full replace**
>
> Document why you deviated. The default position is full replace. Every deviation toward incremental should be a documented decision: what made full replace infeasible, what purity tradeoffs were accepted, and what the plan is for correcting drift. If you can't articulate why you need incremental, you probably don't.

1.9 Idempotency

> **One-liner:** If rerunning the pipeline changes the destination, you have a bug.

1.9.1 What It Means

An idempotent pipeline produces the same destination state whether it runs once, twice, or ten times with the same input. No extra rows, no missing rows, no side effects from the previous run bleeding into the next one. The destination after run N+1 is indistinguishable from the destination after run N – assuming the source didn't change between them.

This sounds obvious until you realize how many pipelines fail it. An append without dedup doubles the data on retry. A cursor that advances before the load confirms creates a permanent gap. A staging table that doesn't get cleaned up causes the next run to load stale data on top of fresh data – in each case, the pipeline works fine on the first run and breaks on the second.

1.9.2 Why It's the Foundation

Every other reliability property – retries, backfills, failure recovery, concurrent runs – depends on idempotency. If your orchestrator retries a failed run, it needs to know that re-executing the pipeline won't corrupt the destination. If you backfill a date range, you need to know that reprocessing already-loaded data won't create duplicates. If two runs overlap because the first one hung, you need to know the destination survives both.

Without idempotency, retries are dangerous, backfills require manual cleanup, and every failure becomes a unique investigation. With it, the recovery playbook for every failure is the same: run it again.

1.9.3 Full Replace Gets It for Free

A pipeline that drops and reloads the entire table on every run is idempotent by construction. There's no prior state to interfere with, no cursor to manage, no accumulated history that a retry could corrupt. Run it once, run it five times – the destination is the same because every run rebuilds it from scratch.

This is the strongest argument for full replace as the default. You don't have to think about idempotency because the architecture hands it to you. The moment you move to incremental, you leave this safe zone and take on the burden of proving that your pipeline still produces the same result regardless of how many times it runs, in what order, or after what failures.

1.9.4 Incremental Has to Earn It

Incremental pipelines accumulate state across runs: a cursor position, a set of previously loaded keys, a log of appended rows. That state creates surface area for idempotency violations. The most common ones:

Cursor advances before load confirms. The high-water mark moves forward, but the data it points past never made it to the destination. The next run starts from the new position and the gap is permanent (unless a lookback window covers it – see 4.6 – Reliable Loads). Fix: advance the cursor only after the destination confirms the load.

Append without dedup. A retry appends the same batch again, and now the destination has two copies of every row. The pipeline "succeeded" both times, but the destination is wrong. Fix: use a dedup mechanism – `INSERT ... ON CONFLICT` on transactional engines, a `ROW_NUMBER()` dedup view on columnar engines (4.4 – Append and Materialize).

Stateful staging that doesn't clean up. The pipeline writes to a staging table, then loads from it. If the staging table isn't truncated before each run, a retry loads the previous batch plus the new one. Fix: truncate staging at the start of every run, not the end.

Non-deterministic extraction. The same query returns different results on different runs – because the source changed between runs, or because the query uses `NOW()` in a way that shifts the window. This is harder to fix because the source is a moving target. Stateless window extraction helps by anchoring the window to a fixed offset rather than tracking state.

1.9.5 The Test

The simplest way to verify idempotency: run the pipeline, snapshot the destination, run the same pipeline again with the same parameters, compare. If anything changed – row count, column values, metadata – the pipeline isn't idempotent and you need to understand why before it goes to production.

> **Automate the idempotency test**
>
> For your critical tables, a scheduled job that runs the pipeline twice on a staging copy and compares the results catches idempotency violations before they hit production. Especially valuable after pipeline changes – a new column, a modified cursor, a changed load strategy can all break idempotency in ways that a single run won't reveal.

1.9.6 Idempotency by Load Strategy

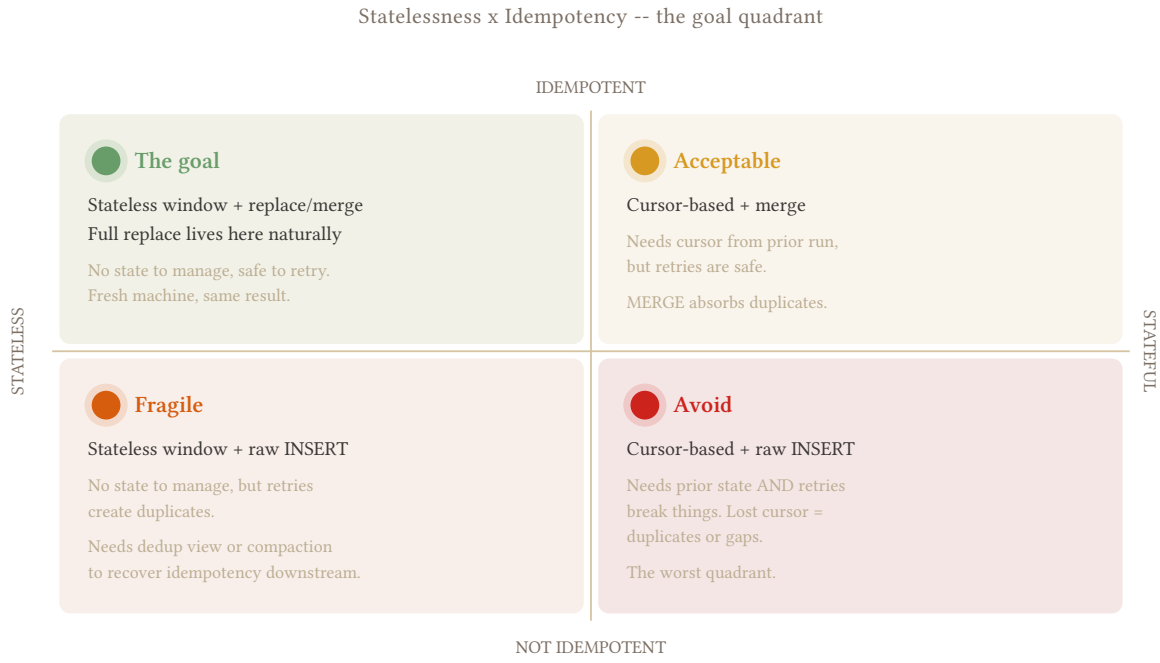
Each load strategy in Part IV has a different relationship with idempotency:

Load strategy	Idempotent?	Why
Full replace	By construction	Every run rebuilds from scratch – no prior state to interfere
Append-only	At the table level, no. At the view level, yes	Retries append duplicates, but the dedup view still returns correct state
MERGE / upsert	Yes	Same key + same data = same result, regardless of how many times it runs
Append and materialize	At the view level, yes	Same as append-only: duplicates in the log, correct state in the view
Hybrid	Yes, if both sides are idempotent	Two destinations means two surfaces to verify

The table reveals the pattern: full replace and MERGE are unconditionally idempotent. Append-based strategies are idempotent *at the consumer level* (the view), not at the storage level (the log). The distinction matters for storage cost and compaction frequency, but not for correctness – which is the thing you actually care about.

1.9.7 Statelessness and Idempotency

These two properties are related but distinct. A pipeline is **stateless** if it can run on a fresh machine with no prior context. A pipeline is **idempotent** if running it multiple times produces the same result. You can have one without the other:



The goal is the top-left quadrant: stateless and idempotent. Full replace lives there naturally. Everything else requires deliberate design to get there – and 4.6 – Reliable Loads covers the mechanics.

2.1 Full Scan Strategies

> **One-liner:** When incremental isn't worth it – or isn't possible – extract everything and replace the destination completely.

Full scan is the simplest pipeline that exists, you extract every row and replace the destination table. Getting to avoid cursor states, hard-deletions and drift is its greatest superpower. Most pipelines reach for incremental when the table doesn't need it. This chapter is about when full scan is the right answer (which hopefully is all times) – and how to do it without killing your source database or leaving a window of empty data in production.

2.1.1 When Full Scan Wins

The decision comes down to one comparison: **full scan cost vs. incremental complexity cost + drift risk**. If the scan is cheap and the table is messy, full scan wins every time.

The table is small enough. Dimensions, configuration tables, reference data, lookup tables. What "small enough" means depends on your source database size and how frequently the table is updated – a 500k-row table on a lightly loaded PostgreSQL replica is different from 500k rows on a production ERP mid-day. The question is whether the source can absorb the scan without impacting application performance, and whether the extraction time fits your schedule window.

No reliable cursor. No `updated_at`, no row version, no sequence. You asked the team and they shrugged. You checked `information_schema` and found nothing useful. You can't borrow a cursor from a header table because this is a standalone table with no parent. Without a cursor, incremental extraction is impossible without hashing every row – which has its own cost and complexity. Full scan is cleaner.

Hard deletes happen and aren't worth tracking separately. Incoming payments, cancelled reservations, temporary staging records – tables where rows get deleted regularly and the deletion is part of the business state you need to reflect. A full scan picks up deletions automatically because the deleted rows simply aren't there when you extract. A cursor-based incremental is blind to deletions by design and requires a separate detection mechanism. If the table is small enough, don't bother.

The source rewrites history. Some applications correct past records in bulk. A pricing table where last quarter's prices get retroactively adjusted. An ERP where a journal entry gets reversed and reposted to a prior period. A more problematic DBA who runs UPDATE scripts directly. A cursor on `updated_at` misses rows that were corrected without bumping the timestamp. Full scan doesn't care – you get the current state of every row, always.

> **Earn incremental complexity**
>
> Full scan is the default – don't assume incremental is needed. Incremental is a performance optimization with a cost in complexity, drift risk, and maintenance. Build full scan first. Switch to incremental only when the scan is genuinely too slow or too expensive for your schedule window. See 1.8 – Purity vs. Freshness.

2.1.2 The Two Shapes of Full Scan

Full table, every run. Extract all rows, replace the destination completely on every execution. The simplest pipeline that exists and the most reliable. No state, no checkpoints, no drift. This should be your default for any table that fits the window.

Full table, periodic + incremental between. Run a full scan nightly or weekly to reset state, run incremental extractions intraday to get freshness. The full scan is the safety net that catches everything the incremental misses – soft rule violations, missed timestamps, retroactive corrections. The incremental is the performance optimization that gives you sub-daily freshness without scanning the whole table every hour. See 3.1 – Timestamp Extraction Foundations for how to design the incremental so it plays well with the periodic full reset.

2.1.3 Source Load and Extraction Etiquette

Full scans hit the source harder than incremental extractions. A few rules:

Schedule during off-peak hours. 2am, weekends, after the monthly close finishes. Know your source system's busy hours before you set the schedule. On a production ERP, mid-morning is when 200 users are posting invoices and confirming orders. That is not when you want to be scanning `order_lines`.

Use a read replica when available. A replica absorbs your scan without touching the primary. Replication lag is a real concern – you might miss rows committed in the last few seconds – but for a nightly full scan this is almost never material. Confirm lag with the DBA.

Chunk large tables. Never pull millions of rows in a single query. Break the extraction into chunks by PK range and append each chunk before replacing the destination. Chunking reduces peak memory, avoids query timeouts, and makes failures recoverable – if chunk 47 fails, you retry chunk 47, not the whole table.

Manual chunking by PK range:

```
-- source: transactional
-- engine: postgresql
-- Chunk 1: rows 1 -- 100000
SELECT * FROM order_lines
WHERE line_id BETWEEN 1 AND 100000
ORDER BY line_id;

-- Chunk 2: rows 100001 -- 200000
SELECT * FROM order_lines
WHERE line_id BETWEEN 100001 AND 200000
ORDER BY line_id;
```

The chunk size is a tunable parameter – start at 100k rows and adjust based on query time and memory pressure. Most orchestrators let you parameterize this per asset or per source.

Most database drivers support streaming modes that yield rows incrementally without loading the full result set into memory. SQLAlchemy's `yield_per` does this consistently across every source engine – the same code works against PostgreSQL, MySQL, SQL Server, and SAP HANA:

```
# orchestrator: python
# SQLAlchemy yield_per: stream results without loading full result set into memory
with engine.connect() as conn:
    result = conn.execution_options(yield_per=10_000).execute(
        text("SELECT * FROM order_lines ORDER BY line_id")
    )
    for chunk in result.partitions():
        stage_chunk(chunk)  # append to staging, not to final table
```

> **Stage all chunks before replacing**
>
> If you chunk the extraction and write each chunk directly to the final destination table, chunk N replaces chunk N-1. You'll end up with only the last chunk in the destination. Extract all chunks to a staging area first, validate, then swap. Always.

See 6.7 – Source System Etiquette for connection limits, timeout coordination, and DBA communication.

2.1.4 At the Destination: Replace Strategies

Full replace should be a staging swap or a partition-level replace. A bare "DELETE everything, INSERT everything" leaves a window where the table is empty, and it's more expensive than necessary on most engines.

Replace strategies at the destination: DROP vs TRUNCATE vs DELETE

	DROP + CREATE	TRUNCATE + INSERT (default)	DELETE + INSERT
Mechanism	recreate table from scratch	empty rows, keep schema	row-by-row delete
What it touches			
metadata (DDL)	✗ replaced (DDL re-run)	✓ untouched	✓ untouched
data rows	✗ replaces all	✗ replaces all	~ scans + writes per row
indexes	✗ rebuilds from scratch	✓ cleared, not rebuilt	~ maintained (slow)
grants	✗ LOST	✓ preserved	✓ preserved
Atomicity	~ engine-dependent	✓ usually transactional	✓ always transactional
Recommended for	last resort	full replace default	scoped / partial replace

✓ safe / preserved ~ partial / conditional ✗ lost / destructive

Staging swap. Load into a staging table, validate, then atomically swap staging to production. Zero downtime – consumers query the production table and see complete data throughout. Rollback is dropping the staging table without touching prod. This is the recommended approach for any table with live consumers. See 2.3 – Staging Swap.

Partition-level replace. When the table is partitioned by date and you're replacing a specific date range, drop and reload only the affected partitions. Still a full replace per partition – you extract all rows for those dates and reload completely – but you don't touch partitions outside the range. See 2.2 – Partition Swap.

Truncate + reload. `TRUNCATE` the table and insert fresh. Simple, but it has a window where the table is empty. Acceptable for overnight runs where no dashboards or queries are running against the table. Never acceptable for tables with intraday consumers.

> **Use bulk loads, not row-by-row inserts**
>
> On every columnar engine, a `LOAD DATA` job or `COPY INTO` from a file is significantly cheaper than a set of `INSERT` statements. BigQuery charges for DML operations; Snowflake burns warehouse time per statement. Load your staging data from Parquet or Avro files, not from repeated inserts. This is especially important at scale – 10M rows via `LOAD DATA` is one job; 10M rows via `INSERT` is 10M jobs.

2.1.5 Data Quality Before the Swap

Never swap staging to production without validating first. The full replace pattern is powerful precisely because it resets state – which means a bad load resets to bad state, with no prior version to fall back on.

Minimum checks before every swap:

```
-- source: columnar
-- engine: bigquery
-- Run these against the staging table before swapping to production

-- 1. Table is not empty
SELECT COUNT(*) AS row_count FROM stg_order_lines;
-- Fail if row_count = 0

-- 2. Row count is within 10% of yesterday's production count
SELECT
    ABS(staging_count - prod_count) * 1.0 / prod_count AS pct_change
FROM (
    SELECT COUNT(*) AS staging_count FROM stg_order_lines
) s,
(
    SELECT COUNT(*) AS prod_count FROM order_lines
) p;
-- Fail if pct_change > 0.10

-- 3. No NULLs on required columns
SELECT COUNT(*) AS null_order_ids
FROM stg_order_lines
WHERE order_id IS NULL;
-- Fail if null_order_ids > 0
```

A full replace that lands zero rows because of a source connection failure is a production disaster. The table goes empty. Every dashboard shows nothing. Every downstream query breaks. The check `row_count > 0` is the single most important gate in a full replace pipeline.

Most orchestrators support post-load validation hooks or checks that run after the staging load and block the swap if any check fails.

See 6.9 – Data Contracts for formalizing these checks into reusable contracts.

2.1.6 What Full Scan Doesn't Solve

Tables too large to scan entirely. When the full scan takes longer than your schedule window, or when the source can't handle the load at any hour, full scan isn't viable. Your options are to scope the scan to the current + previous period (2.4 – Scoped Full Replace) or to switch to a rolling window (2.5 – Rolling Window Replace).

Freshness tighter than scan frequency. If the business needs data every 15 minutes and a full scan takes 2 hours, you need incremental. Part III covers cursor-based extraction, merge patterns, and append strategies for tables that need sub-hourly freshness.

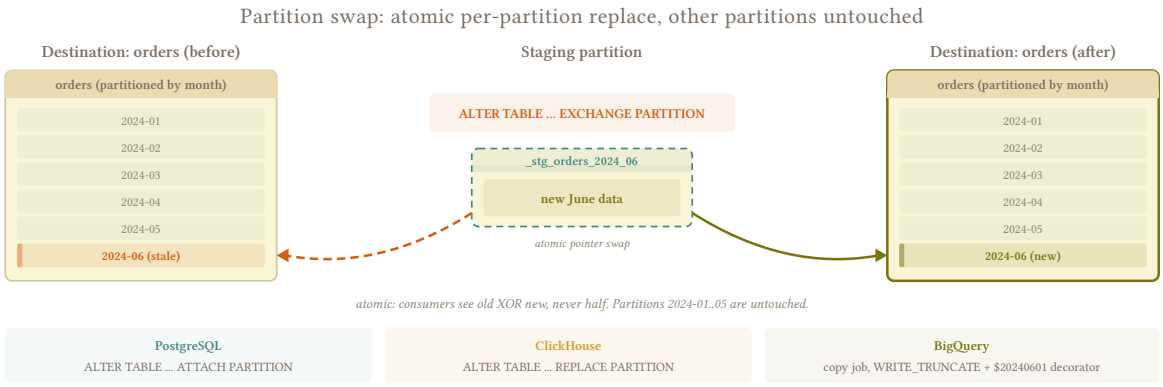
Source that can't absorb the load. Some sources are so sensitive that even an off-hours full scan causes problems. Shared multi-tenant SaaS databases, under-resourced ERPs, systems with hard connection limits. In these cases, extract incrementally and accept the complexity cost. It's cheaper than a production incident.

2.2 Partition Swap

> **One-liner:** Replace data at partition granularity – one partition or thirty, one extraction pass, without touching the rest of the table.

A full table replace is the cleanest option when the table fits the window. When it doesn't – years of historical events, a `metrics_daily` table going back a decade – partition swap is the next cleanest. You still extract everything in the target range in one pass, still load to staging, still validate before touching production. The only difference is the destination operation: instead of replacing the entire table, you replace only the partitions that changed.

Rows outside the target range are never touched; the rest of the table stays exactly as it was.



2.2.1 When to Use It

- The table is partitioned by date and the data is naturally aligned to partition boundaries
- A bounded range needs reloading: yesterday's data was corrected, a backfill covers a month, an upstream pipeline redelivered a week of events

- Full table replace is too expensive – years of history sit in partitions you have no reason to touch
- `metrics_daily`, `events`, `sessions` – any table where each partition is a self-contained, replaceable unit

The partition boundary must be meaningful. If your `events` table has rows for `2026-03-07` scattered across multiple partitions because of a timezone mismatch, partition swap will produce incorrect results. See 5.5 – Timezone Handling.

2.2.2 The Mechanics

One extraction pass covers the full target range:

```
-- source: transactional
-- engine: postgresql
-- Extract the complete target range in a single pass
SELECT *
FROM events
WHERE event_date BETWEEN :start_date AND :end_date;
```

Load everything to a staging table on the destination. Validate. Then replace the affected partitions – all of them, in the same job.

2.2.2.1 Extraction Status as the First Gate

Before anything touches staging, the extraction must have completed successfully. A query that returns 0 rows is not an error – it means the source had no data for that range, and that is correct information. A query that fails with a connection error, a timeout, or an exception is a different outcome entirely.

```
# orchestrator: python
try:
    rows = extract_events(start_date, end_date)  # raises on connection error,
timeout, etc.
    load_to_staging(rows)                         # 0 rows is a valid result here
except Exception as e:
    raise  # propagate -- do not proceed to partition operations
```

If the extraction raised an error, the job fails and every other process for that job stops. Failing fast is a very good practice to avoid overwriting production partitions with NULLs.

> **Silent failures return empty results**
>
> The dangerous case is an extraction layer that swallows exceptions and returns an empty result set instead of raising. Check your database driver and connection wrapper – make sure a dropped connection or a query timeout surfaces as an error, not as an empty iterator. If your extraction layer can return 0 rows on failure, you've lost the signal that makes this safe. See 6.10 – Extraction Status Gates.

2.2.3 Atomicity Per Engine

The extraction is always one pass. The destination-side replacement varies by engine.

2.2.3.1 Snowflake / Redshift

Load to staging, then DELETE + INSERT in a transaction. Delete the full target range by date bounds – not by what's in staging:

```
-- engine: snowflake / redshift
BEGIN;
DELETE FROM events
WHERE partition_date BETWEEN :start_date AND :end_date;
INSERT INTO events SELECT * FROM stg_events;
COMMIT;
```

Atomic: if the INSERT fails, the DELETE rolls back, so it's safe to retry.

The DELETE must cover the full target range, not `IN (SELECT DISTINCT partition_date FROM stg)`. If Saturday had 10 rows last run and the source corrected them to Friday, staging has no Saturday rows – and a DELETE driven by staging would leave the old Saturday data in place. Delete by the declared range; insert whatever staging holds, including nothing for days with no activity.

2.2.3.2 BigQuery

MERGE is the wrong answer here. It scans both tables in full and is the slowest, most expensive DML option BigQuery has. Real-world cases of MERGE consuming hours of slot time on large tables are documented. DELETE + INSERT has no transaction wrapper and leaves an empty-partition window between the two statements.

The right approach: load all data to a staging table partitioned by the same column as the destination, then use **partition copy** per partition – a near-metadata operation that is orders of magnitude faster than any DML:

```
# staging must be partitioned by the same column as destination
# then: copy each staging partition to destination
bq cp --write_disposition=WRITE_TRUNCATE \
  project:dataset.stg_events$20260307 \
  project:dataset.events$20260307
```

N partition copies for N partitions, but each copy is fast. The staging load is one job. The partition copies are the loop – and in BigQuery, copy jobs are near-free in both time and cost compared to DML.

> **Staging must match partition spec**
>
> `bq cp` with a partition decorator requires the source table to be partitioned by the same column and type as the destination. Create staging with `PARTITION BY event_date` – same as the destination – before loading.

2.2.3.3 ClickHouse

DELETE is an async mutation – queued, not inline. `ALTER TABLE ... REPLACE PARTITION` is the right mechanism: it atomically swaps the source partition into the destination.

```
-- engine: clickhouse
-- For each partition in the target range:
ALTER TABLE events REPLACE PARTITION '2026-03-07' FROM stg_events;
ALTER TABLE events REPLACE PARTITION '2026-03-08' FROM stg_events;
```

Sequential within the job, still one orchestrator run. ClickHouse `REPLACE PARTITION` is fast – it operates at the partition level without rewriting rows.

2.2.4 Validation Before Swap

```
-- source: columnar
-- engine: bigquery
-- Run against the staging table before any partition operations

-- No NULLs on the partition key
SELECT COUNT(*) AS null_dates
FROM stg_events
WHERE event_date IS NULL;
-- Fail if null_dates > 0
-- A NULL partition key means a row can't be assigned to any partition.
-- On BigQuery, it lands in the __NULL__ partition. On Snowflake/Redshift,
-- it won't be deleted by the BETWEEN range and won't insert into the right place.
```

The partition list for replacement must come from the **target date range you declared**, not from the distinct dates in staging. If you drive the replacement from `SELECT DISTINCT event_date FROM stg`, you'll skip dates that went to zero – and those partitions will keep their old data.

For Snowflake and Redshift this means the DELETE covers `:start_date` to `:end_date` regardless of what staging contains. For BigQuery, the partition copy loop iterates the declared date range – for dates with no staging rows, copy an empty partition or explicitly delete the destination partition.

2.2.5 One Job

From the outside, this is a single pipeline run: one extraction, one staging load, N destination operations. The orchestrator sees one job succeed or fail – not thirty.

When it fails, you rerun it. The extraction reruns cleanly because it's a bounded range query against the source. The staging load reruns because staging is a throwaway table – truncate and reload. The partition operations rerun because replacing a partition with the same data produces the same result. There's no accumulated state to worry about, no half-applied changes to untangle. Rerun it and move on.

Compare that to an incremental pipeline that fails mid-run: you're left asking what got written, what didn't, whether the cursor advanced, and whether rerunning will duplicate data. With partition swap, the answer to "what do I do if it fails?" is always the same.

When staging is already valid. For large backfills – 30 partitions, say – rerunning the full extraction just to retry two failed partition copies is wasteful. If staging is still intact from the previous run, retry only the failed partition operations against the existing staging data. The staging table didn't change; the per-partition operation is independent and safe to rerun in isolation. BigQuery partition copies and ClickHouse `REPLACE PARTITION` both support this cleanly.

For Snowflake and Redshift, the DELETE + INSERT is a single transaction – it either committed or rolled back entirely. There are no partial partitions to retry. Rerun the full destination step against the existing staging table.

2.2.6 Partition Alignment Is Your Responsibility

The engine partitions by whatever value is in the partition key column. If that value is wrong – because of a timezone mismatch, a bulk load that used server time instead of event time, a late-arriving batch processed with today's date – the row lands in the wrong partition and partition swap will replace the wrong thing.

Handle timezone before determining the partition key, not after. See 5.5 – Timezone Handling.

Late-arriving data adds another dimension: rows for prior dates arriving today belong in their original partition, not today's. Your extraction range must account for this. If yesterday's data is still arriving today, your target range should include yesterday – and your overlap window should be wide enough to catch stragglers. See 3.9 – Late-Arriving Data.

2.2.7 By Corridor

Transactional to Columnar

Primary use case (e.g. PostgreSQL ⟶ BigQuery). Columnar destinations are built for partitioned loads. One staging load + N partition operations per job. BigQuery partition copy is near-free compared to any DML option.

Transactional to Transactional

E.g. PostgreSQL ⟶ PostgreSQL. Transactional destinations have no columnar partition concept. Equivalent: `DELETE WHERE partition_key BETWEEN :start AND :end` then bulk INSERT from staging, inside a transaction. Less elegant but achieves the same scoped replace with the same atomicity guarantee.

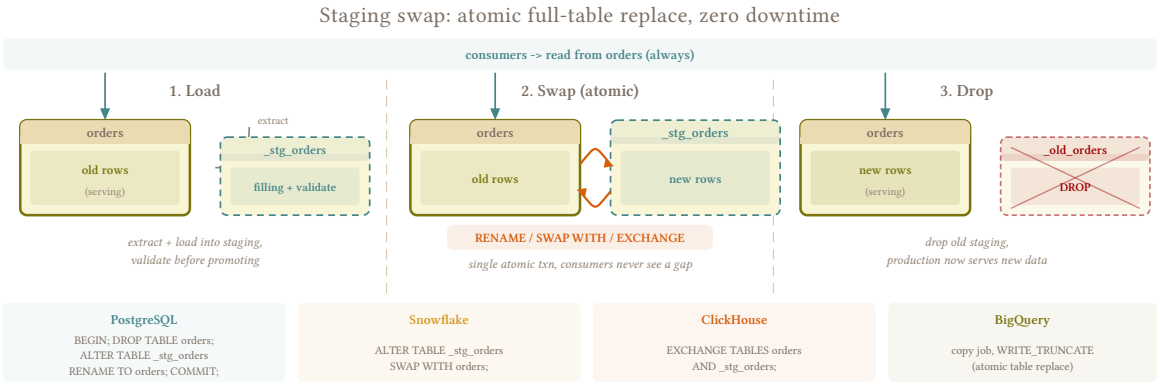
2.3 Staging Swap

> **One-liner:** Load into a staging table, validate, then atomically swap to production. Zero downtime, trivial rollback.

The naive full replace is `TRUNCATE production; INSERT INTO production SELECT * FROM source`. Simple. And it leaves a window where `production` is empty – any dashboard or query that runs between the TRUNCATE and the INSERT sees nothing. On a table with live consumers, that's an incident.

The second problem: if the load fails halfway through, you're left with a half-loaded production table and no clean way back. You can't replay the INSERT without truncating again, which means another empty window.

Staging swap eliminates both problems: consumers see complete data throughout, and rollback is just dropping the staging table without touching production.



2.3.1 The Mechanics

Three steps:

1. Load to staging. Extract from source and load entirely into the staging table. Production is untouched. If the extraction or load fails at any point, nothing has happened to production.

Two conventions for where staging lives, each with real trade-offs:

	Table prefix`public.stg_orders`	Parallel schema`orders_staging.orders`
Namespace	Pollutes production schema	Clean separation
Permissions	Per-table grants	Schema-level grant / revoke
Cleanup	Drop tables individually	`DROP SCHEMA ... CASCADE`
Swap complexity	Simple – rename within schema	Harder – cross-schema move or copy
Snowflake `SWAP WITH`	Works directly	Works across schemas
PostgreSQL swap	`RENAME TO` in transaction	`SET SCHEMA` + rename – 3 steps
BigQuery	`bq cp` or DDL rename (within same dataset)	`bq cp` only – `RENAME TO` doesn't cross datasets.
ClickHouse	No difference	No difference

The parallel schema convention is worth it at scale – permission management alone justifies it when you're running hundreds of tables. But go in with eyes open: the swap step is more involved on PostgreSQL and Redshift, and you'll need to handle it explicitly per engine.

2. Validate. Run checks against `stg_orders` before touching production. At minimum: row count > 0, % change vs. production is within threshold, required columns have no NULLs. See 6.9 – Data Contracts for formalizing these as reusable contracts.

3. Swap. Atomically replace production with staging. The mechanism varies by engine – covered below – but the result is the same: one moment consumers are reading the old data, the next they're reading the new data, with no empty window in between.

2.3.2 The Swap Operation

The swap must be atomic – consumers should never see a missing table. Each engine has its own mechanism.

2.3.2.1 Snowflake

```
-- engine: snowflake
-- Atomic metadata-only swap -- fast regardless of table size
ALTER TABLE stg_orders SWAP WITH orders;
```

`SWAP WITH` is the cleanest option on any engine: metadata-only, instant, truly atomic. One caveat: grants follow the table object, not the name. After the swap, the object that used to be `stg_orders` now carries the name `orders` – but it has `stg_orders`'s original (empty) grant set. Consumers who had access to the old `orders` object lose access to the name `orders` because that name now points to a different object. Re-grant after every swap, or use `FUTURE GRANTS` on the schema so new objects inherit permissions automatically.

2.3.2.2 BigQuery

BigQuery has no native SWAP. In BigQuery, schema = dataset, so the parallel schema convention means `orders_staging.orders` ⟶ `orders.orders`.

Table prefix convention (`dataset.stg_orders` ⟶ `dataset.orders`):

```
bq cp --write_disposition=WRITE_TRUNCATE \
  project:dataset.stg_orders \
  project:dataset.orders
```

Or with DDL rename (brief unavailability window between steps):

```
-- engine: bigquery
ALTER TABLE `project.dataset.orders` RENAME TO orders_old;
ALTER TABLE `project.dataset.stg_orders` RENAME TO orders;
DROP TABLE IF EXISTS `project.dataset.orders_old`;
```

Parallel dataset convention (`orders_staging.orders` ⟶ `orders.orders`):

```
bq cp --write_disposition=WRITE_TRUNCATE \
  project:orders_staging.orders \
  project:orders.orders
```

`ALTER TABLE RENAME TO` does not cross dataset boundaries – DDL rename is not an option with parallel datasets. The copy job works in both conventions.

> **BigQuery copy job performance**
>
> Copy jobs are free (no slot consumption, no bytes-scanned charge) for same-region operations. Cross-region copies incur data transfer charges. Google's documentation explicitly notes that copy job duration "might vary significantly across different runs because the underlying storage is managed dynamically" – there are no guarantees about speed regardless of whether source and destination are in the same dataset or different datasets. Factor this into your schedule window for large tables.

Use the copy job for tables with live consumers. Use DDL rename (same-dataset only) when you control the maintenance window.

2.3.2.3 PostgreSQL / Redshift

Table prefix convention – rename within the same schema:

```
-- engine: postgresql / redshift
BEGIN;
ALTER TABLE orders RENAME TO orders_old;
ALTER TABLE stg_orders RENAME TO orders;
DROP TABLE orders_old;
COMMIT;
```

Parallel schema convention – move across schemas:

```
-- engine: postgresql / redshift
BEGIN;
ALTER TABLE orders RENAME TO orders_old;
ALTER TABLE orders_staging.orders SET SCHEMA public;  -- moves to public schema,
keeps name 'orders'
DROP TABLE orders_old;
COMMIT;
```

`SET SCHEMA` moves the table without copying data – it's a metadata operation, not a rewrite. In both cases, if the transaction rolls back, `orders` is still the original table, unchanged.

2.3.2.4 ClickHouse

```
-- engine: clickhouse
-- EXCHANGE TABLES is atomic -- no intermediate state
EXCHANGE TABLES stg_orders AND orders;
```

`EXCHANGE TABLES` swaps both table names atomically. After the swap, `stg_orders` contains the old production data – useful if you want to keep the previous version for a period before dropping it.

2.3.3 Validation Before Swap

Never skip validation. A staging swap that replaces production with zero rows because of a silent extraction failure is worse than a failed load – it actively corrupts your destination and every consumer sees empty data.

```sql
-- source: columnar
-- engine: bigquery
-- Run against stg_orders before any swap operation

-- 1. Not empty
SELECT COUNT(*) AS row_count FROM stg_orders;
-- Fail if row_count = 0

-- 2. Within 10% of yesterday's production count
SELECT ABS(s.cnt - p.cnt) * 1.0 / p.cnt AS pct_change
FROM (SELECT COUNT(*) AS cnt FROM stg_orders) s,
     (SELECT COUNT(*) AS cnt FROM orders) p;
-- Fail if pct_change > 0.10

-- 3. No NULLs on merge key
SELECT COUNT(*) AS null_keys FROM stg_orders WHERE order_id IS NULL;
-- Fail if null_keys > 0
```

Most orchestrators let you wire these as post-load checks that gate the swap step. If any check fails, the job stops, staging is left intact for inspection, and production is untouched.

> **Keep staging around on failure**
>
> Don't drop staging when validation fails. Leave it for debugging – it's the evidence of what went wrong. Drop it only after the issue is resolved and the next successful run creates a fresh staging table.

2.3.4 Rollback

There is no rollback step. If validation fails, you abort before the swap, failing fast. On the next run, staging is recreated from scratch – it's a throwaway table, not a state you carry forward.

If the swap itself fails mid-operation (rare, but possible on non-atomic engines like BigQuery's DDL rename), check which table exists and which doesn't before deciding how to recover. On atomic engines (Snowflake SWAP, ClickHouse EXCHANGE, PostgreSQL transaction), a failure means the swap didn't happen – production is still the original.

> **Don't reuse staging across runs**
>
> Staging tables are throwaway. Truncate or drop and recreate on every run. A staging table left over from a prior failed run contains stale data – if your validation only checks row count, it might pass against the wrong rows.

2.3.5 By Corridor

Transactional to Columnar

Primary use case for this pattern (e.g. PostgreSQL → BigQuery). Columnar destinations have no cheap in-place UPDATE, so full replace via staging swap is the standard approach for any mutable table that fits the schedule window. On BigQuery, prefer `bq cp` over DDL rename for live tables. On Snowflake, use `SWAP WITH` and re-grant permissions after.

Transactional to Transactional

Equally valid (e.g. PostgreSQL → PostgreSQL). The PostgreSQL RENAME-within-transaction approach is clean and atomic. One additional concern: foreign keys referencing the production table. If other tables have FK constraints pointing to `orders`, the rename sequence may fail or temporarily break referential integrity. Disable FK checks or use `CASCADE` options with care before swapping.

2.4 Scoped Full Replace

One-liner: Declare a scope boundary, apply full-replace semantics inside it, and explicitly freeze everything outside – so you get idempotent reloads without scanning years of history every run.

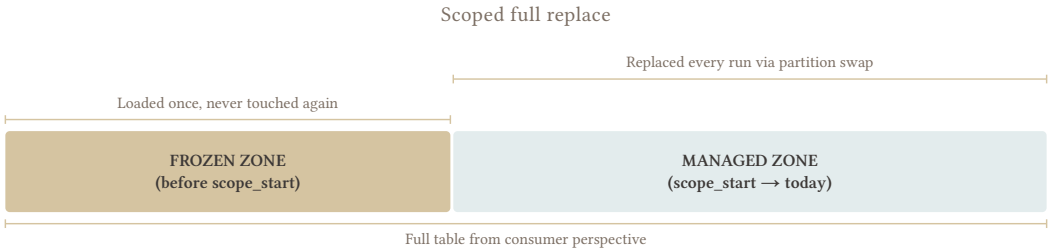
2.4.1 Why Not Just Full Replace?

A full table replace is the cleanest option available. It resets state, eliminates drift, and gives you a complete, verifiable destination every run. The problem is cost. An `orders` table with five years of history might have 200 million rows. A nightly full reload takes hours and burns slot quota. At some point the cost of purity exceeds its value.

The alternative most people reach for is incremental. That trades one problem for another: cursor management, drift accumulation, delete detection – the full weight of Part III. For tables where historical rows rarely change, that complexity is never earned.

Scoped full replace is the middle path. Define a boundary and apply full-replace semantics to everything on the right side of it. Rows to the left are frozen: loaded once via a one-time backfill, never touched again. Within the scope, the pipeline runs a complete, idempotent reload every time. Outside the scope, it owns nothing.

2.4.2 The Mechanics



Declare the scope. `scope_start` is a parameter the pipeline receives at runtime, not a constant baked into SQL. Externalizing it lets you widen the scope for backfills without touching extraction logic.

Extract within scope. Pull only rows where the scope field falls inside the declared window. The source query is bounded – no full-table scan.

Replace the managed zone. Use partition swap (2.2 – Partition Swap) to replace every partition in `scope_start → today`. The frozen zone is never part of the destination operation.

2.4.3 Defining the Scope

```
-- source: transactional
-- engine: postgresql
-- :scope_start injected by the orchestrator
SELECT *
FROM orders
WHERE created_at >= :scope_start;
```

Three ways to anchor `scope_start`:

Anchor	Definition	When to use
Start of last year	`DATE_TRUNC('year', CURRENT_DATE - INTERVAL '1 year')`	Accounting data with open/closed fiscal years. Year boundaries are natural partition boundaries. Window grows Jan⟶Dec then resets.
Fixed date	`'2025-01-01'`	History before that date is known bad, migrated from another system, or simply not needed. Stable until you change it deliberately.
Rolling offset	Last N days	Different pattern – see 2.5 – Rolling Window Replace

The calendar year anchor is particularly useful for transactional systems with formal year-close processes. Once a fiscal year is closed in the source, no document in that year should change. The year boundary is a business invariant backed by a process – align your scope to it.

2.4.3.1 The Field That Defines the Scope

The scope filter doesn't always belong on `created_at`. Some ERP systems define the fiscal year through a document date field that is separate from the record's creation timestamp. In SAP Business One, `DocDate` is the field that places a document in an accounting period – a document created on December 31 with `DocDate` set to January 5 of the next year belongs to the next year, not the current one. Filtering by `created_at` would put it in the wrong scope.

Use whichever date field your source system uses to assign records to fiscal periods. When in doubt, ask the source system owner, not the DBA.

2.4.4 The Assumption You're Making

Scoped full replace rests on one explicit bet: **records created before `scope_start` will not change in ways consumers care about.**

Table	Fits?	Why
`events`	Yes	Append-only. Historical events are immutable by definition.
`metrics_daily`	Yes	Old dates only change during explicit recalculations. Treat those as one-off backfills.
`invoices`	Yes	Closed invoices are frozen. Open invoices are recent. If this soft rule is broken, there could be some legal trouble.
`orders`	Usually	Most old orders are done. Verify with the source team whether support can reopen them.
`customers`	No	A customer created in 2022 can update their email today. Use full scan (see 2.1 – Full Scan Strategies)
`products`	No	Price changes and schema mutations affect all rows regardless of age. Use full scan.
`order_lines`	Indirectly	No reliable own timestamp. Borrow scope from `orders` via cursor from another table (see 3.4 – Cursor from Another Table)

Dimension tables (`customers`, `products`) change across their full history. The right answer for them is a cheap full scan, not an ever-growing scope.

2.4.5 Scope Maintenance

Widening the scope means moving `scope_start` backwards – including a year of history that was previously frozen. This is a one-time manual operation: run the pipeline with the new `scope_start` to reload the newly included range. Subsequent nightly runs extract from the wider window automatically.

Narrowing the scope is dangerous. Moving `scope_start` forward freezes data that may still need correction. If those rows were corrupted or incomplete in the destination, they are now permanently frozen as-is. Only move `scope_start` forward once you're confident the data you're freezing is correct.

> **Don't advance the year boundary early**
>
> Year-end corrections, late-arriving documents, and accounting adjustments routinely arrive in January and February. The fiscal year may be nominally closed, but the data isn't stable yet. A safe rule: don't advance `scope_start` past a year boundary until Q1 is well underway – at least March or April – and only after confirming with the source team that the prior year is closed in the system, with no pending documents or adjustments expected.

2.4.6 Validation

Before any destination operation, verify staging is not empty and reaches the expected end of the window. Whether `scope_start` was set correctly is a parameter-level concern – validate it in your orchestrator, not by interrogating the data boundary, since gaps near the scope edge are legitimate on low-activity days.

```
-- source: columnar
-- engine: bigquery
SELECT
    MAX(DATE(created_at)) AS latest_row,
    COUNT(*)              AS total_rows
FROM stg_orders;
-- Fail if total_rows = 0
-- Fail if latest_row < CURRENT_DATE - INTERVAL '1 day'
```

Document the scope boundary

Every consumer of this table is reading data that may not reflect source state for historical rows. Put `scope_start` in your destination table metadata or documentation. "Complete from 2025-01-01 onwards" is essential information. Leaving it implicit is how you get a silent correctness bug six months later.

2.4.7 Getting Creative

Scoped full replace sets a single boundary: managed vs. frozen. Once you see it as a zone concept, the obvious next step is multiple zones with different replacement cadences – each tuned to how often that slice of data actually changes.

Cold zone (2+ years ago): Data is almost certainly stable. Replace weekly – one extraction pass covers the full cold range, partition swap replaces those partitions. Cost is low because the source query is bounded and runs once a week.

Warm zone (current year including last 7 days): Daily full replace via partition swap, `scope_start → today`. The overlap with the hot zone is intentional – the nightly warm run is the purity reset for the week. Hard deletes, retroactive corrections, and incremental drift all get wiped. Any row the intraday incremental got wrong is corrected by morning.

Hot zone (last 7 days): Intraday incremental runs every hour or few hours, merging only changed rows. It doesn't need delete detection, no lookback window, no complexity – because the nightly warm replace corrects everything the incremental missed. The incremental is a freshness layer, not the source of truth.

Three pipelines, one table, each running at the cadence that matches the data's volatility. The cold run is cheap and slow. The warm run is the core and the cleanup. The hot run is fast and disposable.

Tiered freshness goes further

The building blocks are this pattern, partition swap (2.2 – Partition Swap), and incremental merge (4.3 – Merge / Upsert). The hybrid strategy is introduced in 1.8 – Purity vs. Freshness. For the full architecture – how to wire the three zones together operationally – see 6.8 – Tiered Freshness.

2.4.8 By Corridor

Transactional to Columnar

Natural fit (e.g. PostgreSQL → BigQuery). The frozen zone lives in historical partitions that are never touched. Partition swap handles the managed zone. Ensure `scope_start` aligns with a partition date – splitting a partition between managed and frozen creates a partial-partition edge case on BigQuery.

Transactional to Transactional

Same logic, different destination operation (e.g. PostgreSQL → PostgreSQL): `DELETE FROM orders WHERE created_at >= :scope_start` followed by bulk INSERT from staging, inside a transaction. Rows before `scope_start` are outside the DELETE range and untouched. The same scope documentation requirement applies.

2.5 Rolling Window Replace

> **One-liner:** Drop and reload the last N days every run. The window moves forward with time; everything outside it is frozen.

A full table replace is too expensive. A cursor-based incremental is unreliable or more complexity than the table deserves. But the data changes – corrections arrive, statuses update, late rows trickle in – and those changes cluster in a predictable recent window.

Rolling window replace exploits that clustering. Instead of scanning the full table or tracking individual row changes, it defines a fixed-width window anchored to today, does a complete full replace inside that window every run, and leaves everything older untouched. Within the window, the destination is always correct. Outside it, the data is frozen at whenever it last fell inside the window.

2.5.1 Distinction from Scoped Full Replace

Both patterns maintain a managed zone and a frozen zone. The difference is in how the boundary is defined and what the filter operates on.

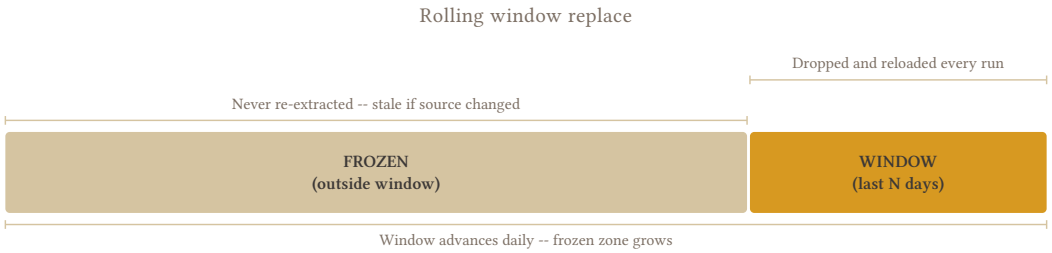
2.4 – Scoped Full Replace uses a calendar anchor – Jan 1 of last year, or a fixed migration date. The boundary is a business date: a fiscal year, a known cutover point. The filter typically operates on `created_at` or `doc_date`. The managed zone grows over the year and resets annually.

Rolling window uses a metadata anchor – `updated_at` or `created_at` relative to today. The window is always the same width. It advances daily. There's no natural hard boundary like a fiscal year close; N is a judgment call based on how long corrections typically take to arrive in the source.

Rolling window also freezes data more aggressively. A 30-day window freezes anything older than a month. That's a much shorter guarantee than 2.4 – Scoped Full Replace's "everything since last January."

This also makes it composable into more stages – a 7-day daily window, a 90-day weekly window, a yearly scoped replace – each tier running at the cadence that matches its data's volatility. See 6.8 – Tiered Freshness.

2.5.2 The Mechanics



Extract by `updated_at`. The filter is on the metadata field that reflects when a row last changed, not when it was created. A 3-year-old order that got its status updated yesterday is inside the 30-day window. A 3-week-old order that hasn't changed is also inside it – you pull it again regardless, because within the window you replace everything, not just what changed.

```
-- source: transactional
-- engine: postgresql
SELECT *
FROM orders
WHERE updated_at >= :window_start;
```

Replace the window in the destination. In a transactional destination, delete by PK – not by `updated_at`. The destination's `updated_at` reflects when the row was last synced, not the current source value. A row updated today in the source still has the old `updated_at` in the destination until you replace it. Deleting by PK covers exactly what was extracted:

```
-- engine: postgresql
BEGIN;
DELETE FROM orders WHERE order_id IN (SELECT order_id FROM stg_orders);
INSERT INTO orders SELECT * FROM stg_orders;
COMMIT;
```

Or collapse into an upsert:

```
-- engine: postgresql
INSERT INTO orders
SELECT * FROM stg_orders
ON CONFLICT (order_id) DO UPDATE SET
    customer_id = EXCLUDED.customer_id,
    status      = EXCLUDED.status,
    updated_at  = EXCLUDED.updated_at;
```

In a columnar destination, the filter field mismatch creates a real cost problem; see By Corridor below.

2.5.3 Choosing N

N must be wider than your source system's actual correction window. The relevant question: how long after a record is created can it still receive updates? That varies by table and by source system behavior.

- Too narrow: corrections arriving on day N+1 miss the window and are permanently invisible.
- Too wide: the extraction approaches a full scan and the pattern loses its cost advantage.

A rough starting point is 2x the maximum expected correction lag – if corrections typically arrive within 7 days, start with 14. Then watch it. Correction windows change when source system behavior changes, and N needs to follow.

> **N is not set-and-forget**
>
> A new bulk update script in the source, a change in how corrections are posted, a migration that backdates rows – any of these can push changes outside your current window and you won't know until a reconciliation catches the drift. Review N when anything significant changes upstream. Complement with a periodic full scan (weekly or monthly) to reset accumulated drift in the frozen zone. See 2.1 – Full Scan Strategies.

2.5.4 The Assumption You're Making

Every row older than N days is either immutable or stale-by-design. The frozen zone grows continuously – a row that was last updated 31 days ago is frozen forever in a 30-day window. Unlike 2.4 – Scoped Full Replace, there's no fiscal year close or business invariant backing this up. N is purely a statistical bet on source behavior.

> **Document the window for consumers**
>
> Consumers querying this table should know that data older than N days may not reflect current source state. The destination is not a complete mirror – it's a rolling-correct-within-window, frozen-outside table. Treat this the same as 2.4 – Scoped Full Replace's scope boundary documentation.

2.5.5 Validation

```
-- source: columnar
-- engine: bigquery
SELECT
    MAX(DATE(updated_at)) AS latest_updated,
    COUNT(*)              AS total_rows
FROM stg_orders;
-- Fail if total_rows = 0
-- Fail if latest_updated < CURRENT_DATE
```

Optionally, compare the window row count against the prior run. A large drop in row count (e.g. >20% fewer rows than yesterday's window) likely signals a source issue, not a real change in data volume.

2.5.6 By Corridor

Transactional to Columnar

E.g. PostgreSQL → BigQuery. Columnar destinations should partition by a stable (hopefully unchangeable) business date – `created_at`, `doc_date`, `event_date`. Never by `updated_at`: a row that gets updated moves to a different partition on each edit, creating duplicates across partition boundaries – deduplication requires a full table scan to resolve. This means the filter field (`updated_at`) and the partition key are misaligned. An order created two years ago that was updated yesterday lives in a two-year-old partition – to replace it, you'd need to replace that partition too. Without scanning the whole table, you can't know which historical partitions are affected. The pattern becomes expensive and unpredictable in columnar. Prefer 2.4 – Scoped Full Replace for columnar destinations.

Transactional to Transactional

Natural fit (e.g. PostgreSQL → PostgreSQL). DELETE by PK from staging, then INSERT – or upsert with `ON CONFLICT (order_id) DO UPDATE`. Precise, no partition mismatch, no overshoot. The destination PK constraint is the safety net. See mechanics above for the full SQL.

2.6 Sparse Table Extraction

One-liner: Cross-product tables where 90%+ of rows are zeros – filter at extraction to pull only meaningful combinations, but know that "empty" is a business definition, not a data one.

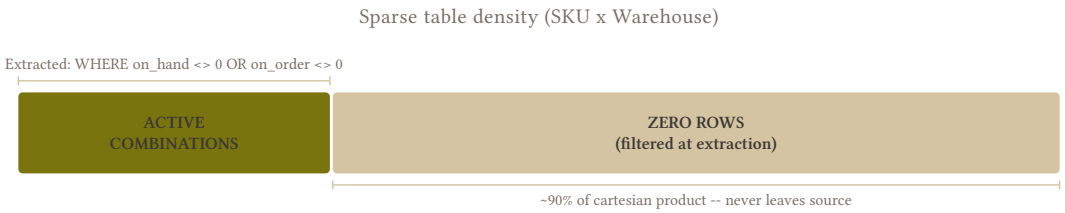
2.6.1 Zeros vs. Missing

Some tables are the cartesian product of two dimensions. Every SKU against every Warehouse. Every Employee against every Benefit. Every Product against every Location. The source system pre-computes all combinations and fills in zeros where nothing is happening.

The result is a table that's technically large but informationally sparse. A retailer with 50,000 SKUs and 200 warehouses has a 10-million-row inventory table – and in most businesses, the vast majority of those rows have `OnHand = 0` and `OnOrder = 0`. Extracting all of them is expensive, slow, and loads mostly noise into the destination.

The obvious fix is to filter: `WHERE OnHand <> 0 OR OnOrder <> 0`. Pull only the combinations with actual activity. The destination shrinks dramatically, queries are faster, and the pipeline runs in a fraction of the time.

The risk is that filtering zeros is not neutral. A zero row and a missing row look identical in the destination but mean different things in the source.



2.6.2 The Filter

```sql
-- source: transactional
-- engine: ansi
SELECT
    sku_id,
    warehouse_id,
    on_hand,
    on_order
FROM inventory
WHERE on_hand <> 0
   OR on_order <> 0;
```

The filter is simple, but the source still scans the full table – the filter reduces the rows transferred, not the rows read. On a large sparse table this is still a significant win: network transfer, staging load size, and destination query cost all drop proportionally to sparsity.

2.6.3 Zero vs. Missing

This is the decision that matters. In the destination, a missing row and a filtered-out zero row look the same. Consumers have no way to distinguish them unless you tell them.

In the source	In the destination (after filter)	What a consumer sees
`on_hand = 5`	Row present	Active combination
`on_hand = 0`	Row absent	???
No row	Row absent	???

The third column is the problem. If a consumer does `COALESCE(on_hand, 0)` on a JOIN, they get zero for both cases – which may be exactly right. But if they're counting rows, or checking for row existence, or relying on the destination having the full cartesian product, the filtered data produces wrong results.

If the source table actually contains the full cartesian product of both dimensions, you can reconstruct existence data in the destination by cross-joining the two dimension tables (`products` and `warehouses`). The sparse table becomes an enrichment on top of a complete baseline, not the source of truth for which combinations exist.

> **Don't filter silently**
>
> A destination that has filtered rows looks exactly like a destination with missing data. Every consumer who queries it will eventually hit this. Document the filter explicitly – in the table description, in a metadata table, in a comment on the asset. "This table excludes rows where on_hand = 0 AND on_order = 0" should be impossible to miss.

2.6.4 When It's Safe

- Consumers only care about active combinations – reporting on what's in stock, not on what's never been stocked
- The filter matches a real business concept ("active inventory") that consumers already think in terms of
- The dimension tables exist separately and can be used to reconstruct the full combination space if needed

2.6.5 When It's Not Safe

- Consumers need to distinguish zero stock from never-tracked – "we have none" vs. "we don't carry this here"
- Downstream aggregations count rows where a zero is a valid data point
- The source uses explicit zeros as a business signal: a zero `on_hand` with a `replenishment_blocked` flag means something different from a row that simply doesn't exist. Filtering removes that signal entirely.

2.6.6 The Filter Is a Business Decision

`WHERE on_hand <> 0 OR on_order <> 0` sounds technical but it encodes a business definition of "active." Who decided that `on_order = 1` with `on_hand = 0` is worth tracking, but `on_hand = 0` and `on_order = 0` is not? Someone did. Find out if that definition matches what consumers expect.

The filter is a contract. If the business changes the definition – "now we also want rows where `min_stock > 0`" – the destination needs a full reload, not an incremental correction. Any row that was filtered out and then became relevant won't be caught by the next run unless you reload.

2.6.7 Relation to Activity-Driven Extraction

2.7 – Activity-Driven Extraction solves a related problem differently. Sparse table extraction still scans the full source table – it just drops most rows before loading. Activity-driven extraction avoids scanning the sparse table at all: it uses recent transaction history to determine which dimension combinations are worth pulling, then queries only those.

This pattern (2.6 – Sparse Table Extraction) is simpler and works for any sparse table. 2.7 – Activity-Driven Extraction is more surgical – it trades source query complexity for a much smaller extraction scope. If your sparse table is large enough that even the filtered extraction is slow, 2.7 – Activity-Driven Extraction is the next step.

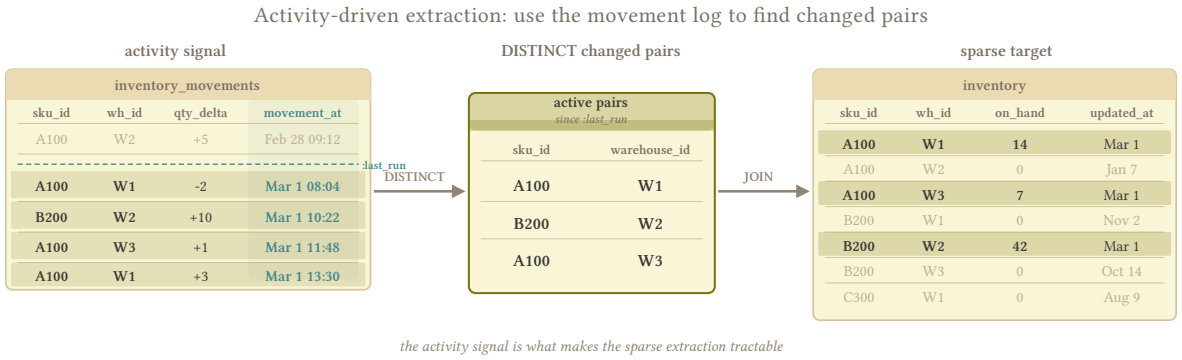
2.7 Activity-Driven Extraction

> **One-liner:** Don't scan the sparse table at all – use recent transaction history to identify which dimension combinations are active, then extract only those rows.

2.6 – Sparse Table Extraction reduces transfer volume by filtering zeros at extraction. The source still scans the full table – it just drops most rows before sending them. For a 10-million-row inventory table that's 95% zeros, you're still reading 10 million rows on the source every run and discarding 9.5 million of them. On a busy production ERP at peak hours, that scan may be a problem.

Activity-driven extraction skips the scan entirely. Instead of asking the sparse table "which of your rows are non-zero?", it asks the transaction table "which dimension combinations have been active recently?" – then pulls only those specific rows from the sparse table. The source reads a few thousand rows instead of millions.

2.7.1 The Mechanics



Step 1: get active combos from movements.

```
-- source: transactional
-- engine: postgresql
SELECT DISTINCT
    sku_id,
    warehouse_id
FROM inventory_movements
WHERE created_at >= :window_start;
```

Using `inventory_movements` rather than `order_lines` matters: movements capture every stock change – sales, manual adjustments, transfers, write-offs. `order_lines` only captures sales. A combo updated through a bulk adjustment script would be invisible to an `order_lines`-based activity filter.

Step 2: pull only those rows from the sparse table.

```
-- source: transactional
-- engine: postgresql
SELECT
    i.sku_id,
    i.warehouse_id,
    i.on_hand,
    i.on_order
FROM inventory i
JOIN (
    SELECT DISTINCT sku_id, warehouse_id
    FROM inventory_movements
    WHERE created_at >= :window_start
) active USING (sku_id, warehouse_id);
```

The JOIN is preferable to an `IN` clause with tuple values – tuple `IN` is valid PostgreSQL but not portable across all engines. The JOIN with a subquery or CTE works everywhere and the query planner handles it cleanly when `(sku_id, warehouse_id)` is indexed on both tables.

The source now reads a small slice of the inventory table via index lookups, not a full scan.

2.7.2 The Assumption

Recent transactions are a reliable proxy for which inventory combinations matter. A combo that had activity in the last N days is worth tracking. A combo with no activity in that window is inactive enough to skip.

This holds for most inventory use cases. It breaks when:

- **Slow movers exist.** A SKU that sells once a quarter won't appear in a 30-day transaction window. It might still have 500 units on hand. If no one queries it, that's fine. If a consumer expects complete on-hand data, it's a blind spot.
- **New combos have no history.** A SKU just added to a warehouse has zero transactions. It won't appear in the active set until its first order.
- **Not all systems log every change to movements.** If a bulk import script updates `inventory` directly without inserting a row into `inventory_movements`, the combo changes but the activity signal doesn't fire. This is a soft rule: "every stock change creates a movement" – until it doesn't. See Domain Model.

2.7.3 Solving Blind Spots: Tiered Windows

A single activity window can't cover all cases without growing large enough to approach a full scan. The solution is the same as 6.8 – Tiered Freshness: tier the cadences.

- **Daily**: short window (e.g. 30 days) – catches everything that moved recently, fast and cheap
- **Weekly**: wider window (e.g. 180 days) – catches slow movers, more expensive but still targeted
- **Monthly**: full scan via 2.1 – Full Scan Strategies – catches everything the activity windows missed, resets any accumulated drift

The monthly full scan is the safety net. It's expensive but infrequent. The daily and weekly runs are fast because their active sets are small. Don't try to size a single window to cover slow movers – a 365-day window defeats the purpose of the pattern.

The full scan makes this safe

Without a periodic full scan, blind spots accumulate silently. A combo that fell outside all activity windows still exists in the source – it's just invisible in the destination until the next full scan corrects it. Schedule the full scan, document its cadence, and treat it as load-bearing – not optional.

2.7.4 By Corridor

Transactional to Columnar

E.g. any source ⟶ BigQuery. Columnar destinations don't enforce PKs or maintain useful indexes for point lookups. MERGE is expensive without them. Inventory tables also rarely have a natural partition key – a stock snapshot has no obvious business date to partition by. If the filtered set is small enough after activity-driven extraction, the cleanest option is a full staging swap (2.3 – Staging Swap): replace the entire destination table, which is now small. The monthly full scan runs the same way. The destination stays small because the extraction is always activity-filtered – the staging swap cost scales with active rows, not total rows.

Transactional to Transactional

Natural fit (e.g. any source ⟶ PostgreSQL). The destination has a composite PK on `(sku_id, warehouse_id)`. Load staging and upsert:

```
-- engine: postgresql
INSERT INTO inventory
SELECT * FROM stg_inventory
ON CONFLICT (sku_id, warehouse_id) DO UPDATE SET
    on_hand  = EXCLUDED.on_hand,
    on_order = EXCLUDED.on_order;
```

The index makes this fast. No full destination scan required – the database resolves each upsert via the PK index.

2.8 Hash-Based Change Detection

One-liner: No `updated_at`? Hash the row, compare to the last extraction, load only what changed.

2.8.1 When Cursors Fail

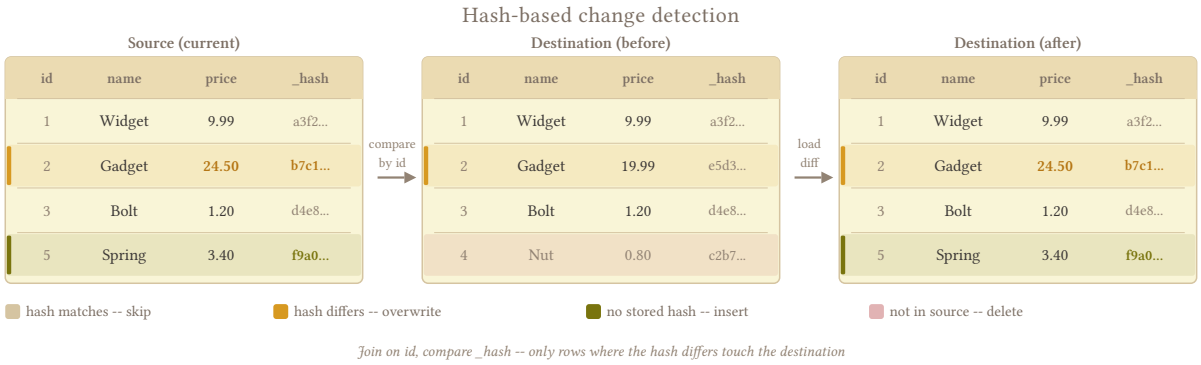
Every incremental pattern in this book assumes the source has a cursor – an `updated_at`, a sequence, a changelog. When that signal doesn't exist or can't be trusted (see 1.5 – The Lies Sources Tell), the standard incremental approach fails silently: you either miss changes, or you fall back to loading everything every run.

A full replace every run is correct but expensive when only a small fraction of rows actually change. A 10-million-row products table where 50 rows change per day doesn't need 10 million destination writes nightly. Hash-based change detection threads the needle: read the full source, but write only the rows that are actually different.

The savings depend on the destination. On transactional engines, upsert cost scales with batch size – writing 50 rows instead of 10 million is a direct win. On columnar engines, it only pays off when the changed rows cluster in a few partitions that you can swap individually (see 2.2 – Partition Swap); an unpartitioned MERGE that touches 50 rows still rewrites the entire table, so the hash comparison saves nothing on the write side.

This is a last resort, not a first choice. Reach for it when there is genuinely no cursor and the table is too large or the destination too expensive to justify a full replace on every run.

2.8.2 The Mechanics



Hash every source row. Concatenate all data columns and compute a hash, fingerprinting the row's current state.

```
-- source: transactional
-- engine: postgresql
SELECT
    product_id,
    name,
    price,
    category,
    MD5(
        COALESCE(name::text,     '') ||
        COALESCE(price::text,    '') ||
        COALESCE(category::text, '')
    ) AS _source_hash
FROM products;
```

Compare against stored hashes. Rows where `_source_hash` differs from the stored value have changed. Rows with no stored hash are new. Rows in the store with no corresponding source row were hard-deleted.

Load only changed rows. Write new and changed rows to the destination. Delete rows that disappeared. Skip everything else. On a table where 1% of rows change per run, 99% of destination writes are eliminated.

2.8.3 Full Source Scan: Avoidable With Scoping

The naive implementation reads every source row to compute hashes – the same cost as a full replace at source. The win is on the destination side: fewer writes, less DML cost, smaller staging loads.

Combined with 2.4 – Scoped Full Replace, the source scan shrinks too. Scope the hash comparison to the managed zone – rows within `scope_start → today`. Frozen history is never read or compared. You get the source-side savings of scoped replace and the destination-side savings of hash filtering in one pipeline.

2.8.4 Where Hash State Lives

The hash comparison requires storing the previous hash somewhere. Two options with real trade-offs:

`_source_hash` on the destination table. The hash travels with the row. Comparison is a JOIN between source hashes and destination hashes: rows where they differ are changed. Simple, no extra infrastructure. The problem on columnar destinations: reading the `_source_hash` column to compare costs money. On BigQuery, that's a full column scan on every run. On Snowflake, it's warehouse compute. The comparison itself has a cost.

Orchestrator state store. Persist hashes in the orchestrator's own storage – a key-value store, a metadata table on a cheap transactional database, or a local file. The destination is never queried for hashes. Comparison happens in the pipeline layer: source hashes vs. stored hashes, entirely outside the destination. More infrastructure to manage, but destination query costs for the comparison step go to zero.

For large columnar destinations where every column scan has a cost, the orchestrator state store pays for itself quickly. For transactional destinations where a column scan is cheap, `_source_hash` on the destination is simpler and sufficient.

2.8.5 Column Selection and NULL Handling

Hash only source data columns. Exclude injected metadata – `_extracted_at`, `_batch_id`, `_source_hash` itself. If a metadata column changes but the source data doesn't, you don't want a false positive.

Column order in the concatenation must be fixed and explicit. The hash is order-sensitive: `MD5('ab')` differs from `MD5('ba')`. Define the column order in code, not dynamically from schema introspection – schema changes can silently reorder columns and invalidate your entire hash store.

NULL handling requires care. A naïve `col1 || col2` in SQL returns NULL if any column is NULL. Use `COALESCE`:

```
-- source: transactional
-- engine: ansi
MD5(
    COALESCE(col1::text, '') ||
    COALESCE(col2::text, '') ||
    COALESCE(col3::text, '')
)
```

There's a subtle trap: `COALESCE(col, '')` makes NULL and empty string indistinguishable. If your source distinguishes between them (a name that was never set vs. a name explicitly set to blank), use a separator that can't appear in the data, or encode NULLs explicitly (`COALESCE(col, '\x00')`).

2.8.6 Hash Function Choice

MD5 is standard, available on every engine, and produces a 32-character string. Collision probability for a data pipeline is negligible – you'd need billions of rows for even a theoretical concern. MD5 is the right default.

SHA-256 is more collision-resistant and produces 64 characters. Use it if regulatory requirements or security policy prohibit MD5, or if the table contains data where even a theoretical collision is unacceptable. The compute cost difference is minor on modern hardware.

2.8.7 When to Use It

Condition	Use hash detection?
No `updated_at` or unreliable cursor	Yes – it's the primary use case
< ~1% of rows change per run	Yes – destination write savings justify the overhead
Destination DML is expensive (columnar)	Yes – hash filtering reduces the write cost significantly
Wide table, narrow change set	Yes – avoid loading hundreds of columns for a handful of changed rows
> 10% of rows change per run	No – overhead exceeds the savings, use full replace
Cheap transactional destination	Probably not – just upsert everything

2.8.8 Combining With Scoped Replace

Hash detection and 2.4 – Scoped Full Replace compose cleanly. Define `scope_start`, scan only the managed zone at source, compute hashes for those rows, compare against stored hashes, load only changed rows within the scope. Frozen history is never touched.

The frozen zone's hashes don't need to be maintained – those rows are immutable by definition. If you ever widen the scope backwards, treat the newly included historical rows as "new" (no stored hash) on the first run and load them fully.

2.8.9 By Corridor

Transactional to Columnar

E.g. PostgreSQL ⟶ BigQuery. Hash comparison reduces the set of rows you need to write – but it does not reduce the cost of writing them. On BigQuery, a MERGE that touches 10 rows still rewrites the entire partition containing those rows. On Snowflake, a MERGE still consumes warehouse time proportional to the scan, not the row count. The win from hash detection in columnar is narrowing **which partitions** you touch, not the cost per partition once you do. The practical approach: after hash comparison, identify which partitions contain changed rows, then use partition swap (2.2 – Partition Swap) to replace only those partitions via staging. You avoid the DML concurrency constraints (BigQuery's 2-concurrent MERGE limit) and replace entire partitions cleanly rather than doing in-place mutations. Reach for MERGE only when the changed rows span too many partitions to swap individually, and accept the cost explicitly.

If using `_source_hash` on the destination for comparison, reading that column on BigQuery costs bytes scanned. For large tables, storing hashes in an orchestrator state store is cheaper.

Transactional to Transactional

E.g. PostgreSQL ⟶ PostgreSQL. `_source_hash` on the destination is cheap – a column scan on a transactional DB with an index is fast. Compare via JOIN, upsert changed rows by PK with `ON CONFLICT DO UPDATE`, delete missing PKs. The destination enforces the PK, so the upsert is safe. Simpler than the columnar case.

2.9 Partial Column Loading

> **One-liner:** When you can't or won't extract all columns, do it explicitly, document what's missing and why, and accept that your destination is no longer a complete clone.

2.9.1 Why Exclude Columns?

Most pipelines extract all columns. `SELECT *` from source, load to destination – a complete clone. Partial column loading is a deliberate departure from that: you extract a subset of columns and leave the rest behind.

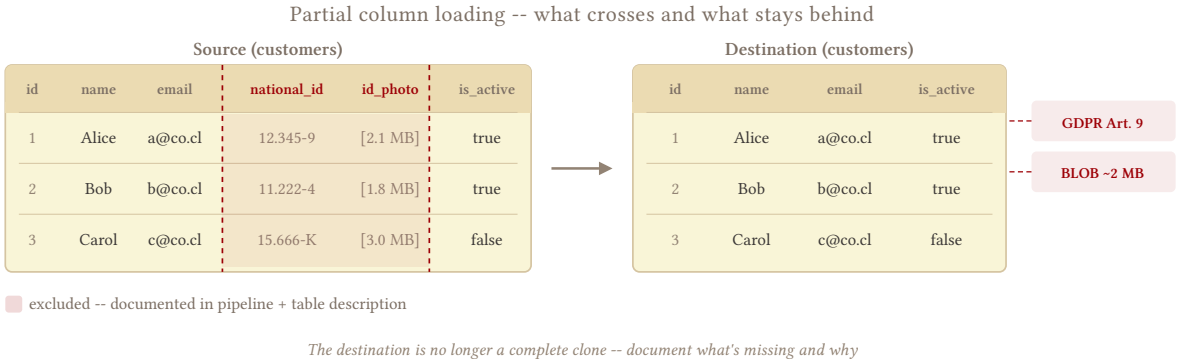
Three situations justify it:

PII and restricted data. GDPR, HIPAA, contractual data processing agreements. Some columns can't land in your analytics destination regardless of what consumers want. `national_id`, `ssn`, `raw_card_number` – these don't belong in BigQuery, period.

BLOBs and binary columns. PDFs, images, audio files, attachments stored in the source database. Extracting them bloats transfer size, explodes storage costs at the destination, and is useless to anyone running SQL. Leave them in the source.

Columns your destination can't represent. A PostgreSQL `geometry` type, a SQL Server `hierarchyid`, a custom SAP compound type. Sometimes there's no clean mapping to the destination's type system. Excluding the column is preferable to a failed extraction or a corrupted value landing silently.

What doesn't justify it: filtering for "relevance." A wide table with 200 columns where analytics only uses 40 is not a reason to exclude 160. That's a semantic transformation, a decision about what matters, and it belongs downstream, not at the extraction layer. Consumers don't understand the difference between "this column has nulls" and "this column was never loaded."



2.9.2 The Trap

The danger is the silence around the exclusion.

A consumer queries `destination.customers` looking for `national_id`. The column doesn't exist. They assume it's null in the source – or worse, they assume the source doesn't have it. Neither is true. The column exists in the source with valid data; it just wasn't loaded.

A pipeline correctness problem becomes a business trust problem the moment the consumer makes a decision based on a gap they didn't know existed.

A second trap: schema drift. When a source table adds a new column, `SELECT *` picks it up automatically on the next run. An explicit column list doesn't. The destination falls silently behind the source – no error, no alert, just a growing gap between what's there and what's available.

2.9.3 When to Use It

Reason	Example columns	Recoverable?
PII / legal restriction	`ssn`, `national_id`, `raw_email`	Yes – with proper access controls at source
Binary / attachment columns	`attachment_blob`, `document_pdf`, `photo`	Yes – if consumers don't need binary data
Unextractable type	`location geometry`, `sap_custom_type`	Sometimes – type casting may be an option first, everything *can* be a string
"Irrelevant" columns	Wide table, only 40 of 200 columns used	No – this is a semantic transformation and belongs downstream

Before excluding a column for type reasons, check 5.3 – Type Casting and Normalization. A type that can't be loaded directly can often be cast to a string or numeric representation. Partial loading is the fallback when casting isn't viable.

2.9.4 The Pattern

Name every column explicitly. Comment every exclusion inline with the reason.

```
-- source: transactional
-- engine: postgresql
SELECT
    customer_id,
    name,
    email,
    is_active,
    created_at,
    updated_at
    -- national_id excluded: GDPR Art. 9 -- special category data, no processing
basis
    -- id_photo excluded: BLOB, ~2MB per row, not used by any downstream consumer
FROM customers;
```

The comments serve two purposes: they document intent for the next engineer who touches this query, and they make the exclusion visible in code review.

At the destination, document what's missing at the table level – not just in the pipeline code. A table description, a metadata entry, a README in the project folder. Wherever consumers go to understand the data, the exclusion needs to be there.

```
-- source: columnar
-- engine: bigquery
-- Destination table description (set via DDL or catalog):
-- "Partial load of source customers table. Excluded: national_id (GDPR Art. 9),
--  id_photo (BLOB). See pipeline docs for details."
```

> **From production: Why we avoid partial extraction**
>
> Across thousands of tables in production we've never been burned by a silently missing column, and the reason is dull on purpose: we avoid partial extraction wherever we possibly can, and on the rare table where it's unavoidable we make the exclusion impossible to miss downstream – in the documentation and in the conversation with whoever consumes it, never as a footnote buried in the pipeline code. We've watched the trap catch other teams precisely because "the destination is not a complete clone" lived only in the extraction query.

2.9.5 Schema Drift Risk

Every time the source schema changes, a `SELECT *` pipeline adapts automatically. A named-column pipeline doesn't.

Add a schema diff check to your pipeline: compare the source column list against your extraction column list before each run and alert on new columns. A new column in the source is either something you should be loading (add it) or something you should be explicitly excluding (add it to the exclusion list with a comment). The only unacceptable outcome is not knowing it appeared.

```
-- source: transactional
-- engine: postgresql
-- Run before extraction to detect new source columns
SELECT column_name
FROM information_schema.columns
WHERE table_name = 'customers'
  AND column_name NOT IN (
      'customer_id', 'name', 'email', 'is_active', 'created_at', 'updated_at',
      'national_id',  -- excluded: GDPR
      'id_photo'      -- excluded: BLOB
  );
-- Non-empty result = new column appeared. Investigate before proceeding.
```

New columns in products

The `products` table in this domain mutates – new columns appear after deploys, making this a known risk. If you're running a partial column extraction on `products`, the schema diff check is not optional. A new `supplier_id` column that appears in the source and gets silently dropped at extraction will be invisible to every downstream consumer.

2.9.6 By Corridor

Transactional to Columnar

E.g. any source → BigQuery. Columnar stores have first-class column-level descriptions in their catalog. Use them. Set the table description and annotate each present column; note which columns are absent and why. Consumers who query the information schema or use a data catalog tool will see it without needing to find the pipeline code.

Transactional to Transactional

E.g. any source → PostgreSQL. Same extraction SQL. At the destination, use `COMMENT ON COLUMN` or `COMMENT ON TABLE` to document the exclusions directly in the schema. It's the closest equivalent to a catalog annotation and it travels with the table.

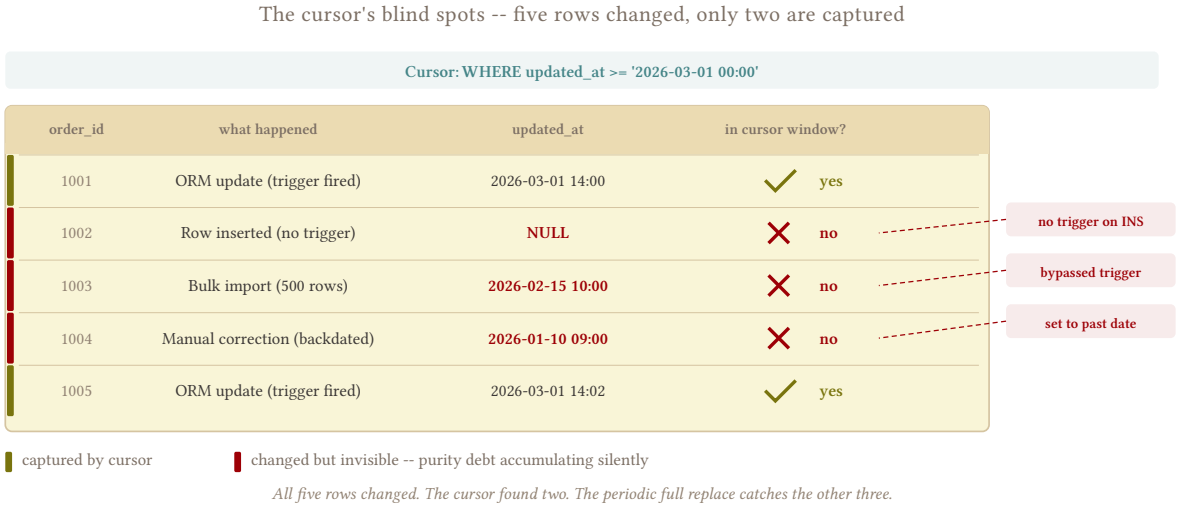
3.1 Timestamp Extraction Foundations

> **One-liner:** `updated_at` is the obvious signal for incremental extraction – and it's exactly as reliable as your application team's discipline.

3.1.1 The Discipline Gap

Incremental extraction needs a signal: which rows changed since the last run? `updated_at` is the obvious answer – it's on most tables, queryable, and cheap to filter. The difficulty is that it's maintained by the application layer, not the database. That means it works only if every write path remembers to update it – triggers, ORMs, admin scripts, bulk imports. In practice, at least one always forgets.

Two patterns build on this signal: 3.2 – Cursor-Based Timestamp Extraction tracks a high-water mark between runs; 3.3 – Stateless Window Extraction always re-extracts a fixed trailing window. Both fail the same way when the signal is wrong.



3.1.2 When `updated_at` Lies

Trigger fires on UPDATE only, not INSERT. A newly inserted row sits there with `updated_at = NULL` – invisible to your cursor, no error, no warning.

```
-- source: transactional
SELECT order_id, created_at, updated_at
FROM orders
WHERE order_id IN (1001, 1002);
```

order_id	created_at	updated_at
1001	2026-01-15 09:00:00	2026-02-20 14:30:00
1002	2026-03-01 11:00:00	NULL

Order 1002 was just inserted. `updated_at` is NULL. Invisible to any `updated_at`-based filter. See 3.10 – Create vs Update Separation.

Bulk operations bypass triggers. Imports, backfills, admin scripts that write directly to the table – they skip the trigger layer entirely and land rows with stale or null `updated_at`. The person running the script rarely knows your trigger exists.

Application sets `updated_at` manually. Some ORMs or legacy apps manage the field in code rather than via trigger. A buggy deploy, a migration script, or a data correction sets it to a past date. Rows change; the signal doesn't.

Clock skew. Source DB clock and extractor clock disagree by a few seconds. Rows updated in that gap fall outside the extraction window.

3.1.3 Validating Before You Trust It

- **Does it populate on INSERT?** Query the most recently inserted rows and check whether `updated_at` is NULL or equals `created_at`.
- **Does a bulk operation update it?** Ask the source team directly. "Do your import scripts update `updated_at`?" is a question worth a 5-minute call.
- **What's the column precision?** `DATETIME` in MySQL defaults to second-level precision. `TIMESTAMP(6)` in PostgreSQL is microsecond. Second-level precision means two rows updated in the same second are indistinguishable at the boundary.
- **Is it indexed?** An unindexed `updated_at` on a 50M-row table will full-scan the source on every run.

If any of these is wrong, document it explicitly and decide whether the failure mode is acceptable given your periodic full replace cadence.

3.1.4 The Periodic Full Replace

Both cursor-based and stateless window extraction freeze everything outside their active range. Hard deletes are invisible to both. Bulk operations that bypassed `updated_at` are invisible to both.

A periodic full replace resets all of it. How often do you see corrections that backdate past your cursor window? That's your full-replace cadence:

Cadence	When it makes sense
Weekly	Most corrections land within days; source is well-maintained
Monthly	Occasional retroactive corrections; ERP with formal period closes
Quarterly	Stable source with rare manual edits

If a full table reload is too expensive, scope the full replace to a rolling window of recent partitions – see 2.4 – Scoped Full Replace.

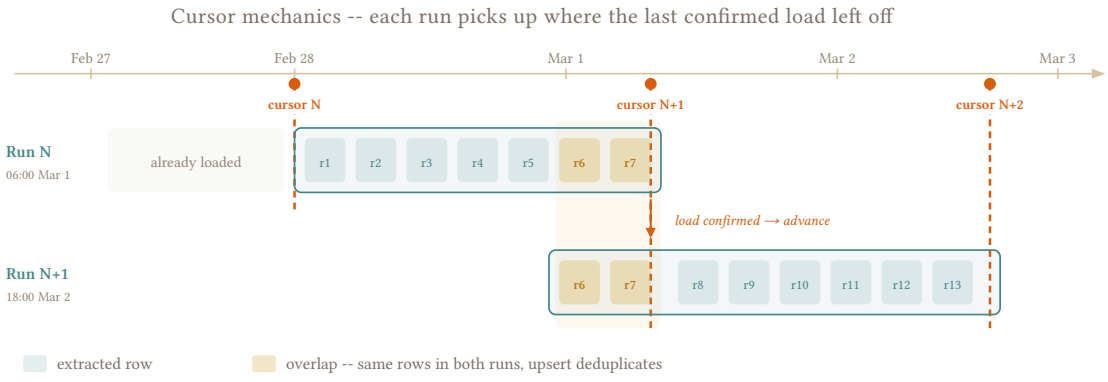
From production: Switching that table to a daily full replace

The client from 1.5.2 – "`updated_at` is reliable" – the one whose closed-month corrections never moved `updated_at` – had so much retroactive rework on that table that no cursor window was ever going to be safe. Any month could change at any time, and the source gave us nothing to detect it. We stopped trying to be incremental and reloaded the whole table with a full replace every day, governed only by a freshness window so it ran often enough for the business. It sounds heavy, but it was the simplest correct option, stateless and idempotent regardless of whatever the source did or didn't track. The table was small enough that "expensive" never materialized, and the entire class of missed-correction bugs disappeared.

3.2 Cursor-Based Timestamp Extraction

One-liner: Track a cursor – the high-water mark of the last successful run. Each run extracts only rows updated after that point.

See 3.1 – Timestamp Extraction Foundations for when `updated_at` lies, how to validate it, and when to run a periodic full replace.



Cursor only advances after the load is confirmed. Partial load + premature advance = permanent gap.

3.2.1 How It Works

```
-- source: transactional
SELECT *
FROM orders
WHERE updated_at >= :last_run;
```

After a confirmed successful load, advance the cursor to the current timestamp. On the next run, use the new value.

3.2.1.1 Where to Store the Cursor

Option 1: `MAX(updated_at)` from the destination.

```
-- source: columnar
SELECT MAX(updated_at) AS last_run
FROM orders;
```

Simple, zero extra infrastructure, self-contained – the cursor lives in the data. The risk: it's tied to what's actually in the destination. If the destination is rebuilt, truncated, or has rows with stale timestamps from a bad load, the max is wrong. A cursor that's too low causes re-extraction (harmless, upsert handles it). A cursor that's too high skips rows permanently.

Option 2: External state store. Orchestrator metadata, a dedicated state table, a key-value store. Survives destination rebuilds and is decoupled from data quality.

The risk: more moving parts. If a load partially succeeds and the cursor advances anyway, you have a permanent gap.

Both are valid. `MAX` from destination is the simpler default. External state earns its overhead when destination rebuilds are a real operational scenario.

> **Advance cursor after confirmed load**
>
> A partial load followed by a cursor advance is a permanent gap. The rows in the failed batch will **never be re-extracted.** Treat cursor advancement as the final step of the pipeline, gated on load confirmation – not something that happens at the start of the next run.

3.2.1.2 Boundary Handling

Always use >= not >. A missed row has no recovery path; a duplicate row is handled by the destination's upsert.

Add a small buffer on the lower bound (5–30 seconds) to absorb clock skew between source and extractor. The overlap mechanism is the same as 3.9 – Late-Arriving Data – just measured in seconds instead of hours.

3.2.2 By Corridor

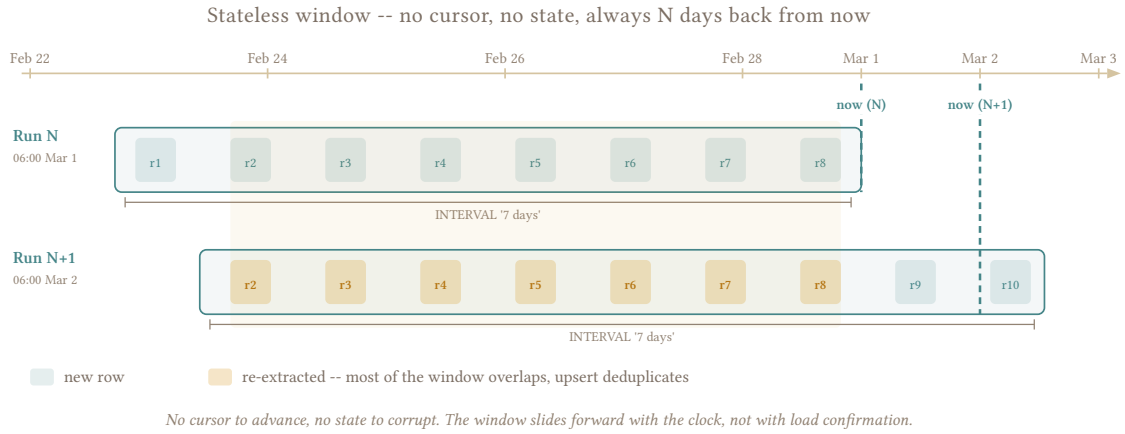
> **Transactional to columnar corridor**
>
> PostgreSQL `TIMESTAMPTZ` maps cleanly to BigQuery `TIMESTAMP`. MySQL `DATETIME` has no timezone and second-level precision – the buffer compensates. A cursor limits the extracted row count but doesn't eliminate the destination load cost – see 4.3 – Merge / Upsert for the MERGE cost anatomy.

> **Transactional to transactional corridor**
>
> `MAX(updated_at)` from the destination is cheap – a simple indexed column scan. The buffer overlap produces duplicates; the destination upsert handles them (see 4.3 – Merge / Upsert).

3.3 Stateless Window Extraction

> **One-liner:** Extract a fixed trailing window on every run. No cursor, no state between runs. This is how I run most of my incremental tables.

See 3.1 – Timestamp Extraction Foundations for when `updated_at` lies, how to validate it, and when to run a periodic full replace.



No cursor to advance, no state to corrupt. The window slides forward with the clock, not with load confirmation.

3.3.1 How It Works

Skip the cursor entirely. Every run extracts a fixed trailing window regardless of what happened last run.

```
-- source: transactional
SELECT *
FROM orders
WHERE updated_at >= CURRENT_DATE - INTERVAL '7 days';
```

No state to manage, no cursor to advance, no orchestrator metadata – run it twice and get the same result, and a failed run leaves nothing behind because the next run picks up from the same window.

A window measured in days absorbs any clock skew between source and extractor. No buffer needed.

3.3.2 Why I Default to This

A cursor-based pipeline can fail in ways that are hard to debug: partial loads that advance the cursor, destination rebuilds that reset the high-water mark, orchestrator metadata that gets out of sync. All of these produce permanent gaps that are invisible until someone notices the counts are off.

A stateless window can't have any of these problems. There's no state to corrupt. Re-run it, get the same result. Retry after failure – just run again. Backfill a date range – change the window bounds. Two runs overlap – upsert or dedup handles it. Every property you want from a pipeline (stateless, idempotent, safe to retry, safe to parallelize) comes for free.

The tradeoff: you always process the full window even when almost nothing changed. For small-to-moderate tables with indexed `updated_at`, that cost is almost always less than the engineering cost of managing cursor state across thousands of tables.

> **Match window to correction lag**
>
> How far back can a correction or late-arriving row realistically land? If support can reopen a 3-day-old order, cover at least 4 days. If the source team runs 2-week backfills, cover that. Query cost comes second.

3.3.3 When a Cursor Earns Its Overhead

Don't use a stateless window when:

- You're running hourly or more on large tables – a 7-day window running 24 times a day reprocesses those 7 days 24 times. The MERGE cost multiplies directly.
- The table is big enough that even an indexed `updated_at` scan on the window is expensive on the source.
- Mutation rate is high and concentrated in recent rows – a cursor extracts only the delta, which might be 0.1% of the window.

If you're running daily or less, or the table is small-to-moderate, the stateless window wins on simplicity every time.

3.3.4 Window Size x Run Frequency

This is the knob that matters. A 7-day window running daily costs X. The same window running hourly costs 24X – both in source query cost and destination load cost (see 4.3 – Merge / Upsert for the cost anatomy).

Size the window for correctness (it must cover your correction lag). Then set run frequency for cost. If the cost is too high, the answer is usually to run less often – not to shrink the window below what correctness requires.

3.3.5 Align Windows to Partition Boundaries

If the destination is partitioned by date, align the window to complete days. A 7-day window that spans 8 calendar days touches 8 partitions instead of 7. See 4.3 – Merge / Upsert for why partition alignment matters in columnar engines.

3.3.6 Multiple Windows

For tables where freshness matters AND corrections land late:

- Narrow window (1 day) running hourly – gives sub-hour latency for recent changes
- Wide window (30 days) running nightly – catches retroactive edits and slow-arriving rows
- Periodic full replace – catches everything outside both windows

All three tiers run without cursor state, each sized independently.

3.3.7 When There's No Timestamp At All

The examples above assume the source has an `updated_at` or similar timestamp to scope the window. Some sources don't – the data is correct as of whenever you query it, figures get revised and disputes get resolved, but there's no column that tells you when. No `updated_at`, no row version, no mechanism to tell you what changed.

The extraction is still a stateless window, but scoped by a business date instead of a modification timestamp:

```
-- source: transactional
-- engine: postgresql
SELECT *
FROM invoices
WHERE invoice_date >= CURRENT_DATE - INTERVAL '90 days';
```

Every source that rewrites history has a horizon – the furthest back a correction can reach. "Sales figures finalize after 60 days." "Invoices can be disputed within 90 days." That horizon defines your window size. If the business says 60 days, extract 90. The stated horizon is a soft rule (1.6 – Hard Rules, Soft Rules) – verify it against actual data before trusting it.

Rows outside the mutable window are immutable by definition. They stay in the destination untouched between runs. Only the window gets re-extracted.

The key difference from a timestamp-based window: you're extracting *every* row in the window, not just rows that changed. A 7-day `updated_at` window returns only rows modified in the last 7 days. A 90-day business-date window returns all 90 days of rows regardless of whether they changed. The append volume is higher, but there's no filtering assumption to get wrong – you're guaranteed to capture every correction within the horizon.

Load with append-and-materialize (4.4 – Append and Materialize) to keep the per-run cost near zero. At intra-day frequency, this is significantly cheaper than a scoped full replace (2.4 – Scoped Full Replace) which would require a partition rewrite on every run.

3.3.8 By Corridor

Transactional to columnar corridor

The source query is cheap (indexed `updated_at` scan). The load cost is where window size and run frequency multiply – see 4.3 – Merge / Upsert and 6.3 – Cost Monitoring. MySQL `DATETIME` second-level precision is a non-issue with a window measured in days.

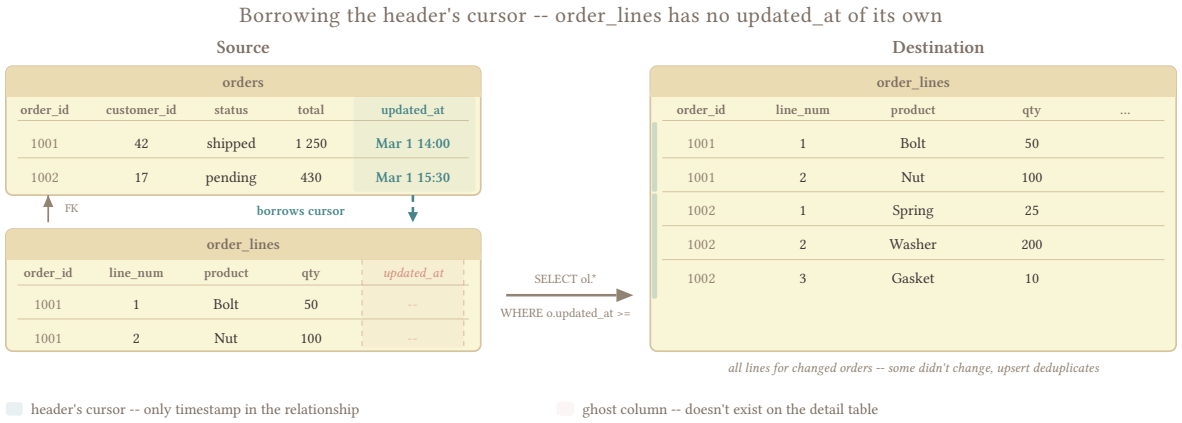
Transactional to transactional corridor

Cheap on both sides. The source query is the same indexed scan. Load cost scales with batch size, not table size – high-frequency runs are viable here. See 4.3 – Merge / Upsert for the upsert mechanics.

3.4 Cursor from Another Table

> **One-liner:** When a detail table has no `updated_at`, borrow the header's timestamp to scope the extraction.

See 3.1 – Timestamp Extraction Foundations for the shared `updated_at` reliability concerns.



Some detail tables like `order_lines` and `invoice_lines` carry no timestamp of their own – the only `updated_at` lives on the header.

Re-extracting all lines on every run works until the table crosses a few million rows and your source DBA starts asking questions. You need to scope.

3.4.1 The Pattern

Use the header's cursor to figure out which detail rows to pull. Two ways to write it – which one is better depends on your source engine and how the query planner handles it.

Subquery filter:

```
-- source: transactional
SELECT ol.*
FROM order_lines ol
WHERE ol.order_id IN (
  SELECT o.order_id
  FROM orders o
  WHERE o.updated_at >= :last_run
);
```

Works well when the subquery returns a small set of IDs. Most transactional engines turn this into a semi-join and it's fast.

Direct join:

```sql
-- source: transactional
SELECT ol.*
FROM order_lines ol
JOIN orders o ON ol.order_id = o.order_id
WHERE o.updated_at >= :last_run;
```

Simpler to read. Can be faster when both tables are indexed on the join key. Check your EXPLAIN – some planners pull unnecessary header columns into the execution plan even with `SELECT ol.*`.

Both pull all lines for every order that changed. Some of those lines didn't actually change. That's fine – the upsert handles duplicates, and you have no way to know which specific lines changed anyway.

3.4.2 Cascading Joins

One hop is straightforward. Two hops gets expensive. Three hops – stop and reconsider.

```sql
-- source: transactional
SELECT sl.*
FROM shipment_lines sl
JOIN shipments s ON sl.shipment_id = s.shipment_id
JOIN orders o ON s.order_id = o.order_id
WHERE o.updated_at >= :last_run;
```

Each join multiplies the row count and the assumptions. You're trusting two foreign key relationships and two intermediate tables to be correct and up to date. At three hops, the scoped full replace in 2.4 – Scoped Full Replace is almost certainly simpler, cheaper, and more reliable.

3.4.3 The False Economy of Re-extracting All Lines

Teams often feel guilty about pulling all lines for changed orders. Don't.

If an order has 5 lines on average and 200 orders changed since the last run, you're extracting 1,000 rows. The upsert handles the unchanged ones. The source barely notices.

This stops being fine when the combination of line count per header and header change rate produces batches large enough to stress the source or the destination MERGE. But that threshold is relative – a wholesale distributor with 100+ lines per document also generates proportionally more data everywhere else, which means their infrastructure budget already accounts for heavier workloads. The absolute cost goes up; the cost relative to the operation stays similar.

Until re-extraction is actually causing problems – slow runs, source contention, MERGE cost spikes – it's the right default. Simple, correct, and the overhead scales with the business.

3.4.4 When the Header Cursor Lies Too

Everything above assumes that when a detail row changes, the header's `updated_at` fires. Two categories of failure break this assumption, and they require different responses.

3.4.4.1 The header doesn't know the line changed

`invoice_lines.status` changes from `approved` to `disputed` – the invoice header's `updated_at` never fires. An admin script reprices 10,000 order lines without touching the header. In SAP B1, the header `UpdateDate`

is a DATE field with no time component, though with a stateless window measured in days (3.3 – Stateless Window Extraction) this particular issue is absorbed.

The common thread: the line mutated, the header didn't, and the cursor is blind to it.

3.4.4.2 The line disappears entirely

`invoice_lines` get hard-deleted independently of the header – not just via cascade. In SAP B1, removing a single line triggers a delete+reinsert of ALL surviving lines with new `LineNum` values. No tombstone, no change log entry. The cursor has nothing to detect because the row is gone and the header may not have registered the event. See 3.6 – Hard Delete Detection.

When this happens, you have two good options:

1. **Accept the blind spot.** The periodic full replace catches everything the cursor misses. If your SLA tolerates the lag, this is the cheapest approach and the one I use most often.
2. **Split by document lifecycle.** Extract all open documents from the source (they're mutable, re-extract everything), combine with only the recently modified closed documents (they're frozen, cursor is reliable). This gives you full coverage of the mutable set without re-extracting the entire table – but the combination logic is nontrivial, especially when documents transition between open and closed between runs, or when lines get hard-deleted from open documents. 3.7 – Open/Closed Documents covers the full pattern.

For detail tables where even these approaches aren't enough, see 3.8 – Detail Without Timestamp.

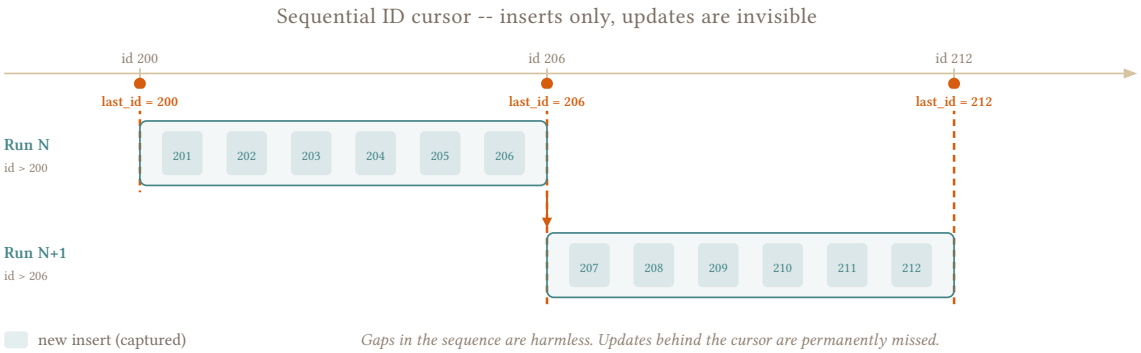
3.4.5 By Corridor

> **Transactional to columnar corridor**
>
> The join runs on the source, so extraction cost is a source-side index scan. Wide detail tables (many columns per line) amplify the load cost even for moderate row counts – see 4.3 – Merge / Upsert and 6.3 – Cost Monitoring.

> **Transactional to transactional corridor**
>
> Cheap on both sides. Extraction is the same index scan. The composite key (`order_id, line_num`) must be indexed on the destination for the upsert to perform – see 4.3 – Merge / Upsert.

3.5 Sequential ID Cursor

> **One-liner:** No `updated_at` anywhere, but the PK is monotonically increasing. `WHERE id > :last_id` detects inserts only – updates are invisible by design.



`events` has no `updated_at` and no `created_at`. `inventory_movements` doesn't either. What they do have is an auto-incrementing primary key that grows with every insert. That's enough to build a cursor on – with an explicit tradeoff.

3.5.1 The Pattern

```
-- source: transactional
SELECT *
FROM events
WHERE event_id > :last_id;
```

After a confirmed successful load, set `:last_id` to the `MAX(event_id)` from the extracted batch. On the next run, pick up where you left off.

3.5.2 The Tradeoff You Accept

This cursor detects inserts only. An existing row that gets modified will **never be re-extracted.** You accept this when:

- The table is append-only in practice – `events` and `inventory_movements` in the domain model are designed this way
- Updates are rare enough that the periodic full replace catches them

Before committing to this pattern, check the table's actual behavior against what the source team claims. "Events are never updated" is likely a soft rule (1.6 – Hard Rules, Soft Rules). If nothing in the schema enforces immutability, someone will eventually run an UPDATE on it – a bulk correction, a backfill, an admin fix. Your pipeline won't notice.

3.5.3 Gap Safety

Sequences produce gaps all the time – rolled-back transactions, failed inserts, reserved-but-unused IDs. Gaps are harmless here: `WHERE id > :last_id` skips the gap and picks up the next real row. No false positives, no missed rows.

The dangerous case is the opposite: a row inserted with an ID *lower* than `:last_id`. This happens with:

- Manually set IDs (bulk imports that override the sequence)
- Sequences with `CACHE` in multi-session environments – IDs are allocated in blocks and committed out of order
- Restored backups that reset the sequence counter

Out-of-order inserts are permanent misses

A row with `id = 500` inserted after the cursor has passed `id = 600` will never be extracted. The periodic full replace is the only safety net.

If you suspect out-of-order inserts are happening (multi-session `CACHE` is the usual cause), add a small overlap buffer the same way 3.2 – Cursor-Based Timestamp Extraction handles clock skew:

```
-- source: transactional
SELECT *
FROM events
WHERE event_id >= :last_id - 100;
```

The overlap re-extracts, at a minimum, the last 100 IDs on every run. With an append-only load (4.2 – Append-Only Load), the duplicates land in the log and the dedup view or next compaction collapses them. With a merge load, the destination upsert handles them directly. Size the buffer to your worst observed out-of-order gap – 100 covers most `CACHE` configurations.

Hard deletes are invisible too, same as with any cursor – see 3.6 – Hard Delete Detection.

3.5.4 Composite Keys

When the primary key is a composite (`order_id + line_num`, `warehouse_id + sku`), there's no natural ordering to build a cursor on. This pattern doesn't apply. See 3.4 – Cursor from Another Table for borrowing a timestamp from a related table, or 3.8 – Detail Without Timestamp when no timestamp is available anywhere in the relationship.

3.5.5 By Corridor

Transactional to columnar corridor

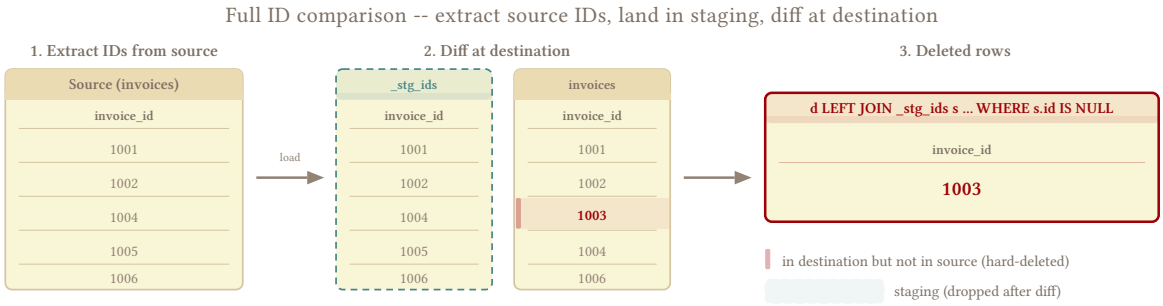
For truly append-only sources, the extraction is a simple indexed range scan. The load can use pure APPEND instead of MERGE – see 4.2 – Append-Only Load. If the table turns out to have occasional updates (the soft rule breaks), a periodic full replace catches them.

Transactional to transactional corridor

Same indexed range scan on the source. The load strategy depends on whether the source is truly immutable – see 4.2 – Append-Only Load for append-only and 4.3 – Merge / Upsert for upsert.

3.6 Hard Delete Detection

> **One-liner:** The row was there yesterday, today it's gone. A cursor never sees a deleted row – you need a separate mechanism.



Source cost: full SELECT id FROM table. Destination cost: single-column scan. Schedule outside business hours.

Every extraction pattern in this chapter – cursors, stateless windows, borrowed timestamps – detects rows that changed. A hard delete leaves nothing behind to detect. The row is gone from the source, still present in the destination, and every cursor-based run confirms zero about its absence. The count drifts silently.

3.6.1 When the Source Cooperates

Two mechanisms make delete detection trivial:

Soft-delete columns (`is_active`, `deleted_at`) – the source marks the row as deleted instead of removing it. The normal cursor captures the flag change like any other update. This is the clean solution, but it's rarer than you'd expect. Many transactional systems – especially ERPs – hard-delete without ceremony.

Tombstone tables – the source writes a record to a separate `deletes` or `audit_log` table when a row is removed. Extract from both tables: the main table for current state, the tombstone table for delete events. Common in CDC-adjacent systems, rare in application databases.

When neither exists – and for most tables in most sources, neither does – you need a detection mechanism that works from the outside.

3.6.2 When It Doesn't

3.6.2.1 Full ID Comparison

Extract the full set of IDs from the source. Extract the full set of IDs from the destination. Compare them – either in the orchestrator, or by landing the source IDs into a staging table in the destination and running the diff there.

```sql
-- destination (after landing source IDs into staging)
SELECT d.invoice_id
FROM invoices d
LEFT JOIN _stg_invoice_ids s ON d.invoice_id = s.invoice_id
WHERE s.invoice_id IS NULL;
```

The rows returned exist in the destination but not in the source – candidates for deletion.

The source-side cost is the expensive part: a full `SELECT id FROM table` on a large transactional table hits every row. Schedule it outside business hours. The destination side is cheap – columnar engines scan a single key column efficiently regardless of table size.

For small-to-medium tables, this is the simplest and most reliable approach. For large tables, use count reconciliation as a cheaper first pass.

3.6.2.2 Count Reconciliation

Compare `COUNT(*)` between source and destination. If the counts match, no deletes happened (or inserts and deletes balanced out – rare but possible). If they diverge, something changed.

```sql
-- source: transactional
SELECT COUNT(*) FROM invoices;

-- destination: columnar
SELECT COUNT(*) FROM invoices;
```

This detects drift but doesn't identify which rows. Two useful responses:

- **Trigger a full replace** when counts diverge – simple, correct, and often the cheapest response for moderate tables
- **Trigger a full ID comparison** – when a full replace is too expensive, use the count mismatch as a signal to run the heavier detection

Run the source count immediately after the load completes – the closer in time the two counts are, the less chance of a concurrent insert or delete skewing the comparison. The direction of the mismatch tells you something:

- **Destination > source:** deletes happened at the source since the last full sync
- **Destination < source:** inserts landed at the source that the extraction missed (cursor gap, late-arriving data, simple delay between extraction and loading)

For partitioned tables, compare counts per partition (`GROUP BY date_partition`) to narrow the scope before running a full ID comparison on only the divergent partitions.

> **Count reconciliation as a gate**
>
> Run `COUNT(*)` on every incremental extraction as a cheap health check. It adds seconds to the run and catches drift early – before it accumulates into a reconciliation problem. See 6.14 – Reconciliation Patterns.

3.6.3 Propagation

Once you've identified deleted IDs, the question is what to do with them in the destination. This is a load concern – see 4.3 – Merge / Upsert for the mechanics.

Three options, in order of preference:

1. **Soft-delete in destination.** Set `_is_deleted = true` and `_deleted_at = CURRENT_TIMESTAMP`. The row stays queryable, downstream consumers can filter on the flag, and the delete is reversible if the source was wrong. If downstream consumers are technical (analysts, dbt models), this is the default.
2. **Hard-delete in destination.** `DELETE FROM destination WHERE id IN (...)`. Matches the source exactly. Simpler for non-technical consumers who don't understand why "deleted" rows still appear in their reports.
3. **Move to a `_deleted` table.** `INSERT INTO invoices_deleted SELECT *, CURRENT_TIMESTAMP AS _deleted_at FROM invoices WHERE id IN (...); DELETE FROM invoices WHERE id IN (...)`. Only when governance or audit requirements demand a record of what was deleted and when. Adds operational complexity.

3.6.4 `invoices` / `invoice_lines`

The domain model case: open `invoices` get hard-deleted regularly. `invoice_lines` get hard-deleted independently of their header – not just via cascade. This creates two detection scopes:

- **Header deletes:** compare `invoice_id` sets between source and destination. The open/closed split from 3.7 – Open/Closed Documents helps – the open-side full extract naturally reveals missing headers.
- **Line deletes:** for each header that still exists, compare `line_num` sets. A header that hasn't changed can still have lines removed underneath it – the header cursor from 3.4 – Cursor from Another Table is blind to this.

In SAP B1, removing a single `invoice_line` triggers a delete+reinsert of ALL surviving lines with new `LineNum` values. The old line numbers are gone, the new ones look like fresh inserts. A full ID comparison catches this while a cursor never will.

3.6.5 By Corridor

Transactional to columnar corridor

The source-side `SELECT id` is the bottleneck – a full table scan on a transactional engine. The destination-side comparison is cheap (single-column scan). Land source IDs into a staging table and run the diff in the destination to avoid pulling large ID sets through the orchestrator. For propagation, soft-delete is a metadata update on the destination – see 4.3 – Merge / Upsert.

Transactional to transactional corridor

Both sides are cheap for ID extraction if the primary key is indexed (it always is). The comparison can run in either system. `DELETE FROM destination WHERE id IN (...)` is a natural fit here – transactional engines handle point deletes efficiently.

3.7 Open/Closed Documents

> **One-liner:** Mutable drafts vs immutable posted documents. Extraction strategy should differ based on document lifecycle state.

See 3.4 – Cursor from Another Table for when the header cursor is enough. This pattern picks up where 3.4 – Cursor from Another Table's "when the header cursor lies" leaves off.

3.7.1 Mutable vs. Frozen

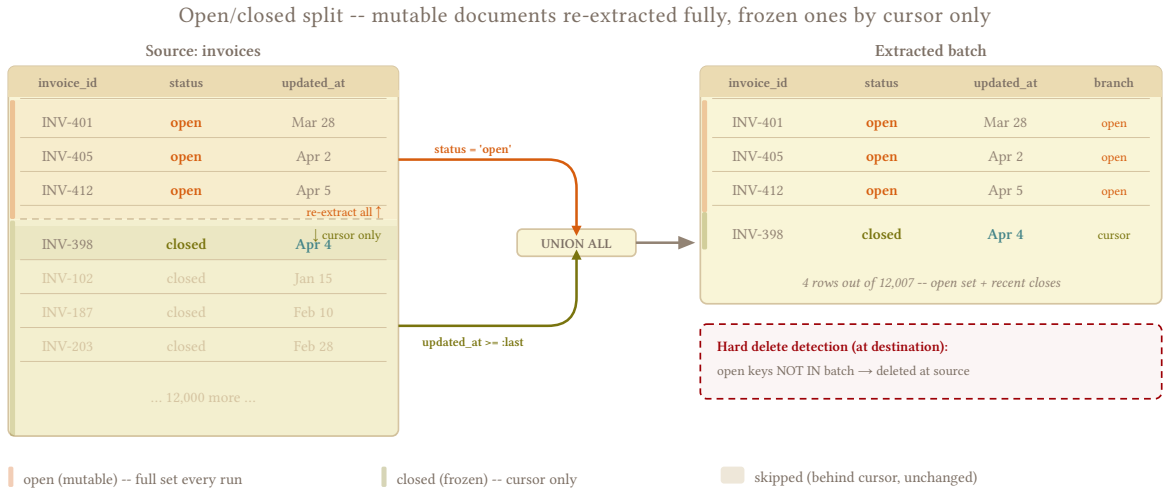
`invoices` are mutable while open – status changes, lines get added or removed, amounts are adjusted. Once posted or closed, they're frozen. Treating both sides the same either wastes resources (re-extracting millions of immutable rows) or misses changes (a cursor can't see mutations on open documents that didn't update the header timestamp).

The business lifecycle itself is the scoping mechanism. Open documents need full re-extraction because anything can change. Closed documents are safe to extract once and never revisit.

3.7.2 The Split

Two extraction strategies for one table:

- **Open documents:** re-extract the full set on every run. They're mutable – lines change, statuses shift, amounts adjust. The only way to be sure you have the current state is to pull it again.
- **Closed documents:** extract only the recently closed. Once posted, a closed invoice is frozen. In many jurisdictions, modifying a closed invoice is illegal – this is one of the rare cases where a soft rule ("we never edit closed invoices") is backed by a hard rule (the law). See 1.6 – Hard Rules, Soft Rules.



3.7.3 The Combination Query

Query against the **source**, combined into one extraction:

```sql
-- source: transactional
SELECT * FROM invoices
  WHERE status = 'open'

UNION ALL

SELECT * FROM invoices
  WHERE status <> 'open'
    AND updated_at >= :last_run;
```

`UNION ALL`, not `OR`. An `OR` across different columns forces the planner to merge index scans (BitmapOr in PostgreSQL, Index Merge in MySQL) – mechanisms that are fragile, statistics-sensitive, and frequently fall back to a full table scan. `UNION ALL` lets each branch use its own optimal index independently: the open branch seeks on `status`, the closed branch seeks on `updated_at` (or a composite `(status, updated_at)`). The branches are mutually exclusive by construction, so no duplicates.

The open set covers all mutations and line changes – everything the header cursor in 3.4 – Cursor from Another Table couldn't see. The closed set is cheap because closed documents don't change.

> **From production: Draft invoices counted as real sales**
>
> A client created draft invoices constantly, deleting and recreating them as documents got revised. We didn't notice from the pipeline – we noticed because we were consistently reporting more sales than the source, since every transient draft landed as a real invoice and never got cleaned up when the source deleted it. The fix was exactly this split: re-scan every document still marked open (`DocType = 'O'`) in the **destination**, take those primary keys back to the source to see which ones survived, and `UNION` that with the incremental window of recently changed documents. One pass then covered everything mutable plus everything new, and the drafts that had vanished at the source dropped out of the destination instead of accumulating as phantom revenue.

The **destination** still has documents that were open last run but have since closed or been deleted at the source. The open-side extract no longer includes them. The closed-side cursor catches transitions (the document appears with `status = 'closed'` and a recent `updated_at`). Deletes need 3.6 – Hard Delete Detection.

3.7.4 Extending to Detail Tables

The same split applies to `invoice_lines`: re-extract all lines for open invoices, cursor-only for closed. Same `UNION ALL` structure:

```sql
-- source: transactional
SELECT il.*
  FROM invoice_lines il
  JOIN invoices i ON il.invoice_id = i.invoice_id
  WHERE i.status = 'open'

UNION ALL

SELECT il.*
  FROM invoice_lines il
  JOIN invoices i ON il.invoice_id = i.invoice_id
  WHERE i.status <> 'open'
    AND i.updated_at >= :last_run;
```

The line extraction joins to the header's status, not just its timestamp. This is the answer to 3.4 – Cursor from Another Table's blind spot: open documents get full line coverage regardless of whether the header's `updated_at` fired.

> **Line status can diverge from header**
>
> `invoice_lines` can have their own `status` – a line marked `disputed` on an otherwise open invoice, or a line already `approved` while the header is still `open`. The split here is on the **header's** lifecycle, not the line's. An open invoice with a mix of approved and disputed lines is still in the open set and gets fully re-extracted. If the line status changes independently after the header closes, neither side of this pattern sees it – 3.8 – Detail Without Timestamp covers the options.

3.7.5 The Transition Moment

A document closes between runs. Two scenarios:

`updated_at` fires on status change. The closed-side cursor captures it. The document appears in the closed-side extract with its final state. Clean.

`updated_at` doesn't fire on status change. The open-side extract had the document in the previous run (it was still open then). The next run's open set won't include it anymore – and the closed-side cursor won't pick it up either (no `updated_at` change). The document falls out of both sides. The destination keeps the last open-side version – with `status = 'open'` permanently. The actual `status = 'closed'` transition never syncs. Any modifications between the last open-side extract and the close are also lost. The periodic full replace is the only thing that corrects both problems.

> **Close plus hard delete in one window**
>
> A document closes AND a line gets hard-deleted in the same window. The open-side extract from the previous run had the line. The closed-side cursor picks up the header (if `updated_at` fired) but the deleted line is gone from the source. The destination keeps the stale line. Either accept this gap until the periodic full replace, or run a line-level reconciliation on recently transitioned documents.

3.7.6 Reopening

"Closed documents don't reopen" – check the legal framework before assuming this is a soft rule. In most jurisdictions, reopening a posted invoice is illegal; the correct process is to issue a credit note or return document. If the system enforces this, reopening is not a concern for the pipeline.

When it does happen (support manually reopens one, or the system allows it), a reopened document appears in the open set on the next run – caught naturally.

The gap is between close and reopen: the document was in neither set (closed cursor already passed it, open set didn't include it yet). The stateless window approach from 3.3 – Stateless Window Extraction absorbs this if the window covers the gap. If the reopen happens within days and the window is 7 days, the document is already covered.

3.7.7 Hard Deletes on Open Documents

Open `invoices` get hard-deleted regularly – the domain model case.

You already have everything you need from the combination query. The extracted batch contains all currently open documents and all recently changed non-open documents. Query the **destination** for the set of keys currently marked as open, then compare against the extracted batch:

- Key is in the batch with `status = 'open'` ⟶ still open, normal upsert
- Key is in the batch with `status <> 'open'` ⟶ recently closed, upsert updates the status
- Key is **not in the batch at all** ⟶ hard-deleted from source, propagate the delete

No extra staging tables, no second source query. The extracted batch is the source of truth for what exists right now.

```sql
-- destination: columnar
-- Keys marked open in destination that don't appear anywhere in the batch
SELECT d.invoice_id
FROM invoices d
LEFT JOIN _stg_extracted_batch b ON d.invoice_id = b.invoice_id
WHERE d.status = 'open'
  AND b.invoice_id IS NULL;
```

> **This assumes the cursor fires on close**
>
> The logic above depends on recently closed documents appearing in the batch – which only happens if `updated_at` fires when the status changes. If closing a document doesn't bump `updated_at`, a closed document disappears from the open set without appearing in the closed-side cursor. The diff incorrectly classifies it as a hard delete, and the pipeline removes a row that still exists in the source. Verify that status transitions update the cursor before enabling delete propagation.

Alternative: cursor + open + destination keys

If the cursor is unreliable on close, a safer extraction is `UNION ALL` of three branches: the normal cursor (`updated_at >= :last_run`), all open documents (`status = 'open'`), and the set of IDs currently marked open in the destination. The destination-side IDs ensure that anything the pipeline previously loaded as open gets re-checked against the source. The tradeoff: the branches are no longer mutually exclusive – a row can appear in multiple branches – so the load must handle duplicates (dedup view, `QUALIFY`, or upsert).

Closed documents that get hard-deleted – the soft rule violation from 1.6 – Hard Rules, Soft Rules – need the general mechanism from 3.6 – Hard Delete Detection.

3.7.8 The Cost Equation

The cost is relative to the alternative. The ratio of open to total matters more than the absolute number: 50,000 open invoices is 0.05% of a 100-million-row table – a fraction of a full replace. The same 50,000 against a 60,000-row table is 83% – at that point, a full replace is simpler.

In systems with long-lived open documents – consulting invoices open for months, construction contracts open for years – the open set grows and the cost advantage over a scoped full replace (2.4 – Scoped Full Replace) shrinks. Evaluate case by case.

3.7.9 By Corridor

Transactional to columnar corridor

Both queries run on the source as indexed scans (`status` should be indexed, or at least selective enough). The open set is small relative to the table, so the source cost is low. The destination load cost depends on the load strategy – see 4.3 – Merge / Upsert. The delete detection query runs entirely in the destination and is cheap (single-column scans).

Transactional to transactional corridor

Cheap on both sides. The open-side extract is a small indexed scan. The delete detection diff can run as a single query joining source and destination if both are accessible from the same connection, or via staging tables if they're not.

3.8 Detail Without Timestamp

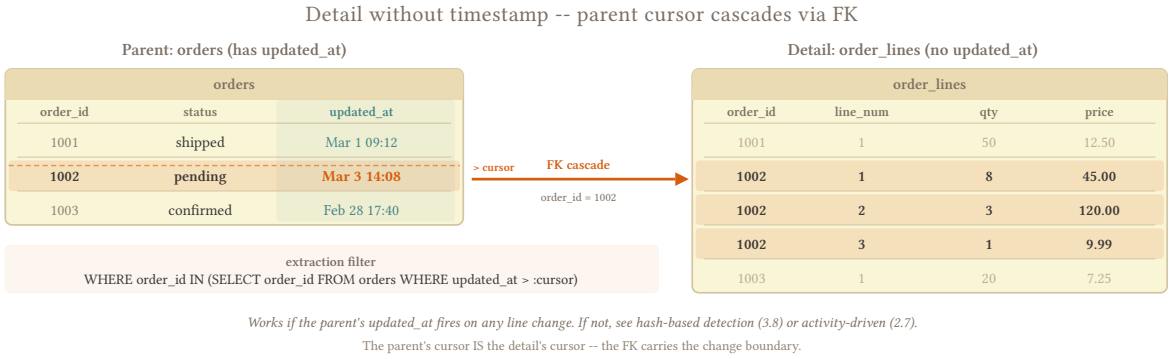
One-liner: `order_lines` and `invoice_lines` have no `updated_at`. They depend on the header for change detection – but what if the detail changes without the header changing?

3.4 – Cursor from Another Table handles this for most tables. The header's `updated_at` scopes the detail extraction, and the periodic full replace (2.1 – Full Scan Strategies) catches anything the cursor missed. The blind spot – a detail row that mutates without the header changing – is real but usually narrow enough that the full replace absorbs it.

When the blind spot is too wide, two responses work better than building detection mechanisms:

Widen the stateless window. A 7-day window on the header might miss a line that changed 10 days ago without touching the header. A 30-day window re-extracts more lines but captures those silent mutations – the cost is proportional to window size, not table size, and the upsert or dedup handles the redundancy.

Increase the full replace cadence. A daily full replace of `order_lines` is often cheaper and simpler than any detection mechanism. Detail tables are bounded by header count × lines per header – they're typically smaller than they look.



3.8.1 Computed Column Signals

Some ERPs maintain computed columns on the header (`DocTotal`, `PaidToDate` in SAP B1) that change when detail rows mutate, even if `updated_at` doesn't fire. If such a column exists, you can detect detail changes after loading the headers: compare the freshly loaded `doc_total` against the previous value at the destination, then go back to the source for detail lines of headers where it differs. This is a two-pass orchestration pattern – earned complexity for tables too large to full-replace where the mutation rate justifies it.

> **Audit computed columns before trusting them**
>
> Verify which detail-level changes actually trigger a recalculation. `DocTotal` changes on quantity/price changes but not on line status changes. A line marked `disputed` without a price change remains invisible.

Hash-based detection (see 2.8 – Hash-Based Change Detection) requires a full source scan to compute hashes – the same cost as a full replace. If you're scanning the full table anyway, just replace it.

3.8.2 By Corridor

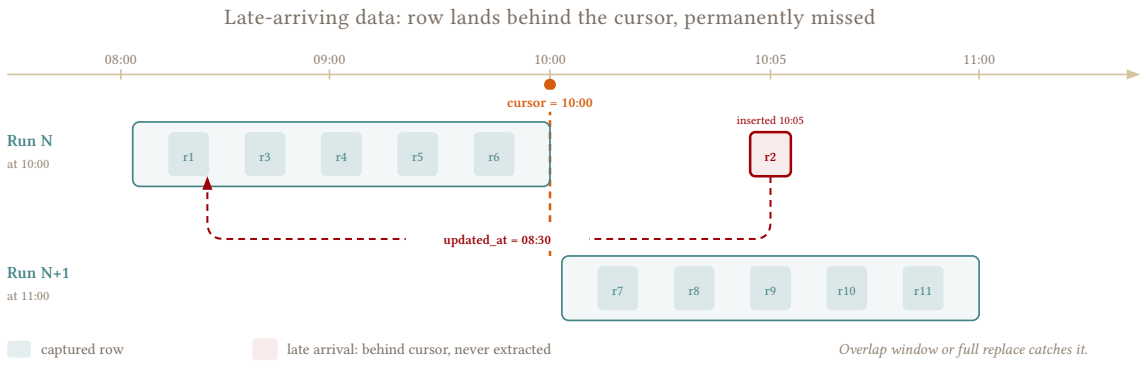
> **Transactional to columnar corridor**
>
> Wider stateless window is the cheapest response. Full replace of detail tables is often viable. See 4.3 – Merge / Upsert for columnar load cost.

Transactional to transactional corridor

Same responses, but the upsert handles duplicates cheaply via PK – wider windows and more frequent full replaces have minimal load overhead. See 4.3 – Merge / Upsert for the upsert mechanics.

3.9 Late-Arriving Data

One-liner: A row's timestamp predates the extraction window. It was modified retroactively, arrived late from a batch job, or was inserted by a slow-committing transaction.



3.9.1 Behind the Cursor

A row lands in the source with an `updated_at` or `created_at` that's already behind your cursor or outside your window. The extraction ran at 10:00, picked up everything through 09:59, and advanced the cursor. At 10:05, a batch job inserts a row with `updated_at = 08:30`. That row is permanently behind the cursor and will never be extracted. Five mechanisms produce this:

- **Retroactive corrections.** Support reopens a 3-day-old order and changes the shipping address. In some systems the correction resets `updated_at` to the original order date rather than the current date, so the row changes while its timestamp moves backward.
- **Batch imports.** An overnight job loads yesterday's POS transactions with `created_at = yesterday`. If your cursor already passed yesterday, those rows are invisible.
- **ERP period closes.** Accounting closes March and runs adjustments. The adjustments land with dates in March, but the close happens in April. A daily cursor in April never looks back at March.
- **Slow-committing transactions.** A long-running transaction inserts a row at 09:50 but doesn't commit until 10:10. The `updated_at` reflects the INSERT time, not the COMMIT time, so the extraction at 10:00 couldn't see the row and the cursor already moved past 09:50.
- **Async replication lag.** The source is a read replica behind the primary by seconds or minutes. Rows committed on the primary during that lag window are invisible to the extraction, and the cursor advances past them.

3.9.2 How Far Back Can It Land?

The overlap window must cover the worst-case late arrival, and that depends entirely on the source system's behavior:

Source behavior	Typical lag	Example
Slow-committing transactions	Seconds to minutes	Long-running INSERT that commits after the extraction
Async replication	Seconds to minutes	Read replica behind the primary
Batch imports	Hours	Overnight POS load with yesterday's timestamps
Retroactive corrections	Days	Support editing a week-old order
ERP period closes	Days to weeks	Accounting adjustments backdated to the closed period
Cross-system reconciliation	Weeks	Finance reconciling invoices from the previous month

> **Measure late arrival, don't guess**
>
> Query the source for rows where `updated_at` predates `created_at` or where `updated_at` is significantly older than the row's actual arrival. Transaction logs, audit tables, or a comparison between `updated_at` and `_extracted_at` over a few weeks will reveal the real distribution. Size the overlap to cover the 99th percentile, not the average.

3.9.3 Overlap Window Sizing

The overlap extends the extraction window backward from the cursor or window start:

```sql
-- source: transactional
-- Cursor-based with overlap
SELECT *
FROM orders
WHERE updated_at >= :last_run - INTERVAL '2 days';
```

```sql
-- source: transactional
-- Stateless window with built-in overlap
SELECT *
FROM orders
WHERE updated_at >= CURRENT_DATE - INTERVAL '9 days';
-- 7 days of intended window + 2 days of overlap
```

The overlap is a parameter to maximize correctness while sacrificing performance, try to size it for the worst-case late arrival, then evaluate cost. If the cost is too high, you have to either run it less often or accept a blind spot which can then be patched with tiered freshness.

The 3.3 – Stateless Window Extraction pattern has overlap built in by design – a 7-day window already covers 7 days of late arrivals, with no overlap parameter to configure and no cursor to worry about. This is one of the strongest arguments for defaulting to stateless windows: the window size itself is the overlap,

and the late-arriving data problem largely disappears. The only case it doesn't cover is rows that land with timestamps older than the window, which requires either a wider window or the periodic full replace. The 3.2 – Cursor-Based Timestamp Extraction pattern needs the overlap added explicitly to the boundary condition.

How large can a window get? I run a 90-day stateless window on a client's transactions because their back-office team routinely edits orders weeks after the fact, backdates corrections, and re-opens closed periods without notice. A 7-day window missed data constantly; 30 days still wasn't enough. At 90 days the source query is heavier, but the table is indexed on `updated_at` and the alternative – constant reconciliation and manual fixes – was more expensive in engineering time.

3.9.4 Oracle EBS PRUNE_DAYS

Oracle BI Applications (OBIA) formalized this pattern as `PRUNE_DAYS` – a configurable parameter that subtracts N days from the high-water mark on every extraction. The parameter exists because Oracle EBS has long-running concurrent programs (batch jobs) that can take hours to complete, inserting rows with timestamps from when the program started, not when it committed. The concept generalizes beyond Oracle: any system where the gap between "when the row's timestamp says it was created" and "when the row became visible" can be large needs an equivalent parameter.

3.9.5 Cost of Overscanning

A wider overlap re-extracts more rows that haven't changed, increasing both source query cost and destination load cost (see 4.3 – Merge / Upsert for the load side). The tradeoff is correctness vs. cost, framed by 1.8 – Purity vs. Freshness: an hours-long overlap adds negligible cost, a days-long overlap is moderate depending on mutation rate, and a weeks-long overlap starts approaching a full replace – at which point a scoped full replace (2.4 – Scoped Full Replace) may be simpler than a cursor with a massive overlap.

3.9.6 Explaining This to Stakeholders

Late-arriving data is one of the hardest pipeline problems to explain to non-technical stakeholders because the failure is invisible: the data looks correct, the pipeline reports success, and the counts are close enough that nobody notices the missing rows until a reconciliation or audit.

What stakeholders need to understand:

"When we extract data incrementally, we ask the source: 'give me everything that changed since the last time I asked.' But some changes arrive with timestamps in the past – a correction from last week, a batch import with yesterday's dates, an adjustment from a period close. Our pipeline already asked for that time range and moved on. Those rows are invisible until the next full reload."

The three questions they'll ask:

1. **"Can't you just get everything?"** Yes – that's a full replace. It's the most correct approach but the slowest and most expensive. I do it periodically as a safety net. The incremental extraction runs between full replaces to keep the data fresh.
2. **"How much data are we missing?"** Depends on the table and the source system. For well-behaved transactional tables with reliable cursors, the gap is bounded by the extraction interval – if you run every 15 minutes, you're at most 15 minutes behind. For tables fed by batch jobs or ERP period closes,

the gap can be days. I size the overlap window to cover the worst case I've measured, and the periodic full replace catches anything beyond that.

3. **"Why can't the data just be right?"** Because "right" has a cost. A 7-day overlap window on a table with 100 million rows re-extracts 7 days of data on every run to catch the rare late arrival. A 30-day overlap re-extracts 30 days. At some point, the cost of absolute correctness exceeds the cost of the occasional missing row. The overlap window is where I draw that line, and the full replace is the safety net behind it.

Frame it as a tradeoff

Stakeholders respond better to "we chose a 7-day safety margin that catches 99% of late arrivals, with a weekly full reload as a backstop" than to "our pipeline might miss some rows." Both are true, but the first version communicates a deliberate engineering decision. See 6.4 – SLA Management for how to formalize these guarantees into measurable SLAs.

3.9.7 By Corridor

Transactional to columnar corridor

The source-side extraction cost scales with the overlap (wider window = more rows scanned on an indexed `updated_at`). The destination-side cost depends on how many partitions the overlap touches – see 4.3 – Merge / Upsert and 1.4 – Columnar Destinations for partition rewrite behavior per engine.

Transactional to transactional corridor

Both sides scale with batch size, not table size, so wider overlaps are cheap. A 7-day overlap on a table with 1,000 changes per day re-extracts ~7,000 rows per run – negligible for a transactional upsert.

3.10 Create vs Update Separation

One-liner: When the trigger fires on UPDATE only and INSERT rows have `updated_at = NULL`, you need two extraction paths.

3.1 – Timestamp Extraction Foundations documents the failure mode: a trigger maintains `updated_at` on UPDATE but not on INSERT, leaving new rows with `updated_at = NULL`. A cursor on `updated_at >= :last_run` catches every modification to existing rows while every new row is permanently invisible.

This happens in `orders` when the trigger was added after the table existed and only wired to the UPDATE event – a common pattern in legacy systems where the trigger was built for auditing, not extraction. The

result is two populations in the same table: rows that have been updated at least once (visible to the cursor) and rows that were inserted but never touched again (invisible).

```
-- source: transactional
SELECT order_id, created_at, updated_at
FROM orders
ORDER BY order_id DESC
LIMIT 5;
```

order_id	created_at	updated_at
1005	2026-03-14 09:30:00	NULL
1004	2026-03-14 08:15:00	NULL
1003	2026-03-13 16:00:00	2026-03-14 10:00:00
1002	2026-03-12 11:00:00	NULL
1001	2026-03-10 09:00:00	2026-03-13 14:30:00

Orders 1005, 1004, and 1002 were inserted but never updated – `updated_at` is NULL and the cursor will never see them.

3.10.1 Detection

Before building any workaround, confirm the problem exists:

```
-- source: transactional
SELECT
  COUNT(*) AS total_rows,
  COUNT(updated_at) AS rows_with_updated_at,
  COUNT(*) - COUNT(updated_at) AS rows_without_updated_at
FROM orders;
```

total_rows	rows_with_updated_at	rows_without_updated_at
84,230	61,507	22,723

If `rows_without_updated_at` is significant, the trigger is UPDATE-only. A second check confirms the pattern – recently created rows should have NULLs:

```
-- source: transactional
SELECT COUNT(*) AS recent_nulls
FROM orders
WHERE created_at >= CURRENT_DATE - INTERVAL '7 days'
  AND updated_at IS NULL;
```

A high count here means the problem is ongoing, not a historical artifact from before the trigger was added.

Check the trigger definition directly

In PostgreSQL, `\dS orders` or `SELECT \* FROM information_schema.triggers WHERE event_object_table = 'orders'` shows exactly which events fire the trigger. In MySQL, `SHOW TRIGGERS LIKE 'orders'` does the same. Take some time to check before debugging.

3.10.2 Why Not COALESCE?

The obvious first attempt is `COALESCE(updated_at, created_at) >= :last_run` – fall back to `created_at` when `updated_at` is NULL. It works, but `COALESCE` wraps both columns in a function, which prevents the optimizer from using indexes on either one. PostgreSQL supports a functional index on the expression; MySQL and SQL Server don't. On large tables without the functional index, this degrades to a full scan. If both columns are NULL on any row, `COALESCE` returns NULL and that row vanishes from every extraction – use the Dual Cursor approach below instead.

3.10.3 Dual Cursor via UNION ALL

Split the two populations into separate queries so each uses its own index:

```
-- source: transactional
SELECT * FROM orders
  WHERE updated_at >= :last_run
UNION ALL
SELECT * FROM orders
  WHERE created_at >= :last_run;
```

The first branch catches updates via `updated_at`. The second branch catches new inserts via `created_at`. Each branch hits its own index cleanly, no function wrapping, no optimizer guesswork.

Overlap. A row inserted at 09:00 and updated at 10:30 appears in both branches. The upsert or dedup at the destination handles the duplicate – the later version wins, which is the correct outcome.

If `created_at` doesn't exist either, use the Cursor + NULL Extraction approach below.

3.10.4 Cursor + NULL Extraction

When `created_at` is also unreliable or missing, extract everything the cursor catches plus every row that has no cursor at all:

```
-- source: transactional
SELECT * FROM orders
  WHERE updated_at >= :last_run
UNION ALL
SELECT * FROM orders
  WHERE updated_at IS NULL;
```

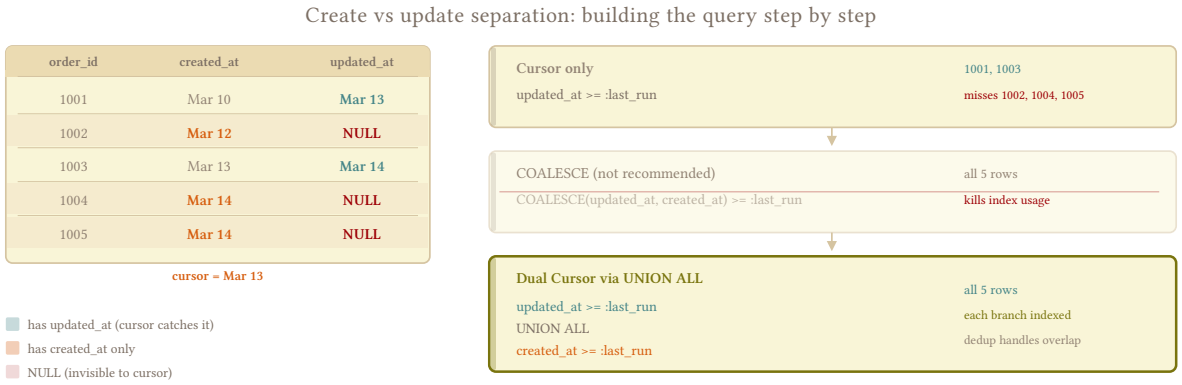
The first branch is the normal cursor. The second branch re-extracts every row where `updated_at` is NULL on every run – these are the rows the cursor will never see. If the NULL population is small (a few

thousand rows from broken inserts), this is cheap. If it's large (the trigger never existed and half the table is NULL), the second branch approaches a full scan and a full replace (2.1 – Full Scan Strategies) is simpler.

The upsert or dedup handles the redundancy from re-extracting NULLs every run. As the source team fixes the trigger and the NULL population shrinks, the second branch gets cheaper until it eventually returns zero rows.

The real fix is upstream

If the source team adds an `AFTER INSERT` trigger that populates `updated_at` and backfills existing NULLs, the standard 3.2 – Cursor-Based Timestamp Extraction cursor works and the strategies above become unnecessary. This is a source-side fix, not a pipeline change – but it's worth pursuing.



If created_at is also missing: replace second branch with WHERE updated_at IS NULL

3.10.5 Choosing a Strategy

Situation	Strategy
`created_at` is reliable	Dual cursor – each branch uses its own index
`created_at` is missing or unreliable	Cursor + NULL extraction
NULL population is large (>50% of table)	Full replace – simpler than re-extracting NULLs every run

In all cases, the periodic full replace (2.1 – Full Scan Strategies) catches anything the workaround misses – rows where both timestamps are NULL, bulk imports that bypassed both triggers, sequences that created gaps the insert cursor didn't cover.

3.10.6 By Corridor

Transactional to columnar corridor

The dual cursor produces two result sets that get UNIONed before loading. The duplicate rows from the overlap between insert and update cursors are handled by the destination's MERGE – see 4.3 – Merge / Upsert. The COALESCE approach benefits from a functional index on the source side; without one, the extraction query is a full scan on every run.

Transactional to transactional corridor

Both cursors should be cheap indexed range scans on the source. The destination upsert (`ON CONFLICT ... DO UPDATE`) absorbs overlap duplicates naturally – see 4.3 – Merge / Upsert.

4.1 Full Replace Load

> **One-liner:** Drop and reload. The simplest load strategy and the default – stateless, idempotent, no merge logic.

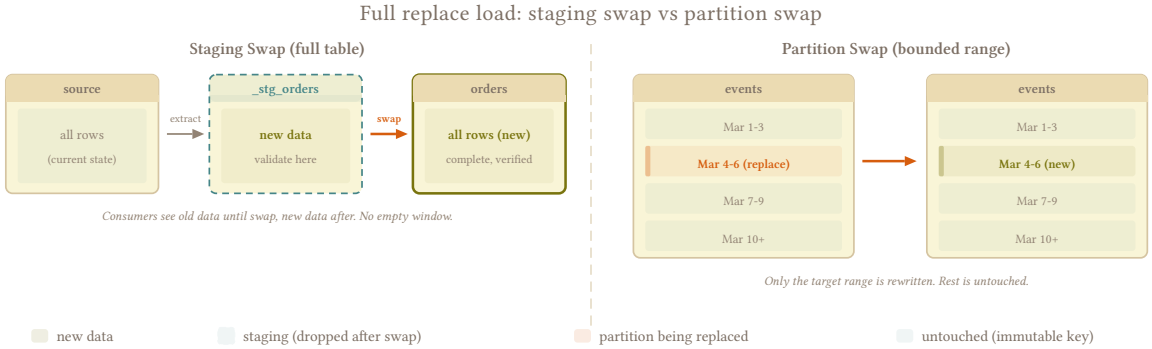
4.1.1 Scope Alignment

The extraction patterns in Part II give you a dataset – full table, scoped range, set of partitions – and this page covers the destination-side mechanics: how to swap it in.

The fundamental constraint is scope alignment: **what you replace must match what you extracted.** All with all (full table extraction replaces the full table), or parts with parts (partition extraction replaces exactly those partitions). What breaks is replacing all with some – if you truncate the destination and load only rows where `updated_at >= :last_run`, every row that wasn't recently modified vanishes. This is the simpler case.

The less obvious case is partition swap. Replacing a partition looks safe – it's parts with parts – but only when the partition key is immutable. If you partition on `event_date` or `created_at`, a row that landed in the `2026-03-01` partition stays there forever; re-extracting and replacing that partition is a clean swap with no data loss. But if the partition key is mutable – say you partition on `updated_at` – a row updated today jumps from last week's partition into today's. You re-extract and replace today's partition, which captures the current version, but the stale copy in last week's partition stays behind untouched. Now you have duplicates, or stale data, depending on how consumers query it. Mutability is what separates "I can replace this slice" from "I have to merge or upsert it," and it applies to partitions just as much as it applies to full tables. When the partition key can change, you need merge or append strategies instead (see 4.3 – Merge / Upsert, 4.4 – Append and Materialize).

The naive TRUNCATE + INSERT leaves a window where the table is empty – bad if anyone's querying it. Safer mechanisms exist, and the choice depends on how much downtime is acceptable and how much validation you want before committing.



4.1.2 Why Not Truncate + Insert?

The naive approach is `TRUNCATE TABLE orders; INSERT INTO orders SELECT * FROM stg_orders`. The two operations aren't atomic – between TRUNCATE and INSERT, the table is empty, and any consumer querying `orders` sees zero rows. If the INSERT fails halfway (connection drop, disk full, timeout), you're left with a partially loaded table and no way back, because TRUNCATE is DDL and can't be rolled back on most engines.

PostgreSQL is the exception: `TRUNCATE` is transactional there, so wrapping both in a transaction gives you atomicity for free. But even on PostgreSQL, staging swap is strictly better – you get the same atomicity plus a validation step before the data reaches production. Use staging swap instead.

4.1.3 Staging Swap

Load into a staging table, validate, then swap to production. Consumers see complete data throughout – the old version until the swap, the new version after.

The validation step between load and swap is the key advantage over truncate + insert. If the extraction returned garbage – zero rows from a silent failure, a schema change that dropped columns, a type mismatch that cast everything to NULL – you catch it before it reaches production.

The swap mechanism varies by engine: Snowflake has `ALTER TABLE SWAP WITH` (atomic, metadata-only), PostgreSQL uses `ALTER TABLE RENAME` inside a transaction, BigQuery uses `bq cp` or DDL rename. See 2.3 – Staging Swap for the per-engine mechanics, including the parallel schema convention for managing staging tables at scale.

4.1.4 Partition Swap

When the table is partitioned and you're replacing a slice – yesterday's data, last week's events, a backfill of a specific month – partition swap replaces only the affected partitions while leaving the rest untouched.

```
-- destination: snowflake / redshift
BEGIN;
DELETE FROM events
WHERE partition_date BETWEEN :start_date AND :end_date;
INSERT INTO events SELECT * FROM stg_events;
COMMIT;
```

```
# destination: bigquery
# Partition copy -- near-metadata operation, near-free
bq cp --write_disposition=WRITE_TRUNCATE \
  project:dataset.stg_events$20260307 \
  project:dataset.events$20260307
```

The cost advantage is proportional to the scope: replacing 7 partitions out of 3,000 touches 0.2% of the table, while a full staging swap rewrites the entire thing. See 2.2 – Partition Swap for extraction-side mechanics, per-engine atomicity guarantees, and the partition alignment pitfalls.

4.1.5 DROP vs TRUNCATE vs DELETE

Three ways to clear destination data before loading, each with different behavior:

Operation	What it removes	DDL or DML	Transactional?	Speed
`DROP TABLE`	Schema + data	DDL	No (except PostgreSQL)	Instant – metadata only
`TRUNCATE TABLE`	All rows, keeps schema	DDL	No (except PostgreSQL)	Fast – no per-row logging
`DELETE FROM table`	All rows, keeps schema	DML	Yes	Slow – logs every row

`DROP` is used in staging swap workflows: drop the old production table, rename staging into its place. The risk: if the rename fails mid-way, the table vanishes. Wrapping both in a transaction (PostgreSQL, Snowflake, Redshift) eliminates this gap.

`TRUNCATE` is used in truncate + insert workflows. It deallocates storage without logging individual row deletions, which makes it orders of magnitude faster than DELETE on large tables. A 50M-row table that takes 30 minutes to DELETE completes a TRUNCATE in under a second – the difference is that DELETE generates WAL/redo entries for every row while TRUNCATE resets the storage allocation in one operation.

`DELETE FROM table` (without a WHERE clause) achieves the same result as TRUNCATE but with full transactional semantics – you can roll it back. The cost is the per-row logging: on a transactional destination, the WAL/redo log grows by the size of the table. On BigQuery, a full-table DELETE rewrites every partition. Use it only when you need the rollback guarantee and the table is small enough that the logging cost is acceptable.

> **TRUNCATE vs DELETE in columnar engines**
>
> BigQuery `TRUNCATE` is a metadata operation that resets the table instantly at zero cost. `DELETE FROM table` without a WHERE clause rewrites every partition and charges for bytes scanned. Snowflake `TRUNCATE` reclaims storage immediately (no Time Travel retention); `DELETE` preserves Time Travel history. Choose based on whether you need the recovery window, and are willing to pay the cost.

4.1.6 Choosing a Mechanism

Situation	Mechanism
Full table replacement	Staging swap
Partitioned table, replacing a bounded range	Partition swap

Both are idempotent – rerunning the same extraction and load produces the same destination state regardless of how many times you run it, with no accumulated state, no cursor, and no merge logic (see 1.9 – Idempotency). The shared failure mode is loading bad data into production before catching the problem, which the staging swap's validation step prevents.

Validate before you swap -- an empty staging table is a silent wipe

A source timeout, a broken query, or a permission change can return zero rows without raising an error. If the pipeline swaps that empty result into production, the table is gone – truncate + insert leaves it empty, staging swap replaces it with nothing, partition swap deletes the target range and inserts zero rows. Always check that the staging table has a sane row count before committing the swap. A simple `SELECT COUNT(\*) FROM stg_orders` compared against a threshold (previous count, minimum expected, or percentage of the current production table) is enough to catch it.

4.1.7 By Corridor

Transactional to columnar

Staging swap is the standard. Partition swap for partitioned tables where only a slice needs replacing. On BigQuery, prefer `bq cp` over DML for both staging swap and partition swap – copy jobs are free (no slot consumption, no bytes-scanned charge) for same-region operations.

Transactional to transactional

Both mechanisms work cleanly. PostgreSQL's transactional DDL means the `RENAME` approach inside a transaction is atomic and instant. One caveat: foreign keys referencing the production table will break during the rename. Disable FK checks or drop and recreate constraints as part of the swap if other tables reference the target.

4.2 Append-Only Load

> **One-liner:** Source is immutable – rows are inserted, never updated or deleted. Append to the destination with pure INSERT, no MERGE needed.

`events`, `inventory_movements`, and clickstream tables only grow. Rows are inserted once and never modified or deleted. A MERGE on every load – matching on a key, checking for existence, deciding between INSERT and UPDATE – is unnecessary work when the source **guarantees** that every extracted row is new.

The append-only load skips all of that: extract the new rows, INSERT them into the destination, done. No key matching, no partition rewriting, no update logic – which also makes it the most naive and fragile load strategy in the book.

4.2.1 The Pattern

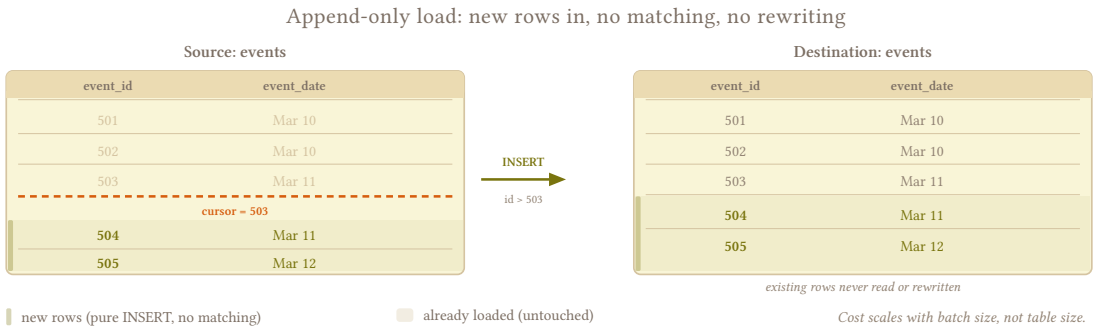
The extraction side uses a sequential ID cursor (3.5 – Sequential ID Cursor) or a `created_at` timestamp cursor (3.2 – Cursor-Based Timestamp Extraction) to scope the new rows:

```
-- source: transactional
SELECT *
FROM events
WHERE event_id > :last_id;
```

The load side is a pure INSERT:

```
-- destination: columnar
INSERT INTO events
SELECT * FROM _stg_events;
```

No `ON CONFLICT`, no `MERGE`, no `MATCHED` / `NOT MATCHED` logic. The destination table grows monotonically, just like the source.



4.2.2 Why This Is the Cheapest Load

In columnar engines, `MERGE` reads the existing table to find matches, then rewrites the affected partitions with the merged result – even when every row in the batch is new. On a table with 500M rows and 50K new rows per run, the MERGE still scans the destination side of the join to confirm that none of the 50K exist. That scan is the cost floor of any MERGE operation, regardless of how many rows actually match.

A pure APPEND writes the new rows into a new partition (or appends to the current one) without reading anything that already exists. BigQuery `INSERT` jobs write to new storage blocks without touching existing partitions. Snowflake appends to micro-partitions. The cost scales with the batch size, not the table size – 50K rows cost the same whether the destination has 1M rows or 500M.

> **Verify the source is actually immutable**
>
> Before committing to this pattern, confirm that "events are never updated" is a **hard rule**, not a soft one (1.6 – Hard Rules, Soft Rules). Unless the schema enforces it, someone will eventually run an UPDATE on `events` – a bulk correction, an admin fix, a backfill that modifies existing rows – and the append-only load will miss the change entirely. Check with the source team, and keep the periodic full replace (2.1 – Full Scan Strategies) as a safety net.

4.2.3 When "Append-Only" Produces Duplicates

Three scenarios where a pure APPEND loads the same row twice:

Pipeline retry. The extraction succeeded and the load partially completed, but the cursor didn't advance because the run was marked as failed. The retry re-extracts the same batch, and the rows that already loaded appear again.

Overlap buffer. 3.5 – Sequential ID Cursor recommends a small overlap (`event_id >= :last_id - 100`) to absorb out-of-order sequence commits. The overlap region is extracted on every run by design.

Upstream replay. The source system replays events – a Kafka consumer rewinds, an API returns the same batch on retry, a file is redelivered. The rows are identical to ones already loaded, but the extraction can't tell.

4.2.3.1 Handling Duplicates

Two approaches, depending on the destination engine:

Transactional destinations – reject at load time:

```
-- destination: transactional
INSERT INTO events (event_id, event_type, event_date, payload)
SELECT event_id, event_type, event_date, payload
FROM _stg_events
ON CONFLICT (event_id) DO NOTHING;
```

`ON CONFLICT DO NOTHING` silently drops duplicates that already exist in the destination. The primary key does the deduplication, and the load cost is negligible because the rejected rows don't generate writes.

Columnar destinations – deduplicate at read time:

```
-- destination: columnar
-- View that exposes only the latest version of each row
CREATE OR REPLACE VIEW events_current AS
SELECT *
FROM events
QUALIFY ROW_NUMBER() OVER (PARTITION BY event_id ORDER BY _extracted_at DESC) = 1;
```

BigQuery and Snowflake don't enforce primary keys, so duplicates will land in the table and the deduplication happens downstream through a view or materialized table. This is the foundation of the 4.4 – Append and Materialize pattern – the difference is that 4.4 – Append and Materialize applies it to mutable data (every version of a row), while here the duplicates are accidental copies of the same immutable row. For your destination, it's the same thing.

> **Don't MERGE to deduplicate immutable data**
>
> Switching from APPEND to MERGE because "duplicates might happen" throws away the cost advantage of append-only loading completely. Handle duplicates at the edges – `ON CONFLICT DO NOTHING` on transactional destinations, a dedup view on columnar – and keep the load path cheap.

4.2.4 The Fragility of Append-Only

The cost advantage of this pattern depends entirely on the source being immutable – and that assumption is fragile. The moment someone runs an UPDATE on `events`, or a backfill modifies existing rows, or a correction script touches historical data, the append-only contract is broken and every row that changed sits silently wrong in the destination.

The recovery path is expensive. You either switch to 4.3 – Merge / Upsert (which rewrites partitions on every load), add a dedup-and-reconcile layer from 4.4 – Append and Materialize, or run a full replace from 4.1 – Full Replace Load to reset the destination. What was the cheapest load pattern in the book becomes one of the most expensive the moment the assumption breaks, because the pipeline has no mechanism to detect or correct the mutation – it just keeps appending new rows while the old ones stay wrong.

Before choosing this pattern, ask how confident you are that the source will stay immutable – not today, but across schema changes, team turnover, and the admin script someone will write at 2am during an incident. If the answer is "pretty confident but not certain," a periodic full replace via 4.1 – Full Replace Load is the safety net, and its cadence should reflect how much damage a silent mutation would cause before the next reload.

4.2.5 Partitioning by Date

Append-only tables are a natural fit for date-based partitioning: `events` partitioned by `event_date`, `inventory_movements` by `movement_date`. Each run's new rows land in the partition corresponding to their date, and old partitions are never touched.

This alignment gives you two operational advantages:

- **Backfill** is a partition replace: re-extract a date range, load into the corresponding partitions using 4.1 – Full Replace Load, done. The rest of the table is untouched.
- **Cost control** in columnar engines: queries that filter on `event_date` scan only the relevant partitions. Without partition pruning, a query over yesterday's events scans the entire table.

> **Late-arriving events land in past partitions**
>
> An event with `event_date = 2026-03-10` arriving on `2026-03-14` lands in the March 10 partition. If that partition was already "closed" by a retention policy or a downstream process that assumed it was complete, the late arrival is either lost or creates an inconsistency. See 3.9 – Late-Arriving Data for overlap sizing that absorbs this.

4.2.6 By Corridor

> **Transactional to columnar**
>
> Pure APPEND is the cheapest load operation available – no partition rewriting, no key matching. BigQuery charges for bytes written on INSERT, not bytes scanned. Snowflake appends to micro-partitions without compaction cost at load time. Dedup view adds read cost only when queried, not on every load.

Transactional to transactional

`INSERT ... ON CONFLICT DO NOTHING` handles duplicates at load time with minimal overhead. The primary key index absorbs the conflict check. For high-volume append tables (`events` with millions of rows per day), ensure the destination is partitioned and that `autovacuum` keeps up with the insert rate.

4.3 Merge / Upsert

One-liner: Match on a key, update if exists, insert if new. The workhorse of incremental loading – and the most expensive operation in columnar engines.

The extraction side (3.2 – Cursor-Based Timestamp Extraction, 3.3 – Stateless Window Extraction) produces a batch of changed rows. The destination already has prior versions of some of those rows. The load needs to reconcile: insert the new ones, update the existing ones, and leave everything else untouched.

4.3.1 MERGE Across Engines

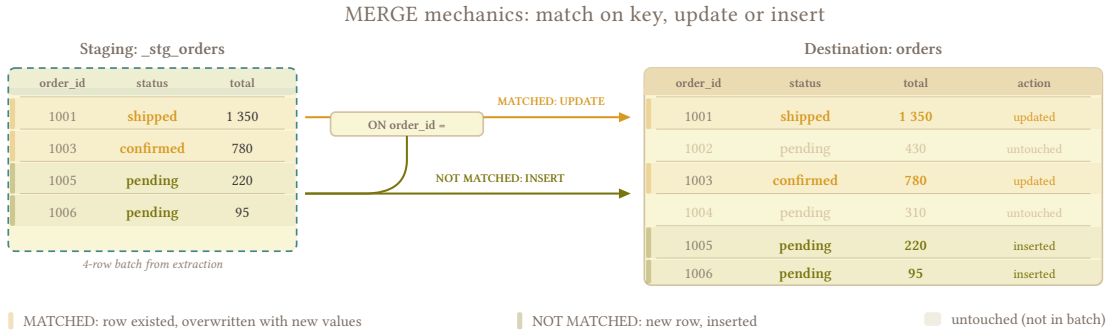
The syntax varies, the semantics are the same – match on a key, decide between INSERT and UPDATE:

```
-- destination: columnar (BigQuery / Snowflake)
MERGE INTO orders AS tgt
USING _stg_orders AS src
ON tgt.order_id = src.order_id
WHEN MATCHED THEN
  UPDATE SET
    tgt.status = src.status,
    tgt.total = src.total,
    tgt.updated_at = src.updated_at
WHEN NOT MATCHED THEN
  INSERT (order_id, status, total, created_at, updated_at)
  VALUES (src.order_id, src.status, src.total, src.created_at, src.updated_at);
```

```
-- destination: transactional (PostgreSQL)
INSERT INTO orders (order_id, status, total, created_at, updated_at)
SELECT order_id, status, total, created_at, updated_at
FROM _stg_orders
ON CONFLICT (order_id)
DO UPDATE SET
  status = EXCLUDED.status,
  total = EXCLUDED.total,
  updated_at = EXCLUDED.updated_at;
```

```
-- destination: transactional (MySQL)
INSERT INTO orders (order_id, status, total, created_at, updated_at)
SELECT order_id, status, total, created_at, updated_at
FROM _stg_orders
ON DUPLICATE KEY UPDATE
  status = VALUES(status),
  total = VALUES(total),
  updated_at = VALUES(updated_at);
```

All three produce the same result: rows that existed get overwritten, rows that didn't get inserted.



The maintenance cost is in the column lists. A table with 5 columns is manageable; a table with 80 columns means 80 entries in the INSERT, 80 in the VALUES, and 79 in the UPDATE SET (everything except the key). Add a column to the source and you need to update three places in BigQuery/Snowflake, two in PostgreSQL. Multiply by however many tables you're loading this way and the MERGE statement itself becomes a schema-drift vector – the exact failure mode covered in 4.3.6 – MERGE and Schema Evolution. Most production pipelines generate these statements dynamically from the staging table's schema at runtime, which eliminates the drift but requires a builder function per engine dialect. See 8.1.4 – Upsert / MERGE for per-engine syntax helpers and 8.1.4.1 – Dynamic MERGE Generation for help on that builder function.

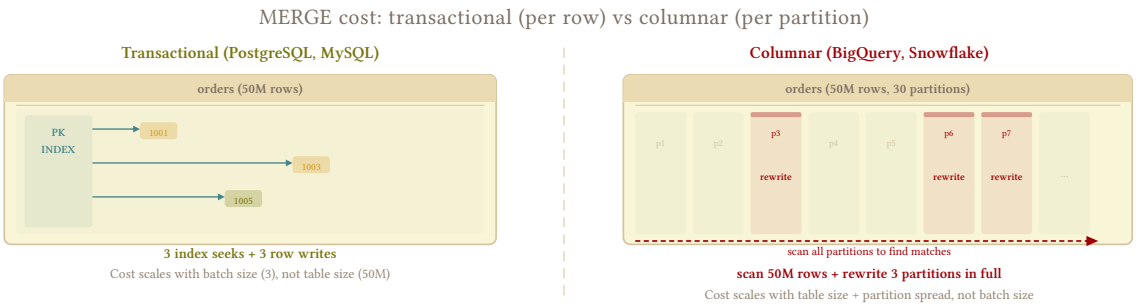
4.3.2 Cost Anatomy

In transactional engines, MERGE cost scales with the batch size – the engine looks up each incoming row by primary key (index seek), decides INSERT or UPDATE, and writes the result. A 10K-row batch against a 50M-row table does 10K index lookups and 10K writes. Cheap.

In columnar engines, the cost structure is fundamentally different. BigQuery's MERGE reads the **entire destination table** (or at minimum every partition that the batch touches) to find matches, then rewrites those partitions with the merged result. A 10K-row batch that touches 30 date partitions rewrites all 30 partitions in full – even if 9,990 of the 10K rows land in a single partition. The read + rewrite cost dominates, and it scales with table size and partition spread, not batch size.

> **BigQuery MERGE partition cost**
>
> Every DML statement in BigQuery rewrites every partition it touches – not just the affected rows within each partition. If your batch contains rows spread across 30 dates, that's 30 full partition rewrites. Keep load batches aligned to as few partitions as possible. See 1.4 – Columnar Destinations for per-engine DML behavior.

Snowflake rewrites affected micro-partitions, which is more granular than BigQuery's date-partition model but still means a MERGE touching scattered micro-partitions across the table is significantly more expensive than one touching a contiguous range.



4.3.3 Key Selection

The merge key determines how the destination identifies "the same row." Two options:

Natural key – a column that uniquely identifies the entity at the source: `order_id`, `invoice_id`, `customer_id`. This is the default and the simplest choice when the source has a single-column primary key. Compound natural keys (`order_id + line_num`) work too but make the ON clause larger.

Surrogate key – a hash or synthetic key generated during extraction (see 5.2 – Synthetic Keys). Necessary when the source has no stable primary key, when the natural key is compound and unwieldy, or when multiple sources feed the same destination table and keys can collide.

Non-unique keys compound duplicates

If the merge key matches more than one row in the destination, the behavior is engine-dependent and **always bad**. BigQuery raises an error when multiple destination rows match a single source row. PostgreSQL's `ON CONFLICT` requires the conflict target to be a unique index – non-unique columns can't be used. Snowflake silently updates all matching rows, which means a single source row can overwrite multiple destination rows. Ensure the merge key is unique in the destination, or duplicates will compound on every run – see 6.13 – Duplicate Detection.

Unenforced PKs cause silent data loss

If the source has no unique constraint on what you're using as the merge key, two rows can share the same key value. The merge collapses them into one – the second overwrites the first, and the destination ends up with fewer rows than the source. **This is data loss** and it's invisible: the pipeline reports success, row counts look close enough, and the missing rows only surface when someone reconciles at the record level. **Verify uniqueness on the actual data** before committing to a merge key (1.5 – The Lies Sources Tell). If the source genuinely has duplicate PKs, you need a synthetic key (5.2 – Synthetic Keys).

4.3.4 Full Row Replace vs. Partial Update

Cloning the source exactly is the goal – `DO UPDATE SET (all columns)` is the simplest approach and matches that goal. Every MERGE overwrites the entire row with the source's current state, which means the destination always reflects the source regardless of which columns changed.

```
-- destination: transactional (PostgreSQL)
INSERT INTO orders (order_id, status, total, created_at, updated_at)
SELECT order_id, status, total, created_at, updated_at
FROM _stg_orders
ON CONFLICT (order_id)
DO UPDATE SET
  status = EXCLUDED.status,
  total = EXCLUDED.total,
  created_at = EXCLUDED.created_at,
  updated_at = EXCLUDED.updated_at;
```

Partial updates – `DO UPDATE SET status = EXCLUDED.status` while leaving other columns untouched – earn their complexity only when partial column loading (2.9 – Partial Column Loading) forces them. If you're extracting all columns, update all columns. Deciding which columns "matter" is a business decision that breaks the syntactic boundary (1.2 – What Is Syntactic Transformation).

4.3.5 Delete-Insert as a MERGE Alternative

An alternative to a true MERGE is delete-insert: delete all destination rows that match the incoming batch's keys, then insert the full batch. The result is identical (destination ends up with the source's current state for every key in the batch), but the execution plan avoids the columnar MERGE cost on engines where DELETE + INSERT is cheaper than a single MERGE statement.

```
-- destination: columnar
-- Delete-insert pattern
DELETE FROM orders
WHERE order_id IN (SELECT order_id FROM _stg_orders);

INSERT INTO orders
SELECT * FROM _stg_orders;
```

Whether delete-insert beats MERGE depends on the engine:

Engine	Recommendation	Why
BigQuery	Delete-insert	Both MERGE and delete-insert rewrite every partition they touch – for scattered key-based deletes the cost is comparable. The advantage is concurrency: MERGE is limited to 2 concurrent operations per table (additional statements queue), while INSERT has no such limit. Under load, MERGE queuing can push lag from minutes to hours.
ClickHouse	`ReplacingMergeTree` append	No native MERGE. Insert every batch into an RMT table – the load is a pure append with no per-insert cost. Dedup happens on background merges, with a per-table policy: read through `FINAL` (partition-selective and parallel since 23.3) or an `argMax`/window dedup view, and trigger partition-scoped `OPTIMIZE PARTITION ... FINAL` on a schedule for hot partitions that get heavy reads.
Snowflake	MERGE if clustered	A well-clustered target table lets MERGE prune 99%+ of micro-partitions, making the scan cheap. Without clustering on the join key, both approaches scan everything – and delete-insert avoids the MATCHED/NOT MATCHED overhead. Snowflake Gen2 warehouses (2025) further reduce MERGE cost by up to 4x for sparse updates.
Redshift	Delete-insert	Redshift's MERGE is a macro around DELETE + INSERT + temp table – it adds overhead without optimizing anything. AWS's own documentation recommends delete-insert in a transaction for upserts.
PostgreSQL	`ON CONFLICT`	`ON CONFLICT DO UPDATE` is a single-pass operation with index-only lookups. Delete-insert rebuilds all index entries for every affected row. Use delete-insert only for partition-scoped replacements where the entire range is being reloaded.
MySQL	`ON DUPLICATE KEY`	Preserves auto-increment PKs, fewer lock escalations than `REPLACE INTO`. Delete-insert has more predictable locking under concurrency but requires explicit transactions.
SQL Server	Delete-insert	MERGE has residual bugs (race conditions without `HOLDLOCK`, assertion errors on partitioned tables) and produces a single execution plan for all branches, which can't be tuned independently.

The pattern in the codeblock above uses `WHERE order_id IN (SELECT ...)`, which scopes the DELETE to matching keys – the rows are scattered across partitions, so every touched partition gets rewritten. This is the incremental case. For partition-scoped full replacement (an immutable date range), see 4.1.4 – Partition Swap where the DELETE covers whole partitions and is free on BigQuery.

4.3.6 MERGE and Schema Evolution

A new column appears in the source. What happens to the MERGE?

Column-explicit MERGE (listing columns in the INSERT and UPDATE clauses) silently ignores the new column – the MERGE succeeds, but the new column's data is dropped on every load. The destination never gets it, and nothing alerts you to the gap.

`SELECT *` extraction + dynamic MERGE (building the MERGE statement from the staging table's schema at runtime) fails with a column mismatch if the staging table has a column that doesn't exist in the destination. The error is loud, which is better than silent data loss, but it breaks the pipeline.

Neither outcome is good. Schema evolution needs handling **before** the MERGE executes:

1. **Detect** – compare the staging table's schema against the destination's schema before running the MERGE. New columns, dropped columns, and type changes are all detectable at this point.

2. **Decide** – a schema policy determines the response. Two modes are compatible with faithful replication:

Entity	`evolve`	`freeze`
New column	Add it via `ALTER TABLE`	Raise error
Type change	Widen if compatible	Raise error

Some loaders offer `discard_row` and `discard_value` modes that drop data silently when the schema doesn't match. These are semantic transformation decisions, deciding what data to keep based on schema fit, and they break the syntactic boundary (1.2 – What Is Syntactic Transformation). If the source sent it, the destination should have it. Either accept the change or reject the load; don't silently drop data.

3. **Apply** – if the policy is `evolve`, add the column to the destination (`ALTER TABLE ADD COLUMN`) before the MERGE runs. If it's `freeze`, the pipeline stops and alerts.

I run `evolve` on both – new columns and type changes. If a type widening breaks something downstream, that's a conversation between the source team and the downstream consumers; the pipeline's job is to land what the source sends, not to gatekeep schema changes. The conservative alternative is `evolve` for new columns and `freeze` for type changes, which stops the pipeline on any type change and forces someone to approve the widening before data flows. See 6.9 – Data Contracts for formalizing either policy into enforceable contracts, and 1.4 – Columnar Destinations for how each engine handles `ALTER TABLE ADD COLUMN`.

> **Column-explicit MERGE silently freezes schema**
>
> If your MERGE statement lists columns explicitly and you don't have a detection step before it, the destination schema is frozen at whatever columns existed when the MERGE was written. New source columns are silently dropped on every load, type changes are never propagated, and the destination drifts further from the source with every schema change. Either build the MERGE dynamically from the staging schema, or add a schema comparison step that catches drift before the MERGE executes.

4.3.7 Staging Deduplication

The extraction batch can contain duplicates: the overlap buffer from 3.2 – Cursor-Based Timestamp Extraction, the dual cursor overlap from 3.10 – Create vs Update Separation, or simply a source that returns the same row twice within the extraction window.

If the staging table contains two rows with the same merge key, the behavior is engine-dependent:

- **BigQuery** raises a runtime error: "UPDATE/MERGE must match at most one source row for each target row"
- **Snowflake** processes both rows non-deterministically – one wins, but which one is undefined
- **PostgreSQL** `ON CONFLICT` processes rows in order, so the last one wins – but "in order" depends on the staging query's sort

Deduplicate the staging table before the MERGE to avoid all three problems:

```
-- destination: columnar
-- Keep the latest version of each key in staging
CREATE OR REPLACE TABLE _stg_orders_deduped AS
SELECT *
FROM _stg_orders
QUALIFY ROW_NUMBER() OVER (PARTITION BY order_id ORDER BY updated_at DESC) = 1;
```

Some loaders deduplicate staging automatically when a primary key is defined on the resource. If yours doesn't, or if you're building the pipeline yourself, add this step explicitly.

4.3.8 By Corridor

Transactional to columnar

MERGE is the most expensive DML operation in columnar engines. The cost scales with the number of partitions touched, not the batch size. Minimize partition spread in each batch, consider delete-insert as an alternative, and evaluate whether 4.4 – Append and Materialize (append + dedup view) is cheaper – **it usually is**, because most tables are read far less often than they're loaded, and the dedup scan on read is cheaper than a MERGE on every write.

Transactional to transactional

`INSERT ... ON CONFLICT` is **very** cheap – each row is an index lookup + point write. Cost scales linearly with batch size. The primary key index handles conflict detection efficiently. For large batches (100K+ rows), load into a staging table first and run the `INSERT ... ON CONFLICT ... SELECT FROM staging` as a single statement rather than row-by-row inserts.

4.4 Append and Materialize

One-liner: Append every extraction as new rows. Deduplicate to current state with a view. Run as often as you want – the load cost is near zero.

4.4.1 The MERGE Cost Ceiling

MERGE cost in columnar engines scales per run: every execution reads the destination, matches keys, and rewrites the affected partitions. If a single MERGE costs X, running it 24 times per day costs $24 \times X$ – and the **cost scales with table size** and partition spread – never batch size (see 4.3 – Merge / Upsert). This creates a ceiling on extraction frequency: you can only afford to run as often as the MERGE budget allows.

That ceiling directly limits purity. The less often you extract, the longer the destination drifts from the source between runs. Missed corrections, late-arriving data, and accumulated cursor gaps all widen with the interval. Running more often closes the gap – but MERGE makes running more often expensive.

This pattern removes the per-run cost ceiling by replacing MERGE with a pure append. The load cost drops to near zero regardless of frequency, and the deduplication cost is paid separately in two places:
1. On compaction, run on a schedule you control, to avoid the table growing too fast
2. On read, whenever the consumers query the dedup view.

4.4.2 The Pattern

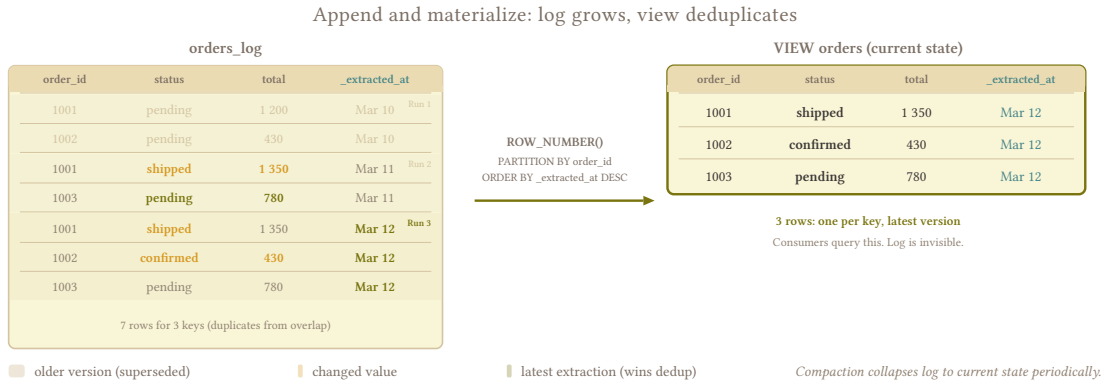
Every extraction run appends its results to a log table with a metadata column (`_extracted_at` or `_batch_id`) that identifies when the row was loaded:

```
-- destination: columnar
INSERT INTO orders_log
SELECT *, CURRENT_TIMESTAMP AS _extracted_at
FROM _stg_orders;
```

A view named `orders` – the same name consumers would use with any other load strategy – deduplicates to the latest version:

```
-- destination: columnar
CREATE OR REPLACE VIEW orders AS
SELECT *
FROM orders_log
QUALIFY ROW_NUMBER() OVER (PARTITION BY order_id ORDER BY _extracted_at DESC) = 1;
```

Consumers query `orders` and see the current state – the view abstracts the log entirely.



4.4.3 Why This Maximizes Purity

The 1.8 – Purity vs. Freshness tradeoff frames purity and freshness as opposing forces – full replace maximizes purity but caps freshness, incremental maximizes freshness but carries purity debt. Append-and-materialize shifts the balance toward both:

Higher frequency = less drift. With near-zero load cost, nothing stops you from extracting every 15 minutes instead of every hour. The shorter the interval between extractions, the smaller the window where the destination can diverge from the source – missed corrections, late-arriving rows, and cursor gaps have less time to accumulate before the next extraction picks them up.

The log is a temporary buffer. The append log holds recent extractions until the next materialization compacts it – a few days or weeks of overlap, scoped to the compaction cycle. Keeping the log short is what makes the pattern affordable: storage stays bounded, and the dedup scan stays fast.

The dedup view absorbs duplicates by design. Regardless of how many redundant copies sit in the log from overlapping windows, pipeline retries, or stateless extractions, `ROW_NUMBER() OVER (PARTITION BY order_id ORDER BY _extracted_at DESC) = 1` always returns the latest version. Duplicates cost storage until the next compaction, but they never corrupt the current state.

4.4.4 The Duplicate Reality

With a cursor-based extraction (3.2 – Cursor-Based Timestamp Extraction), most of the batch is genuinely new or changed rows, and duplicates come from the overlap buffer – a small fraction of each run.

With a stateless window (3.3 – Stateless Window Extraction), the situation inverts. A 7-day window re-extracts 7 days of data on every run, so if the pipeline runs daily, ~6/7ths of each batch is rows the destination already has from previous runs – deliberate duplicates built into the extraction window. The append log grows proportionally to window size × run frequency.

> **Size retention to the extraction window**
>
> A daily run with a 7-day window appends 7× the window volume to the log per week; after a month without compaction, the log holds ~30× the window volume. If the window is a small slice of the table (say, 7 days of changes on a 5-year table), the log overhead is modest. If the window is large relative to the table – say, all open invoices at 40% of the table – the log grows fast. The dedup view handles all of it correctly (the latest `_extracted_at` always wins), but the `ROW_NUMBER()` scan gets heavier with every run. The compaction cycle (covered below) keeps both storage and read cost under control.

4.4.5 The Cost Shift

The cost of reconciling source and destination shifts from load time to read time and storage:

	MERGE (4.3 – Merge / Upsert)	Append and Materialize
Load cost	Scales with table size and partition spread, per run	Near zero – pure INSERT, per run
Query overhead	None – destination is already reconciled at load time	Dedup scan on every query against the view
Materialization cost	N/A	Full dedup scan, but on your schedule
Storage	1× source volume	~1× source volume after compaction + window size × runs until compaction

The shift is favorable when extraction frequency matters more than read frequency. If you load 24 times per day but consumers query the current state 4 times per day, paying for 4 dedup scans is cheaper than paying for 24 MERGEs. It's unfavorable when many consumers query `orders` constantly – the dedup scan runs on every query, and the cost could exceed what the MERGE would have been.

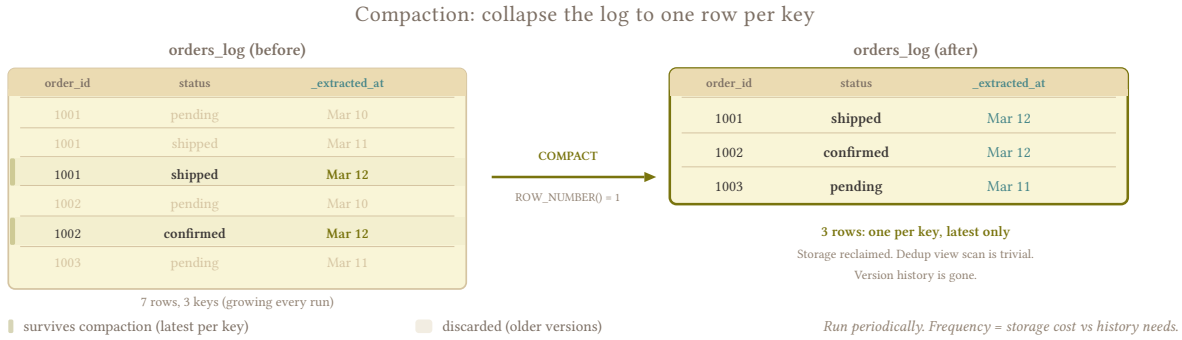
It's usually the case that you want data freshness more frequently than consumption, since most business customers want "New data" whenever they ask for it, but aren't constantly consuming it – more on-demand than live.

Compaction (below) is the lever that controls the read-side cost: compact the log regularly and the view's dedup scan stays fast, regardless of extraction frequency.

4.4.6 Compaction

The dedup view runs `ROW_NUMBER()` against the full log on every query. Without compaction, the log grows with every run – a daily pipeline with a 7-day stateless window adds 7× the window volume per week, and the view's scan grows proportionally. Compaction collapses the log to one row per key, run as a periodic scheduled job:

```
-- destination: columnar
-- Compact to latest-only: one row per key, all extraction history gone
CREATE OR REPLACE TABLE orders_log AS
SELECT *
FROM orders_log
QUALIFY ROW_NUMBER() OVER (PARTITION BY order_id ORDER BY _extracted_at DESC) = 1;
```



Compaction replaces the log with the deduplicated result – every key retains its latest version, all duplicate extractions and historical versions are gone. Storage reclaims completely and the view's `ROW_NUMBER()` scan drops to near-trivial size. Compaction frequency determines how large the log gets between runs and how heavy the dedup scan is at peak, not how stale the view is – the view always reflects the latest version of every row in the log.

The tradeoff is that version history disappears after each compaction. If consumers need point-in-time reconstruction from the log, compaction must run less frequently than their lookback window – or not at all. See 7.6 – Point-in-Time from Events for strategies that preserve history.

> **Partition the log by business date**
>
> After a compact-to-latest, the log holds one row per key – partition it by a business date (`order_date`) so the view's scan benefits from partition pruning on the dimension consumers actually filter on. Before compaction, partitioning by `_extracted_at` is tempting but doesn't help the dedup view.

4.4.7 Historicizing Non-Historical Data

A less common use case, but valuable when it comes up. Most mutable tables in a transactional source overwrite in place without keeping version history – the previous state is gone the moment the row is updated. If someone later asks "what was the product price on March 5?" and you loaded with full replace or MERGE, there's nothing to reconstruct from.

With append-and-materialize, the log already contains the answer. Each extraction captures the state of changed rows at that moment, and historical queries are a `WHERE _extracted_at <= target_date` filter over the log. The version history is a side effect of the load strategy, not an additional mechanism.

This works without changing anything about the load – the mechanism is the same append + dedup view. The only change is the compaction policy: instead of collapsing to latest-only, you either skip compaction entirely (expensive on storage) or use tiered retention to keep recent history at full granularity and compress older history.

4.4.7.1 Tiered Retention

Keeping every extraction indefinitely is a storage problem. Tiered retention sits in between full compaction and no compaction: daily granularity for the recent window, monthly snapshots for older data.

I had a client requesting daily `inventory` snapshots for stock-level analysis across warehouses. After three months the log was large and growing linearly. They realized they only needed daily granularity for the last 60-90 days – further back, a single snapshot per month was enough for seasonal trends and year-over-year comparisons.

```
-- destination: columnar
-- engine: bigquery
-- Monthly job: daily for last 60 days, compress older to monthly
CREATE OR REPLACE TABLE inventory_log AS

-- Recent: keep every daily extraction
SELECT * FROM inventory_log
WHERE _extracted_at >= DATE_SUB(CURRENT_DATE, INTERVAL 60 DAY)

UNION ALL

-- Older: last extraction per PK per month
(SELECT *
FROM inventory_log
WHERE _extracted_at < DATE_SUB(CURRENT_DATE, INTERVAL 60 DAY)
QUALIFY ROW_NUMBER() OVER (
  PARTITION BY sku_id, warehouse_id, DATE_TRUNC(_extracted_at, MONTH)
  ORDER BY _extracted_at DESC
) = 1);
```

The result is two tiers in a single table: daily extractions for the recent window (operational analysis), monthly for older data (trend analysis). The dedup view works identically – `MAX(_extracted_at)` per PK still returns the latest – and downstream queries that filter by date range naturally hit the appropriate granularity.

Match compression boundary to actual needs

What's the shortest period where daily granularity changes a decision? If nobody looks at daily stock levels older than 30 days, compress at 30. If finance needs daily for quarter-close reconciliation, compress at 90. Ask the consumer before picking the number – they usually need less daily granularity than they think.

See 7.6 – Point-in-Time from Events for the full treatment of point-in-time reconstruction from append logs, event tables, and SCD2.

4.4.8 By Corridor

Transactional to columnar

This is the primary corridor for this pattern – columnar engines are where MERGE is expensive and append is cheap. Partition `orders_log` by `_extracted_at` (date) for retention management and cluster by `order_id` for dedup performance. BigQuery storage at ~$0.02/GB/month means the log overhead is affordable when the window is a small fraction of the table, but monitor growth – a large window on a large table accumulates fast.

Transactional to transactional

Less common here because `INSERT ... ON CONFLICT` is already cheap on transactional engines – the MERGE cost ceiling that motivates this pattern doesn't exist. Use append-and-materialize on PostgreSQL when the auditing use case justifies the overhead.

4.5 Hybrid Append-Merge

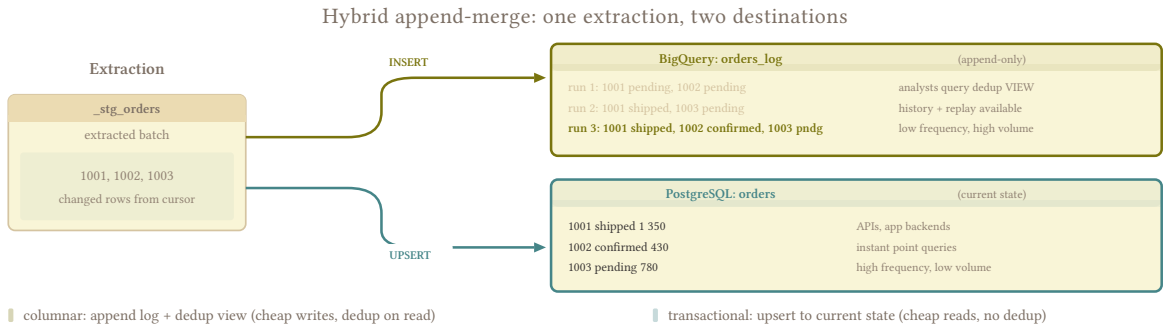
One-liner: Extract once, load to two engines: append-only log in columnar, current-state table in transactional. The complexity ceiling of this book.

4.4 – Append and Materialize gives you cheap appends and a full extraction log, but every read pays a `ROW_NUMBER()` dedup scan – fine for analytical queries that run a few times a day, painful for an API that hits the table hundreds of times per minute. 4.3 – Merge / Upsert gives you a clean current-state table with zero read overhead, but MERGE in columnar engines is expensive per run, which caps extraction frequency. When you have consumers on both sides – analysts who want history and operational systems that need low-latency point queries – neither pattern alone covers both.

The solution is to extract once and load the same batch to two destinations, each playing to its strength:

1. **Columnar** (e.g. BigQuery): append-only log table via pure INSERT. History lives here, and the dedup view from 4.4 – Append and Materialize gives analysts current state when they need it.

2. **Transactional** (e.g. PostgreSQL): current-state table via `INSERT ... ON CONFLICT UPDATE`. Instant point queries for APIs, app backends, and services that validate state before acting.



4.5.1 Why Two Engines

On a single columnar engine, adding a current-state table means running a MERGE alongside the append – strictly worse than choosing one or the other. On a single transactional engine, the append log doesn't give you anything that `INSERT ... ON CONFLICT` doesn't already handle cheaply.

The pattern earns its complexity only when each destination plays to a different engine's strength. If you don't have two engines in your architecture, use 4.4 – Append and Materialize for columnar or 4.3 – Merge / Upsert for transactional and stop there.

4.5.2 Orchestration

The two writes must be treated as a single pipeline unit. If the append to columnar succeeds but the upsert to transactional fails, consumers see different versions of the truth depending on which engine they query.

Idempotency on both sides. The append side is naturally idempotent if combined with the dedup view – duplicate rows in the log don't corrupt the current state. The upsert side is idempotent by design (`INSERT ... ON CONFLICT UPDATE` with the same data produces the same result). A retry of the full pipeline unit is safe as long as both writes use the same batch.

Failure handling. If either write fails, retry the full unit – not just the failed half. Retrying only the failed side risks the two destinations drifting apart on `_extracted_at` if the batch is regenerated between retries.

> **Earn this complexity per table**
>
> This is the most operationally complex load strategy in the book: two destinations, two failure modes, two retention policies, two schema-evolution policies, and the orchestrator treats the pair as a unit. Don't apply it as a default. Most tables don't have both analytical and operational consumers. Promote individual tables to this pattern only when a real consumer can't be served by 4.4 – Append and Materialize or 4.3 – Merge / Upsert alone.

4.6 Reliable Loads

> **One-liner:** Checkpointing, partial failure recovery, idempotent loads. How to make the load step survive failures without losing or duplicating data.

4.6.1 Failure Residue

A pipeline can die after extraction but before load, mid-load with half the batch written, or after load but before the cursor advances. Each failure point leaves different residue – a dangling staging table, a partially written partition, a cursor pointing to data the destination never received. The extraction strategy determines what you pulled; this pattern determines whether the destination survives it.

Full replace (4.1 – Full Replace Load) sidesteps most of this: every run overwrites everything, so there's no residue from a prior failure to clean up. The load is idempotent by construction. The patterns below matter when you're running incremental loads – 4.3 – Merge / Upsert, 4.4 – Append and Materialize, or 4.5 – Hybrid Append-Merge – where the destination accumulates state across runs and a bad load can corrupt that state permanently.

4.6.2 Idempotency at the Load Step

A load is idempotent if running it twice with the same batch leaves the destination unchanged. This is the single most important property for reliability – retries are always safe, and the orchestrator doesn't need to know whether the previous attempt succeeded, partially succeeded, or crashed mid-flight.

MERGE/upsert (4.3 – Merge / Upsert) is naturally idempotent: `INSERT ... ON CONFLICT UPDATE` with the same data produces the same result regardless of how many times it runs. The key match absorbs duplicates, and the update overwrites with identical values.

Append (4.4 – Append and Materialize) is idempotent at the view level but not at the table level. A retry appends the same rows again, doubling them in the log – but the `ROW_NUMBER()` dedup view still returns the correct current state because it picks the latest `_extracted_at` per key. Storage cost goes up, correctness doesn't break. Compaction cleans up the duplicates later.

Full replace (4.1 – Full Replace Load) is idempotent by definition: the destination is rebuilt from scratch on every run, so no prior state can interfere.

> **Test idempotency by running twice**
>
> The simplest validation: run a load, record the destination state, run the exact same load again, compare. If anything changed, the load isn't idempotent and you need to understand why before going to production.

4.6.3 Statelessness

A pipeline that can run on a fresh machine with no local state is valuable – especially when the orchestrator dies at 2am and you're debugging from a laptop: no local files, no SQLite checkpoint databases, no environment variables from a wrapper script; just clone, set credentials, run.

Two things break statelessness:

Local cursor files. If the high-water mark lives in a local file or an in-memory store, a new machine doesn't know where the last successful run left off. Store the cursor in the destination itself (query `MAX(_extracted_at)` from the target table) or in an external state store that survives machine replacement – see 3.2 – Cursor-Based Timestamp Extraction for the tradeoffs.

Local staging artifacts. Some pipelines extract to local disk (Parquet files, CSV dumps) before loading. If the machine dies between extraction and load, the artifacts are gone and the cursor may have already advanced past the data they contained. Either re-extract on retry (stateless window via 3.3 – Stateless Window Extraction handles this naturally) or stage to durable storage (S3, GCS) before advancing any cursor.

> **"Works on my machine" is not stateless**
>
> If the pipeline depends on a prior run having populated a temp directory, a local SQLite checkpoint database, or an environment variable set by a wrapper script, it will fail on a fresh machine. The test is simple: clone the repo, set credentials, run. If it doesn't work, it's not stateless.

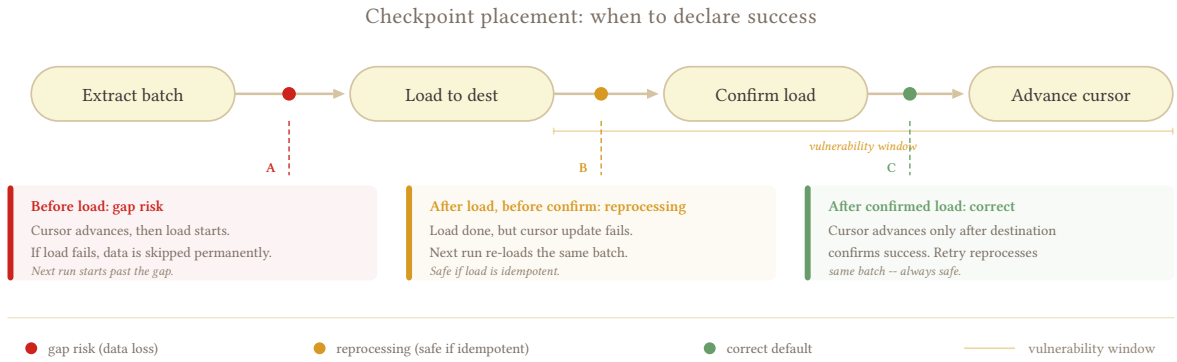
4.6.4 Checkpoint Placement

The checkpoint is when you declare success – advance the cursor, mark a partition materialized. Where you place it determines what breaks when something fails:

Before load (gap risk). The cursor advances, then the load starts. If the load fails, the cursor points past data that was never loaded. The next run starts from the new cursor position and skips the failed batch entirely – unless the extraction uses a lookback window or overlap buffer (see 3.3 – Stateless Window Extraction) that covers the gap. Even with lookback, this placement relies on the safety net catching every failure, which is the wrong default.

After load, before confirmation (reprocessing risk). The load completes, but the cursor update fails (network error, orchestrator crash). The next run re-extracts and re-loads the same batch. With an idempotent load strategy (MERGE or append + dedup view), this is harmless – the data lands twice but the destination state is correct. With a non-idempotent load (raw INSERT without dedup), you get duplicates.

After confirmed load (correct). The cursor advances only after the destination confirms the load succeeded – a successful MERGE, a confirmed partition swap, a row count check on the target. This is the safe default: failures before confirmation mean the next run reprocesses the same batch, which is safe if the load is idempotent.



The gap between "load completes" and "cursor advances" is the vulnerability window. Keep it as small as possible – ideally a single transaction that writes the data and updates the cursor atomically. When that's not possible (columnar engines don't support cross-table transactions), make the load idempotent so the reprocessing path is always safe.

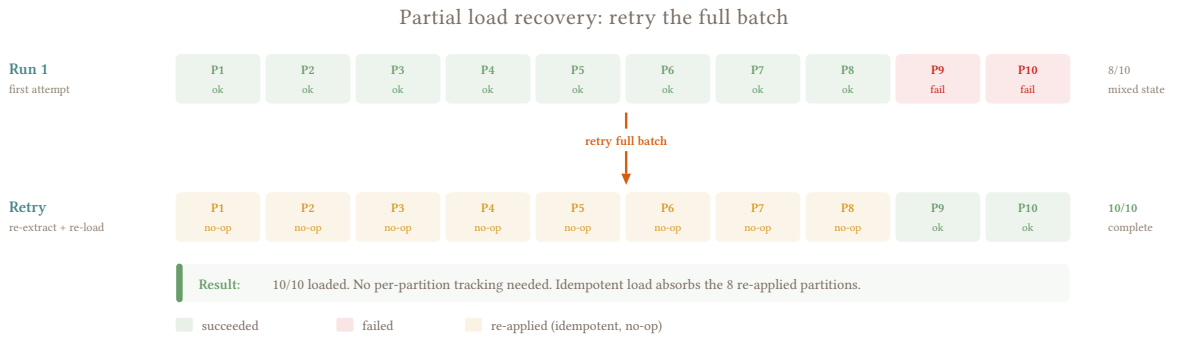
4.6.5 Partial Load Recovery

Not every failure is total. A batch of 10 partitions where 8 succeed and 2 fail leaves the destination in a mixed state: some data is current, some is stale or missing.

MERGE/upsert recovers naturally. Re-running the full batch re-applies all 10 partitions; the 8 that already succeeded are overwritten with identical data (idempotent), and the 2 that failed are applied for the first time. No special handling needed, no data loss.

Append without compaction needs care. Re-running the full batch appends all 10 partitions again, including the 8 that already landed. The dedup view handles it correctly (latest `_extracted_at` wins), but the log now has duplicate copies of the successful partitions. Not a correctness issue, but it inflates storage and slows the dedup scan until the next compaction.

Full replace is immune. The entire destination is rebuilt, so partial state from a prior failure is overwritten completely.



Retry the full batch

Unless the source can't handle it, retrying only the 2 failed partitions introduces complexity: the orchestrator needs to track per-partition success/failure, and the retry batch has a different shape than the original. Reserve per-partition retry for sources that can't afford a second full extraction – an overloaded transactional database, a rate-limited API, or a query that takes hours to run. If the source can give you the full batch again without pain, re-extract and re-load everything; with an idempotent load strategy, the cost of re-applying the successful partitions is just wasted compute, not a correctness risk.

4.6.6 Orchestrator Integration

Orchestrator retries and backfills interact with cursor state in ways that will surprise you.

Automatic retries. Most orchestrators can retry a failed run automatically. If the load is idempotent, automatic retries are safe and you should enable them. If the load is not idempotent (raw INSERT), automatic retries create duplicates – disable them or fix the load strategy first.

Backfills. A backfill replays a date range or partition range, typically to repair corrupted data or to onboard a new table. The backfill should not advance the production cursor – it's filling in historical data, not moving the pipeline forward. Partition-based orchestrators handle this naturally (each partition has its own materialization status). Cursor-based pipelines need a separate code path that loads the backfill range without touching the high-water mark.

Concurrent runs. If the orchestrator allows overlapping runs (a retry starts before the previous attempt finishes), the two runs can race on cursor advancement or produce interleaved writes. Either enforce mutual exclusion (one run at a time per table) or ensure the load is idempotent and the cursor advancement is atomic.

4.6.7 Health Monitoring

A pipeline that fails silently is worse than one that fails loudly. Your orchestrator can tell you when a run fails – but if the orchestrator itself dies, or a run hangs forever, or a run "succeeds" with 0 rows, nobody gets paged. You find out Monday morning when someone asks why the dashboard is stale.

Monitor from outside the pipeline. The destination should be observable independently of the orchestrator. A scheduled query that checks `MAX(_extracted_at)` against the current time and alerts when it exceeds a threshold works regardless of whether the orchestrator is alive. If the orchestrator dies at 2am and the pipeline doesn't run, the freshness check fires at 8am and somebody knows.

Distinguish "0 rows extracted" from "extraction failed." A successful run that returns 0 rows is normal for some tables (no changes since last run, empty table) and a red flag for others (a table that always has activity). 6.10 – Extraction Status Gates covers this in detail – gate the load on extraction status so a silent failure doesn't advance the cursor past a real gap.

Push, then alert on absence. After each successful load, push a heartbeat (a row in a monitoring table, a metric to your observability stack, a timestamp in a health-check endpoint). Alert when the heartbeat stops arriving. This catches every failure mode: orchestrator crash, hung run, infrastructure outage, credential expiration – anything that prevents the pipeline from completing.

4.6.8 By Corridor

Transactional to columnar

The cursor-and-confirmation gap is wider here because columnar engines don't support cross-table transactions – you can't atomically write data and advance a cursor in a single commit. Rely on idempotent loads (MERGE or append + dedup view) so the reprocessing path after a gap is always safe. External freshness monitoring is especially important because columnar loads are often async (BigQuery load jobs, Snowflake COPY INTO) and can fail silently.

Transactional to transactional

Transactional engines allow atomic cursor advancement: write the data and update the cursor in the same transaction, making the confirmation gap effectively zero. This is the simplest path to reliable loads – if you can use it, do. Partial load recovery is also simpler because you can wrap the entire batch in a single transaction and roll back on failure.

5.1 Metadata Column Injection

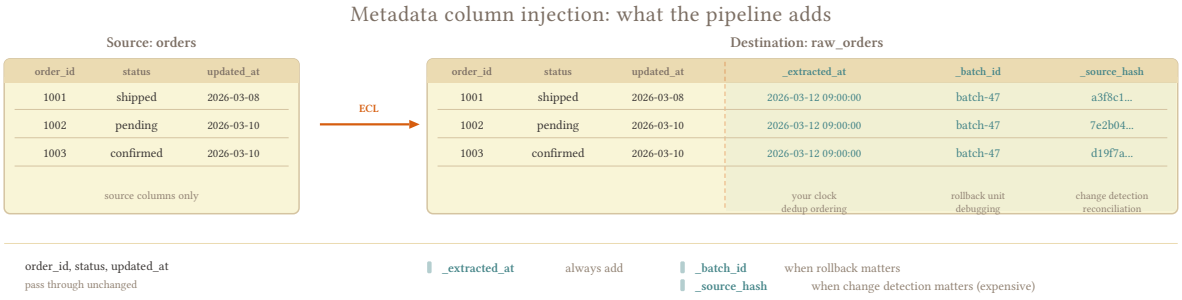
> **One-liner:** `_extracted_at`, `_batch_id`, `_source_hash` – columns the source doesn't have that your pipeline needs for debugging, dedup, and reconciliation.

5.1.1 The Playbook

Metadata columns are just new columns added to the extraction query. Every destination supports them – columnar or transactional, doesn't matter. You're not changing what the data means; you're tagging each row with information about how and when it arrived.

Three metadata columns, each with a different purpose and a different cost/benefit ratio. Not every table needs all three.

```sql
-- source: transactional
SELECT
    order_id,
    customer_id,
    status,
    total,
    updated_at,
    -- metadata columns
    CURRENT_TIMESTAMP                          AS _extracted_at,
    :batch_id                                  AS _batch_id,
    MD5(CONCAT(order_id, '|', status, '|', total)) AS _source_hash
FROM orders
WHERE updated_at >= :last_run;
```



5.1.2 `_extracted_at`

The pipeline's timestamp: when your extraction ran, not when the source row was last modified. A row updated 3 days ago and extracted today has `_extracted_at = today`. This distinction matters because `updated_at` is the source's clock – maintained by the application layer, subject to all the reliability problems covered in 3.1 – Timestamp Extraction Foundations – while `_extracted_at` is your clock, set by your pipeline, and always correct.

Always add this. The cost is trivial (`CURRENT_TIMESTAMP` in the SELECT) and the debugging value is enormous. When something goes wrong – and it will – `_extracted_at` is how you answer "when did this bad data arrive?" and "which extraction run brought it?"

`_extracted_at` is also the foundation for dedup ordering in 4.4 – Append and Materialize. The `ROW_NUMBER() OVER (PARTITION BY order_id ORDER BY _extracted_at DESC) = 1` view depends entirely on this column to determine which version of a row is the latest. Without it, the dedup view has no ordering key and the pattern doesn't work.

> **Share one timestamp per run**
>
> If you extract `orders` and `order_lines` in the same pipeline run, they should share the same `_extracted_at` value. This makes it easy to identify which rows were extracted together and simplifies cross-table debugging. Set the timestamp once at the start of the run and pass it to every extraction query.

5.1.3 `_batch_id`

Correlates all rows from the same extraction run. Where `_extracted_at` tells you *when*, `_batch_id` tells you *which run* – and that distinction matters when you have multiple runs with the same timestamp (retries, overlapping schedules) or when you need to operate on an entire batch at once.

Three use cases earn the column:

Rollback. "Batch 47 loaded bad data. Delete everything from batch 47." With `_batch_id`, that's a single `DELETE WHERE _batch_id = 47`. Without it, you're reverse-engineering which rows came from that run using timestamp ranges and hoping you don't catch rows from adjacent runs.

Debugging. "The destination has 11,998 rows but the source had 12,000. Which batch lost them?" With `_batch_id`, you can trace each row to the run that loaded it and compare batch-level counts against source-side logs.

Reconciliation. A `_batches` table that tracks batch-level metadata – source row count, extraction start/end time, status – gives you an audit trail for every extraction. When 6.14 – Reconciliation Patterns compares source and destination counts, `_batch_id` is the join key.

UUID or sequential integer – consistency matters more than format. If your orchestrator already generates run IDs, reuse those.

5.1.3.1 The `_batches` Table

A lightweight metadata table on the destination that tracks each extraction run:

```
-- destination: any
CREATE TABLE _batches (
    batch_id        TEXT PRIMARY KEY,
    table_name      TEXT NOT NULL,
    extracted_at    TIMESTAMP NOT NULL,
    source_row_count INTEGER,
    dest_row_count  INTEGER,
    status          TEXT DEFAULT 'running',  -- running, completed, failed
    started_at      TIMESTAMP NOT NULL,
    completed_at    TIMESTAMP
);
```

Your pipeline writes a row to `_batches` at the start of each run (`status = 'running'`), updates it after load completes (`status = 'completed'`, `dest_row_count` filled in), and marks it failed on error. This gives you a single place to answer "when did each table last load successfully?" and "which tables are currently loading?" – questions that become surprisingly hard to answer without it.

5.1.4 `_source_hash`

A hash of the source row at extraction time. Enables 2.8 – Hash-Based Change Detection (compare hashes between runs to detect changes without relying on `updated_at`) and post-load reconciliation (compare source-side hash vs destination-side hash to verify the row arrived intact).

```
-- source: transactional
-- Hash all business columns, excluding _extracted_at and _batch_id
SELECT
    *,
    MD5(CONCAT(
        COALESCE(order_id::TEXT, '__NULL__'), '|',
        COALESCE(status, '__NULL__'), '|',
        COALESCE(total::TEXT, '__NULL__')
    )) AS _source_hash
FROM orders;
```

Expensive at scale. Hashing every row adds compute on the source or in the pipeline. At ~800 tables, this can add 20 minutes to an already tight extraction window. The cost is proportional to row count × column count – wide tables with millions of rows are where it hurts.

Tiered approach. Not every table earns the overhead. High-value mutable tables (`invoices`, `orders`) where change detection matters and `updated_at` is unreliable – those earn `_source_hash`. Stable config tables that change once a quarter, append-only tables like `events` where you never need to detect mutations – skip it.

NULL handling. COALESCE every column to a sentinel before hashing. Most hash functions return NULL if any input is NULL, which means a row with a single NULL column produces a NULL hash – indistinguishable from every other row with a NULL in the same position. `COALESCE(col, '__NULL__')` before concatenation prevents this.

Hash business columns, not metadata

Exclude `_extracted_at` and `_batch_id` from the hash input. These change every run by design – including them means the hash changes every run too, defeating the purpose of change detection.

5.1.5 Where to Inject

Every metadata column runs *somewhere* – in the source query, in Python between extraction and load, or in a staging transform on the destination. The choice depends on what the source can handle and how much compute you're willing to add to the extraction.

Source query. Cheapest if the source can handle it. `CURRENT_TIMESTAMP` is free on every engine. `MD5()` is available on PostgreSQL, MySQL, SQL Server, and SAP HANA with slightly different syntax. The syntactic work happens in the same query that extracts the data – no extra hop, no extra infrastructure.

Orchestrator / middleware. Python adds the columns after extraction, before load. More control (you can use a consistent hashing library across all sources regardless of engine), but you're adding an extra data hop and holding the full batch in memory or on disk while you process it.

Staging. Land the raw data without metadata, then add the columns in a staging transform on the destination. Works well when you want to keep the extraction query minimal and offload all syntactic work to the destination's compute. Common in BigQuery workflows where staging + transform is the standard pattern.

For `_extracted_at` and `_batch_id`, the source query is almost always the right place – the cost is negligible. For `_source_hash`, the source query or Python are both reasonable depending on whether your source engine has a convenient hash function and whether the compute cost on the source is acceptable.

5.1.6 By Corridor

Transactional to columnar

No special considerations. Columnar destinations accept new columns without issue. If you're using 4.4 – Append and Materialize, `_extracted_at` is the dedup ordering key – make sure it's populated on every row.

Transactional to transactional

Same approach. One advantage: if you need to add metadata columns to an existing destination table retroactively, `ALTER TABLE ADD COLUMN` is cheap and instant on most transactional engines. On columnar engines it's also cheap, but backfilling the column for historical rows is more expensive.

5.2 Synthetic Keys

> **One-liner:** No PK, composite PK, unstable PK – when the source doesn't give you a reliable key for MERGE, build one from immutable business characteristics.

5.2.1 When You Need One

A MERGE needs a key to match source rows against destination rows. If the source gives you a clean, stable, single-column PK, use it and move on. The problems start when the source doesn't cooperate:

No PK at all. Some tables genuinely have no primary key – log tables, staging tables, tables where the DBA forgot. Without a key, MERGE has nothing to match on and you're stuck with full replace or append-only.

Composite PK of 5+ columns. The table technically has a key, but it's `(company_code, fiscal_year, document_type, document_number, line_number)`. A MERGE matching on 5 columns works but is unwieldy, slow on some engines, and painful to debug when something doesn't match.

Recycled auto-increment. The PK is an `id` that looks stable, but when rows are deleted the ids get reused. Row 42 today is a different entity than row 42 last week. Your MERGE overwrites the new row with the old row's data because the key matches on the wrong entity.

NULLs in key columns. `NULL != NULL` in every SQL engine. A MERGE with NULL in the key column silently misses the row on every run – the match condition never evaluates to true, so the row gets re-inserted as a "new" row every time. You end up with duplicates that accumulate run after run.

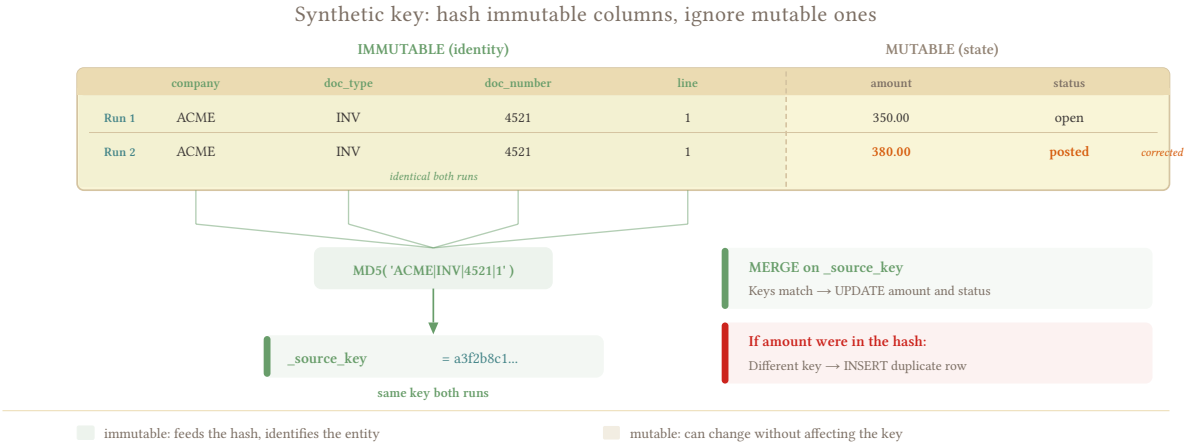
5.2.2 Building the Key

The principle: hash immutable business characteristics – attributes that identify the entity and won't change over its lifetime. The hash becomes `_source_key`, a single column the MERGE matches on.

What's immutable. Natural business identifiers: order number + line number, customer code + document type, SKU + warehouse code. These describe *what* the entity is, not *what happened* to it. If someone corrects the entity later, these columns stay the same.

What's not immutable. Amounts, prices, dates, status fields – anything that can be corrected or updated after the row is created. If you include `total` in the hash and someone corrects an invoice amount, the hash changes and the MERGE treats the corrected row as a new entity instead of an update to the existing one.

```sql
-- source: transactional
-- Build a synthetic key from immutable business identifiers
SELECT
    MD5(CONCAT(
        COALESCE(company_code, '__NULL__'), '|',
        COALESCE(document_type, '__NULL__'), '|',
        COALESCE(document_number::TEXT, '__NULL__'), '|',
        COALESCE(line_number::TEXT, '__NULL__')
    )) AS _source_key,
    *
FROM invoice_lines;
```



5.2.2.1 Concatenation vs Hash

Two approaches, with a clear winner for most cases:

Concatenation. `CONCAT(order_id, '-', line_number)` → `'1001-3'`. Readable and debuggable – you can look at the key and know which entity it represents. The problem is fragility: if the delimiter (`-`) appears in the data, `CONCAT('100', '-', '13')` and `CONCAT('1001', '-', '3')` are distinguishable, but `CONCAT('10-0', '-', '13')` isn't what you expected. Column ordering matters too – reversing the concat order produces different keys for the same entity across pipeline versions.

Hash. `MD5(CONCAT(col1, '|', col2, '|', col3))` → `'a3f2b8c1...'`. Opaque but safe. The output is always the same length, the delimiter problem effectively disappears (collisions from delimiter confusion are astronomically unlikely in the hash space), and column ordering is consistent as long as the pipeline code doesn't change. The downside is that you can't read the key and know which entity it represents – debugging requires recomputing the hash from the source columns.

For most pipelines, hash wins. You look at the key rarely; the pipeline matches on it constantly.

5.2.3 NULL in Key Columns

Most hash functions return NULL if any input is NULL. But whether NULL reaches the hash depends on the engine's `CONCAT` behavior: BigQuery, Snowflake, and MySQL propagate NULL through `CONCAT` (so `MD5(CONCAT('ACME', '|', NULL))` → NULL), while PostgreSQL and SQL Server silently treat NULL as empty string in `CONCAT()` (so `MD5(CONCAT('ACME', '|', NULL))` → a valid hash of `'ACME|'`). The PostgreSQL/SQL Server behavior is arguably worse: two rows with different NULL columns produce the same hash instead of failing loudly, and the MERGE silently collapses them into one entity.

COALESCE every column to a sentinel before hashing:

```
-- source: transactional
MD5(CONCAT(
    COALESCE(company_code, '__NULL__'), '|',
    COALESCE(document_number::TEXT, '__NULL__')
))
```

The sentinel (`'__NULL__'`) must be something that can't appear in real data. `'__NULL__'` works because no business column will contain that literal string. A shorter sentinel like `''` (empty string) is dangerous because empty strings *do* appear in real data and you'd be conflating NULL with empty – the exact problem 5.4 – Null Handling warns against.

Document which columns participate in the key. Downstream consumers, reconciliation queries, and future pipeline maintainers need to know how `_source_key` is built so they can recompute it when debugging.

5.2.4 Hash Function Choice

MD5. 128-bit output, fast on every engine. The standard choice for synthetic keys. This isn't cryptography – you're not protecting against adversarial collisions, you're generating unique identifiers for MERGE matching. MD5's known cryptographic weaknesses are irrelevant here.

SHA-256. 256-bit output, slower. The only reason to use it over MD5 is if regulatory or audit requirements mandate a specific hash function. Some compliance frameworks specify SHA-256 for anything labeled "hash" regardless of context – easier to comply than to argue.

5.2.4.1 When 128 Bits Isn't Enough

The birthday paradox determines collision probability: with a 128-bit hash, you'd need roughly 2^{64} (~18 quintillion) rows before a 50% chance of any two rows colliding. At 1 billion rows, the probability of a single collision is approximately $\frac{1}{3.4 \times 10^{20}}$ – for practical purposes, zero. You'll hit every other failure mode in your pipeline long before you hit an MD5 collision on a synthetic key.

If you're hashing across multiple tables and worry about cross-table collisions, include the table name in the hash input: `MD5(CONCAT('invoice_lines', '|', col1, '|', col2))`. This is more about hygiene than probability.

5.2.5 By Corridor

> **Transactional to columnar**
>
> Columnar destinations don't enforce UNIQUE, so a bad synthetic key doesn't produce an error – it produces silent duplicates. A key built from mutable columns, or a key that doesn't COALESCE NULLs, will generate multiple rows for the same entity with no warning from the engine. The dedup is entirely your responsibility, and the only way to catch it is to monitor for duplicate `_source_key` values after load.

Transactional to transactional

Transactional destinations can enforce UNIQUE on `_source_key`. Add a unique index or constraint on the column, and a collision or a badly constructed key gets rejected at the database level with an explicit error. This is a genuine safety net that columnar destinations can't offer – use it.

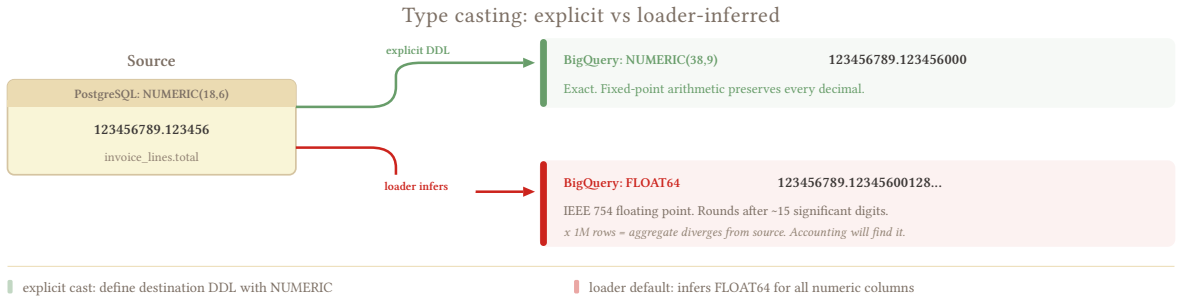
5.3 Type Casting and Normalization

One-liner: Every engine has its own type system and they don't agree on anything. Cast explicitly at extraction or let the loader guess wrong.

5.3.1 The Playbook

Cast explicitly in the extraction query, not at the destination. The source knows its type system better than whatever the loader infers, and implicit casts hide precision loss that you won't notice until someone in accounting finds a discrepancy six months later.

That said, not all precision loss is negative. Very few tables need nanosecond precision – second or microsecond is fine for most business data. The goal is to be deliberate about what you lose and what you keep, not to preserve every bit of precision across every column. A `DATETIME2(7)` truncated to `TIMESTAMP` (microseconds) is almost certainly fine. A `NUMERIC(18,6)` silently cast to `FLOAT64` is almost certainly not.



5.3.2 The Mapping Table

The central reference for type casting across engines. Every source-destination pair has a mapping, and the dangerous ones are the implicit casts that look harmless.

Source type	Source engine	Destination type	Destination engine	Notes
`DATETIME2(7)`	SQL Server	`TIMESTAMP`	BigQuery	Nanosecond → microsecond truncation. Rarely matters
`DATETIME`	MySQL	`TIMESTAMP`	BigQuery	Second precision. Fine for most business data
`NUMERIC(18,6)`	PostgreSQL	`NUMERIC(38,9)`	BigQuery	Safe. Explicit DDL required – loader defaults to `FLOAT64`
`NUMERIC(18,6)`	PostgreSQL	`FLOAT64`	BigQuery	**Dangerous.** Implicit cast loses decimal precision
`MONEY`	SQL Server	`NUMERIC(19,4)`	any	`MONEY` is a fixed-point type with 4 decimal places. Cast to NUMERIC explicitly
`BIT`	SQL Server	`BOOLEAN`	BigQuery	Works if all values are 0/1. See Boolean section below
`TINYINT(1)`	MySQL	`BOOLEAN`	BigQuery	MySQL's pseudo-boolean. May contain values other than 0/1
`NVARCHAR(MAX)`	SQL Server / HANA	`STRING`	BigQuery	No length limit in BigQuery STRING. Safe
`TEXT`	PostgreSQL	`STRING`	BigQuery	PostgreSQL `TEXT` is unlimited. BigQuery `STRING` has a 10MB per-value limit. Rarely hit in practice
`JSONB`	PostgreSQL	`STRING` or `JSON`	BigQuery / Snowflake	See 5.7 – Nested Data and JSON
`TIMESTAMP WITH TIME ZONE`	PostgreSQL	`TIMESTAMP`	BigQuery	BigQuery `TIMESTAMP` is always UTC. See 5.5 – Timezone Handling

Let the mapping table drive DDL

If you define the destination DDL explicitly (rather than letting the loader infer it), the type mapping becomes the source of truth for your schema. New table? Look up each source column in the mapping, generate the DDL. This is mechanical work that belongs in a utility function, not in your head.

5.3.3 Dangerous Implicit Casts

Three categories of implicit cast that loaders get wrong regularly:

5.3.3.1 Numeric Precision

The most financially dangerous cast. `NUMERIC(18,6)` in PostgreSQL is exact – it stores 18 digits with 6 decimal places using fixed-point arithmetic. `FLOAT64` in BigQuery is a 64-bit IEEE 754 floating point that can represent ~15-17 significant digits but rounds everything else.

```
-- What you expect:
SELECT CAST(123456789.123456 AS NUMERIC(18,6));
-- 123456789.123456

-- What FLOAT64 gives you:
SELECT CAST(123456789.123456 AS FLOAT64);
-- 123456789.12345600128173828125
```

Multiply that rounding error by a million invoice lines and the aggregate diverges from the source. Business people are surprisingly tolerant of float precision errors as long as it doesn't affect the big numbers – a `ROUND()` downstream can be risky but worthwhile when you can't avoid `FLOAT64`.

The fix by engine:

Destination	Use instead of FLOAT64
BigQuery	`NUMERIC(38,9)` or `BIGNUMERIC(76,38)`
Snowflake	`NUMBER(18,6)`
ClickHouse	`Decimal(18,6)`
Redshift	`DECIMAL(18,6)`

Explicit type in the destination DDL – never let the loader infer the numeric type. Most loaders default to `FLOAT64` because it's the safest generic choice (it accepts everything), and that's exactly the problem.

Replicating with full precision is worth trying but don't get married to it. Some source engines have types that don't map cleanly to any destination type (SQL Server `MONEY` with implicit currency rounding, Oracle `NUMBER` without scale), and chasing exact precision across every column can burn more time than it's worth for columns where nobody cares about the sixth decimal.

5.3.3.2 Timestamp Precision

`DATETIME2(7)` in SQL Server stores 100-nanosecond precision. `TIMESTAMP` in BigQuery stores microsecond precision. The cast truncates 1 digit of precision, which sounds alarming until you realize that virtually no business process generates or consumes nanosecond-precision timestamps. If your ERP writes `2026-03-15 14:30:00.1234567` and the destination stores `2026-03-15 14:30:00.123456`, nobody will notice – and if they do, the conversation is about why the source has nanosecond precision, not about why the destination doesn't.

MySQL's `DATETIME` defaults to second precision. That's coarser, but it's what the source has – landing it as a higher-precision type at the destination doesn't add information that wasn't there.

5.3.3.3 String Length

PostgreSQL `TEXT` is unlimited. BigQuery `STRING` has a 10MB per-value limit. SQL Server `NVARCHAR(MAX)` can hold 2GB. These limits rarely matter for business data, but they matter for columns that store embedded documents, base64-encoded blobs, or serialized objects. If a value exceeds the destination's limit, the load fails silently or truncates – neither is acceptable. Check the max value length of suspicious columns before committing to a type mapping.

5.3.4 Boolean Representations

The source stores boolean-like values in at least six different ways depending on the engine and the application layer:

Representation	Source engine / system	Notes
`BIT` (0/1)	SQL Server	True boolean. Safe to cast
`TINYINT(1)`	MySQL	Pseudo-boolean. May contain 2, 3, or –1
`BOOLEAN`	PostgreSQL	True boolean
`'Y'` / `'N'`	Various ORMs, SAP B1 (English)	String. Not a boolean at the engine level
`'S'` / `'N'`	SAP B1 (Spanish/Portuguese install)	Same table, different literal depending on install language
`1` / `0` as `INTEGER`	Legacy systems	Integer semantics, boolean intent

The pipeline *can* cast to the destination's native boolean, but only with good justification. "The destination has a `BOOL` type" isn't sufficient reason to reinterpret a `'Y'/'N'` string column – the source stored a string, and reflecting the source faithfully means landing a string. Cast to `BOOLEAN` when the source type is already a boolean (`BIT`, `BOOLEAN`) and the destination has a native equivalent. Leave string representations as strings.

> **Three-state logic**
>
> Some boolean-looking columns actually carry three states: true, false, and unknown (or not applicable). A `BOOLEAN` can't represent this. If the source column has NULL alongside `'Y'/'N'`, and NULL has a distinct business meaning ("not yet evaluated" vs. "evaluated as no"), casting to `BOOLEAN` with COALESCE destroys that distinction. Keep the source type.

5.3.5 Decimal Precision

`NUMERIC(18,6)` in PostgreSQL is exact; `FLOAT64` in BigQuery is not.

Where it hurts: financial data, unit prices, exchange rates. Multiplied by millions of rows, even tiny rounding errors accumulate into visible discrepancies in aggregate reports. An invoice total that's off by $0.00001 per line becomes $10 off on a million-line summary – and accounting will find it.

Where it doesn't hurt: quantities, counts, percentages, scores. If the column is an integer disguised as a decimal (`quantity = 5.000000`), `FLOAT64` is fine. If the column has meaningful decimal places but nobody aggregates it across millions of rows, `FLOAT64` is probably fine. The damage is proportional to row count × aggregation.

The pragmatic approach: explicit `NUMERIC` in the DDL for financial columns, `FLOAT64` for everything else unless proven otherwise. Monitor aggregate differences between source and destination on the critical columns (6.14 – Reconciliation Patterns) and escalate if the divergence exceeds an acceptable threshold.

5.3.6 Engine-Specific Traps

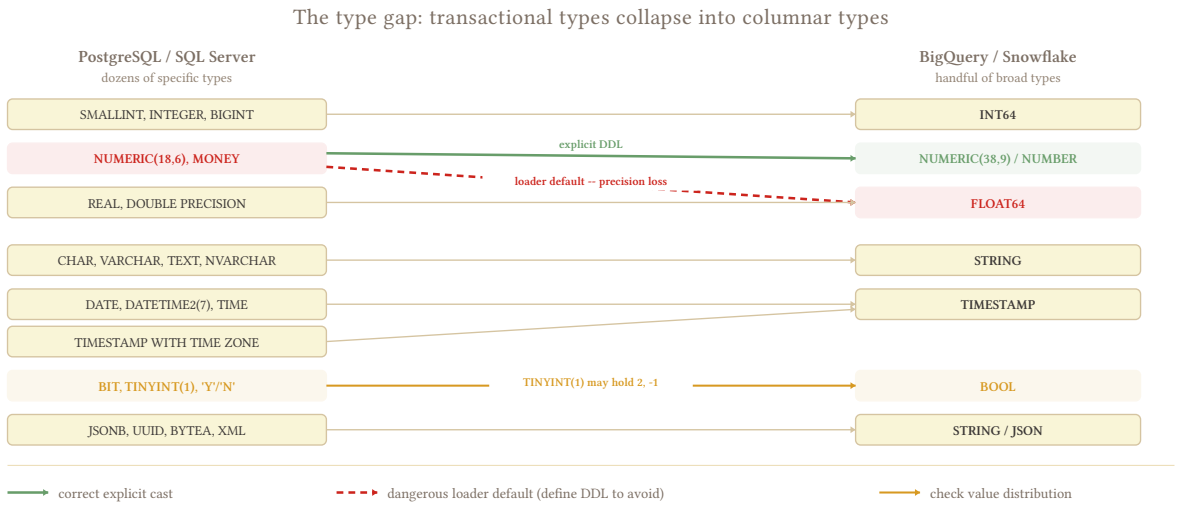
A few combinations that produce surprising behavior:

SQL Server `BIT` vs PostgreSQL `BOOLEAN` vs MySQL `TINYINT(1)`. SQL Server's `BIT` is a true boolean (0 or 1, nothing else). MySQL's `TINYINT(1)` is an integer that the driver *displays* as boolean but happily stores 2, 127, or −1. PostgreSQL's `BOOLEAN` is a true boolean. If you're extracting from MySQL and the column has values outside 0/1, a cast to `BOOLEAN` fails or silently coerces – check the actual value distribution before casting.

SAP HANA `NVARCHAR` vs `VARCHAR`. HANA defaults to `NVARCHAR` (Unicode) for most string columns. When extracting to a destination that distinguishes between Unicode and non-Unicode strings (SQL Server, MySQL), you need to match the encoding or risk truncation on characters outside the ASCII range. When extracting to BigQuery or Snowflake (UTF-8 everywhere), this distinction vanishes.

Schema evolution interaction. A new column appears in the source with a type your cast map doesn't cover. If your extraction uses `SELECT *`, the column arrives with whatever type the loader infers – which might be wrong. If your extraction uses an explicit column list, the column is silently dropped. Both are problems. See 4.3 – Merge / Upsert's schema evolution section for the detect ⟶ decide ⟶ apply workflow.

5.3.7 By Corridor



Transactional to columnar

The widest type gap. Type systems are fundamentally different – transactional engines have dozens of specific types (`MONEY`, `SMALLINT`, `NCHAR(10)`, `DATETIME2(7)`) that columnar engines collapse into a handful (`INT64`, `FLOAT64`, `STRING`, `TIMESTAMP`). Explicit casting is mandatory because the loader's inference maps everything to the broadest compatible type, which is almost always `FLOAT64` for numbers and `STRING` for text. Define your destination DDL explicitly for every table.

Transactional to transactional

Narrower gap, but dialect differences still bite. PostgreSQL `BOOLEAN` vs MySQL `TINYINT(1)`, SQL Server `DATETIME2` vs PostgreSQL `TIMESTAMP`, MySQL `UNSIGNED INT` vs PostgreSQL (no unsigned types). If source and destination run the same engine, the type mapping is nearly 1:1 and explicit casting is rarely needed.

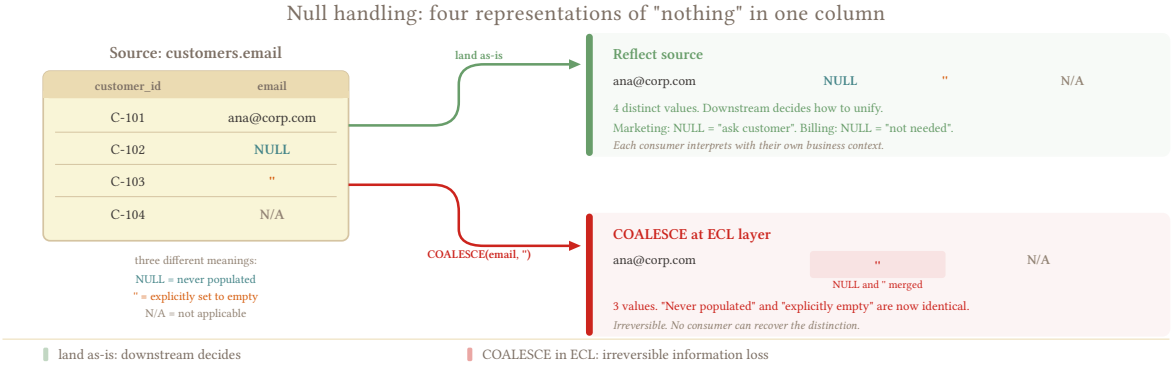
5.4 Null Handling

> **One-liner:** NULL means NULL. Reflect the source as-is – don't COALESCE, don't mix representations, don't solve downstream's problems in the pipeline.

5.4.1 The Playbook

Faithful replication means reflecting the source as accurately as possible. If the source has NULL, land NULL. If the source has empty string, land empty string. If the source has `'N/A'`, land `'N/A'`. These are three different values with potentially three different meanings, and converting one to the other is a business decision – it belongs downstream, not in the syntactic layer.

The temptation to "clean up" NULLs at extraction is strong, especially when you know downstream consumers will struggle with them. Resist it. A COALESCE in the extraction query looks harmless, but it makes an irreversible choice about what NULL means for every consumer of the table, and that choice may be wrong for some of them. A NULL `email` might mean "not provided" for the marketing team and "not applicable" for the billing team – collapsing it to `''` destroys the distinction for both.



5.4.2 The Rule: Don't Mix, Don't Match

Inconsistent representations of "nothing" across tables, columns, or even rows within the same column are the worst outcome – worse than NULLs in the destination.

The source might use any combination of:

Representation	Where you find it
`NULL`	Standard SQL databases, ORMs that use NULL semantics
`''` (empty string)	Legacy systems, some ORMs that default to empty string on form fields
`'N/A'` / `'NONE'` / `'-'`	Manual data entry, ERP display defaults, CSV exports
`0`	Numeric columns where "no value" was entered as zero

If different source tables use different representations, document the inconsistency but don't normalize them to a single representation in the pipeline. Table A uses NULL for "no email" and table B uses `''` for "no email" – that's the source's problem, and it's worth documenting, but converting one to match the other is a semantic decision the downstream team gets to make, because they understand the business context better than the pipeline does.

> **Avoid COALESCE in the syntactic layer**
>
> `COALESCE(email, '')` in the extraction query looks like cleanup. What it actually does: permanently destroys the distinction between "this field was never populated" (NULL) and "this field was explicitly set to empty" (empty string). If that distinction matters to even one consumer, you've lost it for all of them. The only justified COALESCE in the pipeline is in synthetic key hashing (see 5.2 – Synthetic Keys), where NULL would corrupt the hash output – and that's infrastructure, not business logic.

5.4.3 When NULLs Matter at the Pipeline Level

NULLs don't need fixing in the pipeline, but they do need *awareness* – three places where NULL behavior affects the pipeline itself, not downstream consumption:

NULL in synthetic key columns. Most hash functions return NULL if any input is NULL, so a row with a NULL key column produces a NULL `_source_key` and the MERGE can't match it. COALESCE to a sentinel before hashing – see 5.2 – Synthetic Keys. This is the one place where COALESCE is justified because it's protecting pipeline mechanics, not making a business decision.

NULL in cursor columns. A NULL `updated_at` makes the row invisible to incremental extraction – `WHERE updated_at >= :last_run` never evaluates to true for NULL values. This is an extraction problem, not a null handling problem, and 3.10 – Create vs Update Separation covers the strategies.

NULL in partition columns. If you partition `orders` by `order_date` and some rows have `order_date = NULL`, those rows land in a `__NULL__` partition (BigQuery), a default partition (Snowflake), or fail the insert (ClickHouse, depending on config). None of these outcomes are what you want, but the fix belongs in the extraction query (filter or assign a sentinel partition value) – not in a blanket COALESCE policy.

5.4.4 Downstream Consequences

These are real, and you should document them – but they're not your problem to fix in the pipeline.

`GROUP BY` groups NULLs together. The SQL standard treats NULLs as "not distinct" for grouping, and every major engine follows this – BigQuery, Snowflake, PostgreSQL, MySQL, SQL Server, ClickHouse, Redshift. An analyst who writes `GROUP BY status` and gets a NULL group isn't looking at a bug – they're looking at data that has NULLs in the status column, which is what the source has.

`COUNT(column)` vs `COUNT(*)`. `COUNT(*)` counts all rows. `COUNT(status)` excludes rows where `status IS NULL`. Analysts who don't know this will report incorrect counts and blame the data. Document the NULL rate per column if it's significant, but don't COALESCE to inflate the count.

Aggregation with NULLs. `SUM(amount)` ignores NULLs. `AVG(amount)` ignores NULLs in both numerator and denominator. Both of these are correct SQL behavior, but consumers who expect NULLs to be treated as zeros will get different results than they expect. Again: document, don't fix.

> **Surface NULL rates in quality checks**
>
> A table where `email` is 90% NULL is useful information for consumers. Surface it through data contracts or quality checks so downstream teams know what they're working with – but don't change the data to make the numbers look cleaner.

5.4.5 By Corridor

> **Transactional to columnar**
>
> Columnar destinations don't enforce UNIQUE or PK constraints, but they **do** enforce NOT NULL when declared – BigQuery rejects inserting NULLs into REQUIRED columns, Snowflake enforces NOT NULL on standard tables, ClickHouse columns are non-nullable by default (you must explicitly opt into `Nullable()`), and Redshift enforces NOT NULL. If the source has NULLs in a column the destination declared NOT NULL, the load fails. Match the destination DDL to the source's actual nullability.

> **Transactional to transactional**
>
> Transactional destinations *can* enforce NOT NULL. If the destination schema has NOT NULL constraints and the source has NULLs, the load fails – and that's a schema mismatch to resolve by adjusting the destination DDL, not a reason to COALESCE in the extraction. If you genuinely need NOT NULL at the destination (for FK integrity or application requirements), make that a conscious decision and handle the NULLs explicitly in a documented transformation step, not silently in the pipeline.

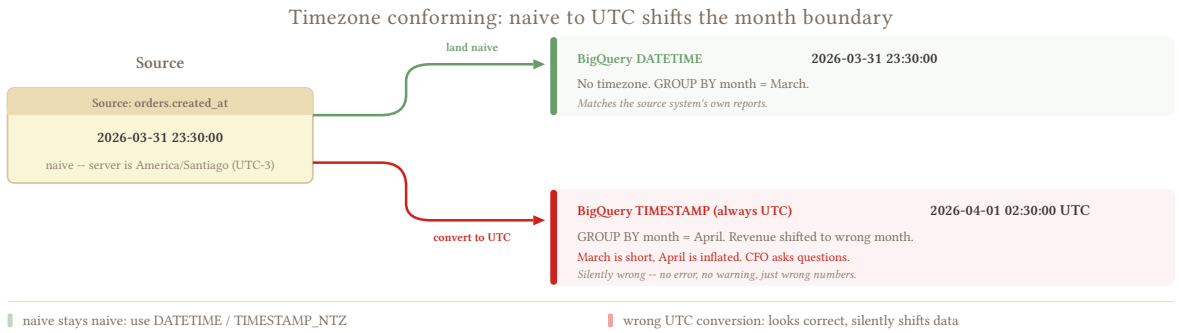
5.5 Timezone Handling

> **One-liner:** TZ stays TZ, naive stays naive. Don't make timezone decisions that aren't in the source data – but know what you're landing.

5.5.1 The Playbook

The rule follows the same principle as 5.4 – Null Handling: reflect the source. If the source stores timezone-aware timestamps, land timezone-aware. If the source stores naive timestamps, land them as datetime – not as timestamp with a timezone you guessed. Converting naive to UTC without being certain of the source timezone is worse than landing naive, because a wrong UTC conversion looks correct in the destination and silently shifts every row by however many hours you got wrong.

Most transactional sources store naive timestamps. The application knows what timezone it means, but the column doesn't say – and often nobody at the source team documented it either. That's the source's data quality problem. Your job is to land what the source gives you, not to retroactively assign timezone semantics that weren't there.



5.5.2 Naive vs Aware

Two fundamentally different types that look similar in query results but behave differently everywhere else:

Naive (`TIMESTAMP WITHOUT TIME ZONE`, `DATETIME`, `DATETIME2`): the value `2026-03-15 14:30:00` with no timezone attached. The source application assumes a timezone – usually the server's local time, sometimes the user's timezone, sometimes something else entirely – but the column itself carries no indication of which one.

Aware (`TIMESTAMP WITH TIME ZONE`, `TIMESTAMPTZ`): the value `2026-03-15 14:30:00+00:00` with an explicit offset or stored as UTC internally. The timezone is part of the data. PostgreSQL's `TIMESTAMPTZ` stores everything as UTC and converts to the session timezone on display; BigQuery's `TIMESTAMP` is always UTC.

The mapping:

Source has	Land as	Why
Aware (TIMESTAMPTZ)	Aware (TIMESTAMP / TIMESTAMPTZ)	The timezone is part of the data – preserve it
Naive (DATETIME)	Datetime / naive equivalent	Don't add timezone info that isn't there
Naive, but you know the timezone with certainty	Convert to aware, document it	Only if the source team confirms and it won't change

> **Not every destination supports naive timestamps**
>
> BigQuery's `TIMESTAMP` is always UTC – there's no naive mode. If you land a naive `14:30:00` as a BigQuery `TIMESTAMP`, the engine treats it as `14:30:00 UTC`, which may be wrong. Use BigQuery `DATETIME` (no timezone) for naive values. Snowflake has both `TIMESTAMP_NTZ` (naive) and `TIMESTAMP_TZ` (aware). Know which one you're targeting.

5.5.3 Discovering the Source Timezone

When you need to know what timezone a naive timestamp represents – for documentation, for downstream, or because you're deciding whether to convert – here's the investigation order:

Ask the source team. "What timezone does your application write timestamps in?" is a 5-minute conversation that saves weeks of debugging. Most teams know the answer even if they never documented it.

Check the database server timezone. `SHOW timezone` (PostgreSQL), `SELECT @@global.time_zone` (MySQL), `SELECT SYSDATETIMEOFFSET()` (SQL Server). Many applications inherit the server's timezone for naive timestamps.

Look at timestamps around DST transitions. If you see a gap at 2:00-3:00 AM on the spring-forward date, the source writes in a timezone that observes DST. If you see two clusters of timestamps at 1:00-2:00 AM on the fall-back date, same conclusion. If timestamps flow smoothly across both transitions, the source writes in UTC or a non-DST timezone.

Multinational ERPs. Each company or branch may write in its own local timezone – same column, different timezone per row, no indicator column. This is genuinely bad source data quality. If the source doesn't provide a timezone indicator per row, avoid assigning timezones in the pipeline. Land naive and let downstream handle it with whatever business context they have about which company operates in which timezone.

5.5.4 DST Traps

Daylight saving transitions create two specific hazards for timestamp data:

Spring forward (the hour that doesn't exist). Clocks jump from 2:00 AM to 3:00 AM. A timestamp like `2026-03-08 02:30:00 America/New_York` refers to a moment that never existed. Some engines reject it, some silently shift it to 3:30 AM, some store it as-is. If the source wrote it, it's probably from a system that doesn't validate timestamps – land it as-is and document that the value is technically invalid.

Fall back (the hour that repeats). Clocks fall from 2:00 AM back to 1:00 AM. A timestamp like `2026-11-01 01:30:00 America/New_York` could refer to two different instants – the first 1:30 AM or the second 1:30 AM. Without an offset, there's no way to distinguish them.

Both of these are reasons to prefer landing naive timestamps as naive rather than converting to UTC at extraction time. A conversion during the ambiguous fall-back hour has a 50% chance of being wrong, and you won't know which rows are affected. A stateless extraction window (3.3 – Stateless Window Extraction) helps here – the overlap naturally re-extracts the ambiguous rows on the next run, and if the source eventually clarifies (some applications write a second-pass correction), the later extraction picks it up.

5.5.5 Downstream Boundary Effects

The most visible consequence of timezone handling shows up in the business reports that consume the data, well downstream of the pipeline itself.

When someone downstream writes `SUM(amount) GROUP BY TRUNC(sale_date, MONTH)`, sales near the month boundary can land in the wrong bucket depending on how the timestamp is interpreted. A sale at `2026-03-31 23:30:00` in the source's local timezone is `2026-04-01 02:30:00 UTC`. If the analyst's report truncates a UTC timestamp, March's revenue is short and April's is inflated. Multiply this across every month boundary and the numbers never match the source system's own reports.

This matters more than partition alignment. A row in the wrong partition is an internal cost issue – a query scans one extra partition. A row in the wrong month in a revenue report gets escalated to the CFO. Document the timezone assumption clearly so downstream teams can adjust their queries accordingly.

Document the timezone assumption per table

Add a comment to the destination DDL or a row in a metadata table: "`orders.created_at` is naive, assumed `America/Santiago` based on source team confirmation (2026-03-14)." When the assumption is wrong – and eventually it will be, because someone changes the server timezone or adds a branch in a different country – at least you'll know what was assumed and when.

From production: A double timezone conversion on Cyber Monday

A retail client converted UTC to local time inside their own database before we ever saw it, so the column already held local time with no marker saying so. We then landed that local value as if it were UTC and translated it back to local for display, applying an eight-hour offset that had already been applied, so every timestamp ended up shifted twice. On Cyber Monday – the one day where hour-by-hour sales matter most – the curve was visibly wrong, with revenue attributed to the wrong hours by a wide margin, because each layer had assumed the other did no conversion. The recovery was clean precisely because the error was consistent: when an entire column is wrong by the same offset, you can build a corrected copy, verify it against the source's own reports, and swap it in (8.1.6 – Table Swap) rather than patching rows in place.

5.5.6 By Corridor

Transactional to columnar

BigQuery and Snowflake handle timezone-aware timestamps well, but only if you give them the right data. BigQuery `TIMESTAMP` = always UTC; use `DATETIME` for naive values. Snowflake has `TIMESTAMP_NTZ` (naive), `TIMESTAMP_LTZ` (session-local), and `TIMESTAMP_TZ` (explicit offset) – pick the one that matches what the source actually stores. Landing a naive value as an aware type silently assigns a wrong timezone with no error and no warning.

Transactional to transactional

If source is naive PostgreSQL and destination is naive PostgreSQL, no conversion needed – the naive value transfers as-is. Document the assumption but don't add complexity. If the destination is a different engine (PostgreSQL to MySQL), check whether the naive type behavior differs – PostgreSQL's `TIMESTAMP WITHOUT TIME ZONE` and MySQL's `DATETIME` are equivalent in practice, but SQL Server's `DATETIME2` has different precision (see 5.3 – Type Casting and Normalization).

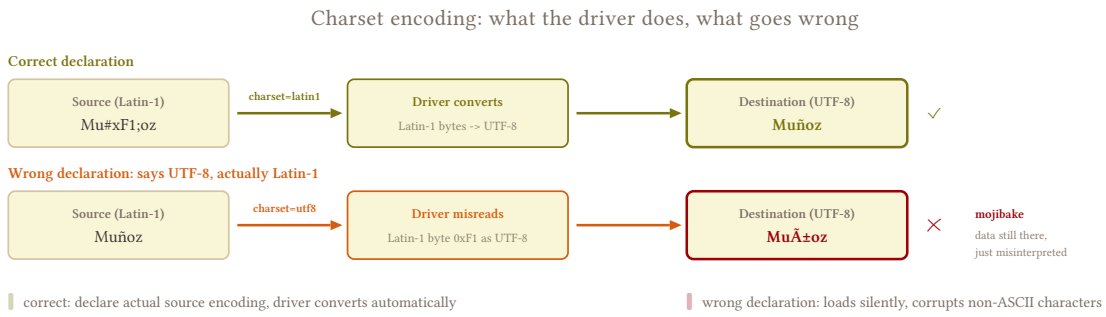
5.6 Charset and Encoding

One-liner: Latin-1 source, UTF-8 destination. Let the library handle the conversion – don't do it by hand.

5.6.1 The Playbook

Charset encoding is one of those syntactic operations that should be invisible. The right approach is to declare the source encoding explicitly on the connection, let the driver handle the conversion to UTF-8, and move on. Don't write manual byte-level conversion code – every database driver and library (SQLAlchemy, JDBC, ODBC) has solved this problem already, and their solution is better tested than yours will be.

The only thing you need to get right is the declaration. If you tell the driver the source is UTF-8 and it's actually Latin-1, the conversion produces mojibake silently. If you tell it Latin-1 and it's actually Windows-1252, you lose a handful of characters (curly quotes, em dashes, ellipsis) that exist in Windows-1252 but not in Latin-1. Get the encoding right on the connection string and the rest takes care of itself.



5.6.2 Where It Happens

Legacy ERPs. SAP (various modules), AS/400, older Oracle installations. These systems predate the UTF-8 consensus and store data in Latin-1, Windows-1252, or vendor-specific encodings. SAP HANA itself is UTF-8, but data migrated from older SAP systems may carry encoding artifacts from the original system.

CSV exports. The encoding header is either missing, wrong, or "it depends on which machine exported it." A CSV that claims UTF-8 but was actually exported from Excel on a Windows machine is Windows-1252. Parquet doesn't have this problem – it's always UTF-8 internally – which is one of many reasons to prefer Parquet over CSV for data exchange when you have the choice.

Older OLTP systems. Any transactional database deployed before ~2010 has a reasonable chance of running a non-UTF-8 encoding. The older the system, the higher the chance – and the more likely it is that nobody remembers what encoding was configured at install time.

5.6.3 Detection

Encoding problems are invisible until they're not. The data loads successfully, the row counts match, and everything looks fine – until someone searches for a customer named "Muñoz" and finds "Mu?oz" or "MuÃ±oz" instead.

Replacement characters. Rows with `?` or `\ufffd` (the Unicode replacement character) in text columns. These appear when the driver encounters a byte sequence that's invalid in the declared encoding and substitutes a placeholder instead of failing. If you see them, the encoding declaration is wrong.

Mojibake. Multi-byte UTF-8 sequences interpreted as single-byte Latin-1 characters. `ñ` becomes `Ã±`, `ü` becomes `Ã¼`, `é` becomes `Ã©`. This happens when the data is actually UTF-8 but the connection declares Latin-1 (or vice versa). The characters are still there – just misinterpreted – and the fix is correcting the encoding declaration, not the data.

The canary columns. Names with `ñ`, `ü`, `ç`, accented characters, or any non-ASCII content. If the accented characters look wrong in the destination, the encoding is wrong. Spot-check these columns after every new source connection setup.

```
-- destination: any
-- Quick canary check after load
SELECT name
FROM customers
WHERE name LIKE '%ñ%'
   OR name LIKE '%ü%'
   OR name LIKE '%ç%'
LIMIT 10;
```

If this returns rows with clean characters, the encoding is working. If it returns mojibake or replacement characters, fix the connection encoding before loading anything else.

5.6.4 The Fix

Declare the encoding on the connection. Every driver has a parameter for this:

```
# SQLAlchemy + PyMySQL -- Latin-1 MySQL source
engine = create_engine(
    "mysql+pymysql://user:pass@host/db?charset=latin1"
)

# SQLAlchemy + pyodbc -- SQL Server with Windows-1252
# Encoding handled via ODBC driver connection string
engine = create_engine(
    "mssql+pyodbc://user:pass@host/db"
    "?driver=ODBC+Driver+18+for+SQL+Server"
    "&ClientCharset=CP1252"
)
```

The driver decodes from the source encoding on read and encodes to UTF-8 for your pipeline. No manual byte manipulation needed.

Validate after load. Run the canary check above on the first load and after any connection configuration change. Encoding problems are deterministic – if the canary passes, every row is fine. If it fails, every non-ASCII row is affected.

CSV-specific. When the encoding isn't declared in the file, try `charset-normalizer` (the modern default, used by `requests` since 2.28) or `chardet` to detect it from the byte content. These aren't 100% accurate but they're better than guessing. Once detected, pass the encoding explicitly to your CSV reader: `pandas.read_csv(path, encoding='cp1252')`. Better yet, convert the source to Parquet first – Parquet is always UTF-8 internally and eliminates the encoding problem entirely.

5.6.5 Collation Traps

Collation is related to encoding but distinct: encoding determines *how bytes map to characters*, collation determines *how characters compare and sort*. A correct encoding with a mismatched collation produces data that loads correctly but behaves differently in queries.

Case sensitivity. PostgreSQL respects per-column `COLLATE` settings – `WHERE name = 'García'` might or might not match `'GARCÍA'` depending on the collation. BigQuery defaults to case-sensitive binary comparison, and while it now supports `COLLATE` clauses (e.g. `'und:ci'` for case-insensitive), most tables use the default. A JOIN that works on a case-insensitive source fails on BigQuery because `'garcia' != 'García'`.

Accent sensitivity. A source with accent-insensitive collation treats `café` and `cafe` as equal. A destination with binary collation (the default on most columnar engines) treats them as different values. A JOIN on a text column that "always worked" on the source returns fewer rows on the destination, and the missing rows are the ones with accented characters.

What to do about it. Document the source collation for text columns used in JOINs or filters. If a collation mismatch causes incorrect query results at the destination, the fix belongs downstream (a `LOWER()` or `COLLATE` clause in the consumer's query), not in the pipeline. Syntactic transformation doesn't change how strings compare – it makes sure the bytes arrive correctly.

5.6.6 Schema Naming

Related but distinct concern: the characters in table and column *names*, not in the data. This is about safety and consistency across engines, not about renaming `OACT` to `chart_of_accounts`.

The problem. SQL Server allows `[Emojis 👽]` as a column name. PostgreSQL allows `"@Table"` with quotes. SAP tables are named `OACT`, `OINV`, `INV1`. These identifiers may contain spaces, special characters, brackets, or characters that are reserved words in the destination engine. A column named `order` in the source breaks every query on the destination unless quoted – and nobody quotes consistently.

What the pipeline should do. Normalize identifiers for *safety*: lowercase, replace spaces with underscores, strip characters that require quoting on the destination engine. This is syntactic normalization, not semantic renaming (`OACT` → `chart_of_accounts`) – it's making sure the identifier doesn't break SQL on the other side. `[Order Lines]` → `order_lines`, `@Status` → `status`, `Column Name With Spaces` → `column_name_with_spaces`.

This deserves its own full treatment – see 8.1 – SQL Dialect Reference for the complete naming convention discussion, including when to rename vs. preserve, schema prefixes, and how to handle identifiers that are reserved words on the destination.

5.6.7 By Corridor

Transactional to columnar

Usually Latin-1 or Windows-1252 to UTF-8, one direction. BigQuery, Snowflake, ClickHouse, and Redshift are all UTF-8 natively. The driver handles the conversion as long as the source encoding is declared correctly. Collation mismatches are more common here because columnar engines default to binary (case-sensitive, accent-sensitive) comparison, while many transactional sources run case-insensitive collations.

Transactional to transactional

Can be UTF-8 to UTF-8 with no encoding conversion needed, but collation differences between engines still bite. PostgreSQL's default collation depends on the OS locale at `initdb` time. MySQL's default depends on the server config and can vary per table. Moving data between them without checking collation equivalence leads to subtle query behavior differences that don't show up until someone reports a missing JOIN match.

5.7 Nested Data and JSON

> **One-liner:** JSON column in the source? Land it as-is. Normalizing nested data into relational tables is semantic transformation – it belongs downstream.

5.7.1 The Playbook

Prefer landing JSON columns as they are – `STRING` or the destination's native JSON type (BigQuery `JSON`, Snowflake `VARIANT`, PostgreSQL `JSONB`). The source has a JSON column, the destination gets a JSON column. That's syntactic.

Flattening JSON into normalized tables (`order`, `order__details`, `order__details__items`) is semantic transformation. Syntactic work makes data survive the crossing without restructuring it. If you have a strong reason to flatten (a consumer that absolutely cannot work with JSON and there's no downstream layer to do it), document the decision and know that you're stepping outside the syntactic boundary. Most of the time, you don't need to.

Avoid the hybrid approach (land raw JSON + flatten to normalized tables) in the pipeline. Two representations means double the storage, double the schema maintenance, and a synchronization problem when one updates and the other doesn't. One representation is enough – pick the simpler one and let downstream build the other if they need it.

5.7.2 Land As-Is

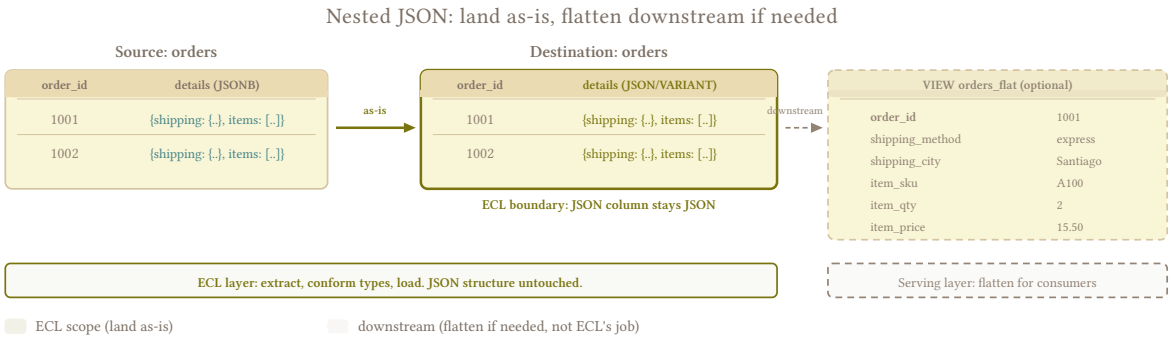
The default. The destination gets what the source has, with no interpretation and no decisions about which fields matter.

```
-- source: transactional (postgresql)
SELECT
    order_id,
    customer_id,
    details,  -- JSONB column, landed as-is
    CURRENT_TIMESTAMP AS _extracted_at
FROM orders
WHERE updated_at >= :last_run;
```

The `details` column might contain:

```
{
  "shipping": {
    "method": "express",
    "address": {"city": "Santiago", "zip": "7500000"}
  },
  "items": [
    {"sku": "A100", "qty": 2, "price": 15.50},
    {"sku": "B200", "qty": 1, "price": 42.00}
  ],
  "notes": "Gift wrap requested"
}
```

Land it as-is. The structure, the nesting, the array of items – all of it arrives at the destination exactly as the source stores it. No field selection, no type inference on nested values, no decision about whether `shipping.address` should be its own table.



This works well when consumers are data-conscious and comfortable with JSON query syntax. BigQuery's `JSON_EXTRACT_SCALAR(details, '$.shipping.method')`, Snowflake's `details:shipping.method`, PostgreSQL's `details->'shipping'->>'method'` – all of these give consumers access to every field without the pipeline making structural decisions on their behalf.

5.7.3 Know Your Consumer

Not all BI tools handle nested data the same way, and that's worth understanding even though it doesn't change the approach.

Tools that handle JSON well. Looker, BigQuery BI Engine, Metabase (with JSON path support), any tool where the query author writes SQL. These consumers can reach into the JSON with path expressions and extract what they need.

Tools that struggle with JSON. Power BI's handling of nested fields is limited – it can expand JSON into columns, but the experience is clunky and the performance degrades with deeply nested structures. Some reporting tools expect flat tabular data and have no JSON path support at all.

But here's the thing: the same consumers who can't handle JSON often can't handle joins either. Normalizing the JSON into 5 relational tables and expecting a business analyst to JOIN `order` → `order__details` → `order__details__items` correctly is optimistic. You've traded one problem (they can't query JSON) for another (they can't join tables), and the second problem is arguably worse because wrong joins produce silently incorrect results while failing to query JSON produces an error.

If the consumer truly can't work with JSON, the answer is a downstream semantic transformation – a view or materialized table that flattens the JSON into the shape the consumer needs. That's a serving concern (7.4 – Query Patterns for Analysts). The pipeline lands the data; the serving layer shapes it for consumption.

> **Flattening views are cheap and reversible**
>
> A `CREATE VIEW orders_flat AS SELECT order_id, JSON_EXTRACT_SCALAR(details, '$.shipping.method') AS shipping_method, ...` gives the consumer a flat table without modifying the landed data. If the JSON structure changes, you update the view. If a new consumer needs a different shape, you create another view. The raw JSON in the landed table is always the source of truth.

5.7.4 Schema Mutation in JSON

JSON columns mutate without warning. A new field appears because the application team shipped a feature. A field disappears because someone removed it from the API response. A field that was always a string is now sometimes a number because a third-party integration changed its output format. None of this is visible in the source schema – the column type is still `JSONB`, the DDL hasn't changed, and your extraction query returns the same column.

This is downstream's problem. Land the JSON as-is and let the consumer or the serving layer handle schema evolution within the blob. The pipeline doesn't parse the JSON, so it doesn't break when the JSON changes – which is exactly the property you want.

The one exception: when schema mutation causes the *load itself* to fail. BigQuery `STRUCT` is schema-on-write – every row must match the declared field names and types. If the JSON gains a new field that the `STRUCT` definition doesn't include, the load rejects the row by default (BigQuery's `ignoreUnknownValues` option silently drops extra fields instead, but that's data loss). Two options:

Land as `STRING` instead of `STRUCT`. The destination stores the raw JSON text with no schema enforcement. Any valid JSON string loads successfully regardless of what fields it contains. Consumers parse the JSON at query time. This is the safest choice for mutating JSON because the schema is the consumer's problem, not the load's problem.

Use a schema-on-read type. Snowflake `VARIANT` accepts arbitrary JSON without a predefined schema. PostgreSQL `JSONB` does the same. These types give you native JSON query syntax without the rigidity of `STRUCT`. If your destination supports schema-on-read, prefer it over `STRING` for the better query ergonomics.

If you must use a typed `STRUCT` (because the destination requires it or because query performance on `STRING` is unacceptable), a full replace (4.1 – Full Replace Load) with an updated `STRUCT` definition handles the schema change cleanly – drop and rebuild the table with the new field included.

5.7.5 By Corridor

Transactional to columnar

Native JSON support varies significantly. **BigQuery**: `JSON` type (schema-on-read, recommended) or `STRUCT/REPEATED` (typed, schema-on-write). Use `JSON` for mutating data, `STRUCT` only when the schema is genuinely stable and you need the query performance. Landing as `STRING` is always safe. **Snowflake**: `VARIANT` is schema-on-read and handles arbitrary JSON natively. The natural choice – flexible, queryable, doesn't break on schema changes. **ClickHouse**: `JSON` type (production-ready since late 2024) or `String`. For older ClickHouse versions, `String` with `JSONExtract\*` functions is the safe fallback. **Redshift**: `SUPER` type accepts semi-structured data. Queryable with `PartiQL` syntax.

Transactional to transactional

Usually straightforward. **PostgreSQL to PostgreSQL**: `JSONB` to `JSONB`. Native, queryable, indexed with GIN indexes. Zero conversion needed. **MySQL to MySQL / PostgreSQL**: MySQL `JSON` to PostgreSQL `JSONB`. Both accept arbitrary JSON. The query syntax differs (`->` vs `->>` semantics) but the data transfers as-is. Both engines accept arbitrary JSON without schema definition, so schema mutation within the JSON is never a load problem.

6.1 Monitoring and Observability

> **One-liner:** Row counts tell you the pipeline ran. They don't tell you it ran *well*, or that the data it produced is worth trusting.

This chapter covers 15 operational concerns in four clusters:

Cluster	Sections
Observability	Monitoring (6.1), Health Table (6.2), Cost Monitoring (6.3), SLA Management (6.4)
Scheduling	Alerting (6.5), Scheduling & Dependencies (6.6), Source Etiquette (6.7), Tiered Freshness (6.8)
Contracts	Data Contracts (6.9), Extraction Status Gates (6.10)
Recovery	Backfill (6.11), Partial Failure (6.12), Duplicates (6.13), Reconciliation (6.14), Corruption Recovery (6.15)

6.1.1 Silent Corruption

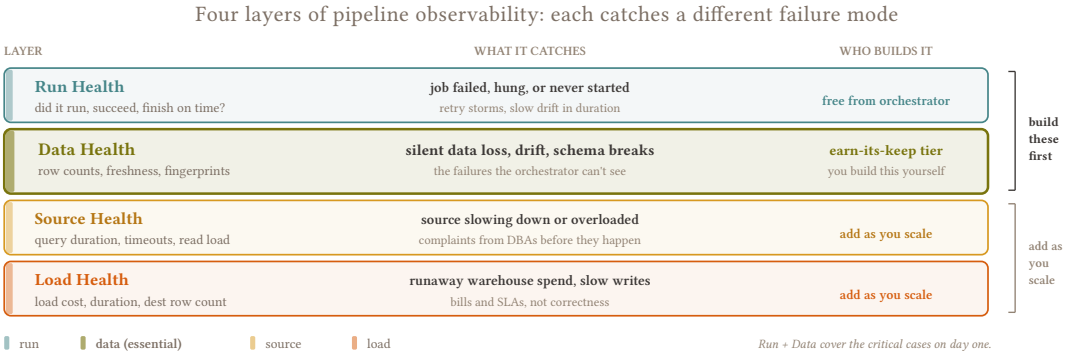
Most pipelines start with a single check: did it succeed? That binary signal covers maybe 40% of what can go wrong. A pipeline can succeed while producing garbage – a query timed out and returned partial results, a full replace that used to take 3 minutes now takes 45 because the table grew 10x, the source schema changed and the loader silently dropped columns, or half the batch loaded while the other half timed out, leaving the destination with rows from two different points in time. Every one of these scenarios reports SUCCESS. Every one of them delivers broken data to consumers.

Without structured **observability**, you discover these problems when a stakeholder asks why the dashboard is wrong – often days after the data actually broke. By that point the blast radius is wide: downstream models have consumed the bad data, reports have been sent, and the person asking is already frustrated. The monitoring approach in this chapter is about catching those failures before anyone else does, ideally within minutes of the pipeline run that caused them.

The key insight is that you need to track more than pass/fail, but you also need to resist the urge to track everything. Every metric you record has a storage cost and a cognitive cost – someone has to look at it, and if the dashboard has 40 numbers, nobody looks at any of them carefully. The goal is a small set of raw measurements that cover the important failure modes, from which you can derive everything else.

6.1.2 Four Layers of Pipeline Observability

Observability breaks into four layers, each covering a different failure mode. You don't need all of them on day one – Run Health and Data Health cover the critical cases, and the other two earn their place as your pipeline count grows.



6.1.2.1 Run Health

The basics: did the pipeline run, did it succeed, and how long did it take? Every orchestrator tracks this natively – run status, duration, dependency graphs – so there's rarely anything to build here. What the orchestrator gives you for free is already enough.

The one thing worth adding is trend tracking on duration. A 3-minute job that creeps to 30 minutes is a signal even when it still succeeds, because it tells you the table is growing or the source is degrading before either becomes an emergency. I had a table silently grow enough that its extraction started overlapping with the next scheduled run, causing 3 PM crashes for weeks before I charted duration and saw it had been climbing steadily for months – the fix was moving heavy tables to a less frequent schedule (6.8 – Tiered Freshness), but the signal was in the health table long before the failure. Without duration trends, you discover these problems when jobs start timing out, which is too late to fix gracefully.

Retry counts are worth recording if your pipeline retries on transient failures. A job that succeeds on the third retry every day is masking an unstable connection or a source system under load.

6.1.2.2 Data Health

This is where monitoring earns its keep. Run Health tells you the pipeline executed; Data Health tells you what the pipeline produced.

Row counts are the single most useful metric. Track three numbers: `source_rows` (counted at the source before extraction), `rows_extracted` (returned by the extraction query), and `destination_rows` (counted at the destination after load). Each pair tells you something different. On a full replace, `rows_extracted` should equal `destination_rows` – you pulled N rows and loaded them, so the destination should have N. If it doesn't, something was lost or duplicated during the load. `source_rows` vs `destination_rows` over time is a drift indicator for incremental tables – if the totals diverge across runs, you're accumulating missed rows or orphaned deletes. A 50% drop in any of the three is a signal worth investigating, but row counts have a blind spot: they measure volume, not composition. I had a client whose `invoices` table hard-deleted draft invoices regularly while new ones replaced them at roughly the same rate – the count stayed stable, but the destination accumulated stale drafts the source had already removed. Only a daily PK comparison (6.14 – Reconciliation Patterns) caught the problem, because row counts told us the right *number* of rows existed without revealing they were the wrong rows.

For incremental tables specifically, `rows_extracted` over time is revealing. It shows big moments of change – month-end closes, batch corrections, seasonal spikes – where you may want to widen your extraction window or shift the schedule to avoid overlapping with the source system's heaviest period.

> **Alert on row count spikes**
>
> If an incremental that usually returns 2k `rows_extracted` suddenly returns 50k, the source had a large batch operation – month-end close, bulk import, data migration. That spike means there may be more rows changed than your window caught. Consider triggering a full replace that night to reset state and catch anything the incremental missed.

Freshness is the other critical data health metric: when was this table last successfully loaded? The health table records `extracted_at` on every run (complementing the per-row `_extracted_at` from 5.1 – Metadata Column Injection, which tags individual records rather than pipeline runs), so staleness is a simple aggregation – 6.4 – SLA Management covers the query and the SLA thresholds that give the number meaning.

Schema fingerprints and null rates are worth tracking here as changes between runs, but enforcement – what to do when they change – belongs in 6.9 – Data Contracts.

6.1.2.3 Source Health

Source health metrics are less about your pipeline and more about the system you're extracting from. Query duration at the source, isolated from load performance, tells you whether the source database is degrading or whether your extraction query needs tuning. Timeout frequency – queries that hit the threshold even when they eventually return on retry – reveals instability before it becomes a failure.

Source system load impact is worth tracking for a less obvious reason: it's a sales tool. If you can demonstrate that your extraction uses less than 1% of the source database's capacity, you can sell the pipeline as a lightweight, non-invasive solution to more technical stakeholders who are nervous about letting you query their production system. See 6.7 – Source System Etiquette for the full treatment.

6.1.2.4 Load Health

Load **cost** generally matters more than load duration. Duration tends to be stable for a given table size and load strategy – it's predictable and boring. Cost is the variable that shifts under your feet: a MERGE on BigQuery at 100k rows costs differently than at 10M, DML pricing changes without warning, and switching from full replace to incremental changes the operation type entirely. Tracking `load_seconds` is still useful for spotting bottlenecks, but if you had to pick one dimension to watch on the load side, it's cost – and 6.3 – Cost Monitoring covers how to capture and attribute it.

The destination row count after load closes the loop on reconciliation. On a full replace, `destination_rows` should match `rows_extracted` – if it doesn't, rows were lost or duplicated during the load. On an incremental, tracking `source_rows` vs `destination_rows` over time reveals whether the totals are drifting apart across runs, which is the signal that your incremental is accumulating missed rows or undetected deletes. See 6.14 – Reconciliation Patterns for the full treatment.

6.1.3 The Morning Routine

Before diving into implementation, it's worth naming what you're actually looking at when you open the dashboard. The sequence matters – it's a triage, not a survey.

Four numbers you check first

(1) How many tables failed overnight. (2) Which tables are stale beyond their SLA. (3) Any row count anomalies – spikes, drops, or reconciliation deltas above threshold. (4) Cost per day. Everything else is drill-down from one of these four.

In a single-orchestrator setup, the orchestrator's native UI covers items 1 and 2 well enough. Items 3 and 4 come from the health table and the cost monitoring layer from 6.3 – Cost Monitoring. In a multi-orchestrator setup, the health table is the only place where all four numbers converge – which is why it exists.

6.1.4 The Pattern

After every pipeline run, append a row to a health table. One row per table per run, with the raw measurements needed to answer the four morning questions. Everything else – dashboards, alerts, SLA reports – is a query on top of this table. 6.2 – The Health Table covers the schema, the column-by-column rationale, and how to populate it.

6.1.5 Anti-Patterns

Don't confuse monitoring with alerting

Monitoring is the dashboard you look at; alerting is the pager that wakes you up. They share data, but the threshold for "worth recording" is much lower than "worth paging someone." Record everything in the health table. Alert on a carefully tuned subset. See 6.5 – Alerting and Notifications for how to calibrate the boundary.

Don't track everything equally

Per-row metrics on a 100M-row table are storage, not observability. The health table is one row per table per run – aggregate metrics only. If you need row-level diagnostics, run them ad hoc against the source or destination, not as part of every pipeline run.

Don't build a custom monitoring stack

You don't need one if you're running a single orchestrator with 50 tables – the orchestrator's native run history, duration tracking, and status page are probably enough. The health table pattern earns its complexity at scale – hundreds of tables, multiple pipelines, or a multi-orchestrator cluster where no single UI gives you the full picture. Build monitoring infrastructure in proportion to the monitoring problem you actually have.

6.1.6 What Comes Next

6.2 – The Health Table covers the health table implementation – the schema, column rationale, derived metrics, and how to populate it reliably. From there, 6.3 – Cost Monitoring extends it with cost attribution,

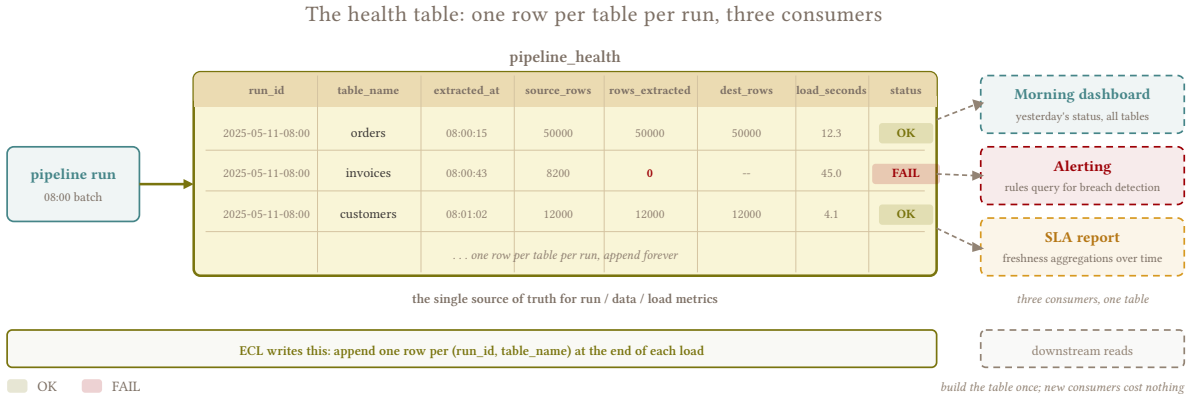
6.4 – SLA Management builds freshness SLAs on the staleness data, and 6.5 – Alerting and Notifications draws the line between what's worth recording and what's worth paging someone about.

6.2 The Health Table

> **One-liner:** One row per table per run, raw measurements only – everything else is a query on top.

6.2.1 What You Can't Measure

The four layers from 6.1 – Monitoring and Observability tell you *what* to watch. This pattern is the *how:* a single append-only table that captures raw measurements from every pipeline run, giving you a queryable history of everything your orchestrator doesn't track natively. Without it, monitoring lives in scattered logs, orchestrator UIs, and tribal knowledge – none of which you can `SELECT` from at 7 AM when something is wrong.



6.2.2 The Pattern

Not every column is equally important. The schema below is ordered by criticality, and the last group is optional depending on how much storage cost you're willing to absorb.

```
-- destination: bigquery
-- One row per table per pipeline run. Append-only.
CREATE TABLE health.runs (
  -- == Identity (always needed) ==
  extracted_at             TIMESTAMP,
  client                   STRING,
  table_id                 STRING, -- Make sure its not only table name, but
identifier in case you query 2 tables of the same name from different sources.
  run_id                   STRING, -- hopefully links back to orchestrator run

  -- == Critical (the metrics you check every morning) ==
  status                   STRING, -- SUCCESS, FAILED, WARNING
  error_message            STRING, -- raw error on failure, NULL on success
  source_rows              INT64,  -- counted at the source before extraction starts
  destination_rows         INT64,  -- counted at the destination after load
completes
  rows_extracted           INT64,  -- rows returned by the extraction query

  -- == Important (phase timing -- where the time goes) ==
  extraction_seconds       FLOAT64,
  normalization_seconds    FLOAT64,
  load_seconds             FLOAT64,
  extraction_strategy      STRING, -- full_replace, incremental, window, etc.

  -- == Nice to have (valuable for debugging, but may be costly at scale) ==
  bytes_extracted          INT64,  -- raw data volume from source
  query_used               STRING, -- the actual extraction query executed
  schema_json              STRING  -- column names + types snapshot, JSON
);
```

Watch storage cost on STRING columns

`query_used` and `schema_json` are STRING columns that grow with query complexity and table width. At thousands of tables running 3x daily, the row count adds up fast – and if each `query_used` averages 2KB, that column alone is 14GB/year before compression. Worth it for debugging, but if cost is really tight, consider storing them in a separate detail table keyed by `run_id` and only joining when you need them. `bytes_extracted` is cheap (INT64) and nearly free to keep.

The guiding principle is **store raw measurements, derive the rest at query time.** Discrepancy percentage, per-row extraction time, average row size, throughput, and total duration are all computable from the columns above and don't need their own storage. A view or a dashboard query handles them.

6.2.2.1 Critical Columns

`status` and `error_message` tell you what failed and why without leaving the health table. Without `error_message`, "12 tables failed overnight" sends you digging through orchestrator logs, job UIs, and possibly multiple systems to find out why each one broke. With it, you can triage severity from a single query – a connection timeout is different from a schema mismatch, and you want to know which you're dealing with before you start investigating. The subtler case is `status = 'SUCCESS'` with `rows_extracted = 0` – normal when an incremental cursor is caught up, alarming when the source table was silently

dropped or permissions changed. 6.10 – Extraction Status Gates covers how to gate the load on extraction status so these two scenarios don't look identical in your health table.

`source_rows` is counted at the source before extraction starts – a snapshot of the total at the moment you begin pulling. `destination_rows` is counted at the destination after the load finishes. `rows_extracted` is the number of rows the extraction query actually returned.

The per-run reconciliation check depends on the strategy. On a **full replace**, `rows_extracted` should equal `destination_rows` – you pulled N rows, loaded N rows, the destination should have N rows. If it doesn't, the load lost or duplicated data. `source_rows` may differ slightly from `rows_extracted` because the source can receive writes between the count and the extraction – transit-time noise, not data loss, typically under 0.1% on a busy table. Set your alert thresholds above this floor to avoid false positives on every run.

On an **incremental**, the per-run check is less direct – `rows_extracted` is a window of change, not the full table, so it won't match `destination_rows`. Instead, track `source_rows` vs `destination_rows` across runs: if the totals drift apart over time, the incremental is accumulating missed rows or undetected deletes, and a full replace is overdue. See 6.14 – Reconciliation Patterns for thresholds and recovery.

6.2.2.2 Important Columns

The timing breakdown stays as three separate columns – `extraction_seconds`, `normalization_seconds`, `load_seconds` – because a single `total_seconds` hides whether the bottleneck is the source query, the syntactic layer, or the destination load. When a pipeline that used to take 5 minutes starts taking 40, you need to know which phase is degrading without digging into logs. The total is trivially computable from the parts; the parts are not recoverable from the total.

`extraction_strategy` records whether this run was `full_replace`, `incremental`, `window`, or something else. The same table can run different strategies on different schedules – a nightly full replace for purity, intraday incremental for freshness (see 6.8 – Tiered Freshness). Without this column, 50k `rows_extracted` is ambiguous: perfectly normal on a full replace, possibly alarming on an incremental that usually returns 2k.

6.2.2.3 Nice-to-Have Columns

`bytes_extracted` is cheap to store and catches a failure mode that row counts miss entirely: rows getting wider. If `rows_extracted` stays flat but `bytes_extracted` climbs, the source table is gaining columns or existing text columns are growing – both of which affect extraction time, network transfer, and destination storage cost. Per-row size (`bytes_extracted / rows_extracted`) and throughput (`bytes_extracted / extraction_seconds`) are both derivable.

`query_used` stores the actual extraction query, which implicitly records the cursor value, window boundaries, and any filters applied. When an incremental returns 0 rows, the query tells you whether the cursor was already caught up or stuck. When a full replace suddenly takes 10x longer, the query tells you if someone added a WHERE clause that forced a full scan at source. It's the single most useful debugging column – and the most expensive to store.

`schema_json` is a JSON snapshot of the column names and types seen during this run. Comparing it to the previous run's snapshot detects schema drift without building a separate fingerprinting system. The policies for what to do when drift is detected – evolve (accept the change) or freeze (reject the load) – belong in 6.9 – Data Contracts. Silently discarding columns that don't match is a semantic decision, not syntactic work – if the source sent it, the destination should have it (see 4.3 – Merge / Upsert).

6.2.2.4 Derived Metrics

None of these need their own column. Build them as a view or compute them in your dashboard:

```
-- destination: bigquery
-- View on top of the health table for common derived metrics.
CREATE VIEW health.runs_derived AS
SELECT
  *,
  extraction_seconds + normalization_seconds + load_seconds
    AS total_seconds,
  -- Per-run check: did everything extracted actually land?
  -- Meaningful on full_replace; less useful on incremental.
  SAFE_DIVIDE(rows_extracted - destination_rows, rows_extracted) * 100
    AS load_loss_pct,
  -- Drift check: are source and destination totals diverging?
  -- Track over time for incremental tables.
  SAFE_DIVIDE(source_rows - destination_rows, source_rows) * 100
    AS drift_pct,
  SAFE_DIVIDE(rows_extracted, extraction_seconds)
    AS rows_per_second,
  SAFE_DIVIDE(bytes_extracted, rows_extracted)
    AS avg_row_bytes,
  SAFE_DIVIDE(bytes_extracted, extraction_seconds)
    AS throughput_bytes_per_sec
FROM health.runs;
```

> **Early warning for source degradation**
>
> On incremental tables, `rows_per_second` should be roughly stable across runs. If it drops by half, the source query is getting slower per row – possibly because the cursor column lost its index, or because the table's physical layout changed. A drop in `rows_per_second` with stable `rows_extracted` points at the source; stable `rows_per_second` with a spike in `rows_extracted` points at a data event.

6.2.2.5 Staleness Report

Once the health table exists, staleness is a `MAX(extracted_at)` grouped by table – the query is straightforward enough that 6.4 – SLA Management covers it in full alongside the SLA thresholds that give the number meaning.

6.2.3 Populating the Health Table

The schema is the easy part; the discipline is harder. Every run – successful or not – must append a row. A missing row in the health table is indistinguishable from "the pipeline didn't run" when you're triaging at 7 AM, and that ambiguity is worse than a recorded failure.

```
-- destination: bigquery
-- Append one row per table per run. Always, even on failure.
INSERT INTO health.runs (
  extracted_at, client, table_id, run_id,
  status, error_message,
  source_rows, rows_extracted, destination_rows,
  extraction_seconds, normalization_seconds, load_seconds,
  extraction_strategy
) VALUES (
  CURRENT_TIMESTAMP(), @client, @table_id, @run_id,
  @status, @error_message,
  @source_rows, @rows_extracted, @destination_rows,
  @extraction_seconds, @normalization_seconds, @load_seconds,
  @extraction_strategy
);
```

On failure, `rows_extracted` and `destination_rows` will likely be NULL – that's expected. The row still captures `status = 'FAILED'`, the error message, and whatever timing was available before the failure point. NULL in `destination_rows` on a FAILED row means the load never completed, which is meaningfully different from zero (the load ran but produced nothing). Both are worth recording and both tell you something different during triage.

The timing columns require wrapping each phase in a timer – most orchestrator SDKs and pipeline frameworks provide hook points (before/after extraction, before/after load) where you can capture deltas. If yours doesn't, a context manager or simple stopwatch around each phase is enough. Sub-second precision doesn't matter here; the value comes from tracking trends across runs, not from any single measurement.

> **Count source rows without punishing the source**
>
> `SELECT COUNT(\*)` on a 50M-row transactional table can lock pages and spike CPU on the source. For drift detection, an approximate count from the database's statistics catalog is often good enough – `pg_stat_user_tables.n_live_tup` in PostgreSQL, `information_schema.TABLES.TABLE_ROWS` in MySQL. You're watching for 10%+ swings, not exact matches. If the approximation is too stale (PostgreSQL's stats depend on autovacuum frequency), schedule a periodic exact count during off-hours and use the approximate count for intraday runs.

The health INSERT itself can fail – destination timeout, permission issue, quota exhaustion – and silently leave a gap in your monitoring. Wrap it in its own error handler with a fallback to local logging (a JSON file, a stderr line, anything durable), so you at least know the health write failed even if the row didn't land. Discovering that your monitoring table has a 3-day gap because the health destination was unreachable is a particularly frustrating way to learn you had no visibility during an incident.

6.2.4 Where Your Orchestrator Fits

6.2.4.1 Generating Metadata on Load

The ideal place to capture health metrics is inside the pipeline run itself – as a side effect of extraction and load, not in a separate job that queries the destination afterward. If your orchestrator lets you attach

custom metadata to each table after a run (row counts, extraction duration, schema fingerprint), that metadata becomes queryable and historically tracked without building a separate system.

This is worth prioritizing when evaluating orchestrators for extract-and-load workloads (see Appendix: Orchestrators). Dagster's custom asset metadata, for example, lets you record these numbers directly on the asset and graph them from the UI – the health table columns above get populated as a side effect of the pipeline run rather than requiring a post-hoc collection step. The less infrastructure you build outside the orchestrator, the less you maintain.

When your orchestrator doesn't support rich metadata attachment – which is the more common case – the health table INSERT becomes an explicit final step in each pipeline run: a wrapper function that captures metrics and writes the row after the load completes (or fails). This works fine and is what most teams end up building. The key is placing the INSERT in a `finally` block or equivalent, so it fires regardless of whether the run succeeded, and giving it its own error handling so a health write failure doesn't mask the original pipeline error.

6.2.4.2 Single Orchestrator

Every orchestrator tracks run status, duration, and dependency graphs natively – the built-in UI covers Run Health almost entirely, and duplicating that visibility in the health table wastes both storage and effort.

The gap is Data Health. Your orchestrator knows the pipeline ran for 4 minutes and succeeded, but it has no idea that `orders` returned 12k rows instead of the usual 450k, or that the source schema lost a column between yesterday and today, or that `destination_rows` doesn't match `rows_extracted`. These are the metrics that justify the health table. Build it to fill the gaps your orchestrator leaves – row counts, reconciliation deltas, schema fingerprints, phase timing if the orchestrator only gives you total duration – and skip what's already there.

Source Health and Load Health slot in the same way: if the orchestrator already provides retry counts and error classification, use those natively and don't duplicate them. If it only gives you pass/fail with no structured error metadata, the health table's `status`, `error_message`, and phase timing columns cover the essentials. The principle is complementary, not redundant – one system of record per metric, and the health table picks up everything the orchestrator drops.

6.2.4.3 Orchestrator-per-Client (Orch. Cluster)

When each client runs its own orchestrator instance, no single UI gives you the full picture. "How many tables failed last night?" requires opening N dashboards, one per client, and mentally aggregating the results – which nobody actually does consistently at 6 AM.

The health table solves this by becoming the unified monitoring layer above the individual orchestrators. Every instance appends to the same destination table after each run, partitioned by the `client` column, and the morning routine works against a single query across all clients rather than N separate UIs. Staleness reports, reconciliation checks, and cost rollups all aggregate naturally because they share a schema.

This setup also unlocks cross-client comparison, which individual orchestrator dashboards can never provide. If `orders` extraction takes 3 minutes for client A but 25 minutes for client B on the same schema version, the health table surfaces that in a single query – and the cause is usually environmental (client B's source database is underpowered, or their `orders` table is 10x larger, or their network path to the extraction server adds latency). Same for schema drift: if client B's source adds a column that client A doesn't have, `schema_json` catches the divergence immediately, which matters when both clients are supposed to be running the same ERP version.

Central health table is a dependency

If the health destination is unreachable, every pipeline run across all clients loses its monitoring write. A local fallback – writing the health row to a staging table in each client's own destination, then syncing centrally on a schedule – mitigates this at the cost of slight staleness in the unified view. At minimum, the health INSERT should log to stderr on failure so the orchestrator's native run output still captures what happened, even if the health table doesn't.

6.2.5 Tradeoffs

Pro	Con
Catches silent failures that pass/fail misses	Storage cost grows linearly with table count and run frequency
Health table provides a single queryable history	Requires discipline to populate on every run, including failures
Raw measurements let you derive new metrics without schema changes	STRING columns (`query_used`, `schema_json`) can dominate storage at scale
Works across orchestrators in a cluster setup	Adds write latency to every pipeline run (one INSERT per table per run)

6.3 Cost Monitoring

> **One-liner:** Per-table, per-query, per-consumer – know where the money goes before the invoice arrives.

6.3.1 Invisible Spend

Cloud data warehouses bill by bytes scanned, slots consumed, or storage volume – and the bill arrives after the damage is done. A single bad pattern can dominate the monthly invoice: an unpartitioned scan that reads the entire table on every query, a MERGE on a table that should be a full replace, or a staging dataset nobody cleaned up accumulating for months. You won't know which one it was until you can attribute cost to individual tables and operations.

Without per-table cost attribution, "costs went up 40%" is a mystery that sends you guessing. With it, you can point at the exact table and the exact operation that caused the spike – and decide whether to fix the pattern, reduce the schedule frequency, or accept the cost because the freshness justifies it.

6.3.2 The Pattern

Cost monitoring extends the health table from 6.2 – The Health Table. The health table captures `extraction_seconds`, `load_seconds`, and `bytes_extracted` per run – time and volume metrics that tell you where the pipeline spends effort. But destination-side cost (bytes scanned, slots consumed, DML

pricing) lives in the destination's own audit logs, and the connection between the two is the job label or run ID you attach to each load operation.

6.3.2.1 What to Track

Compute costs are the volatile dimension. Track bytes scanned per load operation (the MERGE, the DELETE+INSERT, the partition swap), and if your engine charges by slots or query-seconds, track those too. The same data supports three useful aggregations: cost per run (anomaly detection), cost per table (pattern decisions), and cost per schedule (budgeting). MERGE deserves special attention because it's the single most expensive load operation in columnar engines – a MERGE-heavy pipeline loading hundreds of tables **can cost an order of magnitude more** than the same tables loaded via partition swap or append-and-materialize, and the difference only shows up in the bill, not in run duration.

Storage costs are predictable but sneaky at scale. Append logs grow with every run, and without compaction that growth is unbounded (4.4 – Append and Materialize covers compaction). Staging tables that outlive their load job are dead weight, though in practice orphaned staging is more of a schema hygiene issue than a cost problem – at ~$0.02/GB/month in BigQuery, a few hundred GB of staging is annoying but not alarming. The compute cost of accidentally querying unpartitioned staging is usually worse than storing it.

Extraction costs are easy to forget because querying your own PostgreSQL is free. But some sources meter reads: API rate limits, licensed query slots (SAP HANA, some SaaS platforms), or egress charges from cloud-hosted sources. Extracting over VPNs can also incur bandwidth costs. Overlapping extraction windows in stateless patterns (3.3 – Stateless Window Extraction) re-extract the same rows deliberately – the overlap is correct, but its cost in time and source-side load should be visible and known.

6.3.2.2 Cost Attribution

The gap between "the pipeline costs $X/month" and "table Y's MERGE costs $X/month" is a join key: pipeline metadata (table name, run ID, schedule) matched against the destination's query audit log (bytes scanned, cost, duration). Without that join, you're stuck with aggregates that don't point anywhere useful.

Most columnar engines expose per-query metrics through their information schema. BigQuery's `INFORMATION_SCHEMA.JOBS` tracks bytes processed, slot-milliseconds, and the destination table for every DML operation – cost is directly computable from bytes × price. Snowflake's `QUERY_HISTORY` tracks `bytes_scanned` and execution time, but per-query cost requires pro-rating warehouse credits across queries (Snowflake bills by warehouse uptime, not by query). The per-table cost report on BigQuery is a straightforward aggregation:

```sql
-- destination: bigquery
-- Top 20 most expensive destination tables, last 30 days.
-- BigQuery on-demand: $6.25/TB scanned. Adjust for your pricing model.
SELECT
  destination_table.table_id AS table_id,
  COUNT(*) AS load_ops,
  ROUND(SUM(total_bytes_processed) / POW(1024, 3), 2) AS gb_scanned,
  ROUND(SUM(total_bytes_processed) / POW(1024, 4) * 6.25, 2) AS est_cost_usd
FROM `region-us`.INFORMATION_SCHEMA.JOBS
WHERE creation_time >= TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 30 DAY)
  AND job_type = 'QUERY'
  AND state = 'DONE'
  AND error_result IS NULL
GROUP BY table_id
ORDER BY est_cost_usd DESC
LIMIT 20;
```

For richer attribution – by schedule, by client, by run ID – tag each load query with job labels through your orchestrator or pipeline wrapper. These labels appear in the audit log and make the join trivial. If your orchestrator doesn't support job labels natively, a query comment (`-- table=orders run_id=20260323-001`) is a fallback that's parseable with `REGEXP_EXTRACT`.

6.3.2.3 The Drilldown Workflow

Track at maximum granularity – per table, per run, per operation – but check at low frequency. A weekly or monthly glance at the aggregate total is enough. If the aggregate doesn't trigger your curiosity, the individual tables are fine. Drill down when the total spikes, not as a daily ritual.

The practical workflow is top-down: aggregate cost in a dashboard or a weekly scheduled query, filter by table when the number looks off, check whether the pattern changed, the table grew, or the schedule frequency increased. A report showing the top-10 most expensive tables gives you enough signal for most months. **Cost monitoring should cost less attention than it saves money.**

> **Test pattern changes at small scale**
>
> If you're switching a table's load strategy to reduce cost – MERGE to append-and-materialize, full replace to incremental – try it on a small table first, then a large one, then roll it out broadly. Measure the actual cost difference before committing to the migration. The estimated savings from theory and the actual savings in production are rarely the same number.

6.3.3 The Expensive Patterns

Pattern	Why it's expensive	Mitigation
MERGE on large tables	Full scan of both source and destination sides	Partition-scoped merge, or switch to 4.4 – Append and Materialize
Unpartitioned full scan	Every query reads the entire table	Partition by date, enforce `require_partition_filter` (7.3 – Pre-Built Views)
Staging cleanup missed	Orphaned staging datasets accumulate storage	Scheduled cleanup job, weekly or after each run
Append log without compaction	Storage grows linearly with schedule frequency	Periodic compaction to latest-only (4.4 – Append and Materialize)

6.3.4 Tradeoffs

Pro	Con
Per-table attribution turns "costs went up" into an actionable lead	Requires tagging every load query with metadata (job labels, query comments)
Aggregated view catches spikes without daily effort	Destination audit logs have retention limits – store summaries for history
Identifies which patterns to optimize first	Cost optimization can lead to premature complexity if it drives pattern choice

6.3.5 Anti-Patterns

Don't let cost drive pattern selection

Switching from full replace to incremental to "save money" introduces complexity that costs engineering time and creates failure modes (see 1.8 – Purity vs. Freshness). The cheaper pipeline is the one that breaks less, not the one with the lowest bytes-scanned number. Pick the pattern that's correct first, then optimize its cost within that pattern.

Don't optimize without measuring

"MERGE is expensive" is true in general, but *how* expensive depends on table size, partition layout, and update volume. A MERGE on a 10k-row lookup table costs fractions of a cent – switching it to append-and-materialize for cost reasons adds complexity with no real savings. Measure the actual cost per table before deciding anything is worth changing.

6.3.6 What Comes Next

Cost is one input to the freshness decision. 6.4 – SLA Management defines *when* data must be fresh; 6.8 – Tiered Freshness uses cost as one factor in deciding which tables earn high-frequency schedules and which ones run daily.

6.4 SLA Management

> **One-liner:** "The data must be fresh by 8am" – how to define, measure, and enforce freshness commitments.

6.4.1 Implicit Expectations

Stakeholders care about one thing: is the data fresh when they need it? Without an explicit SLA, freshness expectations are implicit – discovered only when violated, usually via an angry email or an angry call from your boss. A pipeline that finishes at 8:15 AM is fine until someone builds a report that refreshes at 8:00 AM, and now you have an SLA you didn't know about.

I had a client who I *told* – but didn't write down – that data updated once daily. They built automated collection emails that fired before midday, but most of their customers had already paid by then. The emails were going out with stale receivables data, and the client blamed the pipeline for the embarrassment. The fix wasn't technical – it was documenting the SLA in the contract so both sides agreed on what "once daily" actually meant: data reflects the previous night's extraction, available by 9 AM, not refreshed throughout the day. **Everything that isn't written down can be reinterpreted against you.** Document the SLA.

6.4.2 Defining an SLA

6.4.2.1 What an SLA Contains

An SLA for a data pipeline is four numbers and a signature:

Component	Example
Table or group	`orders`, `order_lines`, `invoices` – the receivables group
Freshness target	Data reflects source state as of no more than 24 hours ago
Deadline	Available in the destination by 09:00 UTC-3
Measurement point	Last successful load timestamp in the health table, not run start

The measurement point matters. A run that starts at 7 AM but fails and retries until 8:45 AM doesn't meet a 9 AM SLA – it barely makes it, and the next slow day it won't. Measure from `MAX(extracted_at) WHERE status = 'SUCCESS'` in the health table (6.2 – The Health Table), not from when the orchestrator kicked off the job.

6.4.2.2 SLA Tiers

Not every table deserves the same freshness. `metrics_daily` refreshed once a day has a different SLA than `orders` refreshed every 15 minutes or a balance sheet refreshed monthly. Group tables by consumer urgency, not by source system – the tables that most often need more than daily are sales data (especially during Black Friday or seasonal peaks), receivables (for end-of-month collection runs), and inventory stock levels (for in-store availability decisions). Everything else is usually fine at daily.

Daily is the best default. It handles the vast majority of use cases, and the contract should say so explicitly: no more than one scheduled update per day, data reflects the previous night's extraction. When you increase frequency for specific tables – an extra midday refresh for receivables, intraday incremental for sales – make it clear in writing that the increased cadence is outside the base SLA and can be adjusted at any time. This matters because ad-hoc refreshes have a way of becoming expected commitments: you give a consumer an extra midday refresh as a favor, they build a process around it, and now you have an SLA you never agreed to. Give consumers `_extracted_at` in their reports (5.1 – Metadata Column Injection) so they always know how fresh the data actually is, rather than assuming.

> **On-demand refreshes replace high-frequency schedules**
>
> If a consumer needs fresh data once or twice a day at unpredictable times, an on-demand refresh button that triggers the pipeline is often better than scheduling loads every 30 minutes "just in case." One triggered run costs far less than 48 idle runs per day, and the consumer gets exactly-when-needed freshness instead of at-most-30-minutes-stale. On-demand *can* be part of the SLA ("consumer may trigger up to N refreshes per day"), but keep it bounded – without a cap, a trigger-happy user can spam refreshes and compete with scheduled runs for source connections and orchestrator slots. Document the limit, enforce it with a cooldown or queue, and monitor trigger frequency in the health table.

6.4.3 Measuring Freshness

Staleness is the gap between now and the last successful load. The health table gives you this with a single query:

```sql
-- destination: bigquery
-- Freshness report: staleness per table against declared SLA thresholds.
WITH last_success AS (
  SELECT
    table_id,
    MAX(extracted_at) AS last_load,
    TIMESTAMP_DIFF(
      CURRENT_TIMESTAMP(), MAX(extracted_at), HOUR
    ) AS staleness_hours
  FROM health.runs
  WHERE status = 'SUCCESS'
  GROUP BY table_id
)
SELECT
  ls.table_id,
  ls.last_load,
  ls.staleness_hours,
  sla.freshness_hours,
  CASE
    WHEN ls.staleness_hours > sla.freshness_hours THEN 'BREACH'
    WHEN ls.staleness_hours > sla.freshness_hours * 0.8 THEN 'WARNING'
    ELSE 'OK'
  END AS sla_status
FROM last_success ls
JOIN health.sla_config sla USING (table_id)
ORDER BY sla_status DESC, staleness_hours DESC;
```

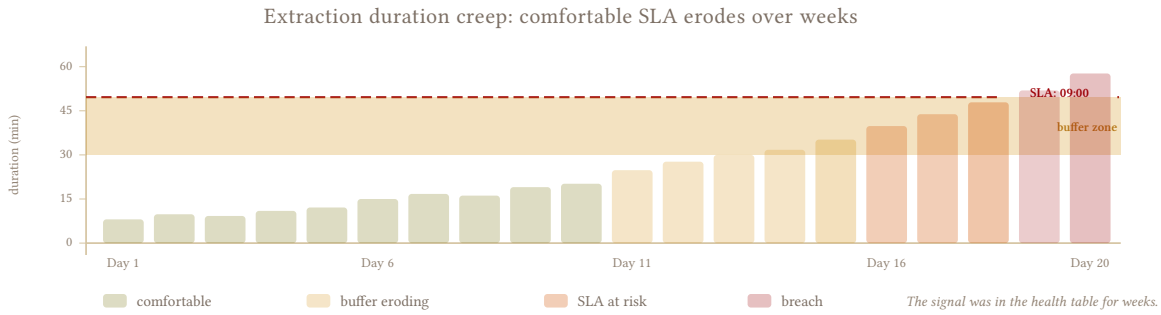
The `sla_config` table is a simple lookup: one row per table or table group, with the `freshness_hours` threshold from the SLA. Hard-code it, load it from a config API, or manage it in a spreadsheet – the mechanism doesn't matter as long as the thresholds are explicit and queryable rather than living in someone's head.

This query is the second item in the morning routine from 6.1 – Monitoring and Observability: after checking failures, check which tables are stale beyond their SLA.

6.4.4 What Erodes SLAs

Upstream delays are the most common cause and the hardest to control. ERP systems run their own batch jobs – posting runs, period closes, nightly aggregation – and those jobs determine when your source data is ready to extract. The ERP itself is rarely the problem; it's the people operating it. When a client has a technical team that runs ad-hoc processes or overloads the database during the window they designated to you, you're the one who gets blamed for stale data. **Build buffer into the SLA** for exactly this – if the source is ready by 7 AM on a good day, don't promise 7:30 AM.

Extraction duration creep turns a comfortable SLA into a tight one over months. The health table's `extraction_seconds` column (6.2 – The Health Table) catches this trend before it becomes a breach – a 3-minute extraction that silently creeps to 25 minutes eats into your buffer without anyone noticing until the SLA breaks.



Stale joins at consumption are the subtler freshness problem. `orders` and `order_lines` can extract and load independently – there's no dependency between them at load time. But if only one of the two refreshes on a given run, consumers joining them will see orphan records: order lines pointing at a non-existent order header, or a refreshed header missing today's new lines. The SLA for header-detail pairs should cover both tables on the same schedule, not because the pipeline requires it, but because the consumer's query does (6.6 – Scheduling and Dependencies).

Backfills that steal capacity from scheduled runs are a less obvious risk. A 6-month backfill running alongside production extractions competes for source connections, orchestrator slots, and destination DML quota (6.11 – Backfill Strategies).

6.4.5 SLA Breach Response

Severity	Trigger	Action
Warning	Staleness > 80% of SLA window	Increase priority of next scheduled run; investigate if it's a trend
Breach	Staleness > SLA window	Alert via 6.5 – Alerting and Notifications, investigate root cause, notify consumers
Sustained breach	Multiple consecutive violations	Escalate – the schedule, the pattern, or the SLA itself needs to change

A single breach is an incident. Sustained breaches mean the SLA is wrong – either the pipeline can't deliver what was promised, or the consumer's actual needs have shifted. Renegotiate the SLA rather than patching around it with increasingly fragile workarounds.

6.4.6 Tradeoffs

Pro	Con
Explicit SLAs set expectations before they're violated	Requires upfront agreement with stakeholders
Staleness query catches breaches before consumers notice	Only measures load completion, not data correctness
Tiered SLAs avoid over-engineering low-priority tables	More tiers means more schedules and more monitoring surface

6.4.7 Anti-Patterns

Don't promise SLAs you can't control

If your pipeline depends on a source system batch job that finishes "sometime between 5 AM and 7 AM," your SLA cannot be 7:30 AM. Build buffer or set the SLA at 9 AM and be honest about it. A missed SLA erodes trust in the pipeline and in you – a conservative SLA that's always met builds more credibility than an aggressive one that breaks monthly.

Don't confuse desire with willingness to pay

I had a client who wanted 15-minute maximum delay on their invoicing data. They weren't willing to pay the increased BigQuery bill, and their source had terrible metadata, hard deletes, and no reliable cursor – making high-frequency extraction expensive to build and expensive to run. After scoping the effort and cost, they realized all they actually needed was one extra on-demand refresh per day. The Head of Sales wanted fresh numbers on his dashboard mid-morning, and a refresh button that triggered the pipeline solved the problem at a fraction of the cost and complexity. Ask what decision the freshness enables before engineering the SLA around it.

6.4.8 What Comes Next

6.5 – Alerting and Notifications covers the mechanics of turning SLA breaches into alerts – the thresholds defined here are the input, and 6.5 – Alerting and Notifications decides who gets paged, how, and at what severity.

6.5 Alerting and Notifications

> **One-liner:** Schema drift, row count drops, partial failures – calibrate severity so not everything is an incident.

6.5.1 Fatigue vs. Blindness

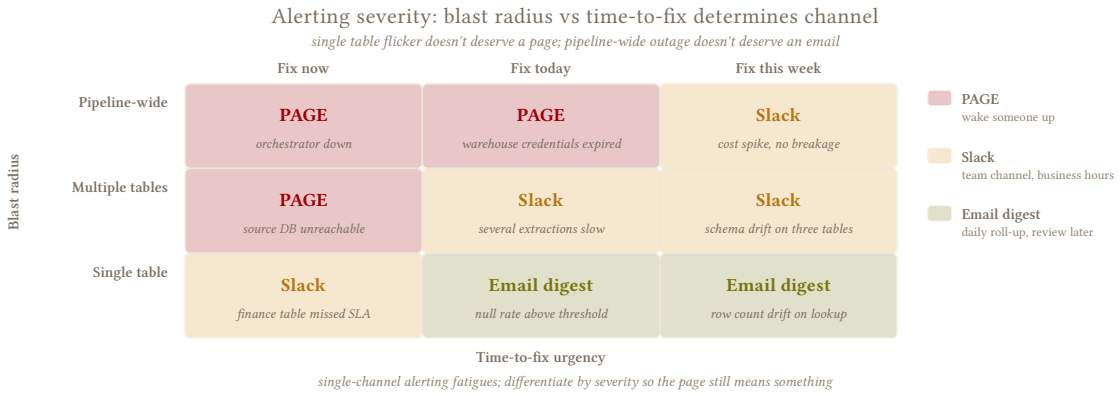
Pipelines fail silently. Zero rows extracted successfully, schema changed upstream, row counts drifting apart between source and destination – all of these can happen while the orchestrator reports SUCCESS. The monitoring layer from 6.1 – Monitoring and Observability and the health table from 6.2 – The Health Table capture these signals; this pattern is about deciding which of them deserve to wake someone up.

The calibration problem has two failure modes. 1. Too many alerts – every run sends a notification, every minor discrepancy triggers a warning – produces alert fatigue, and alert fatigue produces ignored alerts, and ignored alerts produce missed failures. 2. Too few alerts – only page on total outages – means silent data loss accumulates for days before anyone notices.

The goal is a narrow band between the two: alert on conditions that require human attention, monitor everything else on the dashboard – loud enough that silence is genuinely reassuring.

6.5.2 Severity Calibration

Not every failure is equally urgent, and not every table is equally important. A load failure on `orders` during month-end close is a different severity than a stale `item_groups` lookup table on a Saturday. Calibrate on two axes: what broke and how much the table matters.



Severity	Condition	Example
Critical	Destination data lost or significantly diverged from source	Table empty after load, row count dropped 80%, source/destination totals diverged beyond recovery
Error	Load failed, destination stale, SLA breach	Permission denied, query timeout, staleness exceeds SLA from 6.4 – SLA Management
Warning	Anomaly detected but data is present and current	Row count drop > threshold, schema drift (new columns), extraction duration 3x historical average
Info	Nothing wrong	Successful run, counts in range, no drift. Log it, dashboard it, never notify

Table importance is the second axis. Sales and receivables tables failing during end-of-month is critical; a dimension lookup table being 2 hours stale is a warning at most. Classify tables into importance tiers and let the combination of condition severity and table importance determine the alert routing – a WARNING on a critical table might route the same as an ERROR on a low-priority one.

6.5.3 What to Alert On

The rule: alert on things that need human attention before the next morning's monitoring review. At scale – thousands of tables – you can't afford to alert on every condition the pipeline doesn't handle automatically, because there are too many tables where a failure simply doesn't matter overnight. A warehouse dimension table that gets a new row every six months doesn't need to page anyone when it fails on a Tuesday; it'll still be there in the morning. The filter is urgency, not just "unhandled."

If the pipeline already has a pattern that resolves the condition – retry logic, automatic schema evolution, reconciliation with auto-recovery – the alert is redundant. Log and monitor it, but don't notify if there's no urgency for its solution.

6.5.3.1 Always Alert

These are conditions where waiting until morning costs you something real – data loss that compounds, costs that keep burning, or downstream consumers already seeing wrong results. Even here, table importance matters: a load failure on `orders` during month-end close is a page, the same failure on a warehouse lookup table is a line on tomorrow's dashboard.

Data didn't arrive and it matters now – load failure (quota exceeded, permission revoked, timeout) or extraction error on a table that was healthy yesterday. The distinction between "load rejected" and "source query failed" matters for triage but not for urgency – either way, the destination is stale and nothing will fix it automatically. The health table's `status = 'FAILED'` with `error_message` gives you the starting point. Don't confuse extraction errors with "returned 0 rows," which can be normal for quiet incrementals (6.10 – Extraction Status Gates).

SLA breach on a table with consumers waiting – staleness exceeds the threshold defined in 6.4 – SLA Management, and duration is trending in the same direction. A breach means someone downstream is already affected or about to be; check whether it's duration creep, an upstream delay, or a schedule that needs adjustment. Duration anomalies that haven't breached an SLA yet are an early warning – worth surfacing as a warning, not a page, unless the trajectory makes the breach inevitable.

Partial failure across a dependency group – some tables loaded, others didn't, and the successful ones depend on the failed ones or vice versa. This is particularly dangerous because the overall run may report partial success and fly under the radar (6.12 – Partial Failure Recovery). Isolated failures on independent tables can wait for morning; failures that leave the destination in an inconsistent state can't.

Cost spike – daily compute cost exceeds threshold (6.3 – Cost Monitoring). A runaway MERGE or an unpartitioned scan keeps burning money every run until someone intervenes, so this is one of the few conditions where urgency is about the pipeline itself rather than the data.

6.5.3.2 Alert Only When Unhandled

These conditions may or may not need attention depending on two filters: whether the pipeline has automatic recovery, and whether the table's importance justifies a notification over a dashboard entry.

Row count deviation – if the table uses hard-delete detection (3.6 – Hard Delete Detection) or reconciliation with auto-recovery, the pipeline handles it. Alert when the discrepancy exceeds the threshold *and* no automatic pattern resolves it (6.14 – Reconciliation Patterns). On low-importance tables, even an unhandled deviation can wait for the morning review.

Schema drift is nuanced. New columns with an `evolve` policy are accepted automatically – log them, don't alert. Dropped columns deserve an alert even with `evolve`, because a missing column can break downstream queries silently and an `evolve` policy should reject column removal anyway. Type changes depend on direction: widening (INT → BIGINT) is usually safe; narrowing or type-class changes (INT → VARCHAR) are probably a problem. See 6.9 – Data Contracts for the policy framework.

6.5.3.3 Never Alert

Successful runs. Log them, put them on the dashboard, never send a notification. If you get a "success" message for every table on every run, you'll have hundreds of Slack messages per day and you'll stop reading any of them.

Zero rows on an incremental – quiet periods are normal. The cursor is caught up or the source had no changes. This is a data health metric in the health table, not an alert condition.

Minor reconciliation discrepancies within the configured threshold – a 0.05% drift on a busy table is likely to be fixed next run, don't alert but keep it in mind in your dashboard.

Failures on tables that can wait – a warehouse dimension table that gets a new row every six months, a lookup table with no downstream SLA, a staging table for a report that runs weekly. These are real failures that need fixing, but they're morning-coffee problems, not pager problems. The dashboard and health table surface them; a notification adds nothing but noise.

6.5.4 Alert Channels

Route by severity, not by table. Critical alerts go to the pager or a DM – something that demands immediate attention. Warnings go to a Slack channel where they're visible but not intrusive. Info stays on the dashboard where it's available on demand but never pushes a notification.

Your orchestrator's alerting layer handles the routing – configure severity-based rules, not per-table rules. If you find yourself managing per-table routing for more than a handful of exceptions, the severity classification isn't doing its job.

Every alert should tell the responder what to do next – or at least where to look. "Row count anomaly on `events`" is not actionable; the person reading it doesn't know if the anomaly is a 5% dip or a 90% drop, whether it's expected (month-end spike subsiding) or a real problem, or who should investigate. Include the metric value, the threshold it crossed, and a pointer to the relevant health table query or dashboard view. An alert that doesn't guide triage is just noise with a timestamp.

> **Pre-filter before you fix**
>
> When multiple tables fail overnight, resist the urge to investigate all of them at once. Filter to critical failures first, fix those, then work down to warnings. A critical failure on `orders` that blocks month-end reporting matters more than a warning on `products` with a new column. If you try to process every alert in arrival order, the important ones get buried and you burn your morning on problems that could have waited.

6.5.5 Tradeoffs

Pro	Con
Severity tiers prevent alert fatigue	Requires upfront classification of tables and conditions
"Alert only when unhandled" reduces noise	Under-alerting is a real risk if the automatic recovery pattern has a bug
Channel routing keeps critical alerts visible	Warning thresholds need periodic tuning as tables grow and patterns change

6.5.6 Anti-Patterns

> **Don't use one severity for everything**
>
> Schema drift on a lookup table and a total load failure on `orders` are not the same event. If everything is "Error," nothing is – the on-call engineer can't prioritize and will eventually stop responding to any of them.

Don't alert without an escalation path

A warning that persists for 3 consecutive days is no longer a warning – it's either a real problem being ignored or a miscalibrated threshold. Build automatic severity promotion: warning → error after N consecutive violations. If a threshold triggers daily and nobody investigates, the threshold is wrong, not the data.

6.5.7 What Comes Next

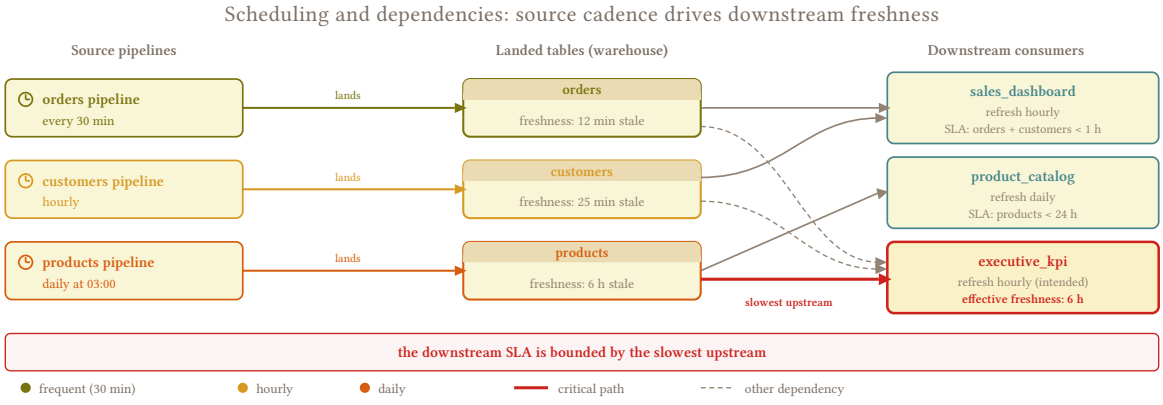
6.6 – Scheduling and Dependencies covers the scheduling layer that determines when pipelines run and in what order – the timing decisions that directly affect whether SLAs from 6.4 – SLA Management are achievable and which alert conditions fire.

6.6 Scheduling and Dependencies

One-liner: Most tables are independent. For the ones that aren't, group them so they update together – but don't enforce strict ordering unless you have a real reason.

6.6.1 Frequency vs. Method

With a handful of tables, scheduling is simple: run everything on a cron, wait for it to finish, done. At hundreds or thousands of tables, three questions dominate your scheduling decisions – how often each table should update, how many extractions your source and infrastructure can handle at once, and which tables need to land in the same window. Get any of these wrong and the consequences are immediate: SLA breaches because heavy tables crowd out critical ones, angry DBAs because you're hammering their production system during business hours, or a pipeline that takes six hours because someone chained 200 independent tables into a single sequence years ago and nobody questioned it.



6.6.2 The Pattern

6.6.2.1 How Often: Schedule Frequency

Every table needs a schedule, and the schedule should reflect how the table is consumed, not how often the source changes. A `customers` table that changes ten times a day but feeds a weekly report doesn't need hourly extraction – once a day is fine. An `orders` table that feeds a real-time dashboard needs to update as frequently as your source and infrastructure can sustain. Watch for schedule pile-ups as tables grow: an extraction that used to finish in 10 minutes may creep to 40 and start overlapping with the next scheduled run, silently turning two clean windows into one messy one.

6.8 – Tiered Freshness covers the framework for assigning freshness tiers. The scheduling implication is straightforward: group tables by the freshness their consumers need, not by their source system or their size.

Most teams evolve through a predictable sequence:

1. **Single cron** – everything runs together. Simple but slow, works when you have few tables and falls apart when it takes longer than ~4 hrs or you need to update within the day.
2. **Weight-based groups** – tables split by size or duration, distributed across time slots. Better throughput, but the groupings don't map to anything the business cares about.
3. **Consumer-driven groups** – tables grouped by the downstream report or dashboard that consumes them, scheduled to meet that consumer's freshness target. If the sales report goes live at 8 AM, its tables update at 6:30. If the warehouse team doesn't check inventory until noon, those tables can run later and spread the source load across a wider window.

The third stage is where you want to end up, but each stage is the right answer at a certain scale – don't over-engineer a consumer-driven architecture when you have three dashboards.

> **Group by consumer, not by source**
>
> Early designs tend to group tables by source connection – "all SAP tables run at midnight." That works until the finance team needs invoice data at 7 AM while the warehouse team doesn't check inventory until noon. Grouping by consumer lets you schedule tighter windows for the tables that matter most and spread the rest across off-peak hours.

6.6.2.2 How Many: Concurrency and Source Load

Every concurrent extraction consumes RAM and CPU on your pipeline infrastructure *and* an open connection plus query load on the source. Getting the concurrency level wrong hurts in both directions: too few concurrent extractions and your pipeline takes hours longer than it should, too many and you overload the source system or exhaust your own memory.

Start conservative – 3 to 5 concurrent extractions per source for a typical transactional database. Larger production setups might run up to 8 tables concurrently against a well-provisioned source, but mostly during off-peak hours when the source has headroom. Monitor source response times and pipeline memory, and increase the limit only when you have evidence that both sides can handle it.

The mechanism is your orchestrator's concurrency controls – run queues, tag-based limits, or pool-based workers. The limit itself comes from knowing your environment: what the source can tolerate and what your infrastructure can sustain.

Set concurrency per source, not per pipeline

Concurrency limits should be set per source system, not per schedule. If three schedules each run 5 extractions against the same database, that's 15 concurrent queries – the source doesn't care that they came from different schedules. However, always keep in mind your orchestrator's limit as a general maximum of available operations.

Lock contention on SQL Server

Some databases handle concurrent reads worse than others. SQL Server in particular can lock tables during long reads, blocking the source application's writes. The usual workaround is `WITH (NOLOCK)`, which avoids locks but introduces dirty reads – rows mid-transaction, partially updated, or about to be rolled back. I've seen dirty reads lead to erroneous business decisions when an in-flight transaction appeared as committed data in the destination. Schedule heavy SQL Server extractions for off-peak hours rather than reaching for `NOLOCK`, and if you must use it, document the risk so downstream consumers know what they're looking at. Please de-duplicate on source, since batched loads with `NOLOCK` can repeat records even when having enforced primary keys.

6.6.2.3 When: Safe Hours

Large extractions during business hours can slow down or even lock the source system (see 6.7 – Source System Etiquette). Gate heavy extractions behind a safe-hours window – typically off-peak, like 19:00 to 06:00 – with a row-count or size threshold that determines which tables qualify as "heavy." Tables below the threshold run during business hours on their normal schedule; tables above it get deferred to the safe window automatically.

A threshold around 100,000 rows is a reasonable starting point, set proactively before an incident forces the decision. The exact number depends on the source – a well-provisioned cloud database tolerates larger reads during business hours than an on-prem ERP running on aging hardware.

Safe hours are per source

If three pipelines each respect their own safe-hours window against the same source, they might all stack into the same off-peak slot. Coordinate safe hours at the source level: one window, one concurrency limit, shared across every pipeline that touches that source.

6.6.2.4 Which Together: Grouping Related Tables

Most tables in a pipeline are independent – `customers` and `events` share no relationship that affects extraction, and there's no reason they need to land at the same time. The few that *are* related – header-detail pairs like `orders`/`order_lines` or `invoices`/`invoice_lines` – should land in the same schedule window so the destination doesn't show today's headers with yesterday's lines.

Within that window, arrival order shouldn't matter. Make sure no table depends on the other's data being present at load time so that joins work regardless of which side finished first. What matters is that both sides reflect roughly the same point in time, which co-scheduling achieves naturally without any dependency graph.

Lookup tables like `customers` and `products` ideally land before `orders` so a consumer querying right after the load sees consistent references, but if `products` is 30 minutes stale while `orders` is fresh, the join still works – the data is slightly behind, not broken. Express this as a preferred ordering in your orchestrator if it supports it, but don't block `orders` on `products` completing unless you want slower loads.

The only time you need strict ordering is when one extraction's *input* depends on another extraction's *output* – which is uncommon because each table is extracted independently from the source. If you do have this case, express it as a real dependency in the orchestrator's DAG, but confirm you actually need it before building the graph.

6.6.2.5 DAG vs. Schedule Groups

For the vast majority of table relationships, co-scheduling is enough: put related tables on the same cron, let them run concurrently within the window, done. No dependency graph, no ordering logic.

Reserve DAG-based dependencies for actual extraction-feeds-extraction cases or for coordinating with downstream semantic transformations that must wait for a group of tables to complete. Building a 200-node extraction DAG when 190 of those nodes are independent is complexity that buys nothing – and a fragile DAG where one table's failure cascades into blocking dozens of unrelated tables is worse than no DAG at all.

If your orchestrator can't group tables into a single schedule that runs them concurrently, that's a serious limitation – grouping related tables for parallel extraction within a window is a basic scheduling requirement, and working around it with cron offsets (`orders` at 6:00, `order_lines` at 6:15) is fragile enough that it should push you toward a better orchestrator rather than deeper into workarounds.

6.6.3 Tradeoffs

Pro	Con
Schedule groups keep related tables coherent without strict ordering	Consumers may briefly see one side of a relationship fresher than the other within the window
Consumer-driven grouping aligns freshness with business needs	Tables needed by multiple consumers may run on multiple schedules, increasing source load
Conservative concurrency limits protect the source	Lower concurrency means longer total pipeline duration
Safe-hours gating prevents source impact during business hours	Heavy tables only update during the off-peak window, which may not meet freshness SLAs

6.6.4 Anti-Patterns

> **Don't serialize everything**
>
> Running 200 tables sequentially "just to be safe" because "it's simpler" turns a 30-minute pipeline into a 6-hour one. Most tables are independent and can run concurrently within your concurrency limits – group the few that are related, and only add explicit dependencies when one extraction actually needs another's output.

Don't model FKs as extraction dependencies

Source tables have foreign keys; that doesn't mean your extraction needs to respect their ordering. The destination's landing layer doesn't enforce FKs, and joins work regardless of which side arrived first. Treating every FK as a hard dependency turns simple co-scheduling into a fragile DAG that blocks unrelated tables on each other.

Don't use sleep as a dependency

"Wait 10 minutes for orders to finish" is a guess that breaks the first time extraction duration changes. Use schedule groups or the orchestrator's native dependency graph.

Don't assume your limit is theirs

Your orchestrator might allow 20 parallel tasks, but the on-prem database you're extracting from might buckle under 8. The constraint is always the weakest link – your infrastructure *or* the source, whichever gives first. Test against the actual source before increasing limits.

6.7 Source System Etiquette

> **One-liner:** Your pipeline is a guest on someone else's production database. Act like it.

6.7.1 Guest on Production

READ-ONLY access doesn't mean zero impact. A full table scan on a 50-million-row table locks pages, consumes I/O, and competes with the application for CPU and memory – and the DBA watching the monitoring dashboard doesn't care that your query is a harmless SELECT. Their job is to keep the application fast for the users who generate revenue; your extraction is a background process that, from their perspective, exists only to slow things down. If you're careless about when and how you extract, you'll lose access – and if you're unlucky, you'll bring the database down on your way out.

I had a client whose IT team didn't mention they ran full database backups between 5 and 6 AM. A load failed overnight, and the automatic retry kicked in at 5:30 AM – right on top of the backup window. The database went down. It was back up within the hour, but the conversation about revoking my access lasted a week. The retry logic was fine – I just didn't know the source's maintenance windows.

6.7.2 Know Your Source

The sensitivity of a source system determines how carefully you need to tread. Before writing the first extraction query, understand what you're connecting to:

Production OLTP – a live transactional database serving the application's users. Every query competes with their transactions. Full scans lock pages, long reads block writes on some engines (see the SQL Server warning in 6.6 – Scheduling and Dependencies), and a bad retry at the wrong time can cascade into an outage. Treat these with maximum care: off-peak scheduling, conservative concurrency, explicit timeouts.

Read replica – lower sensitivity, but not zero. Replicas share storage I/O with the primary or lag behind it on the same hardware. A full scan on a replica can saturate disk throughput, increase replication lag, and degrade the primary indirectly. Treat replicas with the same patterns, just wider tolerances – more concurrent queries, wider safe-hours windows.

Vendor-controlled ERP – systems like SAP where the vendor owns the schema and you have no leverage to change it. You can't add indexes, you can't create views, and the timestamp columns your incremental queries need were designed for the application's audit trail, not for your `WHERE` clause. Tread carefully and accept that some extractions will be slower than you'd like.

6.7.3 The Pattern

6.7.3.1 Check Your Cursor Columns

`updated_at`, `UpdateDate`, `CreateDate` – the columns your incremental queries filter on exist for the application, not for your extraction. Check whether they're indexed before assuming your `WHERE updated_at > :cursor` will be fast. If they're not indexed, you're forcing a full table scan every run, and the DBA will notice before you do.

Ask the DBA to add an index. This is more achievable than it sounds – adding an index on a timestamp column is a low-risk change that benefits anyone querying by date, and technical stakeholders on the source side often stand to gain from it too. I've had clients proactively add indexes after noticing my scans were slow, before I even asked. It's a soft rule – officially read-only, but the performance improvement is large enough that most DBAs will cooperate.

If they can't add an index – vendor-controlled schemas sometimes make this difficult or unsupported – schedule those extractions for off-peak hours and accept that the scan will be heavier than ideal (see 6.6 – Scheduling and Dependencies, safe hours).

6.7.3.2 Respect Business Hours

Whether extraction load during business hours is a problem depends entirely on the database and the client. Some clients proactively ask for intraday updates and are willing to absorb the source load. Others will escalate immediately if they see any query from your pipeline during working hours. This is a conversation for the SLA stage (see 6.4 – SLA Management) – agree on what hours are acceptable before the pipeline goes live, not after the first complaint.

As a baseline: small incremental pulls during business hours are usually fine on a healthy source, because the query filters on a recent cursor and touches a small number of rows. Full table scans and backfills are a different story – they read the entire table and should be gated behind a safe-hours window. Enforce this automatically for very weak or very massive databases by deferring tables above a row-count threshold to the off-peak window (see 6.6 – Scheduling and Dependencies). Sources in the middle ground – decent hardware, moderate table sizes – generally don't need the gate.

> **Know the source's maintenance windows**
>
> Backup jobs, index rebuilds, integrity checks – these run during off-peak hours too, which means your "safe" extraction window may overlap with the source's heaviest internal workload. Ask the DBA for their maintenance schedule and avoid stacking your largest extractions on top of their backup window.

6.7.3.3 Limit Concurrency

Multiple parallel extractions against the same source multiply the load. Cap concurrent connections per source system – not per pipeline or per schedule, because the source doesn't care which schedule spawned the query. 3 to 5 concurrent extractions is a reasonable starting point for a typical transactional database; tune based on the DBA's feedback and the source's monitoring (see 6.6 – Scheduling and Dependencies for the full treatment of parallelism tradeoffs).

6.7.3.4 Set Timeouts

Set query timeouts explicitly. A query that runs for hours without a timeout is holding a connection, consuming source resources, and probably blocking something. When a query times out, fail the table explicitly (see 6.10 – Extraction Status Gates) – don't retry immediately, because the condition that caused the timeout is likely still present.

Timeout thresholds depend on whether you're reading in batches. For unbatched reads, keep timeouts tight: a few minutes for regular tables, longer for known large ones. For batched reads (see below), individual batch timeouts can be shorter since each batch is small, while the overall extraction can run for hours.

6.7.3.5 Batched Reads for Massive Tables

For tables too large to extract in a single query within a reasonable time, SQLAlchemy's `yield_per()` with `stream_results=True` lets you read in batches using a server-side cursor. Each batch is small and fast – 100,000 rows is a solid default – even if the full read takes hours. This keeps your pipeline's memory flat (you're never holding millions of rows at once, just the current batch) and reduces the per-query impact on the source.

The tradeoff: you hold an open connection and server-side cursor for the entire duration, so the source is occupied for longer even though the per-second load is lighter. Schedule batched reads for off-peak hours, and make sure the source's connection pool can accommodate a long-lived session alongside normal application traffic.

```
# Batched read: 100k rows at a time, server-side cursor
# source: transactional (SQLAlchemy)
with engine.connect() as conn:
    result = conn.execution_options(
        stream_results=True
    ).execute(
        text("SELECT * FROM orders")
    )
    for batch in result.yield_per(100_000):
        load_batch(batch)
```

6.7.4 What You Can and Can't Do

Never: triggers, stored procedures, temp tables, or writes of any kind on someone else's production database. You are a reader.

Schema modifications – officially off-limits without DBA approval, but adding an index is worth asking for. It's a low-risk change with high payoff, and framing it as a performance improvement that benefits the application (not just your pipeline) makes the conversation easier.

Views – useful when downstream needs a subset of data that would be expensive to reconstruct from base tables. The recommended approach: build the query in your destination first, validate it works, then send the "translated" query to the DBA and ask them to create the view on the source. This keeps the DBA in control of their schema while giving you a stable, optimized read target.

6.7.5 Building Trust with the DBA

The relationship with the source team determines how much access you keep and how much flexibility you get. A DBA who trusts your pipeline will add indexes, extend your safe hours, and warn you before maintenance windows. A DBA who doesn't trust you will restrict your hours, throttle your connections, and eventually revoke access.

Share your schedule – what you extract, when, how often, how much data. No surprises.

Report your own impact – query duration, rows scanned, connection time. If you can show the source team that your extraction uses a small fraction of their database's capacity, you've answered the question before they ask it. The source health metrics from 6.1 – Monitoring and Observability give you the numbers.

Own your incidents – when your query causes a slowdown, acknowledge it and fix the schedule before they have to ask. Nothing destroys trust faster than a DBA discovering your pipeline caused an issue and you didn't notice or didn't say anything.

6.7.6 Anti-Patterns

> **Read replicas still matter**
>
> Read replicas share storage or lag behind the primary on the same hardware. A full scan on a replica can saturate disk I/O, increase replication lag, and affect the primary indirectly. Treat replicas with the same patterns, just wider tolerances.

> **Don't retry extractions blindly**
>
> A retry that hits the source during a backup window, a peak traffic period, or while the condition that caused the failure is still present makes things worse, not better. Retry logic should respect safe hours and back off on repeated failures rather than hammering the source immediately.

Don't assume the DBA knows you exist

If nobody on the source team knows your pipeline connects to their database, the first time they find out will be during an incident – which is the worst possible time to introduce yourself. Establish the relationship before you go live.

6.8 Tiered Freshness

One-liner: Split a large table's extraction by time range – recent data fast and often, historical data slow and pure.

6.8.1 One Size Fits None

I had an `orders` table that ran a full replace of the entire year's data many times a day. The frequency was right – the table needed intraday updates – but full-replacing twelve months of data every run was killing the source. The DBA noticed before I did. The fix was splitting the extraction by time range: recent data incrementally and often, historical data via partition swap nightly, and a weekly full replace as the safety net.

Most tables never need this – a `products` lookup with 10k rows can full-replace every run without anyone noticing, and full replace at whatever cadence the consumer needs is always simpler and purer. Tiers earn their complexity when a table is too large or too expensive to full-replace at the frequency consumers demand.

6.8.2 The Tiers

The model is three zones, each with its own cadence and extraction method. The boundaries between them depend on the table, the source system, and the consumer's SLA – the names are universal, the numbers are not.

6.8.2.1 Hot (Intraday)

The recent slice – today's `orders`, open `invoices`, the last week of `events`. Refreshed multiple times per day via incremental extraction with a lag window (3.2 – Cursor-Based Timestamp Extraction). The hot tier tolerates impurity: slight gaps from late-arriving data or cursor lag are acceptable because the warm tier catches them on its nightly pass.

6.8.2.2 Warm (Daily)

Current month or current quarter – data that still receives occasional updates but not at high frequency. Refreshed daily, often overnight when the source is under less load. The extraction method is either a partition swap of the warm window (2.5 – Rolling Window Replace, 2.2 – Partition Swap) or incremental with a wider lag.

This tier takes advantage of harder business boundaries – a closed month in an ERP is unlikely to change (though "unlikely" is not "impossible," see 1.6 – Hard Rules, Soft Rules). The warm tier re-reads recent history with enough depth to catch what the hot tier missed: late cursor updates, backdated transactions, documents that changed without bumping `updated_at`. After the warm pass, source and destination should match within the warm window.

6.8.2.3 Cold (Weekly / On-Demand)

Historical data: prior years, closed fiscal periods, archived partitions. Refreshed on a slow cadence – weekly, monthly, or only on demand for backfills and corrections. Full replace is the right method here because the volume is bounded and the frequency is low enough that the cost is negligible.

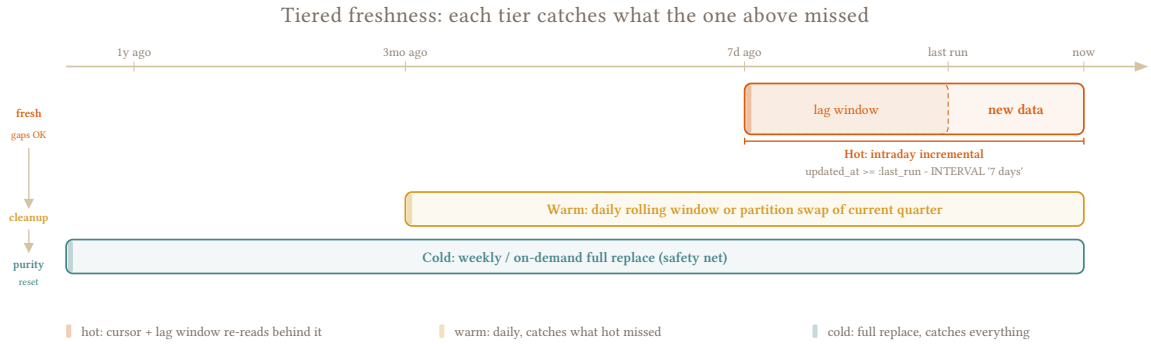
A weekly full replace resets accumulated drift from the hot and warm tiers – any row that was missed by a cursor, any late update that arrived outside the warm window, gets picked up here. The cold tier is the purity checkpoint from 1.8 – Purity vs. Freshness applied on a schedule.

6.8.2.4 The Lag Window

The hot tier's incremental extraction doesn't just grab rows since the last run – it reads a trailing window behind the cursor (`updated_at >= :last_run - INTERVAL '7 days'`). That overlap catches late-arriving data, backdated corrections, and rows the cursor missed on previous runs. Without it, anything that lands behind the cursor after the hot run is permanently invisible until the warm or cold tier picks it up.

The right lag depends on how reliably the source system updates its cursors. For well-organized systems where every modification touches `updated_at`, 7 days covers the common cases. For messier systems where documents get modified without updating any cursor (common in ERPs where back-office edits bypass the application layer), 30 days is safer. The decision is empirical: start at 7, watch for rows that appear in the warm or cold tier's pass but were never picked up by hot, and widen the window if it happens regularly.

The warm tier reads its entire scope (current quarter) on every daily run, so its overlap with the hot zone is inherent – no separate lag parameter needed. The cold tier's full replace covers everything by definition. Each tier downstream is the safety net for the tier above it.



6.8.3 When Tiers Apply

An `orders` table with years of history can't full-replace every 15 minutes – so you split the extraction by time range: intraday incremental on the recent slice (hot), nightly partition swap on the current quarter (warm), weekly full replace on the entire table (cold). Three strategies on the same table, each covering a different time range at a different cadence. About two-thirds of tables in a typical pipeline never need this

split – full replace at the consumer's cadence handles them fine. Reserve tiered extraction for the tables where size or cost forces the compromise.

6.8.4 Month-End and Seasonal Shifts

ERP systems behave differently at month-end and period close. Whether that affects your tiered schedule depends on who consumes the data and why.

If the extracted data drives quick decision-making – collections teams chasing receivables before month-end, sales managers tracking targets – consumers will ask for *more* frequency. Widening the hot tier's extraction window or increasing its cadence during the last week of the month gives them fresher data when the stakes are highest.

If the extracted data feeds a historical analysis engine – a data warehouse that produces reports after the period closes – consumers will often ask for the *opposite*: reduce extraction frequency during month-end to avoid competing with the ERP's own close process for database resources. The source system is already under pressure from period-end batch jobs, and your pipeline hammering it with intraday reads doesn't help anyone.

For pipelines that run overnight only, month-end rarely changes the schedule. The overnight window already avoids the daytime contention, and the warm tier's daily refresh picks up whatever happened during the close.

6.8.5 Schedule Configuration

Each tier maps to a separate schedule or schedule group in your orchestrator:

- **Hot**: frequent cron, interval driven by table volume and source tolerance
- **Warm**: daily cron, typically overnight
- **Cold**: weekly or monthly cron, or triggered manually for backfills

The tier boundaries shift with business cycles. Month-end might widen the hot window or increase its cadence; fiscal year rollover pushes last year's partitions from warm into cold territory; seasonal patterns (Black Friday, harvest season, enrollment periods) can temporarily justify more aggressive extraction on tables that are normally daily. If your orchestrator supports dynamic schedule assignment, encode these transitions as rules rather than manual changes.

> **Don't mix tiers on the same cron**
>
> If the hot extraction shares a cron with everything else, it waits in line behind cold tables that didn't need refreshing. Separate the schedules so the hot tier runs independently at its own cadence.

> **Same frequency, wrong method**
>
> Refreshing a table many times a day is fine. Full-replacing a year's worth of data many times a day is not. If a table needs intraday freshness, the hot tier should extract only the recent window incrementally – not reload the entire history on every run. The frequency is a schedule concern; the method is a pattern concern. Getting one right and the other wrong is how you end up on the phone with the DBA.

6.8.6 Tradeoffs

Pro	Con
Hot data gets to consumers faster without over-refreshing cold data	Three schedules to configure and monitor instead of one
Cold-tier full replace as the purity checkpoint, resetting drift	Lag window tuning is empirical – too short misses rows, too long wastes reads
Tier boundaries shift with business cycles (month-end, seasonal)	Adjusting cadence and window size requires orchestrator flexibility
Cost scales with actual freshness needs, not with table count	Month-end and seasonal shifts require manual or rule-based schedule changes

6.9 Data Contracts

> **One-liner:** Schema drift, row counts, null rates, freshness – what to enforce at the boundary between source and destination.

6.9.1 Schema Drift

Source schemas change without notice. A column gets renamed, a type changes from INT to VARCHAR, a new column appears when someone activates an ERP module, an old one disappears after a migration. The source team doesn't know your pipeline exists – they won't tell you before they deploy a schema migration, and they shouldn't have to. The boundary between their system and yours is your responsibility to defend.

Without a contract, drift propagates silently into the destination – a dropped column becomes NULLs in downstream queries, a type change produces casting errors that surface three layers deep in a dashboard nobody connects back to the source, and a 90% row count drop looks like a quiet day until someone notices the month-end report is missing most of its data. By then the blast radius is wide and the root cause is buried – a data contract makes these boundaries explicit and checkable before the damage spreads.

6.9.2 What a Data Contract Covers

6.9.2.1 Schema Contract

The schema contract defines the expected column names and types – the fingerprint from 6.1 – Monitoring and Observability. It answers three questions when the schema changes:

New columns – accept or reject? The policy is either evolve (add the column to the destination) or freeze (fail the load). Evolve is the right default for almost every table. Source schemas grow – ERPs add columns when modules are activated, applications add fields as features ship. Freezing a schema that legitimately evolves means a manual intervention every time the source team deploys, which is maintenance you don't want and they won't coordinate with you on. Evolve means one less thing to manage, and downstream

consumers shouldn't be doing `SELECT *` against your destination anyway – an added column doesn't break anything for them unless they wrote their queries wrong.

Dropped columns – no decision gets made without the source system. A column disappearing could be a deliberate removal, a migration gone wrong, or a temporary rollback. Set up tolerances: if the column was created yesterday and disappeared today, it was probably a rollback and you can let it go. If a column that's been there for months vanishes, fail the load and investigate. The tolerance depends on how the downstream uses the column – a critical join key disappearing is different from an unused description field being cleaned up.

Type changes – fail, cast, or warn. See 6.9.6 – Type Mapping below for how to handle the mapping itself.

6.9.2.2 Volume Contract

The volume contract defines the expected row count range per extraction, derived from recent history. A table that normally extracts 450k rows and today extracts 12k likely has a problem – even if the pipeline reports SUCCESS. The contract surfaces this before the data reaches consumers.

The threshold should come from observed baselines, not assumptions. A simple approach: track the rolling average and standard deviation of row counts over the last 30 runs, and alert when the current run falls outside 2-3 standard deviations. For tables with predictable seasonality (month-end spikes on `invoices`, weekend dips on `orders`), factor the day-of-week or day-of-month into the baseline.

This feeds directly into 6.10 – Extraction Status Gates for inline enforcement – block the load when the volume looks wrong, rather than discovering the problem downstream.

6.9.2.3 Null Contract

The null contract defines expected null rates on key columns. A cursor column like `updated_at` should never be NULL – if it is, your incremental extraction is blind to those rows. A description column being 40% NULL is probably normal. The contract distinguishes between the two.

The purpose is to protect your pipeline's ability to do its job. A null rate spike on `updated_at` disrupts your extraction; a null rate spike on `customer_name` is the source's problem and downstream's concern. Anything that disrupts your ability to extract and load accurately is alertable. Everything else passes through as-is.

6.9.2.4 Freshness Contract

The freshness contract is the SLA from 6.4 – SLA Management expressed as a checkable rule: maximum acceptable staleness per table, measured from the health table's last successful load timestamp. This is the simplest contract to define and the most visible when violated – a stale table is the one that generates the "why hasn't the dashboard updated" email.

6.9.3 Enforcement Points

6.9.3.1 Pre-Load (Gate)

Check schema, row count, and null rates after extraction but before loading. If the contract is violated, block the load and alert (6.5 – Alerting and Notifications). This is the extraction status gate from 6.10 – Extraction Status Gates extended with richer checks.

Pre-load gates are the strongest enforcement point because they prevent bad data from reaching the destination. The cost is that a false positive blocks a load that was actually fine – which is why baselining

matters. A gate based on assumptions ("this column should never be NULL") fires on the first run and trains you to ignore it.

6.9.3.2 Post-Load (Validation)

Run checks after the load completes: destination row count vs source, schema matches expected, null rates within bounds. Your orchestrator's post-load check primitives are built for this – ideally run them as part of the load job so the check and the data it validates stay in sync. At scale, though, the overhead of inline checks on every table may not fit in the schedule window (see 6.9.3.4 – The Cost of Checking), and running validation on a separate, less frequent cadence becomes the practical tradeoff: you lose immediate detection but keep the pipeline on time.

Post-load validation catches problems that pre-load gates can't see: rows that were lost during the load itself, type coercions that silently truncated values, partition misalignment that put data in the wrong place. The tradeoff is that by the time you detect the problem, the bad data is already in the destination – you're limiting blast radius rather than preventing damage.

6.9.3.3 Continuous (Monitoring)

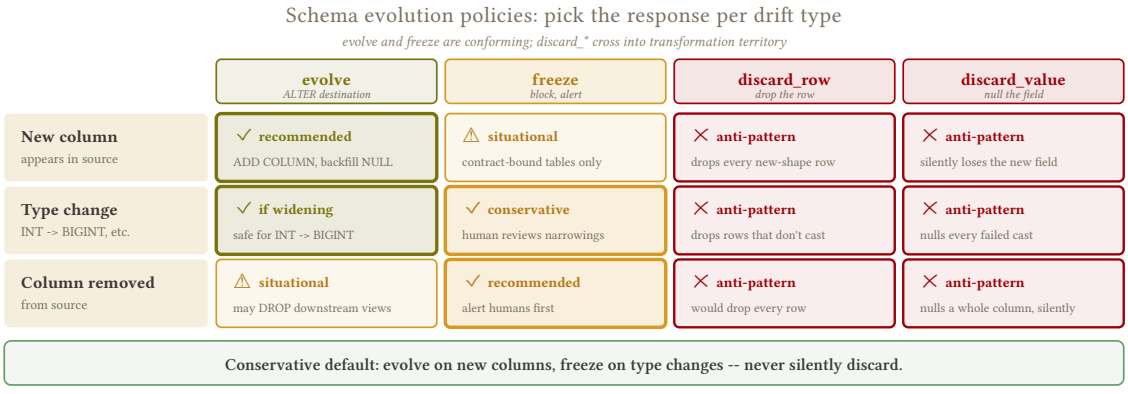
Schema fingerprint comparison on every run, volume trend tracking over time. This feeds the observability layer from 6.1 – Monitoring and Observability and catches slow drift that no single-run check would flag: a table whose row count grows 2% less than expected every week, a column whose null rate creeps from 0.1% to 5% over a quarter.

6.9.3.4 The Cost of Checking

Every contract check adds overhead to every run. A schema fingerprint comparison, a row count validation, a null rate scan – each one might take 10 or 15 seconds on its own, barely noticeable on a single table. Multiply that by 1,000 tables and you've added over 4 hours of load time to your pipeline. The contracts that felt free at 20 tables become a bottleneck at scale.

Contract coverage is a budgeting decision. Not every table needs every check. A critical `orders` table might deserve schema + volume + null rate validation on every run. A 200-row lookup table probably doesn't need anything beyond the run health your orchestrator already provides. Allocate checks where the blast radius of a silent failure justifies the overhead, and leave the rest to the monitoring layer where the cost is amortized across a dashboard glance, not multiplied across every load.

6.9.4 Schema Evolution Policies



Policy	Behavior	When to use
Evolve	Accept new columns, add them to destination	Default for most tables – source schemas grow
Freeze	Reject any schema change, fail the load	Critical tables where downstream depends on exact schema

These are the only two valid policies at the syntactic layer. Some loaders offer `discard_row` and `discard_value` modes that silently drop data when the schema doesn't match – these are semantic decisions, not syntactic work. If the source sent it, the destination should have it. Either accept the change or reject the load; don't silently drop data. See 4.3 – Merge / Upsert for the full reasoning.

6.9.5 Column Naming as a Contract

Your column naming convention – whether you preserve source names verbatim or normalize to `snake_case` – is itself a schema contract, and one of the hardest to change after the fact. Changing the convention on a running pipeline means reloading every table and updating every downstream query that references the old names – a full migration.

The problem gets sharper when you're running multiple pipelines or migrating between systems. A pipeline that loads with source-native names (`@ORDER_VIEW`, `CustomerID`, `línea_factura`) and another that normalizes to `snake_case` produce incompatible destinations. If you plan on running meta-pipelines that handle hundreds of sources, document exactly how you normalize column names and make the convention configurable at two levels: per destination (because consumers expect consistency within the dataset they're querying) and per table (because migrating a source sometimes means fixing individual tables that arrived with a different convention).

This also means you need a documented answer for the edge cases: how do you handle a column named `@ORDER_VIEW` with emoji? A column with spaces? A reserved word? These aren't hypothetical – ERP systems and legacy databases produce all of them. Your naming contract should handle the full range, not just the clean cases.

6.9.6 Type Mapping

Type mismatches between source and destination are universal and varied enough that hand-coding each one is a losing strategy. The corridor determines the severity: transactional-to-transactional pairs usually have close type mappings, while transactional-to-columnar pairs (SQL Server to BigQuery, SAP HANA to Snowflake) produce a steady stream of precision loss, overflow risk, and silent truncation.

Numeric precision is the most dangerous category. SQL Server's `DECIMAL(38,12)` mapped to BigQuery's `NUMERIC(29,9)` silently loses precision on values that fit the source but overflow the destination. Financial data with high-precision decimals is exactly the data where this matters most and where the bug is hardest to catch – the numbers look reasonable until someone reconciles and finds a two-cent discrepancy across a million rows.

The practical approach is to rely on a type-mapping library (SQLAlchemy, your loader's built-in adapters) and override only when you know a specific mapping is wrong for your data. Don't spend time building a comprehensive type-mapping system from scratch – the libraries have already solved the common cases, and the edge cases are specific enough that a generic solution wouldn't help.

Unusual source-destination pairs

If you're extracting from a source where no well-tested adapter exists – a niche ERP, a legacy database with non-standard types, a SaaS API that returns ambiguous JSON types – you may have no alternative to manual type mapping. Document every mapping decision, test with real data (not just the schema), and watch for silent truncation on the first few runs.

6.9.7 Anti-Patterns

Don't enforce unbaselined contracts

A contract based on assumptions ("this column should never be NULL") will fire false positives on the first run. Baseline the actual data first: run a profiling pass, measure real null rates and row counts, then set thresholds from observed behavior. A contract that cries wolf on day one trains everyone to ignore it by day three.

Don't freeze evolving schemas

`products` gains a new attribute column every quarter. Freezing its schema means a load failure every quarter and a manual intervention to update the contract. Use evolve for tables with expected growth; freeze only for tables with stable, critical schemas where a column change would genuinely break something important downstream.

Don't silently discard columns

Silently dropping new or unexpected columns that don't match your schema breaks the syntactic boundary. Wide ERP tables with hundreds of columns are tempting candidates for discard, but the right answer is evolve (accept the column) or 2.9 – Partial Column Loading (explicitly declare which columns you extract and document why). Discarding is implicit partial column loading with no documentation – the worst version of both.

6.9.8 Tradeoffs

Pro	Con
Schema drift caught before it reaches consumers	Every check adds per-table overhead that compounds at scale
Volume anomalies surfaced immediately, not days later	False positives on poorly baselined contracts erode trust
Explicit evolve/freeze policy eliminates ambiguity on schema changes	Evolve means downstream must handle new columns; freeze means manual intervention on legitimate changes
Type mapping libraries handle the common cases transparently	Edge cases on unusual source-destination pairs still require manual work

6.10 Extraction Status Gates

> **One-liner:** 0 rows returned successfully is not the same as a silent failure. Gate the load on extraction status before advancing the cursor.

6.10.1 Zero-Row Ambiguity

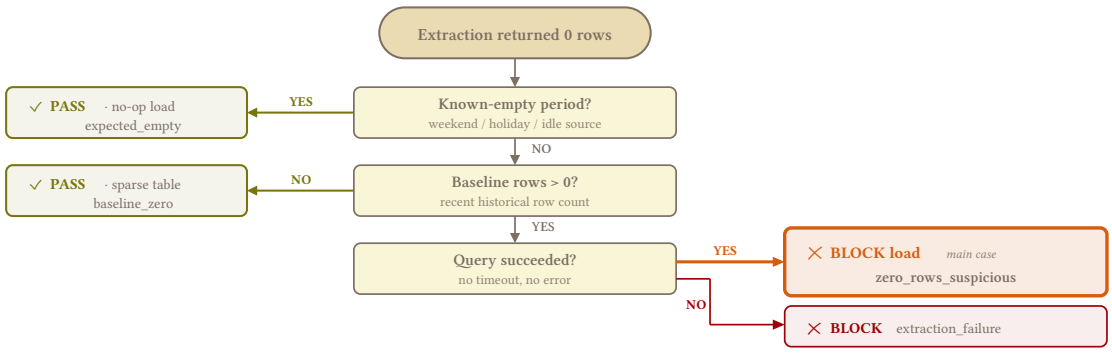
An extraction that returns 0 rows and reports SUCCESS could mean two things: the table genuinely had no changes since the last run, or the source was down, the query timed out silently, or the connection returned an empty result set instead of an error. Without a gate, these two scenarios are indistinguishable – and the pipeline treats them identically, loading nothing and advancing the cursor past data it never read. For incremental tables, that gap is permanent. For full replace tables, it's worse: the destination gets truncated and replaced with nothing.

This happens more often with APIs than with direct SQL connections, but SQL sources aren't immune. I had a client whose upstream team gave us a "database clone" that periodically truncated its tables before reloading them. If my extraction hit the window between truncate and reload, I'd read 0 rows from a table that should have had hundreds of thousands – and my full replace would dutifully wipe the destination clean. It happened more than once before I gated it.

6.10.2 Gate Mechanics

The gate sits between extraction and load. After the extraction query returns, before any data reaches the destination, evaluate whether the result is plausible. The evaluation is per-table – if 3 out of 200 tables return suspect results, block those 3 and let the other 197 proceed. Gating per-run (blocking everything because one table looks wrong) risks your SLA on every other table, and if the blocked table is a heavy one that can only run overnight, you've lost an entire day of data for tables that were fine.

Extraction status gates: resolving zero-row ambiguity



6.10.2.1 What Triggers the Gate

Zero rows from a table that normally returns data. The most common trigger. A table that extracted 450k rows yesterday and 0 today deserves scrutiny. A table that routinely returns 0 rows on weekends does not – the gate needs to know the difference.

Row count outside the expected range. Full replace tables should stay within a percentage of their previous row count. A `customers` table that had 50k rows yesterday and has 50,200 today is normal growth; the same table at 5k rows means something upstream went wrong. The threshold depends on the table's volatility – a `pending_payments` table can legitimately drop by 80% when a batch of payments clears, while `products` is very unlikely to lose half its rows overnight (Also, have they heard of soft deletes?).

Extraction metadata anomalies. Query duration of 0ms on a table that normally takes 30 seconds, or bytes transferred far below the expected range. These can signal a connection that returned immediately without actually querying.

6.10.2.2 What the Gate Does

When the gate fires:

1. **Blocks the load** – the extracted data (or lack of it) does not reach the destination. For full replace tables, the destination retains its current data untouched. **For incremental tables, the decision is less clear-cut** – you may still be getting *some* new data, and a partial update is better than no update at all. Whether to block or load what you got is a case-by-case call based on how wrong the row count looks and how much damage a partial load would cause downstream.
2. **Triggers an alert** (6.5 – Alerting and Notifications) with the extraction metadata: expected row count, actual row count, query duration, and which table.
3. **Logs the event** in the health table (6.2 – The Health Table) so the pattern is visible over time – a table that gates every Monday morning points to a weekend maintenance window nobody told you about.

6.10.2.3 Cursor Safety and Stateless Windows

If you're using stateless window extraction (3.3 – Stateless Window Extraction), cursor advancement is already a non-issue – the next run re-reads the same window regardless. The gate still matters for preventing a bad load, but the recovery is automatic: you have the width of your lag window for the upstream problem to be resolved before data actually falls out of scope. The alert fires on day one; upstream has until the lag window closes to fix it.

For cursor-based extraction, a stuck cursor can become a problem if the window between the cursor and "now" grows large enough that re-extraction becomes expensive. A wide enough lag window (6.8 – Tiered Freshness) mitigates this – the warm tier's daily pass catches what the hot tier missed, and the cold tier's full replace resets everything. Stateless windows avoid this problem entirely, which is one more reason they've become my preferred approach for most incremental extraction.

6.10.3 Full Replace Gates

The stakes for full replace tables are higher than for incremental. An incremental extraction that reads 0 rows leaves a gap in the destination; a full replace that reads 0 rows *empties the destination*. The extraction returned nothing, the pipeline replaced the table with nothing, and now consumers are querying an empty table that had 50k rows an hour ago.

Full replace gates check that the extracted row count is within an expected percentage of the previous load's row count. The percentage depends on the table: a `products` dimension that grows by 1% per month should gate on anything below 90-95% of the last load. A `pending_payments` table that legitimately fluctuates as payments clear needs a wider band. The very few tables that can legitimately approach zero (cleared queues, seasonal staging tables) should be explicitly exempted with documentation explaining why – otherwise the next engineer on call will second-guess the exemption and re-enable the gate.

6.10.4 Baselines

The gate's accuracy depends entirely on knowing what "normal" looks like for each table. The baseline is a range, not a point – flag when outside the range, not when different from last run.

A rolling window of the last 30 runs gives you a reasonable baseline for most tables. Track the min, max, and average row count per table, and gate when the current extraction falls below the historical minimum by a configurable margin. For tables with predictable seasonality – month-end spikes on `invoices`, weekend dips on `orders` – factor the day-of-week or day-of-month into the baseline so the gate doesn't fire every Saturday.

> **Start loose, tighten over time**
>
> A gate that's too tight fires false positives and trains you to ignore it – the exact same failure mode as over-alerting (6.5 – Alerting and Notifications). Start with a generous threshold (block only on 0 rows or >90% drop), observe for a month, then tighten based on the table's actual variance.

6.10.5 Validating Against Source

When the gate fires, the first question is whether the source actually has the data you expected. A `COUNT(*)` against the source during business hours confirms whether the extraction was wrong (source has data, extraction missed it) or the source is genuinely empty (upstream problem). This validation is manual and delayed – the gate fires at 3 AM, someone investigates at 9 AM, and the destination sits stale in the meantime. The SLA clock runs during that gap.

If the source confirms the data is there, the extraction failed silently – re-run it. The truncate-then-reload pattern (source temporarily empty as part of its own load cycle) is a common culprit, and the `COUNT(*)` during business hours distinguishes it from a genuine problem.

If the source is genuinely empty, you have a harder decision with no universal answer:

Hold the gate – the destination keeps its previous data, stale but complete. Consumers see yesterday's numbers, which are wrong but usable. The cost is that you become a silent buffer for upstream's problem: nobody feels the pain, nobody escalates, and the issue can persist for days before anyone outside your team notices.

Load what you got – the destination reflects reality, empty or broken as it is. Consumers see the damage immediately, which hurts but also makes the problem visible to the people who can fix it. A downstream report showing zero revenue generates an escalation in hours; a stale report showing yesterday's revenue generates nothing.

Neither option is always right. Full replace tables almost always deserve a hold – the destination wipeout is too destructive to let through. Incremental tables with partial data lean toward loading what you got, since some fresh data is better than none and the gap is bounded. For everything in between, the decision depends on the table, the consumer, and how much pain you're willing to absorb on upstream's behalf.

Whichever you choose, make the decision explicit: log it in the health table, include it in the alert, and document the policy per table. A gate that silently holds data without anyone knowing it held is a judgment call that nobody can audit – and the next engineer on call will make a different judgment if they don't know yours.

6.10.6 Tradeoffs

Pro	Con
Prevents silent data loss from empty or truncated extractions	Adds per-table overhead (baseline tracking, threshold evaluation)
Per-table gating protects SLA on unaffected tables	Threshold tuning is empirical – too tight fires constantly, too loose misses real failures
Cursor stays safe on incremental tables, destination stays intact on full replace	Stateless windows already mitigate cursor risk, reducing the gate's incremental value
Gated events logged in health table surface recurring upstream patterns	Volatile tables (cleared queues, seasonal) need explicit exemptions

6.10.7 Anti-Patterns

Gate per table, not per run

Blocking 200 tables because 1 returned 0 rows means your entire pipeline misses its SLA. Gate individually. If your orchestrator doesn't support per-asset gating, this is worth building – the alternative is choosing between no gate and an all-or-nothing gate that's too disruptive to enable.

Don't gate without a baseline

A gate that fires on "fewer rows than I expected" without historical data to define "expected" is a guess. Run the pipeline ungated for 30 days, collect baselines, then enable the gate.

6.11 Backfill Strategies

> **One-liner:** Reloading 6 months of data without breaking prod – how to backfill safely alongside live pipelines.

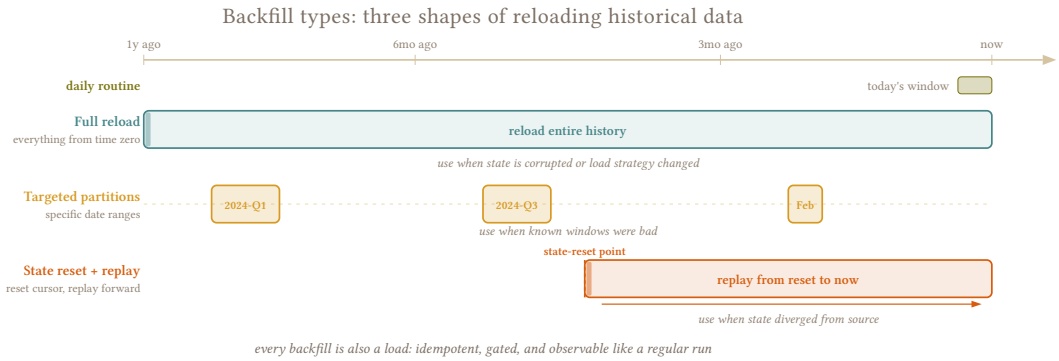
6.11.1 Reloading Without Downtime

Something went wrong upstream – a schema change, a bad deploy, a data corruption that drifted for weeks before anyone noticed – and now you need to reload a historical range. The naive response is "just rerun everything," but a backfill that treats the source like a normal extraction competes with live scheduled runs for source connections, destination quota, and orchestrator capacity. If it runs unchunked during business hours, it violates every rule in 6.7 – Source System Etiquette.

Backfills aren't rare. If you're running hundreds of tables with clients who routinely correct old records, delete and re-enter documents, or run maintenance scripts that touch historical data, backfills are a weekly

operation. A `start_date` override or a `full_refresh: true` flag should be tools you reach for without hesitation – the pipeline that can't backfill safely is the one that drifts furthest from its source.

I had a client with a massive table on a very slow on-prem database – too large to extract in a single overnight window. I loaded two years of data per night, chunked by date range, and it took four nights to complete. The table has been a constant headache since: every backfill is a multi-night operation, and any interruption on night three means deciding whether to restart from scratch or resume from the interrupted chunk.



6.11.2 Backfill Types

6.11.2.1 Date-Range Backfill

The most common type: reload a specific date range – last three months, last fiscal quarter, a single bad week – using partition swap (2.2 – Partition Swap) or rolling window replace (2.5 – Rolling Window Replace). Everything outside the range stays untouched. Scope the range slightly wider than the known corruption – the blast radius of a bad deploy is rarely as precise as the deploy timestamp suggests.

6.11.2.2 Full Table Backfill

Reload the entire table from scratch when corruption is too widespread to scope, when the table is small enough that scoping isn't worth the effort, or when incremental state has drifted so far that a full reset is simpler than diagnosing the gap. Uses full replace (4.1 – Full Replace Load), which resets the destination data, any incremental cursors, pipeline state, and schema versions. After it completes, the next scheduled incremental run picks up from the new baseline.

6.11.2.3 Selective Backfill

Reload specific records by primary key – a handful of corrupted orders, not the entire table. Requires the extraction layer to support PK-based filtering (`WHERE id IN (:ids)`). In practice this is rare: unless you have a short list of known bad PKs and a table large enough that reloading even a date range is expensive, a date-range backfill is simpler and catches records you didn't know were affected.

6.11.3 Execution Strategy

6.11.3.1 Isolation from Live Pipelines

Backfills should never block or delay scheduled runs. Run them as separate jobs in your orchestrator, with their own schedule (or manual trigger) and their own concurrency limits. If your orchestrator supports run priority or queue separation, give scheduled runs higher priority so they proceed even when a backfill is in progress – the backfill can pause between chunks while the scheduled run completes, then resume.

I learned this when a backfill and a scheduled incremental run hit the same table at the same time – both slowed down, both errored, and fixing it meant stopping the backfill, waiting for the scheduled run to finish, and restarting from the interrupted chunk.

6.11.3.2 Chunking

Break large backfills into date-range chunks – one month, one week, or whatever granularity matches the source's partition structure. Each chunk is independently retriable: if chunk 3 of 6 fails, retry only chunk 3. Chunk size trades off per-chunk overhead (connection setup, query parsing, destination writes) against blast radius on failure – smaller chunks lose less work when something goes wrong, larger chunks reduce overhead.

6.11.3.3 Safe Hours

Large backfills belong in the safe-hours window from 6.7 – Source System Etiquette. If the backfill is too large for one window, span it across multiple nights with chunking. Track which chunks completed explicitly – a simple table or config file with chunk boundaries and completion status – so that a failure on night three doesn't force a restart from night one.

6.11.3.4 Staging Persistence

For multi-chunk backfills, staging tables may intentionally persist between chunks so consumers see either the old data or the fully backfilled data, never a half-finished state. Don't clean up staging until the full backfill is validated – the storage cost of a few extra days is negligible compared to restarting a multi-night backfill because you dropped staging prematurely (see 6.3 – Cost Monitoring).

6.11.4 State Reset

After a full backfill, the incremental state – cursor position, high-water mark, schema version – must match the data you just loaded. If the cursor still points to its old position, the next incremental run skips everything between that cursor and the most recent data, leaving an invisible gap. Some pipelines wipe state automatically on a full refresh; others require explicit cleanup (clearing a cursor table, deleting state files, resetting partition metadata). If state cleanup is a manual step, document it prominently – a backfill that reloads the data but leaves the old cursor in place is worse than no backfill, because the pipeline reports success while silently skipping rows.

The risk compounds when pipeline state lives in a separate store. After clearing that state, the next scheduled run starts from scratch – effectively a full refresh of every table, not just the one you backfilled. Engineers who don't expect this find out the hard way. This is one of the strongest arguments for stateless window extraction (3.3 – Stateless Window Extraction): the next scheduled run re-reads its normal trailing window regardless of any backfill, there's no state to reset, and the failure mode of "reload data but forget to fix the cursor" doesn't exist. It's also far simpler to reason about – "the pipeline always grabs the last N days" requires no mental model of cursor state, cleanup procedures, or post-backfill sequencing.

6.11.5 Backfill as Routine

If your clients actively manage their own source data – correcting historical records, deleting and re-entering documents, running maintenance scripts on old rows – backfills are part of the regular operating rhythm, and the pipeline needs to support them without ceremony. Two runtime overrides cover most cases:

`start_date` / `end_date` – override the extraction's date boundaries to re-extract a specific range without pulling everything forward to today. Without an `end_date`, a backfill starting three months back also re-extracts all data between then and now – wasting source load and destination writes on data that's already correct.

Date-range backfills can also clean up hard deletes and orphaned rows within the window if you filter on a stable business date (`order_date`, `invoice_date`) rather than `updated_at`, then swap the destination's partitions for that range with the fresh data (2.2 – Partition Swap). The partition swap fully replaces the slice, so anything that existed in the destination but no longer exists in the source disappears. The business date is the right filter because it's immutable – an order placed on March 5 always has `order_date = 2026-03-05` regardless of when it was last updated – which keeps partition boundaries stable and guarantees you capture every row in the range, not just recently changed ones.

`full_refresh` – ignore all incremental state and reload the entire table using full replace (4.1 – Full Replace Load) instead of a merge. A merge only updates and inserts, so rows hard-deleted at the source survive in the destination indefinitely; a full replace wipes the slate. Useful when the table is small enough that scoping isn't worth the effort, when the incremental state is corrupt, or when you suspect hard deletes have drifted the destination.

Both should be launchable from your orchestrator's UI without modifying code or config files. If a backfill requires editing a config and redeploying, you'll avoid doing it until the problem is too large to ignore. Some orchestrators go further – Dagster's partition-based backfill UI lets you select a date range, kick off the backfill, and track per-partition status from the same interface that shows your scheduled runs (see 8.5 – Orchestrators).

6.11.6 Tradeoffs

Pro	Con
Resets accumulated drift and restores source-destination parity	Large backfills compete with live pipelines for source and destination resources
Chunked backfills are independently retriable – partial failures don't restart from scratch	Multi-night backfills require chunk-tracking state and are fragile over long durations
Date-range scoping limits blast radius to the affected period	Scoping too narrowly may miss corrupted rows at the edges
Full table backfill resets all state to a known-good baseline	Resets incremental cursor – next run after backfill may be heavier than expected
Routine backfill capability reduces time-to-fix for upstream problems	Absorbing upstream messiness via frequent backfills can mask problems that should be fixed at the source

6.11.7 Anti-Patterns

> **Don't run unchunked backfills live**
>
> A 6-month backfill as a single sustained scan at 2pm on a Tuesday during business hours on a live source will get your access revoked. Chunked backfills with indexed reads can coexist with business-hours traffic if the source can handle it, but an unchunked backfill on a production OLTP during peak hours is how you lose source access.

Don't forget the state reset

On cursor-based pipelines, reloading the data while the cursor still points to the old high-water mark means the next incremental run skips everything between the cursor and the new data. Clear the state or force a full refresh. Stateless window extraction avoids this entirely – there's no state to forget.

Don't let backfills compete with schedules

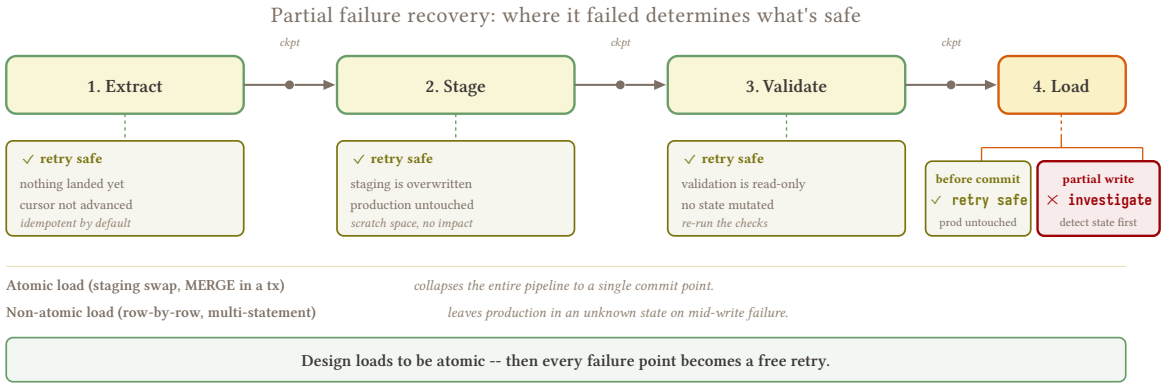
A backfill that blocks a scheduled run isn't fixing the pipeline – it's degrading it. Isolate backfills in separate jobs with lower priority, and design the chunking so a backfill can yield to a scheduled run between chunks.

6.12 Partial Failure Recovery

One-liner: Half the batch loaded, the other half didn't – now what?

A pipeline run that processes multiple tables can fail partway through: 40 tables succeed, 10 fail. Rerunning the entire job wastes time reprocessing the 40 tables that already landed correctly. Not rerunning leaves 10 tables stale, and the staleness compounds with every subsequent run that doesn't fix them. The real problem is knowing which tables failed, at which step, and whether to retry now or wait for the next scheduled run.

At scale, partial failures are daily. With hundreds of tables extracting from multiple sources, something fails every run – a connection timeout on one source, a DML quota hit on the destination, a schema change on a table nobody warned you about. The pipeline that handles partial failures well is the one where failures are visible, scoped, and retriable without disrupting the tables that succeeded.



Cursor safety and partial failures

If your cursors advance only after a confirmed successful load (3.2 – Cursor-Based Timestamp Extraction), partial failures don't create data gaps – the failed tables simply get re-extracted on the next run. Stateless window extraction (3.3 – Stateless Window Extraction) avoids the question entirely.

6.12.1 Failure Modes

6.12.1.1 Extraction Failed for Some Tables

Some tables extracted successfully, others hit a timeout, a connection error, or a source that was temporarily unavailable. The successful tables can proceed to load; the failed ones need re-extraction. Each table should have automatic retry on extraction errors – a connection timeout on the first attempt often succeeds on the second, and waiting for the next scheduled run to discover that wastes an entire cycle. Two or three retries with a short backoff is enough; if the source is genuinely down, retrying indefinitely just adds load to a system that's already struggling (6.7 – Source System Etiquette). Your orchestrator should track per-table status, not just per-run status – the successful tables should proceed to load even though the run as a whole is failed.

The common causes are connection timeouts (especially on slow on-prem sources), connection pool exhaustion when too many tables extract from the same source simultaneously, and source maintenance windows that nobody told you about. A table that fails for the same reason every Monday morning is a scheduling problem, not a retry problem – move it to a different window or investigate the source's maintenance calendar (6.7 – Source System Etiquette).

6.12.1.2 Extraction Succeeded, Load Failed

The data was extracted correctly but the destination rejected it – DML quota exceeded, permission error, schema mismatch, disk full. The extraction is valid and may still be sitting in staging; if it is, you can retry the load without re-extracting. If staging is ephemeral (cleaned up per run), the extraction has to run again.

Destination quotas are the most common cause at scale. BigQuery imposes per-table DML statement limits (1,500 per table per day for non-partitioned tables, per partition per day for partitioned ones), and a pipeline that runs frequent merges against the same tables can exhaust the quota partway through – the first 150 loads land fine, the remaining 50 get rejected. More quota helps, but the real fix is knowing which tables didn't land and retrying them in the next window when the quota resets. This is also where full replace earns its keep: a `DELETE + INSERT` or partition swap avoids the DML-heavy merge path entirely, and quota limits on batch loads are generally higher than on row-level DML.

6.12.1.3 Load Partially Applied

The load started but didn't finish – rows were written but the job died mid-stream. What happens next depends on the load strategy: full replace and partition swaps are idempotent and can be safely rerun since the incomplete load gets overwritten. Append may have produced duplicates that need deduplication (6.13 – Duplicate Detection). A merge may be partially applied – some rows updated, others not – leaving the table in an inconsistent state where the same extraction's data is half-landed. See 4.6 – Reliable Loads for making the load step itself resilient to interruption.

6.12.2 Recovery Strategy

6.12.2.1 Per-Table Retry

The first principle: retry only what failed, not the entire job. If 97/100 tables succeeded, rerunning all 100 wastes compute, risks introducing new failures on previously successful tables, and delays recovery. Your orchestrator should support re-running individual tables from a failed run – if it doesn't, this is worth building, because the alternative is choosing between "rerun everything" and "wait for the next schedule." Some orchestrators support this natively – Dagster lets you retry individual failed assets from a run's status page without touching the ones that succeeded (see 8.5 – Orchestrators).

The retry should also target the right step. A table that failed at extraction needs re-extraction; a table that extracted successfully but failed at load only needs the load retried – preferably from the data already in staging, not from a fresh extraction that hits the source again for no reason.

6.12.2.2 Staging as a Safety Net

If staging tables persist after extraction, a load failure can be retried from staging without hitting the source again. This is the faster recovery path and the one that's gentler on the source system – the data is already extracted, you just need to land it. The tradeoff is storage cost: persistent staging means keeping a copy of every extracted table until the load confirms success (see 6.3 – Cost Monitoring). For most tables the cost is trivial; for a few massive ones it may matter.

If staging is ephemeral, a failed load requires full re-extraction. Whether that's acceptable depends on how expensive the extraction is and how soon the data needs to land. For small tables on a healthy source, re-extraction is fast and harmless. For a 50M-row table on a slow on-prem database during business hours, you may have to wait until the next safe window (6.7 – Source System Etiquette).

6.12.2.3 Per-Table Status Tracking

Track each table's lifecycle explicitly: `extracting` -> `extracted` -> `loading` -> `loaded` / `failed`. On restart, tables stuck in `loading` are failed tables, not running ones – treat them accordingly. A table that's been in `loading` for longer than its expected load duration either crashed or is hanging, and leaving it in limbo means nobody investigates.

The health table (6.2 – The Health Table) should record the outcome per table per run – not just `success` / `failure` but which step failed and why. This is what makes per-table retry possible: without a record of where each table stopped, every retry is a guess.

6.12.3 Alerting on Partial Failures

Any failure, no matter how small, should mark the pipeline run as failed. A run where 197 tables succeeded and 3 failed is a failed run – not a successful run with caveats. If your orchestrator reports it as success, the 3 broken tables disappear into the noise and nobody investigates until a consumer complains. The run status should be unambiguous: if anything didn't land, the run failed.

The tension is failure fatigue. If the pipeline fails every single run because one flaky table times out on Mondays, the team learns to ignore the failure status – and the one time 50 tables fail for a real reason, nobody notices because the alert looks the same as every other Monday. Your alerting (6.5 – Alerting and Notifications) needs to distinguish between the two: include the count of failed tables, which ones, which step failed, and whether the failure is retryable. "Run failed: 3 tables (invoices, order_lines, products) – extraction timeout, auto-retry exhausted" is actionable. "Run failed" with no context trains people to click dismiss.

6.12.4 Anti-Patterns

Don't rerun everything

Retry only the failed tables. Rerunning the entire pipeline to fix 3 failures wastes compute, risks new failures on previously successful tables, and delays recovery.

Don't leave tables stuck in loading

A table in `loading` after the run process has died is a failed table. If your recovery logic doesn't detect and reset orphaned states, those tables sit in limbo indefinitely – neither loaded nor marked for retry.

6.13 Duplicate Detection

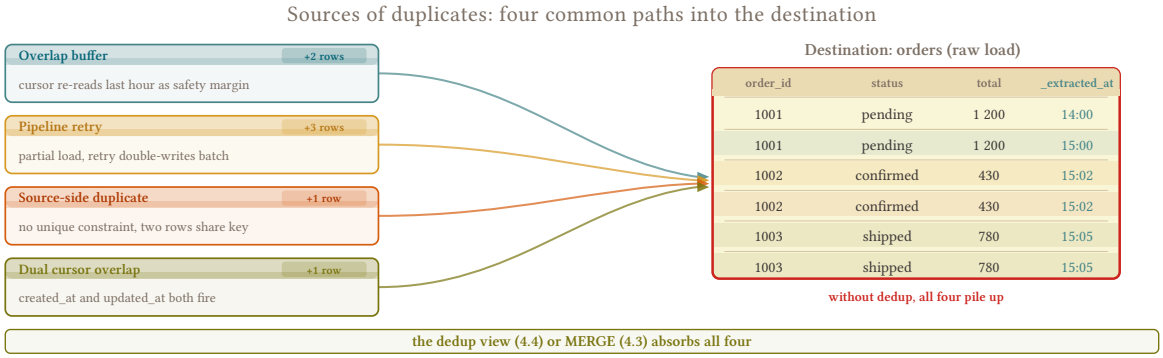
One-liner: Duplicates already landed. How to find them, quantify the damage, and deduplicate without losing data.

6.13.1 Silent Duplication

Duplicates in the destination are a symptom, not a root cause – they indicate a load strategy mismatch, a failed retry that double-wrote, or an append that should have been a merge. If you followed the patterns in this book (merge with the correct key, full replace where possible, append-and-materialize with a dedup view), duplicates should be rare. But when they happen, the damage is disproportionate: consumers don't notice until aggregations are wrong – revenue doubled, counts inflated, joins producing unexpected fan-out – and once they catch it, your data's credibility takes a hit that's hard to recover from. One episode of duplicates, even if you fix it in an hour, can make consumers question every number you produce for months.

Checking for duplicates is fast – a `GROUP BY pk HAVING COUNT(*) > 1` takes seconds. Run it before anything else. If the table is clean, the problem is downstream: most "duplicate" reports turn out to be bad JOINs on the consumer's side (a one-to-many fanout they didn't expect, a missing GROUP BY). But verify your side first – it's cheaper than asking for their query.

6.13.2 How Duplicates Arrive



Cause	Mechanism
Append without dedup handling	Append-only done right (4.2 – Append-Only Load) handles edge cases with `ON CONFLICT DO NOTHING` or a dedup view. Raw INSERT with no conflict handling and no dedup layer produces duplicates from retries, overlap buffers, or upstream replays
Merge key too specific	The merge key includes a column that changes between extractions (e.g., `_extracted_at`, a hash that incorporates load metadata), so the merge never matches existing rows and every re-extraction INSERTs instead of UPDATing
NOLOCK page desync	SQL Server NOLOCK reads can return the same row twice if a page split moves it mid-scan – duplicates arrive in a single extraction, before the load strategy even runs

Cross-partition duplicates

Partitioning the destination by `updated_at` or another mutable date makes cross-partition duplicates likely: a row lands in the March partition, gets updated in April, and the next extraction writes the updated version to the April partition while the March copy persists. Partitioning by an immutable business date (`order_date`, `invoice_date`) prevents the row from scattering across partitions – every re-extraction targets the same partition, which is cheaper and correctly scoped. But **partitioning alone doesn't deduplicate**: columnar engines don't enforce uniqueness, so you still need your load strategy (merge with the correct key, or a dedup view from 4.4 – Append and Materialize) to handle duplicates within the partition.

6.13.3 Detection

6.13.3.1 Row Count Comparison

The simplest signal: run `COUNT(*)` on the source and destination separately (different engines, no cross-query), then compare in your orchestrator. If the destination has more rows, you either have duplicates or you're missing hard-delete detection. Run hard-delete detection first (3.6 – Hard Delete Detection) – if after cleaning up deleted rows the destination still has a surplus, the excess can only be duplicate PKs. If 6.14 – Reconciliation Patterns is already running on a schedule, it surfaces the count mismatch before anyone downstream notices.

6.13.3.2 By Primary Key

The definitive test. Group by PK, count > 1 = duplicate.

```
-- destination: columnar
SELECT order_id, COUNT(*) AS dupes
FROM orders
GROUP BY order_id
HAVING COUNT(*) > 1;
```

If you're particularly worried about duplicates and you have overhead to spare, add this at the end of your pipeline, after loading finishes.

6.13.3.3 By Content Hash

When there's no natural PK, hash the columns that identify the entity and group by hash – count > 1 means multiple rows for the same entity. Fix the key definition (5.2 – Synthetic Keys) so it uses only the columns that define identity (revise your synthetic keys, maybe?), then deduplicate.

6.13.3.4 Narrowing the Root Cause

Once you've found duplicates, `_extracted_at` or `_batch_id` from 5.1 – Metadata Column Injection narrow down which load introduced them. "All duplicates share `_batch_id = 47`" points to a specific run and limits where to look.

6.13.4 Deduplication

6.13.4.1 Dedup in Place

Keep one row per PK, delete the rest. A `MERGE` against the deduplicated version of itself preserves table permissions, policies, and metadata that a `CREATE OR REPLACE` would wipe:

```
-- destination: columnar (BigQuery)
MERGE INTO orders AS tgt
USING (
    SELECT * EXCEPT(_rn) FROM (
        SELECT *, ROW_NUMBER() OVER (PARTITION BY order_id ORDER BY _extracted_at
DESC) AS _rn
        FROM orders
    )
    WHERE _rn = 1
) AS deduped
ON tgt.order_id = deduped.order_id
WHEN MATCHED THEN
    UPDATE SET tgt._extracted_at = deduped._extracted_at
WHEN NOT MATCHED BY SOURCE THEN
    DELETE;
```

Expensive on large tables – it rewrites every partition – but should be a one-off. Fix the pipeline first so duplicates stop arriving, then clean up the destination.

6.13.4.2 Dedup via Rebuild

Re-extract the table with a full replace (4.1 – Full Replace Load) or rebuild from staging. Cleaner than in-place dedup because it resets to a known-good state with no residual risk of missed duplicates. Prefer this when the duplication is widespread or when the table is small enough that a full reload is cheap.

6.13.4.3 Dedup View

Leave the base table as-is and create a view that deduplicates:

```
-- destination: columnar
CREATE VIEW orders AS
SELECT * FROM (
    SELECT *, ROW_NUMBER() OVER (
        PARTITION BY order_id ORDER BY _extracted_at DESC
    ) AS _rn
    FROM orders_raw
) WHERE _rn = 1;
```

Fast to deploy, no DML, no data loss risk. If you rename the base table to `orders_raw` and create the view as `orders`, downstream queries don't need to change – this is the same mechanism that 4.4 – Append and Materialize uses permanently. As a temporary fix it buys you time to investigate the root cause while consumers see clean data immediately.

> **Consider append-and-materialize permanently**
>
> If you're reaching for the dedup view often, consider switching to append-and-materialize. The dedup view is the core of 4.4 – Append and Materialize. Append-and-materialize removes the duplicate problem structurally – every extraction appends, the view always deduplicates – and it's cheaper than merge in columnar engines because a pure INSERT never rewrites existing partitions. The dedup cost is paid at read time, not at load time, and only for the rows the consumer actually queries.

6.13.5 Anti-Patterns

> **Don't deduplicate without finding the cause**
>
> Deduplication fixes the symptom. If you don't fix the load strategy that produced the duplicates, they'll come back on the next run. Find the cause first, fix the pipeline, then clean up the data.

> **Verify duplicates before blaming the pipeline**
>
> Run the `GROUP BY pk HAVING COUNT(\*) > 1` check first – it takes seconds. If the table is clean, the problem is downstream.

6.14 Reconciliation Patterns

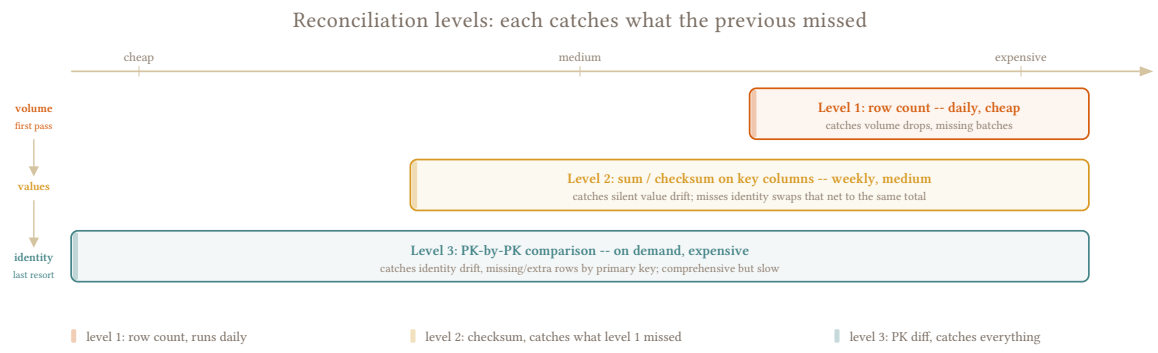
> **One-liner:** Source count vs destination count – row-level, hash-level, and aggregate reconciliation.

6.14.1 Undetected Drift

A load that completes without errors can still produce wrong results: missing rows, duplicate rows, stale data, wrong values. The pipeline reports success; the destination is quietly wrong. Without a verification step, discrepancies surface only when a consumer notices something off – a report that doesn't tie out, a dashboard metric that jumped overnight, an analyst who ran the numbers twice and got different answers.

Reconciliation is the scheduled check that the destination actually reflects the source. It runs after the load, compares what arrived against what should have arrived, and alerts when the numbers don't add up. It's not a replacement for staging validation (2.1 – Full Scan Strategies) or extraction gates (6.10 – Extraction Status Gates) – those catch failures before the load commits. Reconciliation catches what those gates didn't see: rows that fell within tolerance, drift that accumulated over multiple runs, or discrepancies that only surface after downstream queries start running.

6.14.2 Reconciliation Levels



6.14.2.1 Row Count Reconciliation (Cheapest)

Compare `COUNT(*)` at the source against `COUNT(*)` at the destination. A count mismatch is the first and cheapest signal that something went wrong – missing rows from a failed extraction, duplicates from a retry that double-wrote, a dropped partition that nobody noticed.

Catches: missing rows, duplicates, dropped partitions, undetected hard deletes (destination surplus).
Misses: rows with wrong values, rows updated at the source but not reloaded.

6.14.2.2 Aggregate Reconciliation (Medium)

Compare key aggregates – `SUM(amount)`, `MAX(updated_at)`, `COUNT(DISTINCT customer_id)` – between source and destination. A row that changed from $100 to $0 has the same count but a different sum; row count alone misses it. Choose aggregates that are meaningful for the table: financial totals for transaction tables, max timestamps for activity tables.

The cost caveat: running `SUM(amount)` against a transactional source during business hours is expensive. Columnar destinations handle aggregation cheaply; transactional sources don't. If you're running aggregate reconciliation, run it against a read replica or during off-peak hours – or accept that it runs less frequently than row count checks. At scale, with tables spanning dozens of clients and schemas, the variety makes it impractical to standardize meaningful aggregates across the board. Row count reconciliation runs on everything; aggregate reconciliation is reserved for critical tables where value integrity matters.

6.14.2.3 Hash Reconciliation (Expensive)

Hash every row at source and destination and compare the hashes. Any difference at any column surfaces – this is the nuclear option. At scale it's too expensive to run on every load; reserve it for critical tables or periodic audits, not routine runs.

6.14.3 Configuring Thresholds

An exact match between source and destination count is rarely achievable in practice. In-flight transactions committed after the extraction window but before the destination count is taken create natural discrepancy on live transactional sources. A tolerance threshold absorbs this noise without masking real failures.

Two asymmetric rules drive threshold configuration:

Destination has fewer rows than source – acceptable within a threshold, but the right threshold depends on when you extract. For off-peak extractions – overnight, early morning – in-flight transactions are rare and the tolerance should be tight; a deficit of more than a handful of rows is a real signal. For extractions running during business hours, the threshold needs to cover transactions committed after the extraction window closes; calibrate it from your actual discrepancy history, not a guess. 100 rows is a starting point for a busy source during business hours, but the right number varies by table and schedule.

Destination has more rows than source – interpretation depends on whether hard delete detection is running. If it is, a surplus means duplicates and warrants an immediate alert. If it isn't, the surplus is expected: rows deleted from the source still exist in the destination. Know which case you're in before alerting.

When a deficit above threshold surfaces, the resolution depends on the gap size. A small gap is a candidate for pk-to-pk detection (3.6 – Hard Delete Detection) to identify exactly which rows are missing without reloading the whole table. A large gap points to a structural failure – a missed extraction window, a dropped partition, a load strategy mismatch – and the right fix is a full reload or partition swap (2.1 – Full Scan Strategies, 2.2 – Partition Swap).

6.14.4 Timing Matters

Source count and destination count must be taken as close together as possible. A gap between counting the source and counting the destination lets new rows arrive at the source in between, generating a false discrepancy that looks like a pipeline failure but is just timing.

The right approach: record the source count at extraction time, before new data arrives; record the destination count after the load completes; compare in your orchestrator. The comparison cannot happen inside either database – source and destination are different engines with no shared query context.

6.14.5 Reconciliation Jobs

A per-run inline reconciliation check is ideal but expensive at scale. An alternative is a dedicated reconciliation job on a schedule – typically once daily – that iterates all tables, compares counts against source, and produces a summary report. Run it before business hours: discrepancies surface to operators before downstream consumers act on bad data. 07:00 is a reasonable default if your pipeline runs overnight.

The tradeoff: inline checks catch discrepancies before downstream queries run on bad data. A scheduled reconciliation job may surface an issue hours after downstream consumers have already seen it. For critical tables, inline is worth the overhead. For the long tail of lower-priority tables, a daily reconciliation job is the right balance.

Store the results in the health table (6.2 – The Health Table): table name, source count, destination count, delta, status (OK / warning / critical). Storing results historically lets you detect drift trends – a table that's consistently 50 rows short is a different problem from one that's suddenly 50,000 rows short. If you store per-run source and destination counts as part of normal pipeline operation, the dedicated reconciliation job becomes a comparison of already-collected numbers rather than a fresh round of queries against both systems – which makes it cheap enough to run across everything, every morning.

6.14.6 By Corridor

Transactional to columnar

Source count: `SELECT COUNT(\*) FROM schema.table` at source. Destination count: `SELECT COUNT(\*)` – BigQuery bills 0 bytes for it (resolved from table metadata internally); Snowflake resolves it from micro-partition headers without scanning data. Both engines also expose row counts in `INFORMATION_SCHEMA`, but those update asynchronously and can lag after a recent load, making them unreliable for post-load verification.

Transactional to transactional

Both sides support `COUNT(\*)` efficiently – no reason not to use it.

6.14.7 Anti-Pattern

Don't reconcile only on count

A table with 1M rows at source and 1M rows at destination can still be wrong: 1,000 rows missing, 1,000 duplicates. Count matches, data doesn't. Use aggregate reconciliation for critical tables.

6.15 Recovery from Corruption

> **One-liner:** A bad deploy corrupted 3 months of data – identifying the blast radius and rebuilding.

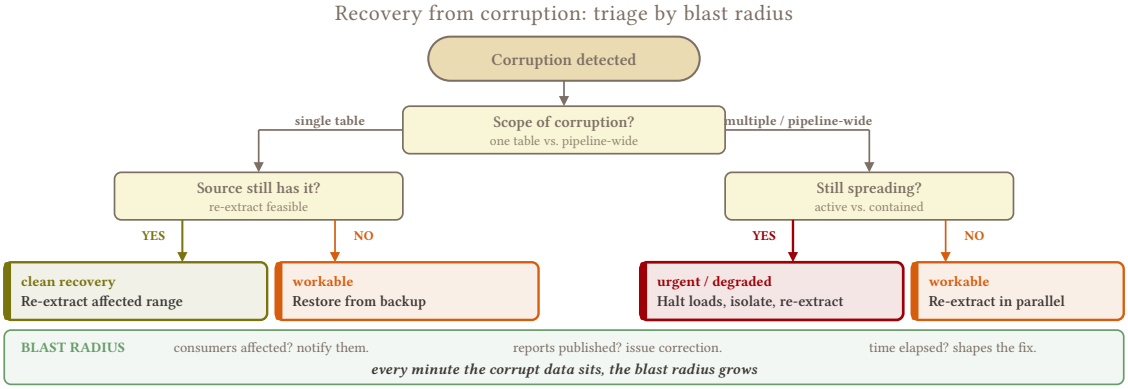
6.15.1 Plausible Wrong Data

Something broke and bad data has been landing for a while. A schema migration that silently changed types, a cursor that skipped a range, a load strategy that dropped a column, a syntactic bug that mangled values. The pipeline reported success on every run because the failure was in the data, not in the execution – no errors, no alerts, no signal that anything was wrong until someone downstream noticed the numbers didn't add up.

The gap between when corruption starts and when it's detected is the blast radius. A bug introduced three months ago that nobody caught until today means three months of data in the destination is suspect, every downstream model that consumed it is suspect, and every report built on those models has been wrong for three months. The recovery isn't just reloading the data – it's scoping the damage, fixing the root cause, rebuilding what's affected, and communicating what happened so consumers can reassess decisions they made on bad data.

The worst corruptions are the ones that look plausible. A date format that flipped from D-M-Y to M-D-Y after a source ERP version upgrade produces dates that parse successfully – January through December, day 1 through 12 with no alerts or failures, and everything becomes silently wrong. You'll discover it when someone notices March orders arriving in May, and by then the entire destination may be corrupt and you'll get more than an earful about it.

6.15.2 Triage: Assess the Blast Radius



6.15.2.1 When Did It Start?

`_extracted_at` from 5.1 – Metadata Column Injection narrows the window. Filter destination rows by `_extracted_at` ranges and compare against the source to find where the data starts diverging – the first batch where values don't match is the start of the corruption window. Cross-reference that timestamp with your deploy history and git log: a commit that shipped on the same day as the first bad batch is the likely root cause.

If `_batch_id` is populated, the scoping is even tighter – "all rows from batch 47 onward are corrupted" is a precise statement that drives the recovery scope. Without metadata columns, you're left correlating deploy dates with destination anomalies by hand, which is slower and less certain.

6.15.2.2 What Tables Are Affected?

The blast radius depends on where the root cause lives. A pipeline code change that affects the syntactic layer corrupts every table processed by that code path – potentially hundreds. A source schema change affects only tables from that source. A destination-side issue (quota exhaustion, permission change) affects only tables on that destination.

Start with the narrowest plausible scope and widen if evidence points further. Checking a handful of tables from each source against their current source state is faster than assuming everything is wrong and rebuilding the world.

6.15.2.3 What's Downstream?

Every downstream model, materialized view, dashboard, and report that reads from the corrupted tables is also affected. Map the lineage from corrupted tables to downstream consumers – if the destination feeds a semantic transformation layer that builds aggregates, those aggregates are wrong too, and they need rebuilding after the source tables are clean.

6.15.3 Recovery Strategies

Three strategies, from broadest to most surgical. The right choice depends on how much data is affected and how precisely you can scope it.

6.15.3.1 Full Replace (Simplest)

Reload the entire table from source using `full_refresh: true`. Resets the destination to the current source state – every row, every column, clean baseline. Downstream models rebuild from the clean data. This is the right default when the table is small enough to reload within the schedule window, when the corruption is widespread, or when you can't precisely scope the damage.

Full replace always works for current state. The source has the correct data right now, so reloading it produces a correct destination. The caveat is historical state: if the source is transactional and rows were modified or deleted since the corruption started, the full replace reflects the source's current state, not the state at any point during the corruption window. For most tables this is exactly what you want – the destination should match the source as of now, not as of three months ago.

6.15.3.2 Date-Range Rebuild

Reload only the corruption window via backfill (6.11 – Backfill Strategies). Less disruptive than full replace because data outside the window stays untouched, but it requires knowing the exact corruption range. Scope the range slightly wider than the first bad batch – corruption boundaries are rarely as precise as a single timestamp suggests, and a few extra days of reload is cheap insurance against missing rows at the edges.

Use partition swap (2.2 – Partition Swap) for the destination-side replacement so the rebuild is atomic per partition and the rest of the table stays live throughout. For tables too large to full-replace but where the corruption window is bounded, this is the sweet spot.

6.15.3.3 PK-to-PK Repair

Compare primary keys between source and destination to identify exactly which rows are wrong – missing, surplus, or mismatched values. Fix only the discrepancies: insert missing rows, delete surplus rows, update changed values. This is the same mechanism as hard delete detection (3.6 – Hard Delete Detection) and the small-gap resolution described in 6.14 – Reconciliation Patterns.

Use this when the corruption is narrow – a handful of rows in a large table, a specific set of PKs identified during triage – and reloading an entire table or date range would be disproportionate. The tradeoff is that you need to know exactly which rows are affected, which requires either a full PK comparison against the source or a reconciliation pass that identified the discrepancies.

All three strategies may require a state reset if the table uses cursor-based extraction. A full replace or date-range rebuild that reloads the data but leaves the old cursor in place means the next incremental run skips everything between the stale high-water mark and now – the same problem 6.11 – Backfill Strategies warns about. Stateless window extraction (3.3 – Stateless Window Extraction) sidesteps this entirely – the next run re-reads its normal trailing window regardless of what the rebuild did, and there's no cursor to forget about. This is one of the operational arguments for defaulting to stateless: recovery is simpler because there's less state to manage.

6.15.4 Recovery Checklist

Regardless of which strategy you choose, the sequence is the same: confirm the fix, verify the source, rebuild, verify the result, notify.

- ☐ If consumers have already acted on corrupted data (reports sent, decisions made), notify them now – they need to know before you start, not after
- ☐ Confirm the root cause is fixed and deployed
- ☐ Test the fix on a small range before committing to the full rebuild
- ☐ Verify source connectivity and schema haven't changed since the corruption started
- ☐ If the table uses cursor-based extraction: reset incremental state (cursor position, schema versions) so the rebuild sets a clean baseline – not needed for stateless window extraction (3.3 – Stateless Window Extraction)
- ☐ Run the rebuild (full replace, date-range backfill, or PK-to-PK repair depending on scope)
- ☐ Reconcile post-rebuild: source count vs destination count (6.14 – Reconciliation Patterns)
- ☐ Notify downstream consumers that data is clean and they can rebuild dependent models

6.15.5 Prevention

None of these prevent corruption from happening – source schemas change, bugs ship, scripts run without warning. What they do is make corruption detectable early and recoverable fast, which limits the blast radius.

Metadata columns (`_extracted_at`, `_batch_id`) make triage possible. Without them you can't scope the corruption to specific batches – you're left guessing which runs introduced the bad data based on deploy dates and git blame. See 5.1 – Metadata Column Injection.

Schema contracts catch drift before it corrupts data. A new column appearing is harmless; a column disappearing or a type changing is a signal that something upstream changed without coordination. Contracts surface these changes before the load commits, not after consumers have already consumed the result. See 6.9 – Data Contracts.

Reconciliation catches silent count and value drift between source and destination. A table that's consistently 50 rows short is a different problem from one that's suddenly 50,000 rows short, and both are problems that row-level pipeline success doesn't reveal. See 6.14 – Reconciliation Patterns.

Stateless, idempotent pipelines reduce the recovery surface. Pipeline state – cursors, schema version tracking, checkpoint files – is itself a corruption vector. When the state is wrong, the pipeline produces wrong output from correct source data, and the failure mode is invisible because no query failed and no error fired. The less state your pipeline carries between runs, the fewer ways it can silently break. Full replace and stateless window extraction (3.3 – Stateless Window Extraction) both minimize carried state; cursor-based extraction with external state stores maximizes it.

6.15.6 Anti-Patterns

> **Don't fix forward without fixing backward**
>
> Fixing the pipeline so future runs are correct doesn't fix the corrupted historical data already in the destination. You need both: fix the code AND rebuild the affected range. A pipeline that's producing correct data going forward while three months of bad data sits in the destination is a pipeline that's still wrong – it's just wrong in a way that's harder to notice.

> **Don't rebuild before confirming the fix**
>
> Reloading 3 months of data only to have the same bug corrupt it again is wasted work and a wasted weekend. Confirm the fix is deployed, test it on a small range, then run the full rebuild. The checklist above puts "test on a small range" before the rebuild for exactly this reason.

7.1 Don't Pre-Aggregate

> **One-liner:** Land the movements, build the snapshot downstream. Resist the pressure to transform at extraction.

7.1.1 One-Way Transformation

The first request from every non-technical consumer sounds the same: "how much did we sell?" They want a total. They want it per month, per product, per warehouse. The temptation is to build that aggregation into the extraction – `SELECT product_id, SUM(quantity) FROM order_lines GROUP BY product_id` – and hand them exactly what they asked for. Clean, simple, one table with the numbers they need.

Then they ask "which orders drove the spike in product X?" And the detail isn't in your warehouse, because you extracted the SUM and threw away the rows. **You can't drill down from a total to its components**. You can't recompute the aggregation with a different grouping. You can't debug a number that looks wrong because the individual records that produced it were never loaded. Every client who starts with "just give me the totals" eventually asks for the detail – and if you aggregated at extraction, the only way to answer is to rebuild the pipeline.

The defense is simple: land the detail. Build the totals downstream – in views consumers can query, change when the business logic shifts, and rebuild without touching the pipeline when someone asks a question you didn't anticipate.

7.1.2 Where the Boundary Is

The line between syntactic and semantic transformation runs through aggregation. Landing `inventory_movements` as-is is syntactic – the source has those rows and you're cloning them. Building `inventory_current` by summing movements is semantic transformation – you're computing a derived state that encodes business logic: which movement types to include, how to handle negative quantities, whether to count pending transfers.

The same applies to derived columns. `revenue = quantity * unit_price` **looks** harmless in the extraction query, but it's a business calculation. The moment discounts apply, taxes enter the formula, or currency conversion becomes relevant, that column is wrong in every historical row and the only fix is a full backfill. Land `quantity` and `unit_price` as separate columns and let downstream compute whatever formula the business currently uses.

The distinction matters because aggregation and derivation encode decisions that belong to the people who understand the business context – and those decisions change. A grouping that makes sense today ("revenue by product category") stops making sense when the category taxonomy changes. A formula that's correct today is wrong next quarter when the pricing model shifts. If the pipeline made those decisions at extraction, every change requires a pipeline change. If a downstream view made them, the view changes and the pipeline keeps running untouched.

See 1.2 – What Is Syntactic Transformation for the full framework.

7.1.3 The Exception: `metrics_daily`

Some source tables are already pre-aggregated. `metrics_daily` in the domain model is computed by the source system – the aggregation decision was made upstream, not by your pipeline. Landing a pre-aggregated table as-is is faithful replication because you're cloning what the source has, aggregation included. The rule: don't aggregate in the pipeline – land what the source gives you.

7.1.4 Movements vs. Snapshots

Two kinds of data, two different representations of the same reality:

Movements are append-only event records: `inventory_movements` (stock received, sold, adjusted), `order_lines` (items ordered), `events` (clickstream, transactions). Each row is something that happened. The history is in the rows themselves.

Snapshots are point-in-time state captures: `inventory` (current stock levels), `metrics_daily` (today's aggregated numbers). Each row is the state of something right now. The history is gone the moment the next snapshot overwrites it.

Land both when both exist at the source. The `inventory` table and the `inventory_movements` table are different data – the snapshot and the movements don't always agree (bulk imports that update `inventory` without logging a movement, the soft rule from Domain Model), and it's your job to make that discrepancy visible to consumers rather than hiding it by building one from the other.

Downstream can reconstruct snapshots from movements if they want to (see 7.6 – Point-in-Time from Events) – stock as of any date is a `SUM(quantity) WHERE created_at <= target_date`. The inverse isn't possible: you can't recover individual movements from a snapshot total. Detail produces aggregates; aggregates don't produce detail.

7.1.5 The Conversation

The request always starts simple. "How much did we sell last month?" You land `order_lines`, build a view that sums `quantity * unit_price` grouped by month, and the consumer is happy. Then:

"Can I see that by product?" – change the GROUP BY in the view. No pipeline change.

"Can I see which orders had returns?" – filter on `quantity < 0` in the detail. Only possible because the detail is there.

"Can I see the monthly trend for this specific SKU?" – filter the view by SKU. Still no pipeline change.

"Wait, these numbers don't match the ERP's report" – compare your `order_lines` detail against the source, row by row. Only possible because you have the rows, not just the total.

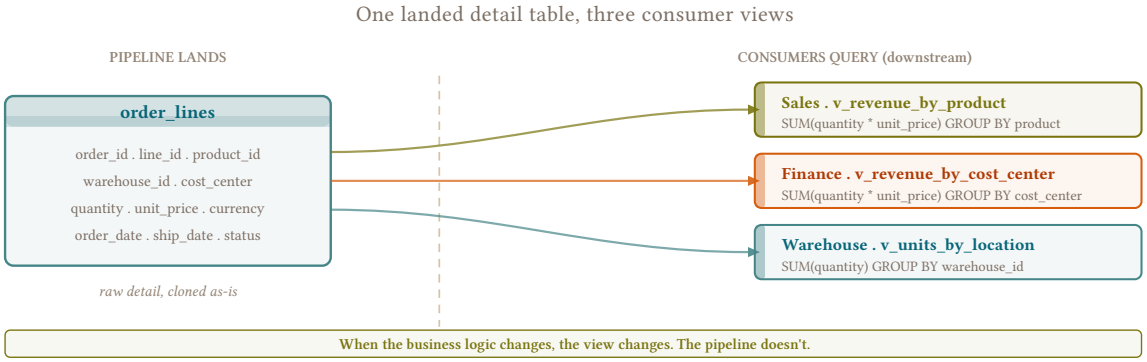
Every one of these follow-up questions is answerable because the detail was landed. None of them would be answerable if the pipeline had extracted `SUM(quantity * unit_price) GROUP BY month`. The consumer who said "I just need the monthly total" needed the detail all along – they just didn't know it yet.

7.1.6 What Consumers Actually Need

A pre-built view (7.4 – Query Patterns for Analysts) that aggregates the raw data for their specific use case. The view is downstream, documented, and changeable without touching the pipeline. Different consumers can have different aggregations over the same raw data: the sales team sees revenue by

product, the finance team sees revenue by cost center, the warehouse team sees units shipped by location – all from the same `order_lines` table, each through their own view.

When the business logic changes – a new product category, a different grouping, a revised pricing formula – the view changes. The pipeline doesn't.



7.1.7 Anti-Patterns

Don't extract SUMs instead of rows

`SELECT product_id, SUM(quantity) FROM order_lines GROUP BY product_id` as your extraction query means the per-line detail never reaches the destination. The total looks correct until someone needs to drill down, and then there's nothing to drill into.

Don't compute derived columns at extraction

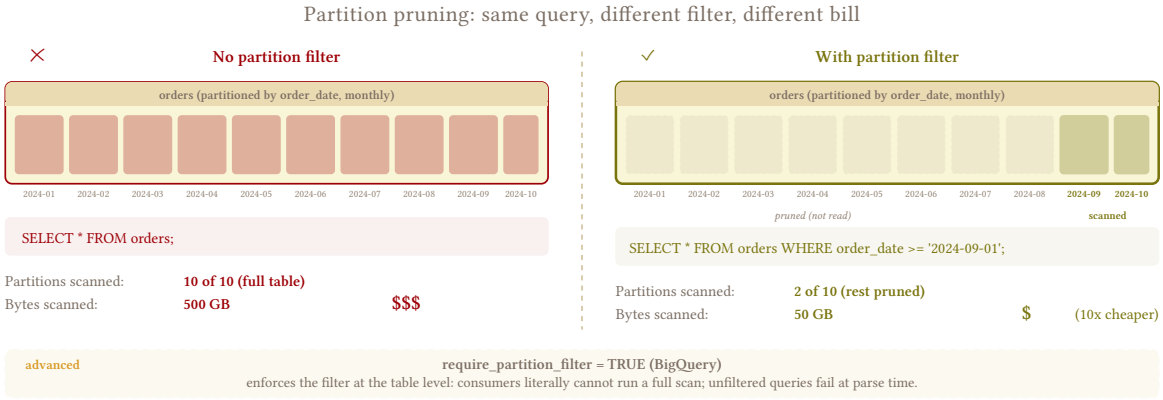
`revenue = quantity \* unit_price` in the extraction query is a business calculation baked into the pipeline. When the formula changes – and it will – every historical row is wrong and the only fix is a full backfill of the entire table. Land the raw columns, compute downstream.

7.2 Partitioning, Clustering, and Pruning

> **One-liner:** Partition by business date, cluster by consumer filters, enforce partition filters. The physical layout decisions that protect every downstream query.

A table without a partition scheme in a columnar engine forces a full scan on every query. An analyst filtering `orders` by last week's dates scans the entire table – five years of history, every row, every column they selected – and the bill reflects it. Partitioning by date means that same query reads only the seven partitions that contain last week's data, and the engine skips everything else. Clustering goes one level deeper: within those seven partitions, it organizes data so a filter on `customer_id` reads fewer blocks instead of scanning every row in the partition.

Both decisions are made at load time, and both affect every downstream query for the lifetime of the table. The engineer picks the partition key and the cluster keys – two of the few load-time choices that directly shape what consumers pay.



7.2.1 Choosing the Partition Key

Partition by the column consumers filter on most. For transactional data, that's almost always an immutable business date: `order_date`, `event_date`, `invoice_date`. These dates describe when the business event happened – not when the row was last modified or when the pipeline extracted it – and they never change. An order placed on March 5 always has `order_date = 2026-03-05` regardless of how many times its status, amount, or shipping address gets updated. That stability is what makes it safe as a partition key: the row stays in the same partition across every load.

Never partition by `updated_at` or `_extracted_at`. These are mutable – `updated_at` changes on every source modification, `_extracted_at` changes on every extraction. A row updated today lands in a different partition than its previous version, which forces the MERGE to touch both the old and new partition. In BigQuery, every partition touched in a DML statement is a full partition rewrite (4.3 – Merge / Upsert), so a batch of 10,000 rows scattered across 200 dates rewrites 200 partitions. If the load strategy doesn't clean up the old version in the previous partition, you also end up with cross-partition duplicates (6.13 – Duplicate Detection).

7.2.2 Partition Granularity

Daily is the default, and it works for most tables. `events`, `order_lines`, `invoices` – daily partitions give tight pruning for date-range queries and align naturally with the schedule (one partition per day's extraction). Tables rarely have enough history to hit partition limits at daily granularity – 30 years of daily partitions is ~11,000, which exceeds BigQuery's 10,000-partition cap, but most tables don't span 30 years.

Monthly is the fallback when daily creates too many partitions or when the table's query patterns are month-oriented. If consumers always aggregate by month and never filter by individual days, monthly partitions match their access pattern and reduce partition management overhead. It's also the right choice when daily granularity hits engine limits – a table with 30+ years of history at daily granularity exceeds BigQuery's cap, and switching to monthly brings it well under.

Yearly is rare – only for archival tables with low query frequency where even monthly is more granularity than anyone uses.

The practical approach: start with daily. If you hit the partition limit or discover the table has decades of history, switch to monthly. The rebuild is a one-off `CREATE TABLE ... AS SELECT` with the new partition clause.

7.2.3 Clustering

Partitioning controls which date slices a query reads. Clustering controls how data is physically organized within those slices. A query that filters `orders` by `customer_id` within a single day's partition still reads every row in that partition without clustering – with it, the engine skips blocks that don't contain the target customer.

Choose cluster keys based on how consumers filter: `customer_id`, `product_id`, `status` – the columns that appear in WHERE clauses and JOIN conditions. Column order matters on BigQuery: the first column clusters most effectively, so put the highest-cardinality filter first. Don't cluster by columns nobody filters on – it costs storage reorganization for no query benefit, and don't cluster by pipeline metadata like `_extracted_at`.

Clustering interacts with load strategy. Append-only loads naturally cluster by ingestion time – good for time-series queries, bad for entity lookups. Full replace rebuilds clustering from scratch on every load. MERGE can fragment clustering over time as updates scatter across micro-partitions – BigQuery auto-reclusters in the background, Snowflake reclustering costs warehouse credits.

7.2.4 `require_partition_filter`

BigQuery's `require_partition_filter = true` rejects any query that doesn't include the partition column in the `WHERE` clause. It's the single most effective cost-protection mechanism for large tables – an analyst who forgets to filter by date gets an error instead of a bill for scanning 3TB.

The tradeoff is friction. Consumers who are used to `SELECT * FROM orders LIMIT 100` for a quick look now get an error and have to add a date filter. BI tools that generate queries without partition awareness fail until someone configures the date filter in the tool's connection settings. For tables where consumers query frequently and know the schema, the protection is worth the friction. For tables where non-technical consumers explore ad hoc and the hand-holding cost is high, consider whether the enforcement helps more than it annoys – and whether a pre-built view (7.4 – Query Patterns for Analysts) with a built-in default date range is a better answer than forcing the filter on the raw table.

No other columnar engine has an equivalent enforcement mechanism. Snowflake, ClickHouse, and Redshift rely on documentation, query review, and cost monitoring (6.3 – Cost Monitoring) to catch unfiltered scans after they happen.

7.2.5 Per Engine

BigQuery. `PARTITION BY` on date, timestamp, datetime, or integer range. `CLUSTER BY col1, col2` up to 4 columns – auto-reclusters in the background at no explicit cost. `require_partition_filter = true` for enforcement. Hard limit of 10,000 partitions per table and 4,000 partitions per DML job. Every DML statement rewrites every partition it touches in full.

Snowflake. No explicit partition key – Snowflake manages micro-partitions automatically based on ingestion order. `CLUSTER BY (col1, col2)` influences how micro-partitions are organized; pruning happens automatically when queries filter on clustered columns. Reclustering costs warehouse credits. No partition filter enforcement.

ClickHouse. `PARTITION BY` expression in the MergeTree definition, fixed at table creation. `ORDER BY` in the MergeTree definition is the cluster key – the most important physical layout decision in ClickHouse, also fixed at creation. Partition pruning and block skipping are automatic on filtered queries.

Redshift. No native partitioning in the columnar sense. Sort keys determine scan efficiency for range queries (a sort key on `order_date` lets Redshift skip blocks outside the filtered range). Dist keys control how data is distributed across nodes for JOIN performance. Both can be changed via `ALTER TABLE` (sort key, dist key, and dist style), but the operation rewrites data in the background – plan them carefully at creation.

7.2.6 Partition Alignment with Load Strategy

The partition scheme and the load strategy interact directly – a mismatch between them turns a cheap operation into an expensive one.

Full replace via partition swap (2.2 – Partition Swap) is partition-native: you replace entire partitions atomically, and the partition key determines which slices get swapped. BigQuery partition copies are near-free metadata operations; Snowflake and Redshift use DELETE + INSERT within a transaction scoped to the partition range.

Incremental MERGE cost scales with the number of partitions the batch touches (4.3 – Merge / Upsert). A batch aligned to a single day's partition rewrites one partition. A batch scattered across 30 dates rewrites 30. Keep load batches as aligned to partition boundaries as the data allows.

Append-and-materialize (4.4 – Append and Materialize) introduces a split: partition the log table by `_extracted_at` for cheap retention drops (each day's extraction is its own partition, dropping old extractions is a metadata operation). The dedup view sits on top and can't be partitioned itself – but if consumers filter by a business date in their query, the engine still prunes the underlying log's partitions. If read cost becomes a problem, materialize the dedup result into a separate table partitioned by business date (7.4 – Query Patterns for Analysts).

7.2.7 Retrofitting a Partition Scheme

If you didn't partition a table at creation and it's grown large enough to matter, the fix is a table rebuild: `CREATE TABLE` with the partition clause from a `SELECT` on the original, then rename.

```
-- destination: bigquery
CREATE TABLE orders_partitioned
PARTITION BY order_date AS
SELECT * FROM orders;
```

Follow up by renaming the original and swapping the new table in. A script that renames the original tables, rebuilds them with partitions, and drops the originals can run across dozens of tables in a single overnight window – it's a one-off that doesn't need to be elegant, just correct. Verify row counts match before dropping anything.

The rebuild is a full table rewrite, so it costs bytes scanned on the read and bytes written on the write. For a 10GB table that's a few dollars; for a 10TB table it's a conversation worth having before running. Partitioning at creation costs nothing and avoids the rebuild entirely – but if you're inheriting a destination that was built without partitions, the retrofit is straightforward and the improvement in query cost pays for itself within days.

7.2.8 Anti-Patterns

Don't partition by updated_at

A row that gets updated lands in a different partition than its previous version. The MERGE touches both partitions – the old one to find the existing row and the new one to write the updated version. In BigQuery, both partitions are fully rewritten. The cost scales with how scattered the updates are across dates, not with how many rows changed.

Don't cluster by _extracted_at

Pipeline metadata isn't a consumer filter. Cluster by business columns that appear in downstream WHERE clauses.

7.3 Pre-Built Views

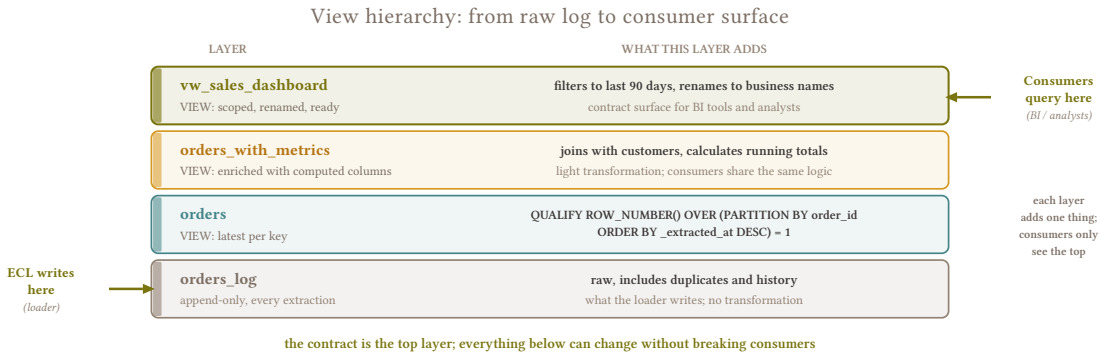
One-liner: Materialized views, scheduled queries, and pre-cooked tables – the serving layer you build on top of landed data to protect consumers from themselves.

7.3.1 Consumer Query Mistakes

The pipeline did its job: the data landed correctly, partitioned, with metadata columns and clean types. The destination is a faithful clone of the source. Now an analyst opens their query editor, writes `SELECT * FROM orders_log`, and gets back 90 million rows – every version of every order from the append log, duplicates and all. They aggregate on it, get numbers that are 3x what the source shows, and file a bug against your pipeline. The data is correct; the query is wrong.

This is the gap the serving layer fills. The pipeline lands raw data. The serving layer builds clean, queryable surfaces on top of it – dedup views that expose current state from append logs, flattening views that extract fields from JSON columns, materialized tables that pre-compute expensive aggregations so consumers don't pay for them on every query. None of this is in the pipeline. It's what you build after the data lands, as a service to the people who consume it.

The goal is to put a guardrail between the consumer and the raw data. Not because the raw data is wrong – it's exactly what the source has – but because raw data in a columnar engine is expensive to misuse, and most consumers don't know how their queries translate into bytes scanned or warehouse time. A well-built view costs you minutes to create and saves consumers thousands of dollars in accidental full scans over the life of the table.



In practice, the serving layer is smaller than you'd expect. A typical client with 70 base tables needs around 15 views for their entire reporting surface, and many of those are variations on the same core query with different filters or groupings – maybe 5 distinct view designs that cover the full reporting need. When the client runs multiple companies on the same ERP schema (separate databases, identical structure), the base tables multiply but the views don't – each view UNIONs the same table across databases with a `_database` column to distinguish the source. The effort is low; the impact on consumer experience and cost control is disproportionately high.

7.3.2 The Hierarchy

Four tools, from lightest to heaviest. Start at the top and move down only when the lighter option doesn't serve the consumer well enough.

SQL views. A saved query, computed fresh on every read. The dedup view from 4.4 – Append and Materialize is the most common example: a `ROW_NUMBER()` over the append log that exposes only the latest version of each row. Column-filtering views that hide internal metadata (`_extracted_at`, `_batch_id`) are another. Free to create, not free to consume – every query against the view scans the underlying table. A well-written view can reduce cost by baking in partition filters and column selection that consumers would otherwise forget, but it doesn't pre-compute anything.

Materialized views. Pre-computed and stored. The engine refreshes them on a schedule or on data change, and routes queries to the materialized result instead of recomputing from the base table. The query cost drops to scanning the materialized result (generally smaller than the base table), at the expense of storage and refresh overhead. This is where the cost savings happen – the consumer queries the pre-built result, not the raw data.

Materialized views work best when the view has a single base table or a fact table with a few dimension lookups – one source of truth driving the refresh. When the view joins multiple independently-refreshed fact tables, the "update on data change" trigger gets messy: every participating table's load triggers a refresh, and if five tables contribute to one view, you're refreshing it five times per pipeline run with partially-stale data each time. For views like these, scheduled query tables are the cleaner option.

Scheduled query tables. A query that runs on a schedule and writes its results to a destination table. The simplest form of materialization – no special engine feature needed, works on every engine. Your orchestrator or a cron job runs the query after all the participating tables have landed, and consumers query the output table directly. Less elegant than a native materialized view, but more portable, easier to debug, and the right choice when the view joins many tables that refresh at different times – one scheduled run after all sources have landed produces a consistent result without multiple partial refreshes.

Consumer-specific tables. A table shaped for a specific dashboard, report, or API. Pre-joined, pre-filtered, pre-aggregated – exactly the columns and rows the consumer needs, nothing else. The most

expensive to maintain (a pipeline change or a business logic change can invalidate it) and the most efficient to query (consumers scan only what they need with zero overhead). Reserve these for high-frequency, high-cost query patterns where even a materialized view isn't cheap enough.

7.3.3 When to Materialize

The dedup view from 4.4 – Append and Materialize is a SQL view by default, and during development that's fine – you're the only one querying it. Once analysts start using it daily, the cost shifts: 50 queries per day against a view that scans 90 days of append log means 50 full scans of that log per day. At that point the materialization cost (one refresh per load) is a fraction of the repeated read cost, and the switch is justified.

A view over 90 days of append log is an even clearer case – every query scans 90x the base table volume to find the latest version per key. Materialization is almost always worth it here, even at low query frequency.

The rule: don't materialize speculatively. Wait until the query cost shows up in 6.3 – Cost Monitoring, then materialize the views that actually get hit. A materialized view for a table queried once a week is wasted storage and refresh compute – and at 15 views across 70 base tables, most of the serving layer stays as simple SQL views that never need materialization.

7.3.4 Flattening Views for JSON

JSON and nested data land as-is (5.7 – Nested Data and JSON) – the pipeline doesn't parse or restructure them. Consumers who need tabular access get a flattening view. The syntax depends on how the data landed:

JSON string columns (landed as `STRING` or `JSON` type):

```
-- destination: bigquery
CREATE VIEW orders_flat AS
SELECT
    order_id,
    customer_id,
    status,
    JSON_EXTRACT_SCALAR(details, '$.shipping.method') AS shipping_method,
    JSON_EXTRACT_SCALAR(details, '$.shipping.address.city') AS shipping_city,
    order_date
FROM orders;
```

Repeated records / STRUCTs (BigQuery's native nested types, common when loading from Avro or when the loader normalizes JSON into typed structs):

```
-- destination: bigquery
CREATE VIEW order_items_flat AS
SELECT
    o.order_id,
    o.customer_id,
    item.sku,
    item.qty,
    item.price
FROM orders o, UNNEST(o.items) AS item;
```

`UNNEST` explodes the array into rows – one row per item per order. This is the BigQuery-native way to flatten repeated fields; `JSON_EXTRACT_SCALAR` only works on string-typed JSON, not on STRUCTs or repeated records.

Different consumer groups can have different flattening views over the same nested data – the sales team sees shipping and pricing fields, logistics sees warehouse and carrier fields – each shaped for their use case without duplicating the underlying data.

When the nested schema mutates – a new field appears, a field is renamed – the view definition changes. The pipeline doesn't. This is the same principle as 7.2 – Partitioning, Clustering, and Pruning: the pipeline lands what the source has, the serving layer adapts it for consumption.

7.3.5 Per Engine

BigQuery. SQL views are free to create but every query scans the underlying table and bills for bytes read. Materialized views auto-refresh and BigQuery routes queries to the MV when it can – this is where the cost savings happen, because the query scans the pre-computed result instead of the full base table. Scheduled queries write to destination tables on a cron and are the workhorse for consumer-facing aggregation tables that join multiple sources.

Snowflake. SQL views are free to create, same caveat – every query costs warehouse time against the underlying table. Materialized views refresh automatically on data change, costing warehouse credits for each refresh. Snowflake's `SECURE VIEW` hides the view definition from consumers, useful when the view encodes business logic you don't want exposed.

PostgreSQL. `CREATE MATERIALIZED VIEW` with `REFRESH MATERIALIZED VIEW CONCURRENTLY` for zero-downtime refreshes. No auto-refresh – schedule via cron, orchestrator, or a post-load hook. Standard SQL views are free and fast for simple cases.

ClickHouse. Materialized views trigger on INSERT and pre-compute aggregations at write time – a fundamentally different model from the others. The compute happens at ingestion, not at refresh time, so the materialized result is always current with zero read-time overhead. Powerful for dashboards that need real-time aggregations, but the logic is baked into the write path, making it harder to change than a post-hoc refresh.

7.3.6 Anti-Patterns

> **Don't build consumer-specific landed tables**
>
> A table shaped for one dashboard is semantic transformation. The pipeline lands data generically; the serving layer shapes it for consumption. If the dashboard needs a different shape, change the view, not the pipeline.

> **Don't materialize before you measure**
>
> A materialized view for every table "just in case" is wasted storage and refresh compute. Materialize the views that actually get queried, based on observed cost from 6.3 – Cost Monitoring.

7.4 Query Patterns for Analysts

> **One-liner:** Cheat sheet: how to query append-only tables, how to get latest state, how to not blow up costs.

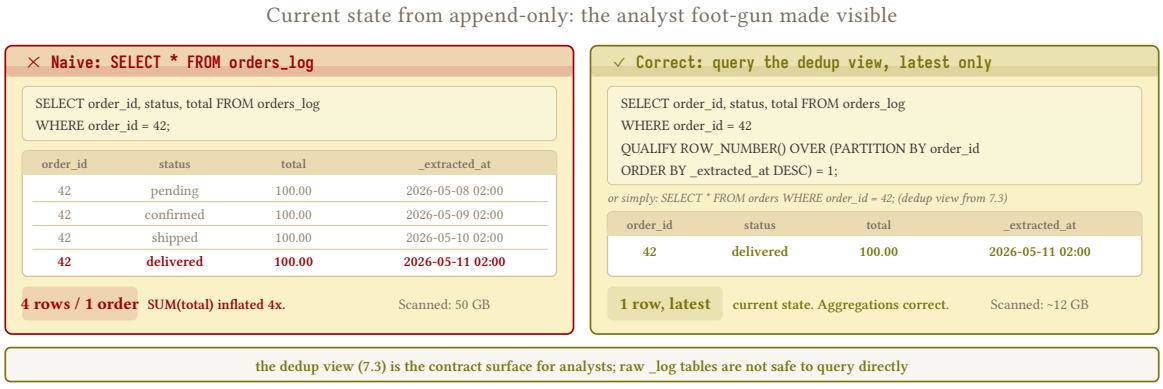
7.4.1 Who This Is For

This is the reference you hand analysts when they get access to the destination. They didn't design the schema, they don't know what append-and-materialize means, and they will `SELECT *` on a 3TB table if nobody tells them not to. The patterns below are the minimum they need to query landed data correctly and cheaply.

One thing to internalize before querying: the destination is not a moment-to-moment replica of the source. Data has to be extracted and loaded before it appears – that takes time, and the freshness depends on the table's schedule (6.4 – SLA Management). If you need transactional-level freshness for point lookups ("is this order shipped right now?"), query the source directly. Columnar destinations are for analysis, not real-time lookups.

7.4.2 Current State from Append-Only Tables

Some tables in the destination are append logs – every extraction appends new rows without overwriting old ones (4.4 – Append and Materialize). The log contains every version of every row your pipeline has ever seen: order 123 with `status = pending`, then order 123 with `status = shipped`, then order 123 with `status = delivered`. All three rows are in the log. The current state is the latest one.



If the table has a dedup view, query the view. The view handles the deduplication logic and returns one row per entity – the latest version. The view is named after the business object (`orders`), and the log is suffixed (`orders_log` or `orders_raw`). If you're not sure which is which, the one with fewer rows is the view.

```
-- destination: columnar
-- Use the view -- always one row per order
SELECT * FROM orders WHERE order_id = 123;
```

If no dedup view exists, dedup manually:

```
-- destination: columnar
SELECT * FROM (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY order_id ORDER BY _extracted_at DESC)
AS rn
    FROM orders_log
) WHERE rn = 1 AND order_id = 123;
```

The `ROW_NUMBER()` picks the row with the most recent `_extracted_at` for each `order_id`. Everything else is a prior version that's been superseded.

> **Don't aggregate on the log table**
>
> `SELECT SUM(total) FROM orders_log` sums every version of every order – if an order was extracted 5 times, its total appears 5 times in the sum. Use the dedup view, or wrap the aggregation around a deduped subquery.

7.4.3 `_extracted_at` vs `updated_at`

Two timestamp columns, two different clocks:

`_extracted_at` is when the pipeline pulled the row. It's set by the pipeline, not the source, and it's always accurate – it reflects when this version of the row arrived in the destination.

`updated_at` is when the source last modified the row. It's maintained by the source application – triggers, ORMs, manual updates – and its reliability varies by table (3.1 – Timestamp Extraction Foundations).

Which one to filter on depends on the question:

Question	Filter on
"What changed at the source this week?"	`updated_at`
"What arrived in our warehouse today?"	`_extracted_at`
"Show me the freshest version of each row"	`ORDER BY _extracted_at DESC` (the dedup view does this)

A row with `updated_at = 2026-03-01` and `_extracted_at = 2026-03-15` was modified at the source two weeks before it was extracted – maybe the pipeline runs weekly, maybe the row fell outside the extraction window until a periodic full replace picked it up. Both timestamps are correct; they answer different questions.

7.4.4 Partition Filters

Most tables in the destination are partitioned by a business date – `order_date`, `event_date`, `invoice_date`. This partition key controls how much data the engine reads: a query with `WHERE order_date >= '2026-01-01'` reads only partitions from January onward, while a query without a date filter reads the entire table.

On BigQuery with `require_partition_filter = true`, the engine rejects queries that don't include the partition column in the WHERE clause – you'll get an error instead of a surprise bill. On other engines, the filter isn't enforced but the cost difference is the same.

```
-- destination: bigquery
-- This works and scans only 2026 data
SELECT order_id, status, total
FROM orders
WHERE order_date >= '2026-01-01';

-- This is rejected (require_partition_filter = true)
-- or scans the entire table (no enforcement)
SELECT order_id, status, total
FROM orders;
```

The partition filter is a cost filter. `WHERE order_date >= '2026-01-01'` doesn't just narrow your business results – it tells the engine to skip every partition before January 2026. Always include it.

7.4.5 Querying JSON Columns

Some source tables have JSON or nested data columns that land as-is (5.7 – Nested Data and JSON). If a flattening view exists (7.4 – Query Patterns for Analysts), use it – the view extracts the fields you need into regular columns. If not, use the engine's JSON path syntax:

```
-- destination: bigquery (JSON string column)
SELECT
    order_id,
    JSON_EXTRACT_SCALAR(details, '$.shipping.method') AS shipping_method
FROM orders
WHERE order_date = '2026-03-15';

-- destination: bigquery (repeated records / STRUCTs)
SELECT
    o.order_id,
    item.sku,
    item.qty
FROM orders o, UNNEST(o.items) AS item
WHERE o.order_date = '2026-03-15';

-- destination: snowflake
SELECT
    order_id,
    details:shipping.method::STRING AS shipping_method
FROM orders
WHERE order_date = '2026-03-15';
```

`JSON_EXTRACT_SCALAR` works on string-typed JSON. `UNNEST` works on BigQuery's native repeated records and STRUCTs. Snowflake uses `:` path notation on `VARIANT` columns.

7.4.6 JOINs on Landed Tables

Header-detail JOINs (`orders` JOIN `order_lines`) work the same as on the source – the foreign keys are the same, the column names are the same, the relationship is the same.

The one difference is freshness. `orders` and `order_lines` may have been extracted minutes apart within the same pipeline run. A very recent order might exist in `orders` but not yet have its lines in `order_lines`, or vice versa. For any analysis that doesn't require real-time accuracy – which is virtually all analysis on a columnar destination – this gap is invisible.

```
-- destination: columnar
SELECT
    o.order_id,
    o.status,
    ol.product_id,
    ol.quantity,
    ol.unit_price
FROM orders o
JOIN order_lines ol ON o.order_id = ol.order_id
WHERE o.order_date >= '2026-03-01';
```

7.4.7 Cost Traps

`SELECT *` scans every column. Columnar engines store each column separately and only read the columns you name. `SELECT order_id, status` reads two columns. `SELECT *` reads all of them – including that 2MB JSON blob you didn't need.

`COUNT(*)` is free (or nearly free). BigQuery resolves it from metadata at zero bytes scanned. Snowflake resolves it from micro-partition headers. Use it freely for row counts.

`LIMIT` does NOT reduce cost on BigQuery. `SELECT * FROM events LIMIT 10` still scans the full table; the LIMIT only caps the result set, not the bytes read. Filter with WHERE first, then LIMIT.

Preview modes scan less. BigQuery's query preview and Snowflake's `SAMPLE` function read a subset of the table for exploration. Use these for "what does the data look like?" instead of `SELECT * LIMIT 100`.

7.4.8 Anti-Patterns

Don't assume LIMIT reduces cost

In BigQuery, `SELECT \* FROM events LIMIT 10` scans the full table. Filter by partition first, select only the columns you need, then LIMIT.

Don't expect real-time destination data

The destination reflects the source as of the last successful extraction, not as of right now. Check `_extracted_at` or the health table (6.2 – The Health Table) to know how fresh the data is. If you need live data, query the source.

7.5 Cost Optimization by Engine

> **One-liner:** Engine-specific strategies for keeping query costs under control – because BigQuery bills differently from Snowflake, and the optimizations don't transfer.

7.5.1 Engine-Specific Costs

Cost optimization is engine-specific. What saves money on BigQuery (reducing bytes scanned) is irrelevant on Snowflake (which bills by warehouse time). Generic advice like "use partitions" applies everywhere, but the specifics – what to partition on, how clustering interacts with the billing model, which operations are free and which are traps – differ enough across engines that generic advice doesn't help with the decisions that actually move the bill.

Load-time decisions have permanent cost consequences on every consumer query. A partition key chosen at table creation, a clustering configuration, a table format – these compound across every query for the lifetime of the table. This chapter is the engine-specific reference for making those decisions correctly, and for knowing which levers to pull when the cost monitoring from 6.3 – Cost Monitoring surfaces a table that's too expensive.

I once wasted $500 in a single night because of unlimited retries on a badly merged table. The retries ran all night, rescanning the table roughly 30 times a minute. By next morning the bill was already in, and the lesson was clear: set per-day cost limits on the project, and understand what each query costs before you let it retry indefinitely.

7.5.2 BigQuery (Bytes Scanned)

BigQuery on-demand billing charges per byte scanned: $6.25/TB. Every query pays for the bytes it reads from the columns it touches, regardless of how many rows the result returns. The optimization target is reducing bytes scanned per query.

> **BigQuery documentation**
>
> Pricing – Partitioned tables – Clustered tables – Materialized views – Reservations (slots)

Partitioning + `require_partition_filter`. Mandatory cost control for any table over a few GB. A query that filters on the partition column reads only the partitions that match; everything else is skipped at zero cost. `require_partition_filter = true` rejects queries that forget the filter, turning a potential $50 full scan into an error message. See 7.2 – Partitioning, Clustering, and Pruning for partition key selection.

Clustering. Reduces bytes scanned within partitions. Up to 4 columns, ordered by filtering priority – the first column clusters most effectively. A query filtering on a clustered column reads fewer storage blocks because the engine skips blocks whose min/max range doesn't include the target value. BigQuery auto-reclusters in the background at no explicit cost.

Column selection. `SELECT col1, col2` scans only those two columns. `SELECT *` scans every column in the table, including that 2MB JSON blob nobody needed. Columnar storage physically separates columns – the engine literally reads fewer bytes when you name fewer columns.

`COUNT(*)`. Free – resolved from table metadata at 0 bytes scanned. Use it for row counts without cost anxiety.

`LIMIT`. Does NOT reduce bytes scanned. `SELECT * FROM events LIMIT 10` still scans the full table; the LIMIT only caps the result set.

Materialized views. BigQuery auto-refreshes materialized views and can route queries to the MV when the optimizer determines the MV can answer the query. The consumer queries the base table, but the engine reads the smaller MV instead. Effective for repetitive aggregation patterns – dashboards that always GROUP BY the same dimensions.

Slots vs on-demand. Flat-rate pricing (reservations/slots) makes bytes-scanned irrelevant – you pay for compute capacity, not data read. The optimization target shifts from "scan fewer bytes" to "avoid slot contention." Most of the advice above still helps because it reduces execution time, which frees slots for other queries.

Per-day cost limits. BigQuery supports custom cost controls natively: `maximum_bytes_billed` per query (set in the job config, rejects the query before it runs if it would exceed the limit), plus daily byte caps at both the project and per-user level (configured in BigQuery's quota settings, resets at midnight PT). Set these before your first production run, not after the first surprise bill. A runaway retry loop is bounded by the daily limit instead of running until someone notices.

7.5.3 Snowflake (Warehouse Time)

Snowflake bills per second of warehouse compute. No per-byte charge – a query that scans 1TB and one that scans 1GB cost the same if they run for the same duration on the same warehouse size. The optimization target is reducing query runtime and minimizing idle warehouse time.

> **Snowflake documentation**
>
> Warehouses overview – Micro-partitions and clustering – Persisted query results (caching)

Warehouse sizing. Right-size the warehouse for the workload. A larger warehouse (XL) finishes queries faster but costs more per second; a smaller warehouse (XS) costs less but takes longer. For batch loads, an XS or S warehouse running for 10 minutes is usually cheaper than an XL running for 2 minutes – the XL's per-second rate is higher and the minimum billing increment (60 seconds) means short bursts on a large warehouse are disproportionately expensive.

Auto-suspend and auto-resume. Set auto-suspend aggressively – 60 seconds is reasonable for most workloads. An idle warehouse that stays running burns credits for nothing. Auto-resume starts the warehouse on the next query, so the only cost of aggressive suspend is a brief cold-start delay.

Clustering. Snowflake manages micro-partitions automatically, but declaring cluster keys helps when the natural ingestion order doesn't match how consumers filter. Reclustering costs warehouse credits, so don't cluster tables where the natural order already matches the query pattern.

Result caching. Identical queries within 24 hours return cached results at zero compute cost. Significant for dashboards that refresh periodically with the same query – the first execution pays, subsequent ones are free until the underlying data changes or 24 hours pass.

Query queuing. Too many concurrent queries on a small warehouse queue instead of fail. Queued queries wait for a slot, which is fine for batch loads but terrible for interactive dashboards. Monitor queue times and scale the warehouse if interactive queries are consistently queuing.

7.5.4 ClickHouse (Self-Hosted Compute)

ClickHouse is self-hosted (or managed via ClickHouse Cloud) – cost is infrastructure (CPU, memory, disk), not per-query metering. The optimization target is making queries fast enough that the infrastructure you're already paying for can handle the workload.

> **ClickHouse documentation**
>
> MergeTree engine – Projections

MergeTree `ORDER BY`. The primary cost lever. The `ORDER BY` clause in the MergeTree definition determines how data is physically sorted on disk. Queries that filter on the `ORDER BY` prefix skip entire granules (~8,192 rows) that don't match – the ClickHouse equivalent of partition pruning. Choose the `ORDER BY` to match the most common consumer query pattern. Fixed at creation – can't be changed without recreating the table.

Compression. ClickHouse compresses aggressively by default, and column types affect compression ratio. `LowCardinality(String)` for columns with a small number of distinct values (status fields, country codes, category names) replaces each value with a dictionary-encoded integer, reducing both storage and scan time. Apply it at table creation for columns with fewer than ~10,000 distinct values.

Materialized views. ClickHouse materialized views trigger on INSERT and pre-compute aggregations at write time – a fundamentally different model from the others. The materialized result is always current with zero read-time overhead, which makes them ideal for dashboards that need real-time aggregations. The tradeoff is that the aggregation logic runs on every insert, adding load-time overhead.

Projections. Alternative physical orderings of the same data. If your table is `ORDER BY (event_date, event_type)` but some queries filter only on `user_id`, a projection ordered by `user_id` lets those queries skip granules efficiently. Multiple projections optimize multiple query patterns simultaneously, at the cost of additional storage and insert overhead.

7.5.5 Redshift (Cluster Compute)

Redshift bills per node per hour (provisioned) or per RPU-second (Serverless). Provisioned clusters pay for the hardware regardless of utilization; Serverless pays per compute consumed. The optimization target depends on the model: provisioned optimizes for query efficiency (get more done on the same nodes), Serverless optimizes for query cost (reduce compute time per query).

> **Redshift documentation**
>
> Sort keys – Distribution styles – VACUUM – Redshift Spectrum

Sort keys. The equivalent of clustering – they determine physical sort order on disk and enable block skipping on filtered queries. Compound sort keys work for range queries on a prefix of columns. Interleaved sort keys work for multi-column filters where queries might filter on any combination, at the

cost of slower VACUUM. Changeable via `ALTER TABLE ... ALTER SORTKEY`, but the operation rewrites data in the background – plan them at creation when possible.

Dist keys. Control how data is distributed across nodes. When two tables are distributed on the same key (e.g., both `orders` and `order_lines` on `order_id`), JOINs between them don't need to redistribute data across the network – co-located rows are already on the same node.

VACUUM and ANALYZE. Redshift runs automatic VACUUM DELETE in the background for routine cleanup, and automatic ANALYZE keeps statistics current. For pipelines with heavy bulk deletes (hard delete detection, large DELETE+INSERT merge batches), automatic VACUUM may not keep up – dead rows accumulate faster than the background process clears them. Monitor `SVV_TABLE_INFO` for unsorted-row percentage and dead-row bloat, and schedule manual VACUUM during off-peak if the numbers drift.

Spectrum. Query data in S3 directly without loading it into the cluster. Useful for cold data that's too large or too infrequent to justify cluster storage. Spectrum bills per byte scanned (like BigQuery), so the optimization advice for S3-resident data is the same: partition, use Parquet, select only the columns you need.

7.5.6 Cross-Engine Principles

Partition by business date, cluster/sort by consumer filter columns. Universal. The partition key controls which slices the engine reads; the cluster/sort key controls how efficiently it reads within those slices.

Select only the columns you need. Matters most on BigQuery (bytes scanned = money), still reduces I/O and speeds up queries on every engine.

Monitor before optimizing. Cost attribution from 6.3 – Cost Monitoring tells you which tables and queries to focus on. Optimizing a table that costs $0.02/month is wasted effort.

Set cost guardrails early. BigQuery's per-day cost limits, Snowflake's resource monitors, Redshift's query monitoring rules – every engine has a mechanism to cap runaway costs. Configure them before production, not after.

7.5.7 Anti-Patterns

> **Don't apply BigQuery optimizations to Snowflake**
>
> Reducing bytes scanned doesn't affect Snowflake's bill – it's warehouse time that matters. Conversely, warehouse sizing is irrelevant on BigQuery's serverless model. Know which billing model you're optimizing for.

> **Don't optimize tables that aren't expensive**
>
> A 10k-row lookup table costs fractions of a cent per query regardless of partitioning or clustering. Optimize what shows up in the top-10 cost report from 6.3 – Cost Monitoring.

Don't let unlimited retries run unbound

A retry loop on BigQuery rescans the table on every attempt. 30 retries per minute on a 100GB table is 4.3TB scanned per hour – $27/hour, $216 overnight. Set retry limits and per-day cost caps before the first production run.

7.6 Point-in-Time from Events

> **One-liner:** Reconstruct past state from event tables, not snapshots. Events are cheaper to store and replay than periodic copies of the full state.

7.6.1 Snapshot Gaps

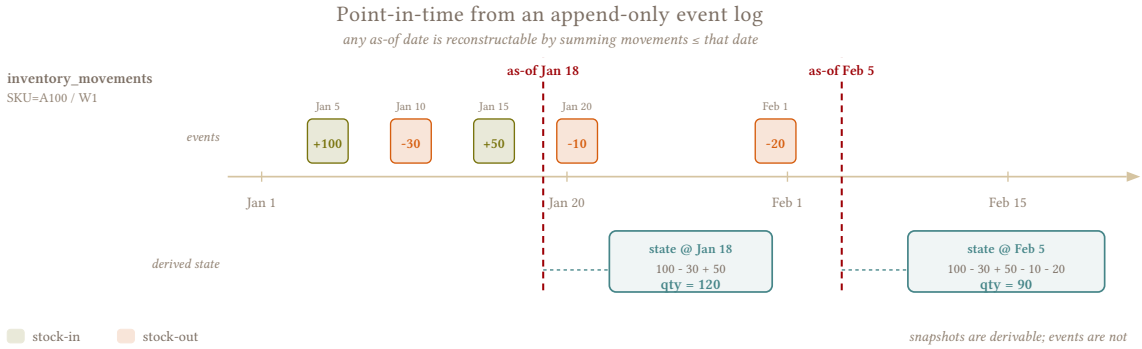
A consumer asks "what was the inventory level on March 5?" or "what was the order status at 2pm last Tuesday?" If you only have the current state – the latest version of each row from a full replace or a dedup view – the answer is gone, overwritten by subsequent updates. The destination reflects right now, not any point in the past.

Two mechanisms preserve history. An append-and-materialize log (4.4 – Append and Materialize) accumulates extracted versions over time – each extraction appends rows tagged with `_extracted_at`, and prior versions survive alongside current ones until compaction. Event tables take a different approach – `inventory_movements`, append-only `events` – each row is something that happened, and the full history is in the log itself. Any point-in-time state is computable by replaying events up to the target date, without storing a single snapshot.

In practice, the most common use case is inventory auditing. A client wants to reconcile their physical stock count against what the system said the stock was on the count date. That's a point-in-time reconstruction: sum all movements up to the count date, compare against the physical count, and the difference tells you whether the system or the warehouse is wrong.

7.6.2 Movements to Snapshots

`inventory_movements` records every stock change: +50 received, −3 sold, −1 adjustment, −10 transferred out. Current stock is the running sum of all movements per SKU per warehouse. Stock as of any date is the same sum, filtered to movements before that date.




```
-- destination: columnar
-- Current inventory, reconstructed from movements
SELECT
    sku_id,
    warehouse_id,
    SUM(quantity) AS on_hand
FROM inventory_movements
GROUP BY sku_id, warehouse_id;
```

```
-- destination: columnar
-- Inventory as of March 5, reconstructed from movements
SELECT
    sku_id,
    warehouse_id,
    SUM(quantity) AS on_hand
FROM inventory_movements
WHERE created_at <= '2026-03-05'
GROUP BY sku_id, warehouse_id;
```

The two queries are identical except for the WHERE clause. Any point-in-time snapshot is computable from the event log by moving the date boundary – this is why 7.2 – Partitioning, Clustering, and Pruning insists on landing the movements: the detail produces any aggregate, but the aggregate can't reproduce the detail.

For high-frequency point-in-time queries – a dashboard showing stock levels at close-of-business for each day of the month – replaying the full movement history on every query gets expensive fast. A materialized table built from movements avoids the rescan: a scheduled query (7.4 – Query Patterns for Analysts) runs after each extraction, replays movements up to each date, and writes the result.

The trap is materializing the full grid. 200 warehouses, 100,000 SKUs, 365 days – that's 7.3 billion rows for a single year, most of them zeros because a given SKU doesn't move every day in every warehouse. Materialize sparse: only SKU/warehouse/date combinations where a movement actually occurred. A consumer who needs "stock on March 5 for SKU X in warehouse Y" gets the answer from the most recent materialized row on or before that date, not from a row for every day.

Even sparse, the table grows with activity volume over time. Tiered granularity keeps it manageable: daily materialization for the current month, monthly rollups for anything older. You can also scope by warehouse type – sales warehouses that move thousands of SKUs daily need daily granularity, while a low-activity storage warehouse that sees a handful of movements per week is fine at monthly resolution.

The goal is to pre-compute the queries consumers actually run, not every possible combination of dimensions and dates.

7.6.3 Status History from Append Logs

Not every table has a natural event log. `orders` doesn't have a changelog – it's a mutable table that gets updated in place. But if `orders` is loaded via append-and-materialize (4.4 – Append and Materialize), the log table has every extracted version of each order: order 123 with `status = pending` from Monday's extraction, order 123 with `status = shipped` from Wednesday's. The extraction log becomes an implicit event trail, with one "event" per extraction run.

Status at a point in time is the latest extracted version before the target timestamp:

```
-- destination: columnar
-- Order status as of March 5 at 2pm
SELECT * FROM (
    SELECT *,
        ROW_NUMBER() OVER (
            PARTITION BY order_id
            ORDER BY _extracted_at DESC
        ) AS rn
    FROM orders_log
    WHERE _extracted_at <= '2026-03-05 14:00:00'
)
WHERE rn = 1;
```

The granularity is limited to extraction frequency. If you extract daily, you can reconstruct state at daily resolution – you know what the order looked like at each extraction, not at every moment between them. For most analytical use cases this is sufficient; for audit-level granularity, the source system's own changelog is the authoritative record.

> **Compaction destroys version history**
>
> 4.4 – Append and Materialize recommends compacting the log – trimming old extractions or collapsing to latest-only – to keep the dedup view fast and storage bounded. That compaction deletes the version history this section depends on. If consumers need point-in-time reconstruction from the append log, the compaction retention window must be longer than their lookback requirement. A 90-day lookback needs at least 90 days of log retention, which means 90 days of extraction overlap sitting in storage. That's a real cost on a large table – decide upfront whether the log is a temporary buffer or a historical record, because it can't cheaply be both.

7.6.4 When Events Aren't Enough

Not all state changes produce events. A `customers` table updated in-place with no changelog has no event trail – the previous state is gone the moment the row is overwritten. `products` has the same problem: a price change replaces the old price, and unless someone stored the before-and-after, the history is lost.

For tables where point-in-time matters but no event log exists:

Append-and-materialize with history retention (4.4 – Append and Materialize). Skip compaction (or compact less frequently) and the append log becomes an explicit version history. Each extraction

appends the current state of changed rows, and prior versions accumulate. Storage grows with extraction frequency, but the history is queryable – point-in-time state is the latest extracted version before the target date.

Append-and-materialize log (4.4 – Append and Materialize). The extraction log provides event-like histories as a side effect, while being a lot cheaper than full snapshots since you're only appending changed rows. But remember, it's not a dedicated backup and will drop rows when compacting the log. Once you compact to latest-only, the prior versions are gone. If consumers depend on point-in-time queries against the log, the compaction retention window must be longer than their lookback requirement – and they need to know that compaction is happening so they don't build a process that assumes the history is permanent.

SCD Type 2 (Slowly Changing Dimension). When point-in-time queries are a first-class requirement – not an occasional audit but something dashboards and reports depend on daily – an SCD2 structure makes the history explicit in the schema itself. Each row gets `valid_from` and `valid_to` columns, and a query for "what did this customer look like on March 5?" becomes a range filter instead of a window function over an extraction log:

```
-- destination: columnar
-- Customer record as of March 5
SELECT *
FROM customers_scd2
WHERE customer_id = 42
  AND valid_from <= '2026-03-05'
  AND (valid_to > '2026-03-05' OR valid_to IS NULL);
```

Building the SCD2 table is semantic transformation; it lives in the serving layer – the pipeline lands the current state or the append log, and a scheduled job compares consecutive extractions to detect changes and maintain the `valid_from`/`valid_to` bookkeeping. The mechanics are well-documented elsewhere; what matters for this pattern is that SCD2 gives you point-in-time queries that are cheap to run (a range filter that benefits from partitioning and clustering – 7.3 – Pre-Built Views), explicit in their semantics (no ambiguity about what `_extracted_at` means versus when the change actually happened), and immune to compaction, since the history is the table itself rather than a side effect of a retention window.

The cost is maintaining the SCD2 pipeline itself. Every extraction needs to be diffed against the previous state to detect what changed, close out old rows, and open new ones. For a `customers` table with 100K rows that changes slowly (hence the name), this is trivial. For an `orders` table with millions of rows and high mutation rates, the daily diff becomes expensive. SCD2 earns its place on tables where the change rate is low relative to the table size and the point-in-time queries are frequent – dimension tables like `customers`, `products`, `warehouses`. For high-mutation fact tables, the append log or snapshot approaches are usually cheaper to maintain.

7.6.5 Storage: Events vs Snapshots

Approach	Storage grows with	Point-in-time granularity	Completeness
Event log (`inventory_movements`)	Activity volume	Per-event (every change)	Only as complete as the event source
Snapshot append (`_snapshot_at`)	Snapshot frequency x table size	Per-snapshot (daily, hourly)	Always complete – it's a full copy
Append-and-materialize log	Extraction frequency x change volume	Per-extraction	Only changes captured by the extraction
SCD Type 2	Change volume (one row per change per key)	Per-extraction (when the diff detected it)	Only changes captured between consecutive extractions

Low-mutation tables store far less with events than with snapshots – 10 changes per day adds 10 rows, while a daily snapshot adds the entire table. High-mutation tables may store more with events – the break-even depends on the mutation rate relative to the table size.

Tiered retention applies to all approaches: keep daily granularity for the recent window, compress older data to monthly, drop anything beyond the retention requirement.

7.6.6 Completeness

Replay is only as accurate as the event log, and event logs have gaps. The domain model's `inventory_movements` table has a soft rule: "every stock change creates a movement." But bulk import scripts that update `inventory` directly without logging a movement violate this silently (Domain Model). The reconstructed snapshot from movements will differ from the actual `inventory` table, and the difference is the sum of all unlogged changes.

I had a client whose `inventory` table and the reconstructed-from-movements inventory diverged by hundreds of units on certain SKUs. The client refused to believe my data was correct – their expectation was that movements and inventory should always match. I had to pull both from the source, show the same discrepancy in the source system itself, and demonstrate that the gap came from bulk operations that bypassed the movement log. The pipeline was cloning faithfully; the source was inconsistent.

The periodic full replace of the `inventory` table catches the drift – it reflects the source's current state, including unlogged changes, while the event-based reconstruction doesn't. When both exist in the destination, consumers should understand which one to trust: the `inventory` table for current state (it's what the source says right now), the movement log for historical reconstruction (it's what the source recorded happening). When they disagree, the source has unlogged changes – that's a source data quality problem, not a pipeline problem.

7.6.7 Anti-Patterns

Don't assume every table has events

`customers`, `products`, dimension tables are overwritten in place with no changelog. Point-in-time reconstruction requires either snapshots or an append-and-materialize log. Choose the load strategy before the consumer asks for the history – retrofitting history onto a table that was loaded with full replace from day one means there's nothing to reconstruct.

Don't replay without knowing completeness

If the event log has gaps (bulk operations that bypass it, the soft rule from the domain model), the reconstructed state is wrong. Document which event sources are incomplete and surface the discrepancy rather than hiding it.

Don't compact without considering consumers

Compacting the append log to latest-only destroys version history. If consumers depend on point-in-time queries against the log, the compaction retention window must be longer than their lookback requirement.

7.7 Schema Naming Conventions

> **One-liner:** Table and column naming at the destination: as-is from source, snake_case, normalized? Pick a convention and apply it consistently – changing it later is a full migration.

7.7.1 Names Are Permanent

Source systems name things however they want: `OrderID`, `@ORDER_VIEW`, `invoice_line`, `OACT`, `Column Name With Spaces`. The destination needs identifiers that are consistent and queryable without quoting gymnastics. A column called `order` clashes with a reserved word on every engine, `@Status` collides with SQL Server's variable syntax, and `Column Name With Spaces` demands quotes everywhere it appears.

This is a one-time decision with permanent consequences – changing a naming convention on a running pipeline means rebuilding every table and rewriting every downstream query, view, and dashboard that touches those names. Identifier normalization is syntactic: it happens at load time, not downstream. Get it right and consistently applied, and downstream teams will (mostly) follow your lead. Get it wrong, and you'll find `Vw_Sales-backup_FINAL-JSmith_2026-05` in your catalog within the year.

7.7.2 Naming Schemes

Three schemes see real use, and each behaves differently depending on the destination engine.

7.7.2.1 Preserve source names

Land `OrderID` as `OrderID`, `OACT` as `OACT`, `invoice_line` as `invoice_line`. Anyone looking at the destination can trace a column straight back to the source without a mapping table.

The real argument for this approach comes from upstream teams. They send you queries written against their system and expect them to run against yours – it's one of the most common requests you'll get. When the destination preserves `OrderId`, adapting a source query is mechanical: quote the identifiers, adjust the `FROM` clause, done. When the destination has normalized to `order_id`, every column in a 30-column query needs translating back, and someone will get one wrong. If your consumers regularly cross-reference with the source system, preserving names saves everyone time.

The cost is that five sources produce five different conventions in the same destination. `OrderID` sits next to `order_id` sits next to `ORDER_STATUS`, and every consumer has to know which source uses which style.

Destination	What happens
BigQuery	Case-sensitive – names land exactly as provided, preserve works cleanly
Snowflake	Folds to uppercase by default. `OrderID` quietly becomes `ORDERID` unless you double-quote at create time *and* in every query
PostgreSQL	Folds to lowercase by default. `OrderID` becomes `orderid` unless quoted
ClickHouse	Case-sensitive – names preserved exactly
SQL Server	Case-insensitive (collation-dependent). `OrderID` and `orderid` resolve to the same column; the original casing is stored but not enforced

> **Snowflake and PostgreSQL destroy mixed case**
>
> Committing to preserve-source-names on either engine means double-quoting every identifier in every DDL and every query. Most teams that start here end up quoting nothing and losing the casing by default – arriving at lowercase-only by accident rather than by choice.

7.7.2.2 Normalize to snake_case

`OrderID` becomes `order_id`, `Column Name With Spaces` becomes `column_name_with_spaces`, and `invoice_line` stays put. This is the standard analytics warehouse convention – consistent, quoting-free, and what analysts expect when they write SQL by hand.

Destination	What happens
BigQuery	The ecosystem convention – BigQuery's own `INFORMATION_SCHEMA` uses snake_case. Store original source names in column descriptions for traceability
Snowflake	Lands as `ORDER_ID` due to the uppercase fold, but `order_id` and `ORDER_ID` resolve identically so it reads fine
PostgreSQL	The native convention. System catalogs use it, `psql` tab-completion expects it
ClickHouse	Works, though ClickHouse's own system tables mix camelCase (`query_id` alongside `formatDateTime`). No strong ecosystem standard
SQL Server	Technically fine, but the SQL Server world expects PascalCase (`OrderId`, `CustomerName`). Landing snake_case puts your landed tables at odds with every system table and most existing schemas. Right call if consumers write ad-hoc SQL; friction if they're .NET applications expecting `dbo.Orders.OrderId`

The cost is irreversibility. Once `OrderID` becomes `order_id`, the original casing is gone – and if two source columns normalize to the same string (`OrderID` and `Order_ID` both become `order_id`), you have a collision to detect and resolve at load time.

For most pipelines, snake_case is still the better default – it reads clean, requires no quoting on case-insensitive engines, and it's what analysts expect to find. I use it across the board and it's never been the wrong call. But I've also worked with clients whose upstream teams live in the source system and send us queries daily, and for those cases preserve-source-names would have saved us hours of translation work every week.

7.7.2.3 Lowercase only

`OrderID` becomes `orderid`, `Column Name With Spaces` becomes `columnamewithspaces`. Fold to lowercase, strip illegal characters, done.

Many teams are already doing this without realizing it – PostgreSQL and Snowflake both fold unquoted identifiers automatically. Single-word names survive fine (`orderid` is readable enough), but multi-word names lose all structure. Try parsing `inventorymovementlogs` or `abortedsessioncount` at a glance.

Lowercase-only is viable when the source uses short, single-word identifiers (common in legacy ERPs: `OACT`, `BUKRS`, `WAERS`). For anything with compound names, snake_case is worth the extra transformation.

7.7.3 Handling Special Characters and Reserved Words

No naming scheme saves you from these – they need explicit handling regardless of which convention you pick.

Reserved words like `order`, `select`, `from`, `table`, and `group` break unquoted queries on every engine, and snake_case doesn't help because `order` stays `order`. Prefix with the source context (`source_order`), suffix with an underscore (`order_`), or accept that the column will always need quoting.

Syntactic characters – `@Status`, `#Temp`, `$Amount` – carry engine-specific meaning. SQL Server interprets `@` as a variable prefix and `#` as a temp table marker, so a column named `@Status` requires quoting there even

though PostgreSQL handles it fine. Strip or replace any character that has syntactic meaning on your destination.

Spaces require quoting on every engine without exception. Replace with underscores – the one normalization everyone agrees on.

Accented characters like `línea_factura` or `straße` are valid UTF-8 and every modern engine supports them, but BI tools and older ODBC connectors can choke. Replace accents at load time (`línea` → `linea`, `straße` → `strasse`), as the readability cost is negligible and you avoid your non-technical analyst blaming you for "breaking the data".

Collisions after normalization happen when a case-sensitive source has columns like `OrderID` and `orderid` that collapse to the same string after any normalization. Detect these at load time and fail loudly – a silent overwrite is worse than a broken load. Resolve by suffixing (`orderid`, `orderid_1`) and document the original-to-normalized mapping in column descriptions or a schema contract (6.9 – Data Contracts). Ugly, but it preserves every source column.

7.7.4 Schema Naming

Column naming decides what identifiers look like. Schema naming decides where tables live – which schema (PostgreSQL, Snowflake, SQL Server) or dataset (BigQuery) holds each source's data, and how consumers know where to look.

7.7.4.1 Connection as schema

One destination schema per source connection, named to encode both the server and the specific database. `production` as a schema name is useless when you pull from three production databases on two servers. `erp_prod_finance` tells you the server and the database in one glance.

When a single connection exposes multiple schemas – SQL Server defaults to `dbo`, PostgreSQL to `public`, MySQL has no schema layer at all – flatten them into the destination with a double underscore separator:

```
connection__schema.table
```

Single underscores already appear inside names (`order_lines`), so `__` is an unambiguous boundary:

Source	Source table	Destination table
PostgreSQL `erp_prod`, schema `public`	`public.orders`	`erp_prod__public.orders`
PostgreSQL `erp_prod`, schema `accounting`	`accounting.invoices`	`erp_prod__accounting.invoices`
SQL Server `crm_main`, schema `dbo`	`dbo.customers`	`crm_main__dbo.customers`
SQL Server `crm_main`, schema `sales`	`sales.leads`	`crm_main__sales.leads`
MySQL `shopify_prod` (database = schema)	`orders`	`shopify_prod.orders`
SAP B1 `sap_prod`, schema `dbo`	`dbo.OACT`	`sap_prod__dbo.OACT`

For MySQL and other engines where database and schema are the same thing, `connection__schema` collapses naturally – `shopify_prod.orders` instead of `shopify_prod__shopify_prod.orders`. But when a connection has a single schema that isn't the only one it *could* have (SQL Server with just `dbo`, PostgreSQL with just `public`), keep the full `connection__schema` form anyway. `erp_prod.orders` reads cleaner than

`erp_prod__public.orders` today, but the moment that server gets a second schema you're facing a rename across every table and every downstream reference. Use `connection__schema` from day one and the second schema slots in without touching anything that already exists.

You can extend the prefix with a business domain (`finance__erp_prod__accounting.invoices`) to group related schemas alphabetically, but this extra nesting is rarely worth it until your schema list outgrows a single screen.

7.7.4.2 Layer prefixes

Prefixing schemas with `raw_`, `bronze_`, or `landing_` marks the data layer: `raw__erp_prod.orders` versus `curated__erp_prod.orders`. The benefit is alphabetic grouping in catalog UIs and `INFORMATION_SCHEMA` queries – all raw schemas cluster together, all curated schemas cluster together. Apply layer prefixes consistently across every schema or not at all; a mix of prefixed and bare names is worse than no prefixes.

7.7.4.3 Opaque sources and layered schemas

Systems like SAP name every table with codes that mean nothing outside the source – `OACT`, `OINV`, `INV1`. The temptation to rename `OACT` to `chart_of_accounts` at load time is strong, especially when your analysts keep asking "what's OACT?", but that rename is semantic transformation and belongs in the serving layer, not the pipeline. Land the source name, use table metadata (column descriptions, table comments) to explain what it means, and let consumers discover the mapping without a separate lookup table.

I run a SAP B1 deployment where I landed everything raw at first – one schema, hundreds of opaque tables. It worked until it didn't scale. The approach that survived:

Schema	What lives here	How tables are named
`bronze__sap_b1__schema_1`	Raw landing from SAP schema 1	Source codes: `OACT`, `OINV`, `INV1`, `ORDR`, `RDR1`
`bronze__sap_b1__schema_2`	Raw landing from SAP schema 2	Source codes
`bronze__sap_b1__schema_3`	Raw landing from SAP schema 3	Source codes
`silver__sap_b1`	All bronze layers consolidated, enriched with metadata	Still source codes: `OACT`, `OINV` – same table, more columns
`gold__sap_b1`	Business-facing models	Human names: `chart_of_accounts`, `ar_documents`, `balance`

Bronze lands what SAP calls it, separated by source schema. Silver consolidates across schemas and enriches with metadata, joins, and deduplication, but the tables keep their opaque names – `OACT` is still `OACT`, just with more columns. Gold is where `OACT` finally becomes `chart_of_accounts`, because at that layer the consumers are analysts who have never logged into SAP. The semantic rename belongs here, not in the pipeline.

7.7.4.4 Staging conventions

Staging tables need their own namespace to avoid colliding with production. Table prefix (`stg_orders` in the same schema) or parallel schema (`orders_staging.orders`) – the tradeoffs are covered in 2.3 – Staging Swap.

7.7.5 Per-Table Overrides

The convention should be configurable at two levels: a destination-wide default that covers the common case, and per-table overrides for the exceptions. Collisions from character stripping are the most common reason you'll need them.

I learned this the hard way with a client who had `ProductStock` and `ProductStock$` in the same source – identical structure, one holding unit quantities and the other monetary values. My stripping rule removed the `$`, both tables landed as `product_stock`, and whichever loaded second silently overwrote the first. I didn't catch it until the numbers stopped making sense downstream. The fix was a per-table override renaming one to `product_stock_value` – a borderline semantic transformation, but better than losing data. The general rule works until it doesn't, and when it doesn't, the alternative to a per-table escape hatch is rewriting the entire convention.

6.9 – Data Contracts treats the naming convention as a schema contract – any change to it, including per-table overrides, is a breaking change that should go through the contract process.

7.7.6 Migrating a Convention

I've done this once. It was a week of hell – rebuilding tables, rewriting queries, repointing every report and dashboard that referenced the old names. I thought I was done by Friday. I wasn't. For three months afterward, people came back from vacation to broken dashboards, scheduled exports failed silently because nobody had updated the column references, and ad-hoc queries saved in personal notebooks kept surfacing the old names. Every time I thought I'd caught the last one, someone opened a ticket.

Don't do it if you can avoid it. If you can't, treat it as a formal breaking change: announce a cutover window, run a deprecation period where both conventions coexist (old names as views over new tables), and set a hard deadline for tearing down the aliases. And budget three months of intermittent cleanup after the deadline, because you will need them.

7.7.7 Anti-Patterns

Don't change convention on a running pipeline

Changing from camelCase to snake_case across 200 tables means rebuilding every table and updating every downstream query, view, and dashboard. The only thing better than perfect is **standardized**.

Don't rename source tables for readability

`OACT` ⟶ `chart_of_accounts` is a semantic rename; keep it in the serving layer. The pipeline lands what the source calls it. If consumers need readable names, build an alias layer downstream.

Don't mix conventions in one destination

Tables from source A in snake_case and tables from source B in camelCase within the same dataset confuses every consumer. Avoid when possible.

PART VIII: APPENDIX

8.1 SQL Dialect Reference

The lookup table for every operation that differs between engines. When a pattern in the book says "syntax varies by engine," it points here. Six engines are covered: PostgreSQL, MySQL, and SQL Server as sources and transactional destinations; BigQuery, Snowflake, ClickHouse, and Redshift as columnar destinations.

Quick nav

- #Identifier Quoting and Case Sensitivity
- #Timestamp and Datetime Types
- #Date and Time Functions
- MERGE
- #Append and Materialize
- #Table Swap
- #Partition Operations
- #Partition and Clustering DDL
- #Deduplication (QUALIFY vs Subquery)
- #Bulk Loading
- #JSON and Semi-Structured Data
- #Schema Evolution
- #Source-Specific Traps
- #Engine Quirks

8.1.1 Identifier Quoting and Case Sensitivity

Engine	Default case	Quote character	Example
PostgreSQL	Folds to lowercase	`"double quotes"`	`"OrderID"` preserves case
MySQL	Case depends on OS (Linux: sensitive, Windows: insensitive)	`` `backticks` ``	`` `Order ID` ``
SQL Server	Case-insensitive (collation-dependent)	`[brackets]` or `"double quotes"`	`[Order ID]`
BigQuery	Case-sensitive	`` `backticks` ``	`` `project.dataset.table` ``
Snowflake	Folds to uppercase	`"double quotes"`	`"order_id"` preserves lowercase
ClickHouse	Case-sensitive	`` `backticks` `` or `"double quotes"`	Names preserved exactly
Redshift	Folds to lowercase	`"double quotes"`	Same as PostgreSQL

See 8.1 – SQL Dialect Reference for naming strategy.

8.1.2 Timestamp and Datetime Types

Type	PostgreSQL	MySQL	SQL Server	BigQuery	Snowflake	ClickHouse	Redshift
Naive (no TZ)	`TIMESTAMP`	`DATETIME`	`DATETIME2(n)`	–	`TIMESTAMP_NTZ`	`DateTime`	`TIMESTAMP`
Aware (with TZ)	`TIMESTAMPTZ`	–	`DATETIMEOFFSET`	`TIMESTAMP`	`TIMESTAMP_TZ`	`DateTime64` with tz	`TIMESTAMPTZ`
Max precision	Microseconds	Microseconds	100 nanoseconds	Microseconds	Nanoseconds	Nanoseconds	Microseconds

> **BigQuery has no naive datetime**
>
> Every `TIMESTAMP` in BigQuery is UTC. Naive timestamps from the source land as UTC – if they were actually in `America/Santiago` or `Europe/Berlin`, every value is wrong from the moment it lands. Handle the timezone during load. See 5.5 – Timezone Handling.

> **DATETIME2 precision truncates on load**
>
> SQL Server DATETIME2(7) 100-nanosecond precision truncates to microseconds on BigQuery and Redshift. Snowflake's `TIMESTAMP_NTZ(9)` and ClickHouse's `DateTime64(7)` can preserve it.

See 5.3 – Type Casting and Normalization for the full type mapping.

8.1.3 Date and Time Functions

Operation	PostgreSQL	MySQL	SQL Server	BigQuery	Snowflake	ClickHouse
Subtract interval	`date - INTERVAL '1 day'`	`DATE_SUB(d, INTERVAL 1 DAY)`	`DATEADD(day, -1, d)`	`DATE_SUB(d, INTERVAL 1 DAY)`	`DATEADD(day, -1, d)`	`d - INTERVAL 1 DAY`
Add interval	`date + INTERVAL '1 day'`	`DATE_ADD(d, INTERVAL 1 DAY)`	`DATEADD(day, 1, d)`	`DATE_ADD(d, INTERVAL 1 DAY)`	`DATEADD(day, 1, d)`	`d + INTERVAL 1 DAY`
Truncate to month	`date_trunc('month', d)`	`DATE_FORMAT(d, '%Y-%m-01')`	`DATEFROMPARTS(YEAR(d), MONTH(d), 1)`	`DATE_TRUNC(d, MONTH)`	`DATE_TRUNC('month', d)`	`toStartOfMonth(d)`
Difference (days)	`d2 - d1` (returns integer)	`DATEDIFF(d2, d1)`	`DATEDIFF(day, d1, d2)`	`DATE_DIFF(d2, d1, DAY)`	`DATEDIFF(day, d1, d2)`	`dateDiff('day', d1, d2)`
Extract part	`EXTRACT(YEAR FROM d)`	`EXTRACT(YEAR FROM d)`	`DATEPART(year, d)` or `YEAR(d)`	`EXTRACT(YEAR FROM d)`	`EXTRACT(YEAR FROM d)` or `YEAR(d)`	`toYear(d)`
Current timestamp	`NOW()` or `CURRENT_TIMESTAMP`	`NOW()` or `CURRENT_TIMESTAMP`	`GETDATE()` or `SYSDATETIME()`	`CURRENT_TIMESTAMP`	`CURRENT_TIMESTAMP()`	`now()`

DATEDIFF argument order varies

MySQL and BigQuery put `DATEDIFF(end, start)`. SQL Server, Snowflake, and ClickHouse put the unit first: `DATEDIFF(day, start, end)`. PostgreSQL skips the function entirely and uses subtraction. Getting the argument order wrong produces results with the wrong sign.

8.1.4 Upsert / MERGE

BigQuery / Snowflake / SQL Server

```
MERGE INTO orders AS tgt
USING _stg_orders AS src
ON tgt.order_id = src.order_id
WHEN MATCHED THEN
  UPDATE SET
    tgt.status = src.status,
    tgt.total = src.total,
    tgt.updated_at = src.updated_at
WHEN NOT MATCHED THEN
  INSERT (order_id, status, total, created_at, updated_at)
  VALUES (src.order_id, src.status, src.total, src.created_at, src.updated_at);
```

PostgreSQL

```
INSERT INTO orders (order_id, status, total, created_at, updated_at)
SELECT order_id, status, total, created_at, updated_at
FROM _stg_orders
ON CONFLICT (order_id)
DO UPDATE SET
  status = EXCLUDED.status,
  total = EXCLUDED.total,
  updated_at = EXCLUDED.updated_at;
```

MySQL

```
INSERT INTO orders (order_id, status, total, created_at, updated_at)
SELECT order_id, status, total, created_at, updated_at
FROM _stg_orders
ON DUPLICATE KEY UPDATE
  status = VALUES(status),
  total = VALUES(total),
  updated_at = VALUES(updated_at);
```

ClickHouse – no MERGE statement, by design. The idiomatic upsert is to insert into a `ReplacingMergeTree` and let the engine collapse duplicates on merge (paid lazily, or eagerly via partition-scoped `OPTIMIZE`), with reads going through `SELECT ... FINAL` or an `argMax` view. The load is a pure append – no per-row cost, no partition rewrite. See 4.4 – Append and Materialize for the read-time dedup pattern this maps to.

```
-- engine: clickhouse
CREATE TABLE orders (
  order_id UInt64,
  status String,
  total Decimal(18,2),
  updated_at DateTime,
  _extracted_at DateTime DEFAULT now()
)
ENGINE = ReplacingMergeTree(_extracted_at)  -- version column: latest _extracted_at
wins
PARTITION BY toYYYYMM(updated_at)
ORDER BY order_id;

-- Every batch is a pure append:
INSERT INTO orders SELECT * FROM _stg_orders;

-- Reads see latest-per-key:
SELECT * FROM orders FINAL WHERE order_id = 42;
```

RMT gotchas

The version column must **not** be part of `ORDER BY` – if it is, `SELECT ... FINAL` returns duplicates (ClickHouse issue #69552). Pass it as the engine parameter only. Also: if your `PARTITION BY` expression depends on a column that can change between versions (e.g. partitioning on `updated_at` when updates land in a later month), the old and new versions live in different partitions and dedup is partition-scoped – the duplicates are permanent. Partition on something immutable for the key's lifetime.

Redshift – `MERGE` added in late 2023, same syntax as Snowflake/BigQuery. For older clusters or performance-sensitive loads, the classic pattern is DELETE + INSERT in a transaction.

Engine	Duplicate key in staging
BigQuery	Runtime error if multiple source rows match one destination row
Snowflake	Processes both rows, nondeterministic – undefined which wins
PostgreSQL	Processes rows in insertion order, last one wins
SQL Server	Runtime error on multiple matches (same as BigQuery)

See 4.3 – Merge / Upsert for cost analysis and when to use MERGE vs alternatives.

8.1.4.1 Dynamic MERGE Generation

Writing column-explicit MERGE statements by hand doesn't scale – 80-column tables mean 80 entries in three places, and every schema change is a drift risk (4.3.6 – MERGE and Schema Evolution). In production, generate the statement from the staging table's schema at runtime.

The building blocks per engine:

Engine	How to introspect staging columns
PostgreSQL	`SELECT column_name FROM information_schema.columns WHERE table_name = '_stg_orders'` – then build the `INSERT ... ON CONFLICT DO UPDATE SET` string in your language of choice
MySQL	Same `information_schema.columns` query. `ON DUPLICATE KEY UPDATE` needs `col = VALUES(col)` per non-key column
SQL Server	`sys.columns` joined to `sys.tables` – same approach, different catalog
BigQuery	`INFORMATION_SCHEMA.COLUMNS` per dataset, or the `bq show --schema` CLI. Build the MERGE string in Python/SQL
Snowflake	`INFORMATION_SCHEMA.COLUMNS` or `DESCRIBE TABLE _stg_orders` – same pattern

With SQLAlchemy, `inspect(engine).get_columns('_stg_orders')` returns the column list for any supported engine – one function call, no engine-specific catalog queries. Build the MERGE template once, parameterize per table.

8.1.5 Append and Materialize

The alternative to MERGE on columnar engines: append every extraction to a log table, deduplicate with a view. Load cost drops to near-zero (pure INSERT), and the dedup cost shifts to read time.

Append to log (all engines)

```
INSERT INTO orders_log
SELECT *, CURRENT_TIMESTAMP AS _extracted_at
FROM _stg_orders;
```

Dedup view – BigQuery / Snowflake / ClickHouse

```
CREATE OR REPLACE VIEW orders AS
SELECT *
FROM orders_log
QUALIFY ROW_NUMBER() OVER (PARTITION BY order_id ORDER BY _extracted_at DESC) = 1;
```

Dedup view – PostgreSQL / MySQL / SQL Server / Redshift

```
CREATE OR REPLACE VIEW orders AS
SELECT * FROM (
  SELECT *,
    ROW_NUMBER() OVER (
      PARTITION BY order_id
      ORDER BY _extracted_at DESC
    ) AS rn
  FROM orders_log
) sub
WHERE rn = 1;
```

Compaction – collapse to latest-only (BigQuery / Snowflake / ClickHouse)

```sql
CREATE OR REPLACE TABLE orders_log AS
SELECT *
FROM orders_log
QUALIFY ROW_NUMBER() OVER (PARTITION BY order_id ORDER BY _extracted_at DESC) = 1;
```

Keeps exactly one row per key regardless of age – safe with any extraction strategy. All version history is gone, but every current row survives. On engines without `QUALIFY`, use the subquery wrapper inside the `CREATE TABLE ... AS SELECT`.

See 4.4 – Append and Materialize for the full pattern, cost tradeoffs, and retention sizing.

8.1.6 Table Swap

Snowflake

```sql
ALTER TABLE stg_orders SWAP WITH orders;
```

Atomic, metadata-only. Grants follow the table object, not the name – after the swap, consumers querying the name `orders` see the old staging table's (empty) grants. Re-grant after every swap, or use `FUTURE GRANTS` on the schema.

BigQuery

```bash
# Copy job: works across datasets, free for same-region
bq cp --write_disposition=WRITE_TRUNCATE \
  project:dataset.stg_orders \
  project:dataset.orders
```

```sql
-- DDL rename: same dataset only, brief unavailability window
ALTER TABLE `project.dataset.orders` RENAME TO orders_old;
ALTER TABLE `project.dataset.stg_orders` RENAME TO orders;
DROP TABLE IF EXISTS `project.dataset.orders_old`;
```

`ALTER TABLE RENAME TO` does not cross dataset boundaries. Use the copy job for cross-dataset swaps or when consumers can't tolerate unavailability.

PostgreSQL / Redshift

```sql
BEGIN;
ALTER TABLE orders RENAME TO orders_old;
ALTER TABLE stg_orders RENAME TO orders;
DROP TABLE orders_old;
COMMIT;
```

Atomic within the transaction. If the transaction rolls back, `orders` is untouched.

ClickHouse

```
EXCHANGE TABLES stg_orders AND orders;
```

Atomic swap of both table names. The old production data moves to `stg_orders` after the swap.

See 2.3 – Staging Swap for the full pattern.

8.1.7 Partition Operations

BigQuery – partition copy

```
# Near-metadata operation, orders of magnitude faster than DML
bq cp --write_disposition=WRITE_TRUNCATE \
  project:dataset.stg_events$20260307 \
  project:dataset.events$20260307
```

Staging must be partitioned by the same column and type as the destination. One copy per partition, but each copy is near-free.

Snowflake / Redshift – DELETE + INSERT in transaction

```
BEGIN;
DELETE FROM events
WHERE partition_date BETWEEN :start_date AND :end_date;
INSERT INTO events SELECT * FROM stg_events;
COMMIT;
```

Delete by the declared range, not by what's in staging. If Saturday had rows last run and the source corrected them to Friday, a staging-driven delete would leave stale Saturday data in place.

ClickHouse – REPLACE PARTITION

```
ALTER TABLE events REPLACE PARTITION '2026-03-07' FROM stg_events;
ALTER TABLE events REPLACE PARTITION '2026-03-08' FROM stg_events;
```

Atomic per partition, operates at the partition level without rewriting rows.

See 2.2 – Partition Swap for the full pattern.

8.1.8 Partition and Clustering DDL

BigQuery

```
CREATE TABLE `project.dataset.events` (
  event_id STRING,
  event_type STRING,
  event_date DATE,
  payload JSON
)
PARTITION BY event_date
CLUSTER BY event_type
OPTIONS (require_partition_filter = true);
```

Up to 4 cluster columns. `require_partition_filter` rejects queries without a partition filter – mandatory cost protection on large tables. Limit: 10,000 partitions per table, 4,000 per job.

Snowflake

```
CREATE TABLE events (
  event_id VARCHAR,
  event_type VARCHAR,
  event_date DATE,
  payload VARIANT
)
CLUSTER BY (event_date, event_type);
```

Snowflake has no traditional partitions – micro-partitions are managed automatically. Clustering keys guide the physical layout. Snowflake auto-reclusters in the background (costs warehouse time).

ClickHouse

```
CREATE TABLE events (
  event_id String,
  event_type String,
  event_date Date,
  payload String
)
ENGINE = ReplacingMergeTree()
PARTITION BY toYYYYMM(event_date)
ORDER BY (event_date, event_id);
```

`ENGINE` is required. `ORDER BY` is the primary sort key and cannot be changed after creation. `PARTITION BY` supports expressions (`toYYYYMM`, `toDate`).

Redshift

```
CREATE TABLE events (
  event_id VARCHAR,
  event_type VARCHAR,
  event_date DATE,
  payload SUPER
)
SORTKEY (event_date)
DISTSTYLE KEY
DISTKEY (event_id);
```

Sort keys and dist keys are changeable via `ALTER TABLE`, but the rewrite runs in the background and can be slow on large tables – choose well at creation. Sort key serves the role of a partition/cluster key for scan pruning. Dist key controls how data distributes across nodes for join performance.

See 1.4 – Columnar Destinations for storage mechanics and 7.3 – Pre-Built Views for key selection.

8.1.9 Deduplication (QUALIFY vs Subquery)

BigQuery / Snowflake / ClickHouse – QUALIFY

```
SELECT *
FROM orders_log
QUALIFY ROW_NUMBER() OVER (
  PARTITION BY order_id
  ORDER BY _extracted_at DESC
) = 1;
```

PostgreSQL / MySQL / SQL Server / Redshift – subquery wrapper

```
SELECT * FROM (
  SELECT *,
    ROW_NUMBER() OVER (
      PARTITION BY order_id
      ORDER BY _extracted_at DESC
    ) AS rn
  FROM orders_log
) sub
WHERE rn = 1;
```

Same result, different syntax. `QUALIFY` filters directly on window functions without a subquery. Engines that don't support it need the subquery wrapper.

See 4.4 – Append and Materialize for dedup views and 6.13 – Duplicate Detection for detection patterns.

8.1.10 Bulk Loading

Engine	Primary method	Preferred format	Key constraint
BigQuery	`bq load` / `LOAD DATA` / streaming	Avro (handles JSON natively)	JSON columns can't load from Parquet
Snowflake	`COPY INTO` from stage	Parquet	`VARIANT` from Parquet lands as string, needs `PARSE_JSON`
ClickHouse	`INSERT INTO ... SELECT` (batch)	Parquet or native format	Small inserts cause too-many-parts; batch aggressively
Redshift	`COPY` from S3	Parquet	Row-by-row INSERT is orders of magnitude slower than COPY
PostgreSQL	`COPY` / `\copy`	CSV or binary	Binary is faster but not human-readable
MySQL	`LOAD DATA INFILE`	CSV	`LOAD DATA LOCAL INFILE` has security restrictions

See 1.4 – Columnar Destinations for format compatibility and gotchas.

8.1.11 JSON and Semi-Structured Data

Engine	Native type	Load from Parquet?	Query syntax
BigQuery	`JSON`	No – use JSONL or Avro	`JSON_VALUE(col, '$.key')`, dot notation
Snowflake	`VARIANT`	Lands as string, needs `PARSE_JSON`	`col:key::type`, `:` path notation
ClickHouse	`JSON` (v25.3+) or `String`	Yes (as string)	`JSONExtractString(col, 'key')` or native path access on `JSON` type
Redshift	`SUPER`	Yes	PartiQL syntax: `col.key`
PostgreSQL	`JSONB` / `JSON`	N/A (not a bulk load format)	`col->>'key'`, `col @> '{}'` operators
MySQL	`JSON`	N/A	`JSON_EXTRACT(col, '$.key')`, `col->>'$.key'`

See 5.7 – Nested Data and JSON for the syntactic strategy.

8.1.12 Schema Evolution

Operation	BigQuery	Snowflake	ClickHouse	Redshift
ADD COLUMN	Instant	Fast (metadata)	Metadata-only for MergeTree	Cheap if added at end
Type widening	Compatible pairs (`INT64` → `NUMERIC`)	VARCHAR width increase OK	Some widening via `MODIFY COLUMN`	Requires full table rebuild
DROP COLUMN	Destructive (breaks `SELECT *` downstream)	Fast	Supported	Supported
Column limit	10,000	No hard limit	No hard limit	1,600
Key limitation	–	CLONE picks up new columns automatically	`ORDER BY` key fixed at creation	Sort/dist keys changeable via `ALTER TABLE` (background rewrite)

See 1.4 – Columnar Destinations for full details and 6.9 – Data Contracts for schema policies.

8.1.13 Source-Specific Traps

- **PostgreSQL**: `TIMESTAMP` vs `TIMESTAMPTZ` confusion – both exist, applications mix them. TOAST compression on large columns can slow extraction on wide tables.
- **MySQL**: `utf8` is 3-byte UTF-8, not real UTF-8. `utf8mb4` is real UTF-8. If the source uses `utf8`, you might be getting truncated data. `DATETIME` has no timezone at all. `TINYINT(1)` is commonly used as a boolean but it's still an integer.
- **SQL Server**: `WITH (NOLOCK)` avoids blocking writers during extraction but reads dirty data (rows mid-transaction). `DATETIME2(7)` nanosecond precision truncates on most destinations. Getting read access to a production SQL Server often involves procurement, security reviews, and a DBA who has 47 other priorities.
- **SAP HANA**: Proprietary SQL dialect. Legally restricted access to some tables (S/4HANA). Varies by SAP module – extraction patterns that work for B1 may not apply to S/4. If you're extracting from SAP, you already know.

See 1.3 – Transactional Sources for the full terrain.

8.1.14 Engine Quirks

BigQuery - DML concurrency: max 2 mutating statements per table concurrently, up to 20 queued. Flood it and statements fail outright - Every DML rewrites entire partitions it touches – 10K rows across 30 dates = 30 full partition rewrites - Copy jobs are free for same-region operations - Streaming inserts: rows may be briefly invisible to `EXPORT DATA` and table copies (typically minutes, up to 90)

Snowflake - `PRIMARY KEY` and `UNIQUE` constraints are not enforced – they're metadata hints only. Deduplication is your problem - `VARIANT` from Parquet loads as string, not queryable JSON, until you `PARSE_JSON` - Result cache: identical queries within 24h return cached results at no warehouse cost - Grants follow the table object, not the name – after `SWAP WITH` or `CLONE`, re-grant or use `FUTURE GRANTS`

ClickHouse - `ALTER TABLE ... UPDATE` and `ALTER TABLE ... DELETE` are async – they return immediately, actual work happens during the next merge - `ReplacingMergeTree` deduplicates on merge rather than on insert, so duplicates coexist until the merge scheduler runs; `SELECT ... FINAL` forces read-

time dedup at a performance cost - Small inserts cause a “too many parts” error. Batch inserts into blocks of at least tens of thousands of rows - `ENGINE` is required in every `CREATE TABLE`. `ORDER BY` is fixed at creation

Redshift - `COPY` from S3 is the only performant bulk load. Row-by-row `INSERT` is orders of magnitude slower - Automatic VACUUM DELETE runs in the background for most cases, but manual VACUUM may still be needed after heavy bulk deletes or to reclaim sort order - Sort keys and dist keys are changeable via `ALTER TABLE`, but the background rewrite can be slow on large tables – plan them at creation - Hard limit of 1,600 columns per table

8.1.15 Engine Cheat Sheet

Engine	Load Method	Partition Mechanism	Mutation Cost	Key Gotcha
BigQuery	`bq load` / `LOAD DATA` / streaming	Date/integer/ ingestion-time	Entire partition rewrite	DML quotas; JSON + Parquet = failure
Snowflake	`COPY INTO` from stage	Micro-partitions + clustering keys	Warehouse time per merge	`VARIANT` -> string in Parquet; PK/unique not enforced
ClickHouse	`INSERT INTO` (batch)	Partition by expression	Async, no ACID	Duplicates until merge; `FINAL` is expensive
Redshift	`COPY` from S3	Sort key + dist key	Delete + VACUUM cycle	Row-by-row INSERT is orders of magnitude slower

8.2 Decision Flowchart

Three decisions drive every pipeline: how to extract, how to load, and how often to refresh. These flowcharts walk through each one, then map every table in the domain model to its recommended pattern combination.

8.2.1 Extraction Strategy

The default path is the shortest: if the table fits a full scan, use full replace and stop thinking. Every branch to the right adds complexity that should be earned, not assumed.

8.2.2 Load Strategy

On transactional destinations, MERGE is cheap – use it by default. On columnar destinations, append-and-materialize avoids the per-run MERGE cost and shifts deduplication to read time or a scheduled compaction job.

8.2.3 Freshness Tier

See 6.8 – Tiered Freshness for the full framework.

8.2.4 Domain Model Mapping

Every table in the domain model mapped to its recommended extraction, load, and freshness pattern:

Table	Extraction	Load	Freshness	Why
`orders`	Stateless window 7d (3.3 – Stateless Window Extraction)	Append-and-materialize (4.4 – Append and Materialize)	Hot + warm nightly reset	`updated_at` unreliable, hard deletes unlikely, high mutation rate
`order_lines`	Cursor from header (3.4 – Cursor from Another Table)	Same as `orders`	Same schedule as `orders`	No own timestamp, borrows from `orders`
`customers`	Full replace (2.1 – Full Scan Strategies)	Full replace (4.1 – Full Replace Load)	Warm (daily)	Dimension table, changes across full history, small enough to scan
`products`	Full replace (2.1 – Full Scan Strategies)	Full replace (4.1 – Full Replace Load)	Warm (daily)	Schema mutates, full replace catches everything
`invoices`	Open/closed split (3.7 – Open/Closed Documents)	Merge (4.3 – Merge / Upsert)	Hot for open, cold for closed	Hard deletes on open invoices, closed invoices frozen
`invoice_lines`	Open/closed from header (3.7 – Open/Closed Documents) + detail handling (3.8 – Detail Without Timestamp)	Same as `invoices`	Same schedule as `invoices`	Independent status changes, hard deletes not just cascade
`events`	Sequential ID cursor (3.5 – Sequential ID Cursor)	Append-only (4.2 – Append-Only Load)	Hot	Append-only, partitioned by date, never updated
`sessions`	Sequential ID or `created_at` cursor	Append-only (4.2 – Append-Only Load)	Hot	Late-arriving events need wider window (3.9 – Late-Arriving Data)
`metrics_daily`	Scoped full replace (2.4 – Scoped Full Replace)	Partition swap (2.2 – Partition Swap)	Warm (daily)	Pre-aggregated, overwritten daily, partition-aligned
`inventory`	Activity-driven (2.7 – Activity-Driven Extraction)	Staging swap (2.3 – Staging Swap)	Warm (daily) + monthly full	Sparse cross-product, activity-filtered extraction
`inventory_movements`	Sequential ID cursor (3.5 – Sequential ID Cursor)	Append-only (4.2 – Append-Only Load)	Hot	Append-only activity log

> **This mapping is a starting point**
>
> The recommended combination depends on the source system, the destination engine, and the consumer's SLA. A `customers` table with 500 rows doesn't need the same treatment as one with 5 million. Use the flowcharts to classify, then adjust based on what you learn about the source during the first few weeks of extraction.

8.3 Glossary

Append-and-materialize – Load strategy that appends every extraction as new rows to a log table and deduplicates to current state via a view. Avoids MERGE cost on columnar engines. See 4.4 – Append and Materialize.

Backfill – Reloading a historical date range or an entire table to correct accumulated drift, recover from corruption, or onboard a new table. See 6.11 – Backfill Strategies.

Batch ID (`_batch_id`) – Metadata column that correlates all rows from the same extraction run. Used for rollback, debugging, and reconciliation. See 5.1 – Metadata Column Injection.

Cold tier – Freshness tier for historical data refreshed weekly or monthly via full replace. Acts as the purity safety net. See 6.8 – Tiered Freshness.

Compaction – Collapsing an append log to one row per key, removing all historical versions. Always collapse-to-latest (`QUALIFY ROW_NUMBER() = 1`), never trim-by-date. See 4.4 – Append and Materialize.

Syntactic transformation – The form-level work data needs to survive moving between systems without changing meaning: type casting, metadata injection, null handling, charset encoding, key synthesis. If it changes business meaning, it belongs downstream in the serving layer. See 1.2 – What Is Syntactic Transformation.

Corridor – The combination of source type and destination type. Transactional -> Columnar (e.g. PostgreSQL -> BigQuery) or Transactional -> Transactional (e.g. PostgreSQL -> PostgreSQL). Same pattern, different trade-offs. See 1.7 – Corridors.

Cursor – A high-water mark (typically `MAX(updated_at)` or `MAX(id)`) used to extract only rows that changed since the last run. See 3.2 – Cursor-Based Timestamp Extraction.

Data contract – Explicit, checkable rules at the boundary between source and destination: schema shape, volume range, null rates, freshness. See 6.9 – Data Contracts.

Dedup view – A SQL view over an append log that uses `ROW_NUMBER() OVER (PARTITION BY pk ORDER BY _extracted_at DESC) = 1` to expose only the latest version of each row. See 4.4 – Append and Materialize.

Faithful replication – The goal of the pipeline: landing data in the destination so it matches the source exactly, with only the form-level changes needed to survive the crossing. See 1.1 – The EL Myth.

EL – Extract-Load with zero transformation. The theoretical ideal that never survives contact with real systems. See 1.1 – The EL Myth.

Evolve – Schema policy that accepts new columns from the source and adds them to the destination automatically. The recommended default for most tables. See 6.9 – Data Contracts.

Extracted at (`_extracted_at`) – Metadata column recording when the pipeline pulled the row, not when the source last modified it. Foundation for dedup ordering in append-and-materialize. See 5.1 – Metadata Column Injection.

Extraction gate – A check between extraction and load that blocks the load when the result looks implausible (0 rows from a table that normally has data, row count outside expected range). See 6.10 – Extraction Status Gates.

Freeze – Schema policy that rejects any schema change and fails the load. Reserved for tables with stable, critical schemas. See 6.9 – Data Contracts.

Freshness – How recently the destination reflects the source. The other end of the purity tradeoff. See 1.8 – Purity vs. Freshness.

Full replace – Drop and reload the entire table on every run. Stateless, idempotent, catches everything. The default until the table outgrows the scan window. See 2.1 – Full Scan Strategies.

Hard delete – A source row that was physically removed. Invisible to any cursor-based extraction. Requires a separate detection mechanism. See 3.6 – Hard Delete Detection.

Hard rule – A constraint enforced by the database: PK, UNIQUE, NOT NULL, FK, CHECK. If the system rejects violations at write time, it's hard. See 1.6 – Hard Rules, Soft Rules.

Health table – Append-only table with one row per table per pipeline run, capturing raw measurements (row counts, timing, status, schema fingerprint). See 6.2 – The Health Table.

Hot tier – Freshness tier for actively changing data refreshed multiple times per day via incremental extraction. See 6.8 – Tiered Freshness.

Idempotent – A pipeline that produces the same destination state whether it runs once or ten times with the same input. Full replace gets it for free; incremental has to earn it. See 1.9 – Idempotency.

Metadata columns – Columns injected during extraction that don't exist in the source: `_extracted_at`, `_batch_id`, `_source_hash`. See 5.1 – Metadata Column Injection.

Open document – A record that can still be modified (e.g. draft invoice, pending order). Contrast with closed document. See 3.7 – Open/Closed Documents.

Closed document – A record that is immutable (e.g. posted invoice). In many jurisdictions, modifying a closed invoice is illegal. See 3.7 – Open/Closed Documents.

Partition swap – Replace data at partition granularity without touching the rest of the table. See 2.2 – Partition Swap.

Purity – The degree to which the destination is an exact clone of the source at a given point in time. Full replace maximizes it; incremental carries purity debt. See 1.8 – Purity vs. Freshness.

QUALIFY – SQL clause that filters directly on window functions without a subquery. Native on BigQuery, Snowflake, ClickHouse. Not supported on PostgreSQL, MySQL, SQL Server, Redshift. See 8.2 – Decision Flowchart.

Reconciliation – Post-load verification that the destination matches the source: row count comparison, aggregate checks, hash comparison. See 6.14 – Reconciliation Patterns.

Schema policy – How the pipeline responds when the source schema changes. Two valid modes: evolve (accept) or freeze (reject). See 6.9 – Data Contracts.

Scoped full replace – Full-replace semantics applied to a declared scope (e.g. current year) while historical data outside the scope is frozen. See 2.4 – Scoped Full Replace.

SLA – Service Level Agreement. Four components: table/group, freshness target, deadline, measurement point. See 6.4 – SLA Management.

Soft rule – A business expectation with no database enforcement. "Quantities are always positive," "only open invoices get deleted." Your pipeline must survive these being wrong. See 1.6 – Hard Rules, Soft Rules.

Source hash (`_source_hash`) – Hash of all business columns at extraction time. Enables change detection without relying on `updated_at`. See 5.1 – Metadata Column Injection, 2.8 – Hash-Based Change Detection.

Staging swap – Load into a staging table, validate, then atomically swap to production. Zero downtime, trivial rollback. See 2.3 – Staging Swap.

Stateless window – Extract a fixed trailing window on every run with no cursor state between runs. The default incremental approach for most tables. See 3.3 – Stateless Window Extraction.

Synthetic key (`_source_key`) – A hash of immutable business columns, used as the merge key when the source has no stable primary key. See 5.2 – Synthetic Keys.

Tiered freshness – Splitting a pipeline into hot, warm, and cold tiers so tables are refreshed at the cadence that matches their consumption, not at a uniform schedule. See 6.8 – Tiered Freshness.

Warm tier – Freshness tier for recent data refreshed daily, typically overnight. The purity layer that catches what the hot tier missed. See 6.8 – Tiered Freshness.

8.4 Domain Model Quick Reference

Condensed reference for the shared fictional schema used in every SQL example. For the full description, ERD, and soft rule explanations, see Domain Model.

8.4.1 Tables at a Glance

Table	PK	Key columns	Pipeline role	Primary patterns
orders	order_id	customer_id, status, total, created_at, updated_at	Broken cursor showcase	3.1 – Timestamp Extraction Foundations, 3.3 – Stateless Window Extraction, 3.10 – Create vs Update Separation
order_lines	line_id	order_id, product_id, line_num, quantity, unit_price	Detail with no timestamp	3.4 – Cursor from Another Table, 3.8 – Detail Without Timestamp
customers	customer_id	name, email, is_active	Soft-delete dimension	2.1 – Full Scan Strategies, 1.6 – Hard Rules, Soft Rules
products	product_id	name, price, category	Schema drift case	2.1 – Full Scan Strategies, 1.5 – The Lies Sources Tell, 2.9 – Partial Column Loading
invoices	invoice_id	customer_id, status, doc_status, created_at, updated_at	Open/closed + hard deletes	3.6 – Hard Delete Detection, 3.7 – Open/Closed Documents
invoice_lines	line_id	invoice_id, product_id, quantity, unit_price, status	Independent detail lifecycle	3.8 – Detail Without Timestamp, 3.6 – Hard Delete Detection
events	event_id	event_type, event_date, payload	Append-only, partitioned	3.5 – Sequential ID Cursor, 4.2 – Append-Only Load
sessions	(implicit)	session_id, user_id, started_at	Late-arriving data	3.9 – Late-Arriving Data
metrics_daily	(composite)	metric_date, metric_name, value	Pre-aggregated, partition-replace	2.2 – Partition Swap, 2.4 – Scoped Full Replace
inventory	(sku_id, warehouse_id)	on_hand, on_order	Sparse cross-product	2.6 – Sparse Table Extraction, 2.7 – Activity-Driven Extraction
inventory_movements	movement_id	sku_id, warehouse_id, movement_type, quantity, movement_date	Activity signal, append-only	2.7 – Activity-Driven Extraction, 4.2 – Append-Only Load, 7.7 – Schema Naming Conventions

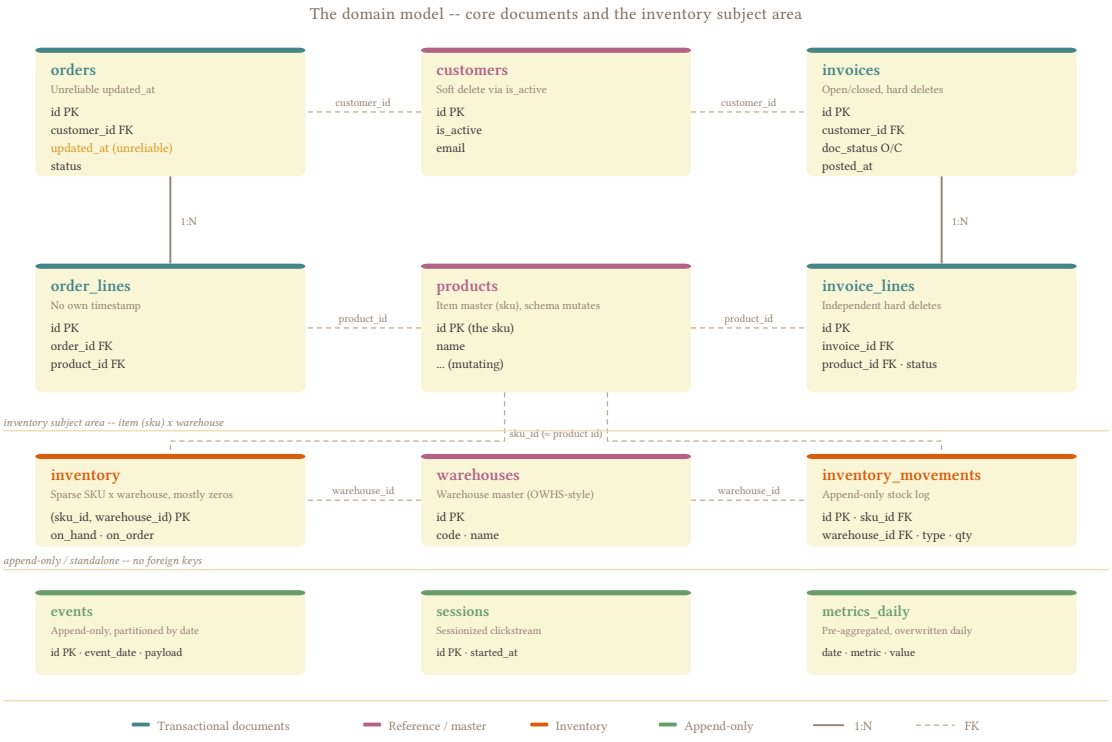
8.4.2 Soft Rules

Every "always true" business rule in the domain model is a soft rule – none have a database constraint enforcing them.

Table	Soft rule	How it breaks
`orders`	"Always has at least one line"	UI bug creates empty order
`orders`	"Status goes `pending` -> `confirmed` -> `shipped`"	Support resets manually
`order_lines`	"Quantities are always positive"	Return entered as `-1`
`invoices`	"Only open invoices get deleted"	Year-end cleanup script
`invoice_lines`	"Line status always matches header"	One line disputed independently
`customers`	"Emails are unique"	Duplicate registration, no unique index
`inventory`	"`on_hand` is always >= 0"	Write-off creates negative balance
`inventory_movements`	"Every stock change creates a movement"	Bulk import bypasses movement log

See 1.6 – Hard Rules, Soft Rules for why these matter and how your pipeline should handle violations.

8.4.3 Relationships



`events`, `sessions`, and `metrics_daily` have no foreign keys into the schema above. `inventory` and `inventory_movements` connect to `products` via `sku_id` but have no `warehouses` table – `warehouse_id` is a plain integer key.

8.5 Orchestrators

Every pattern in the book works regardless of tooling. This page names names.

An orchestrator schedules extractions, retries failures, and tracks what happened on each run. The relevant concerns are scheduling cadence (6.6 – Scheduling and Dependencies), tiered freshness (6.8 – Tiered Freshness), backfill execution (6.11 – Backfill Strategies), and health table population (6.2 – The Health Table). The three serious options for a Python-based stack are Dagster, Airflow, and Prefect – each models work differently, and the model shapes what's easy.

8.5.1 Feature Comparison

Concern	Dagster	Airflow 3	Prefect 3
Pipeline unit	Software-defined asset	`@asset` decorator (creates a DAG per asset) or traditional DAG + tasks	Flow + tasks
Scheduling	Schedules + Sensors	Cron, data-aware triggers, asset-aware scheduling	Deployment schedules, automations
Freshness	Freshness policies per asset, violations in UI	Deadline alerts (3.1), SLA callbacks on task duration	No native staleness tracking
Data quality	`@asset_check` inline after materialization	Custom operators, external tools	Artifacts + assertions
Backfill	Partition-based: select range in UI, per-partition retry	Scheduler-managed from UI (3.0): missing/all/failed runs, DAG-scoped	Parameterized reruns, no native partition concept
Metadata	`context.add_output_metadata({...})` per materialization	XComs, asset metadata for lineage	Artifacts on flow/task runs
Concurrency	Per-resource limits (e.g. 2 connections to source X)	Pool-based (N slots per pool)	Work pool limits
Managed offering	Dagster Cloud	Astronomer, MWAA, Cloud Composer	Prefect Cloud
Task SDK	Python	Python, Go, Java, TypeScript (3.0 Task SDK)	Python
Learning curve	Moderate – asset model requires rethinking	Low for DAG users, moderate for asset model	Low – decorators, minimal concepts

8.5.2 Dagster

Dagster's core abstraction is the **software-defined asset**: a function that produces a named data artifact, declared in code. One asset maps to one destination table – `orders`, `customers`, `events` – and the orchestrator tracks when each was last materialized, whether it's fresh, and what metadata the last run attached to it.

- **Partitioned assets** let you declare that `events` is partitioned by date, then backfill a range by selecting it in the UI. The orchestrator chunks the range into partition runs, respects concurrency limits, and tracks success per partition. Prefer monthly partitions over daily – a yearly backfill with daily partitions

spawns 365 individual runs with their own metadata and UI entries, while monthly gives you 12 with the same per-partition retry.

- **Asset checks** (`@asset_check`) run inline after materialization: row count validation, null rate thresholds, schema drift detection. Maps directly to 6.9 – Data Contracts and 6.10 – Extraction Status Gates.
- **Freshness policies** declare how stale an asset is allowed to be. Violations surface in the UI and trigger alerts – the 6.4 – SLA Management SLA expressed as a one-liner in the asset definition.
- **Custom metadata per materialization** (`context.add_output_metadata({"row_count": n})`) feeds the health table (6.2 – The Health Table) as a side effect of every run, with no explicit INSERT required.
- **Sensors** trigger runs from external events. I use sensors to let dashboard admins trigger an on-demand refresh of the tables behind their reports, which means the pipeline only needs to run once daily while consumers who need fresher data pull it when they actually need it – without a high-frequency schedule running for data nobody checks until 10 AM.
- **Concurrency limits per resource** cap concurrent extractions against a single source without global semaphores. At scale – thousands of tables across dozens of sources – this is what keeps the pipeline from overloading its own clients.

> **Stateless by default**
>
> Dagster's asset model encourages stateless pipelines: each materialization reads from the source and writes to the destination with no persisted cursor between runs. Incremental cursors (3.2 – Cursor-Based Timestamp Extraction) can live in Dagster's built-in cursor mechanism or in the destination itself, but the orchestrator doesn't force a state store. This aligns with the 1.9 – Idempotency goal.

Where it costs you:

- The asset abstraction requires rethinking pipeline structure, especially coming from a DAG/task mental model. The learning curve is real and takes a few weeks.
- Smaller community and fewer pre-built connectors than Airflow's ecosystem.
- Multi-table operations (extract 5 tables from one API call, split them) need multi-asset functions, which are more awkward than single-asset definitions.
- Dagster Cloud's pricing is credit-based (per materialization), which can add up at high table counts with frequent schedules. Self-hosting on Kubernetes is the alternative but requires platform engineering.

8.5.3 Airflow

Airflow is the most widely deployed orchestrator in the data ecosystem. Its traditional model is the **DAG** – a directed acyclic graph of tasks – and Airflow 3.0 (April 2025) added an `@asset` decorator that brings asset-oriented thinking into the framework alongside the existing DAG model.

Airflow 3 is a substantial release: asset-aware scheduling, scheduler-managed backfills with a UI, a new Task SDK that supports Go/Java/TypeScript alongside Python, DAG versioning, event-driven scheduling, and deadline alerts in 3.1. The gap between Airflow and Dagster narrowed significantly with this release.

- Widest connector ecosystem of any orchestrator – if a source system has an API, there's probably an Airflow provider package for it.
- Pool-based concurrency control is straightforward: define a pool with N slots, assign tasks to it, and Airflow queues the rest.
- Backfills in 3.0 are scheduler-managed and triggerable from the UI with configurable reprocessing (missing, all, or failed runs) – a major improvement over 2.x's CLI-only `airflow dags backfill`.

They're still DAG-scoped, so backfilling `orders` for March reruns the entire DAG for that range including other tables in it.
- Mature managed offerings (Astronomer, AWS MWAA, Cloud Composer) all support Airflow 3 and handle infrastructure.
- The `@asset` decorator creates a DAG per asset with asset-aware scheduling, which means you can trigger downstream work when an upstream asset updates. The model is conceptually similar to Dagster's assets but architecturally different – each `@asset` is its own DAG, and cross-asset data passes through XComs rather than through a shared graph context.
- The team already knows it, and that matters more than any feature comparison.

Where it needs more wiring:

- Populating the health table (6.2 – The Health Table) with structured run metrics (row counts, durations, schema hashes) still requires explicit code per task. Asset metadata in 3.0 is oriented toward lineage tracking rather than the kind of per-run operational metrics that Dagster's `add_output_metadata` captures.
- SLA miss callbacks track task duration, and 3.1's deadline alerts add proactive monitoring on schedules – but neither directly measures data freshness as 6.4 – SLA Management defines it. You still need your own staleness query.
- XComs improved in 3.0 but remain the primary mechanism for passing structured data between tasks, and at scale (hundreds of tables) the ergonomics for metadata like row counts and schema hashes feel heavier than Dagster's built-in approach.

> **One DAG per source system**
>
> Group tables by source system, with each table as a task within the DAG. One DAG per table creates hundreds of DAGs that overwhelm the scheduler and UI. One monolithic DAG creates a single point of failure where a stuck extraction blocks everything downstream. The per-source structure groups tables that share connection limits and scheduling cadence while keeping the blast radius of a failure scoped to one source.

8.5.4 Prefect

Prefect 3 (September 2024) brought the events and automation system to open source, added a transactional interface for idempotent pipelines, and significantly improved performance for distributed workloads. The API is genuinely pleasant – `@flow` on a Python function and it's orchestrated – and Prefect Cloud removes infrastructure concerns for small-to-medium deployments.

- Python-native API with minimal boilerplate. The gap between "script that works" and "orchestrated pipeline" is the smallest of the three tools.
- Automations (trigger actions on flow/task state changes, external events) provide flexible alerting and event-driven scheduling.
- Ephemeral infrastructure via work pools – Prefect spins up ECS tasks or Kubernetes jobs per run and deprovisions after completion, which keeps costs low for bursty workloads.
- The transactional interface lets you group tasks into transactions with automatic rollback on failure, which helps with the idempotency goals from 1.9 – Idempotency.

Where it's limited at scale:

- No native partition concept. Backfilling a date range means parameterizing the flow and triggering N runs manually – the orchestrator doesn't know they form a logical unit.

- No first-class freshness tracking or per-asset metadata. The health table (6.2 – The Health Table) is entirely your responsibility.
- Flow-level concurrency from Prefect 2 was removed in 3.0, replaced by a combination of global concurrency limits, work pool limits, and work queue limits – functional but less ergonomic.
- At scale (thousands of tables), the flow-per-table model generates UI clutter without the asset lineage graph that helps navigate large Dagster installations.

8.5.5 Other Tools

- **Kestra** – Event-driven, YAML-based. Good for non-Python teams and polyglot pipelines, with a visual flow editor. The tradeoff is losing Python-native advantages for data engineering.
- **Mage** – Notebook-like UI, promising for exploratory work but less mature for production at scale. The UI also gets painfully slow over time, and is by far the most volatile one.
- **cron + scripts** – Acceptable for < 10 tables with no dependencies and no backfill needs. Falls apart the moment you need retries, visibility, or any coordination between jobs.

> **Never build your own orchestrator**
>
> Every team that builds a custom orchestrator eventually rebuilds 60% of Airflow, poorly. The "we just need a simple scheduler" conversation leads to a homegrown system with no UI, no backfill capability, no alerting, and a bus factor of one. Use a real orchestrator and spend the engineering time on the pipelines.

8.5.6 My Recommendation

For a new project, start with **Dagster**. The asset model maps 1:1 to the "one asset = one destination table" structure this book is built around, partition-based backfills are the hardest thing to build from scratch, and inline asset checks plus freshness policies implement half of Part VI as configuration. The asset graph and partition-based backfills have justified the learning curve many times over.

Airflow is a strong choice when the team already runs it, when you need the widest connector ecosystem, or when Airflow 3's asset model and managed backfills cover your needs without Dagster's steeper learning curve. The 3.0 release closed many of the gaps that used to make the comparison one-sided – asset-aware scheduling, UI-managed backfills, and the Task SDK are real improvements. Structure one DAG per source system, add health table inserts per task, and it works well. If you're still on Airflow 2.x, upgrading to 3 is worth the effort before considering a migration to a different tool.

Prefect is the right pick for smaller teams (< 500 tables) that value developer velocity and don't need partition-aware backfills or per-asset freshness tracking. Prefect Cloud removes infrastructure overhead entirely, and the transactional interface aligns well with idempotency goals. Move to Dagster when backfill complexity or table count outgrows it.

Scenario	Recommendation
New project, pipeline-focused	Dagster
Existing Airflow, already on 3.x or upgrading	Stay on Airflow 3
Existing Airflow 2.x	Upgrade to 3 before considering migration
Small team, < 500 tables, no platform engineer	Prefect Cloud
Non-Python team or polyglot stack	Kestra or Airflow 3 (Task SDK supports Go/Java/TypeScript)

8.6 Extractors and Loaders

8.6.1 The Spectrum

Extractor/loader tools sit on a spectrum from fully managed to fully custom. On the managed end, Fivetran handles everything – connectors, scheduling, schema decisions, infrastructure – and you accept whatever it decides. On the custom end, you write Python with SQLAlchemy, own every line, and maintain every failure mode. In between, Airbyte gives you managed connectors with more visibility into what they do, and dlt gives you a Python library that handles the plumbing while leaving schema control, deployment, and orchestration in your hands.

Where you belong on this spectrum depends on how many sources you need to cover, how much control you need over the syntactic layer (1.2 – What Is Syntactic Transformation), whether someone else's schema decisions are acceptable for your destination, and price. A self-built stack running dlt on your own infrastructure with BigQuery as the destination can run thousands of tables for a few hundred dollars a month in compute and storage. The same workload on Fivetran costs an order of magnitude more because you're paying per row, per sync, per connector – and you're paying for the engineering you didn't have to do, which is a valid tradeoff only if you genuinely don't have the engineering capacity.

8.6.2 Comparison

Tool	Type	Schema control	Incremental	Naming	Deployment	Best for
Fivetran	Fully managed	None – Fivetran decides	Built-in cursors	Fivetran decides	SaaS	Teams without engineering capacity
Airbyte	Semi-managed	Limited – normalization layer	Built-in per connector	Configurable	Cloud or self-hosted	SaaS sources (Salesforce, Stripe)
dlt	Python library	Full – schema contracts, naming conventions	Cursor or stateless window	Configurable (`snake_case` default)	You deploy it	SQL sources, custom APIs, full control
Custom Python	Code	Total	You build it	You decide	You deploy it	Legacy/niche sources, extreme requirements

8.6.3 dlt

dlt is an open-source Python library for loading data into warehouses. It handles type inference, Parquet/JSONL serialization, destination-specific load jobs, and schema evolution – the plumbing that every loader needs and nobody wants to build twice.

8.6.3.1 The Standard Way: sql_database and sql_table

dlt ships with a `sql_database` source that reflects an entire database via SQLAlchemy and yields every table as a resource, and a `sql_table` function for extracting individual tables. For most teams getting started, this is the right entry point:

```
from dlt.sources.sql_database import sql_database, sql_table

# Full database: reflect all tables, load everything
source = sql_database(
    connection_url="postgresql://user:pass@host/db",
    schema="public",
    backend="pyarrow",  # or "sqlalchemy", "pandas", "connectorx"
)

# Single table with incremental merge
orders = sql_table(
    connection_url="postgresql://user:pass@host/db",
    table="orders",
    incremental=dlt.sources.incremental("updated_at"),
    primary_key="order_id",
    write_disposition="merge",
    backend="pyarrow",
)

pipeline = dlt.pipeline(
    pipeline_name="my_pipeline",
    destination="bigquery",
    dataset_name="raw_erp",
)
pipeline.run(source)
```

The `sql_database` source handles schema reflection, type mapping, and batching automatically. Four backends are available: `sqlalchemy` (default, yields Python dicts), `pyarrow` (yields Arrow tables – significantly faster for columnar destinations), `pandas` (yields DataFrames), and `connectorx` (parallel reads for large tables).

Callbacks let you customize behavior per table without writing custom extraction code:

- **`table_adapter_callback`** – receives each reflected table and lets you modify which columns get extracted, add computed columns, or skip tables entirely. This is where you'd exclude PII columns (2.9 – Partial Column Loading) or add metadata columns.
- **`type_adapter_callback`** – overrides SQLAlchemy type mappings. If your source has `FLOAT` columns that should land as `DECIMAL`, this is where you fix it before any data moves.
- **`query_adapter_callback`** – modifies the SELECT query before execution. Add WHERE clauses for scoped extraction (2.4 – Scoped Full Replace), change the ORDER BY for cursor alignment, or inject hints for the source query planner.

For incremental loading, dlt tracks cursor state internally via `dlt.sources.incremental()` – it stores the last value in a `_dlt_pipeline_state` table on the destination and picks up where it left off on the next run. Incremental requires a primary key on the resource so dlt can merge correctly; without one, you're appending duplicates on every run. This works well for simple cursor-based patterns (3.2 – Cursor-Based Timestamp Extraction) where you trust `updated_at` and want the library to manage state for you.

8.6.3.2 Schema Contracts

dlt's schema contract system controls what happens when the source sends something unexpected. Four modes per entity (`tables`, `columns`, `data_type`): `evolve` (accept it), `freeze` (fail the pipeline), `discard_row` (drop the row), `discard_value` (drop the value).

```
# Permissive: evolve everything, let the pipeline adapt
pipeline.run(source, schema_contract={"tables": "evolve", "columns": "evolve"})

# Conservative: freeze tables and types, evolve columns only
pipeline.run(source, schema_contract={"tables": "freeze", "columns": "freeze"})
```

I run permissive (`evolve`/`evolve`) in production because at scale the alternative is a constant stream of freeze-triggered failures from ERP modules being activated, schema migrations, and column additions that are all legitimate. The monitoring layer (6.9 – Data Contracts) catches what matters; the pipeline keeps running.

The conservative option makes sense when you have a small number of high-value tables where a schema surprise should stop the pipeline – freeze tables to prevent junk table creation from source bugs, freeze types so a `VARCHAR` that suddenly arrives as `INT64` doesn't silently corrupt downstream queries.

One thing to know about `data_type: "evolve"`: when a column's type changes, dlt creates a variant column alongside it – `amount__v_text` next to the original `amount` – so old data stays intact while new rows land in the variant. Variant columns can accumulate if the source is messy with types.

> **Discard modes break the replication boundary**
>
> Silently dropping rows or values means your destination no longer mirrors the source – you've introduced an invisible filter that nobody downstream knows about. For extract-and-load workloads where the goal is faithful replication, stick to `evolve` and `freeze`. See 1.2 – What Is Syntactic Transformation.

8.6.3.3 Naming Conventions

dlt normalizes all identifiers through a naming convention before they reach the destination. The default is `snake_case` – lowercased, ASCII only, special characters stripped. Other options include `duck_case` (case-sensitive Unicode), `direct` (preserve as-is), and SQL-safe variants (`sql_cs_v1`, `sql_ci_v1`).

This is a one-time decision with permanent consequences – the same tradeoff described in 8.1 – SQL Dialect Reference. Changing the convention after data exists is destructive: dlt re-normalizes already-normalized identifiers (it doesn't store the originals), which means every table and column name in your destination could change.

Normalization can collide source keys

dlt detects some collision types (case-sensitive convention on a case-insensitive destination, convention changes on existing tables) but does not detect collisions in the source data itself. If two dictionary keys or column names normalize to the same identifier under `snake_case`, they merge silently – the last value wins. This is rare in SQL tables (column names are unique at the source) but common in JSON/dict sources where keys like `ProductID` and `product_id` can coexist. Audit nested or dict-based sources before the first load.

8.6.3.4 Destination Gotchas

Destination engines have format-specific limitations that dlt inherits:

- **BigQuery**: cannot load JSON columns from Parquet files – the job fails permanently. Use JSONL or Avro for tables with JSON columns.
- **Snowflake**: `VARIANT` columns loaded from Parquet land as strings, not queryable JSON. Downstream queries need `PARSE_JSON()` to unwrap them. `PRIMARY KEY` and `UNIQUE` constraints are metadata-only.
- **PostgreSQL**: the default `insert_values` loader generates large INSERT statements. Switching to the CSV loader (`COPY` command) is several times faster.

8.6.3.5 Stateless Operation

dlt persists pipeline state in a local directory (schema cache, pending packages, load history) and in the destination (`_dlt_version`, `_dlt_pipeline_state`). For stateless operation (1.9 – Idempotency), delete the pipeline directory before every run to prevent stale caches from causing errors on staging tables that were cleaned up after the last merge.

Even with a clean local directory, dlt caches schema metadata in the destination's `_dlt_version` table. If a staging table is deleted after merge but the destination-side cache survives, the next load can skip table creation and fail. Use dlt's `refresh="drop_resources"` mechanism or delete cache entries before each load.

Combined with a stateless trailing-window extraction (3.3 – Stateless Window Extraction), the pipeline has no persisted state between runs – every execution is independent and idempotent.

8.6.3.6 Going Custom

At scale, I don't use `sql_database` or `sql_table` – I use dlt as a loader only and build extraction, merge, and schema evolution myself. The reasons are specific to my workload (thousands of tables, custom partition-pruned merges, PyArrow batching for performance), and most teams won't need this level of control. But if you outgrow the standard `sql_table` path, here's what I replaced and why:

- **Extraction**: custom `@dlt.resource` functions with manual SQL via SQLAlchemy instead of `sql_table`. I build the query myself (including the WHERE clause for trailing-window extraction) and yield PyArrow tables via dlt's `row_tuples_to_arrow` helper – significantly faster than dict-based iteration for large tables.
- **Merge**: custom DELETE+INSERT+QUALIFY in a BigQuery transaction instead of dlt's built-in merge. dlt's merge rewrites all touched partitions; ours prunes the DELETE to only the months that appear in the staging data, which matters when a 7-day trailing window touches rows across 2-3 partition months on a table with years of history.
- **Schema evolution**: custom `ALTER TABLE ADD COLUMN` before the merge step, with a mapping for BigQuery's legacy type names (`FLOAT` ⟶ `FLOAT64`, `INTEGER` ⟶ `INT64`) that the schema API returns.

- **Incremental state**: no `_dlt_pipeline_state` table. The trailing window (3.3 – Stateless Window Extraction) means every run re-extracts the same N-day range regardless of what happened before – no cursor to track, no state to corrupt.

dlt still handles the load job itself (`pipeline.run()` with `write_disposition="replace"` to staging), Parquet serialization, `_dlt_id`/`_dlt_load_id` generation, schema contracts, and naming conventions. The library earns its place even when you bypass most of its extraction and merge machinery.

> **dlt and append-and-materialize**
>
> dlt's three write dispositions are `replace`, `append`, and `merge`. There's no built-in support for the append-and-materialize pattern from 4.4 – Append and Materialize – appending every extraction to a log table and deduplicating via a view. You can use `write_disposition="append"` to build the log, but the dedup view, compaction job, and materialization schedule are entirely yours to build and maintain outside of dlt. If append-and-materialize is your primary load strategy for columnar destinations, know that dlt handles the append step but everything after it – the view, the compaction, the partition management – is custom SQL you manage separately.

8.6.4 Airbyte

Airbyte provides a catalog of managed connectors – pre-built extractors for SaaS APIs (Salesforce, HubSpot, Stripe, Jira) and databases (PostgreSQL, MySQL). Each connector handles authentication, pagination, rate limiting, and incremental state. Available as a cloud service or self-hosted via Docker.

Where it works well: SaaS sources where you don't have direct database access and the API is the only option. Writing a Salesforce extractor from scratch means handling OAuth refresh, query pagination, bulk API vs REST API selection, and field-level security. Airbyte's connector does this, and when it works, it saves weeks. CDC support for PostgreSQL and MySQL is available through Debezium-backed connectors, which gives you change streams without managing Debezium infrastructure directly.

Where it gets complicated: Airbyte applies a normalization step after extraction – flattening nested JSON, renaming columns, and creating sub-tables for arrays. This is a transformation step you may not want, sitting between your source and your destination without your explicit control. Connector quality varies significantly; some are maintained by Airbyte's core team, others by the community, and community connectors break on edge cases that the core team never tested. The self-hosted (OSS) version requires Docker infrastructure and has no built-in orchestration – you schedule syncs externally or use the cloud tier, which imposes sync frequency minimums that may not match your freshness requirements.

> **Check the connector support level**
>
> Airbyte classifies connectors as Generally Available, Beta, or Alpha. For production pipelines, stick to GA connectors. Beta and Alpha connectors change their schemas across versions, which means your downstream queries break when Airbyte pushes an update.

8.6.5 Fivetran

Fully managed, zero code, zero infrastructure. You authenticate a source, pick a destination, set a sync schedule, and Fivetran handles everything else. For teams without engineering capacity or for SaaS sources where the connector exists and works well, this is the fastest path to having data in your warehouse.

The tradeoff is control. Fivetran decides column types, naming conventions, and how to handle nested data. You can't inject metadata columns (5.1 – Metadata Column Injection), can't control the schema contract (6.9 – Data Contracts), and can't customize the merge strategy. What lands in your destination is what Fivetran decided, and if that decision is wrong for your use case, your only recourse is a support ticket.

Fivetran does add its own metadata columns (`_fivetran_synced`, `_fivetran_deleted`) and handles soft deletes for some connectors. These are useful but non-standard – your downstream queries become Fivetran-aware, which creates coupling that matters if you ever migrate off the platform.

Cost: priced by Monthly Active Rows (MAR). Affordable for small volumes, expensive at scale – a table that re-extracts 10 million rows monthly on a trailing window costs the same as 10 million unique rows. Sync frequency minimum is 5 minutes on the standard tier, 1 minute on business/enterprise. At scale – hundreds or thousands of tables – Fivetran's pricing becomes a serious constraint; the math works best when you have a few dozen high-value SaaS sources and the engineering team to maintain them doesn't exist.

8.6.6 Custom Python + SQLAlchemy

When the source is niche enough that no connector exists – a legacy ERP with a proprietary database, a vendor-specific API with no public documentation, a mainframe behind three layers of VPN – you write it yourself.

SQLAlchemy is the universal connector for SQL sources. It covers PostgreSQL, MySQL, SQL Server, SAP HANA, and dozens of other databases with a unified API for connection management, query execution, and type introspection. For extraction specifically, three backends cover most needs:

- **SQLAlchemy** (universal): works everywhere, reasonable performance, handles all types.
- **PyArrow**: fast columnar reads, good for wide tables headed to columnar destinations. Doesn't handle every type (JSONB on PostgreSQL, for example).
- **ConnectorX**: parallel reads that saturate the network. Best for large tables where single-threaded extraction is the bottleneck.

> **Custom extractors accumulate**
>
> Every custom extractor is a maintenance surface. After a year, you'll have 15 of them, each with slightly different error handling, slightly different retry logic, and slightly different assumptions about how types map. If you find yourself writing the third custom extractor, evaluate whether dlt or another library can absorb the common plumbing before the codebase becomes a collection of snowflakes.

The cost is everything else. Schema evolution, error handling, retry logic, state management, observability – dlt and Airbyte handle these as features, and with custom code, they're your problem. You also own type mapping: deciding that a SQL Server `DATETIME2(7)` should land as `TIMESTAMP` in BigQuery (truncating nanoseconds to microseconds) is now an explicit choice you make in code, not something a library infers for you.

Worth it when no alternative exists or when the extraction logic is complex enough that a generic tool gets in the way. Most production pipelines end up with at least a few custom extractors for the sources that no tool covers.

8.6.7 Decision Table

Source type	Recommended	Why
Direct DB access, SQL sources	dlt or custom SQLAlchemy	Full control over extraction and syntactic transformation
SaaS APIs (Salesforce, Stripe)	Airbyte or Fivetran	Managed connectors handle auth, pagination, rate limits
File-based (S3, SFTP, CSV drops)	dlt or custom	Connector overhead not justified for file reads
Legacy/niche sources	Custom SQLAlchemy	No connector exists
Team without engineering capacity	Fivetran	Zero code, zero ops

> **Mix and match tools freely**
>
> Running dlt for your SQL sources and Fivetran for two SaaS APIs is a perfectly valid architecture. The destination doesn't care which tool loaded the data, as long as your naming convention and metadata columns are consistent across all of them.

8.7 Destinations

1.4 – Columnar Destinations covers how columnar engines store, partition, and price data. 7.5 – Cost Optimization by Engine covers the cost levers once data is loaded. This page is the decision: which engine for which workload, and what to watch out for when running pipelines against each one.

8.7.1 Cost Model Comparison

Engine	Billing model	What you optimize	Cost guardrails
BigQuery	Per TB scanned (on-demand) or slots (reservations)	Bytes scanned per query	Per-query and per-day byte limits, `require_partition_filter`
Snowflake	Per second of warehouse compute	Query runtime, warehouse idle time	Auto-suspend, resource monitors, warehouse sizing
ClickHouse	Self-hosted infrastructure (or ClickHouse Cloud RPU)	Query speed on fixed hardware	Infrastructure budget
Redshift	Per node per hour (provisioned) or RPU-second (Serverless)	Cluster utilization or query compute time	Query monitoring rules, WLM queues
PostgreSQL	Self-hosted or managed instance (RDS, Cloud SQL)	Instance size, connection count	Fixed monthly cost regardless of query volume
DuckDB / MotherDuck	Free locally; MotherDuck $0.15/GB scanned (higher unit price than BQ, lower total bills on moderate data)	Query efficiency (local); GB scanned (MotherDuck)	Per-second billing, no idle tax, Duckling size limits

8.7.2 BigQuery

Best for: serverless pay-per-query, many ad-hoc consumers, Google Cloud native stacks. This is my primary destination – BigQuery's cost model rewards exactly what these pipelines produce: partition-scoped writes, partition-filtered reads, and bulk loads over row-by-row DML.

Pipeline strengths:

- `require_partition_filter` is the only engine with query-cost enforcement built into the table definition – consumers literally cannot full-scan without a partition predicate
- Copy jobs are free for same-region operations, making partition swap and staging swap nearly zero-cost
- `QUALIFY` is native, so dedup views and compaction queries are clean single statements
- Per-day cost limits prevent runaway retry loops from burning through the budget overnight

Pipeline weaknesses:

- DML concurrency caps at 2 concurrent mutating statements per table, with up to 20 queued – flood it and statements fail outright
- Every MERGE or UPDATE rewrites entire partitions it touches, so a 10-row update across 30 dates triggers 30 full partition rewrites
- JSON columns can't load from Parquet – use Avro or JSONL for tables with JSON fields
- 10,000 partition limit per table (4,000 per single job), constraining daily-partitioned tables to 27 years of history
- Rows inserted via the streaming buffer may be invisible to `EXPORT DATA` and table copy jobs for up to 90 minutes – use batch load jobs instead of streaming inserts if your pipeline chains a load with an immediate copy

8.7.3 Snowflake

Best for: predictable budgets, multi-workload isolation, data sharing, semi-structured data. Good for teams that need warehouse-level isolation between workloads: a small warehouse for extract-and-load, a medium one for analyst queries, a large one for dashboard refreshes, each with its own auto-suspend and budget ceiling.

Pipeline strengths:

- `VARIANT` handles arbitrary JSON natively with `:` path notation, no schema needed at load time
- `SWAP WITH` is atomic metadata-only swap – staging swap completes in milliseconds regardless of table size
- Result cache returns identical queries within 24 hours at zero warehouse cost
- Micro-partition pruning is automatic without explicit partition DDL

Pipeline weaknesses:

- `PRIMARY KEY` and `UNIQUE` constraints are metadata hints only – deduplication is entirely your responsibility
- Grants follow the table object, not the name – after `SWAP WITH` or `CLONE`, consumers lose access unless you re-grant or use `FUTURE GRANTS`
- Reclustering costs warehouse credits in the background; heavily mutated tables accumulate significant charges
- No partition filter enforcement – consumers can full-scan any table without warning

8.7.4 ClickHouse

Best for: append-heavy analytical workloads, high-frequency incremental loads, real-time dashboards, self-hosted control. ClickHouse is the strongest fit for pipelines that need many cheap incremental loads per day: `ReplacingMergeTree` (RMT) turns every upsert into a pure append and gives you an explicit knob for when to pay the deduplication cost. On a per-incremental-load basis, RMT is more economical than BigQuery's MERGE (which rewrites every partition the batch touches) and Snowflake's MERGE (which burns warehouse time on every run).

Pipeline strengths:

- Fastest raw INSERT throughput of any engine on this list – bulk inserts into `MergeTree` engines are limited by disk I/O, not the engine
- `ReplacingMergeTree` collapses duplicates by key on merge, turning upserts into pure appends – no MERGE bytes-scanned bill, no partition rewrite. Read-time correctness via `SELECT ... FINAL` (partition-selective and parallel since 23.3) or an `argMax`/window dedup view
- Self-hosted (or fixed-price Cloud) cost model has no per-query or per-insert charge – high load frequency doesn't increase the bill the way it does on BigQuery's bytes-scanned or Snowflake's warehouse-time models
- `REPLACE PARTITION` is atomic and operates at the partition level without rewriting other partitions
- Materialized views trigger on INSERT, enabling real-time pre-aggregation without a separate scheduling layer

Pipeline weaknesses:

- No ACID guarantees for mutations – `ALTER TABLE ... UPDATE` and `DELETE` are async, queued for the next merge cycle. Stay in the RMT append model and avoid them
- Duplicates coexist in RMT until the merge scheduler runs – correctness on reads requires `FINAL` or a dedup view. Avoid blanket `OPTIMIZE FINAL` on large tables; use partition-scoped optimize on a schedule

- RMT footguns: the version column must **not** be in `ORDER BY` (silently returns duplicates with `FINAL` if it is); partitioning on a column that mutates between versions creates permanent cross-partition duplicates
- `ORDER BY` is fixed at table creation – changing it requires rebuilding the table
- Small frequent inserts cause "too many parts" errors – batch aggressively (tens of thousands of rows minimum)

8.7.5 Redshift

Best for: AWS-native shops with existing infrastructure, teams that want PostgreSQL-compatible SQL in a columnar engine. The legacy choice – still viable for teams already invested in AWS, but BigQuery and Snowflake have moved ahead in pipeline ergonomics around DML flexibility, schema evolution, and operational overhead.

Pipeline strengths:
- `COPY` from S3 is fast bulk load with automatic compression, and S3 is the natural staging area for AWS pipelines
- PostgreSQL dialect means familiar SQL for teams coming from transactional databases
- `MERGE` added in late 2023, same syntax as BigQuery/Snowflake
- Spectrum queries S3 data directly without loading, useful for cold-tier data

Pipeline weaknesses:
- Sort keys and dist keys are changeable via `ALTER TABLE`, but the background rewrite can take hours on large tables – plan them at creation
- Automatic VACUUM DELETE handles most cleanup, but manual VACUUM may still be needed after heavy bulk deletes or to restore sort order
- Row-by-row `INSERT` is orders of magnitude slower than `COPY` – every load path must stage through S3
- Hard limit of 1,600 columns per table, and type changes require table rebuilds

8.7.6 DuckDB / MotherDuck

Best for: small-to-medium analytical workloads, local-first development, startups that want a warehouse without the bill.

DuckDB is an embedded columnar engine that runs in-process – no server, no cluster, no infrastructure. MotherDuck adds a cloud layer on top: managed storage, sharing, and read scaling via "Ducklings" (isolated compute instances per user). The combination gives you BigQuery-class query performance on datasets up to a few TB.

Pipeline strengths:
- Reads and writes Parquet and CSV natively from S3/GCS/Azure – no separate load job needed
- `INSERT ON CONFLICT` and `MERGE INTO` (DuckDB 1.4+) support the upsert and merge patterns from 4.3 – Merge / Upsert
- Develop locally with the exact same SQL that runs in MotherDuck cloud – the dev-to-prod gap is zero
- Local DuckDB is free. MotherDuck's per-GB price ($0.15/GB) is higher than BigQuery's on-demand rate ($0.006/GB), but the total bill is often lower because DuckDB's single-node engine scans less data per query – no distributed overhead, no shuffle. The savings come from efficiency, not a cheaper unit price

Pipeline weaknesses:
- Single-writer architecture – concurrent pipeline runs writing to the same database need external coordination (one run at a time, or separate databases per table)

- No partitioning in the BigQuery/Snowflake sense. Hive-partitioned Parquet on object storage or min/max index pruning, but no `PARTITION BY` in DDL, no partition-level replace, no `require_partition_filter`
- `QUALIFY` is supported (since v0.5) – dedup queries work the same as BigQuery and Snowflake
- At multi-TB scale with many concurrent dashboard users, MotherDuck costs converge toward Snowflake territory. The cost advantage is strongest for small teams with moderate data
- Self-hosting DuckDB on a dedicated server (Hetzner, bare metal) is zero-cost-per-query for a single client, but for multi-client pipelines the single-writer constraint means one database file per client with no shared users, roles, access control, or high availability – at that point the engineering overhead of building isolation exceeds the hosting savings

For self-hosted columnar beyond ClickHouse, **StarRocks** and **Apache Doris** are worth evaluating – both are FOSS MPP databases with MySQL wire protocol, real MERGE/upsert, ACID transactions, and better write concurrency than ClickHouse. Younger ecosystems, but they solve the concurrent-write limitations that make ClickHouse awkward for mutable extract-and-load workloads.

> **PostgreSQL as a destination**
>
> For pipelines with fewer than 100 tables, PostgreSQL with real PK enforcement, transactional `TRUNCATE`, and cheap `INSERT ON CONFLICT` is simpler and more forgiving than any columnar engine. The complexity tax of columnar only pays off when you need partition pruning, bytes-scanned billing, or warehouse-scale analytics. See 1.7 – Corridors.

8.7.7 Load Pattern Compatibility

How each engine handles the load strategies from Part IV, and what each costs relative to the others. Cost is per-run relative cost for the same data volume – not absolute pricing, which depends on your contract and usage tier.

Engine	Full replace (4.1 – Full Replace Load)	Append-only (4.2 – Append-Only Load)	Merge / upsert (4.3 – Merge / Upsert)	Append-and-materialize (4.4 – Append and Materialize)
BigQuery	Partition copy or `CREATE OR REPLACE`. Near-free (copy jobs cost nothing same-region)	`INSERT` via load jobs. Cheapest load operation – no partition rewrite	`MERGE` rewrites every partition touched. Expensive at scale – cost proportional to partitions, not rows	`QUALIFY` dedup view + `CREATE OR REPLACE` compaction. Load is cheap (append); read cost depends on log size
Snowflake	`SWAP WITH` (metadata-only, instant). Free beyond warehouse startup	`COPY INTO` from stage. Fast, warehouse time only	`MERGE` consumes warehouse time. Moderate – more predictable than BigQuery's partition model	`QUALIFY` dedup view + `CREATE TABLE ... AS` compaction. Warehouse time on reads and compaction
ClickHouse	`EXCHANGE TABLES` (atomic). Minimal cost on self-hosted	Native strength – fastest INSERT throughput of any engine	`ReplacingMergeTree` insert – pure append, no MERGE statement. Cheapest per-load of any engine in this table; read-time correctness via `FINAL` or `argMax` view	`ReplacingMergeTree` is this pattern, natively. Write is free; dedup cost is configurable between background merges, scheduled partition-scoped `OPTIMIZE`, and read-time `FINAL`
Redshift	`TRUNCATE` + `COPY` in transaction. Fast via S3 staging	`COPY` from S3. Row-by-row `INSERT` is orders of magnitude slower	`MERGE` (late 2023) or DELETE + INSERT in transaction. Moderate – cluster compute	Subquery dedup view + `CREATE TABLE ... AS` compaction. No `QUALIFY`
PostgreSQL	`TRUNCATE` + `INSERT` in transaction. Atomic, transactional, cheap	Standard `INSERT`. Cheap at moderate volumes	`INSERT ON CONFLICT` with real PK enforcement. Cheapest upsert of any engine – index lookup per row	Subquery dedup view or materialized view. Read overhead on view; `REFRESH MATERIALIZED VIEW CONCURRENTLY` for zero-downtime
DuckDB	`CREATE OR REPLACE` or `TRUNCATE` + `INSERT`. Free locally	`INSERT` or `COPY FROM` Parquet. Free locally	`MERGE INTO` (1.4+) or `INSERT ON CONFLICT`. Free locally; MotherDuck charges per GB scanned	`QUALIFY` dedup view + `CREATE OR REPLACE` compaction. Free locally

8.7.8 Decision Matrix

Workload	Recommended	Why
Many ad-hoc analysts, pay-per-query	BigQuery	Cost scales with actual usage; partition filter enforcement protects the bill
Predictable budget, multi-team	Snowflake	Warehouse isolation, fixed compute costs, data sharing
Append-heavy, real-time dashboards	ClickHouse	Fastest inserts, materialized views on write
High-frequency incremental loads, many upserts per day	ClickHouse	`ReplacingMergeTree` turns upserts into pure appends – no MERGE cost, no per-insert charge, dedup paid lazily or at read time
AWS-native, existing infrastructure	Redshift	Familiar PostgreSQL dialect, `COPY` from S3, Spectrum for cold data
Small team, PostgreSQL expertise	PostgreSQL	Cheapest, real constraint enforcement, transactional `TRUNCATE`
Startup, small team, moderate data	DuckDB / MotherDuck	Lowest cost, local-first dev, no infrastructure to manage
Mixed analytical + operational consumers	Snowflake or BigQuery + PostgreSQL	Columnar for analytics, transactional for point queries (4.5 – Hybrid Append-Merge)

Start with load strategy, not engine

The decision matrix above is a starting point, but the more productive question is often: which load strategies does my pipeline need, and which engines support them cheaply? If every table can be fully replaced, all five engines work fine and the choice comes down to your cloud provider and team expertise. The engine choice starts to matter when you need high-concurrency MERGE, append-and-materialize with dedup views, or partition-level atomic swaps – that's when the compatibility table above narrows the field.

About the Author

Alonso Burón

Writing your own bio is inherently awkward, so I'll keep it short and honest.

I studied music composition at the Pontifical Catholic University of Chile for five years. I wrote scores for a French indie game studio, directed sound for a theater company, and coordinated concert recordings. None of that prepared me for data engineering – except that it taught me to think in systems, manage complexity under pressure, and work with people who speak a completely different technical language than I do.

The pivot happened by accident. I started tutoring Stata and Python for economics students – a side job that came from helping my girlfriend with her homework. That pulled me into data analysis, then into BI, then into building pipelines. I joined a data consultancy as an intern in 2024. Within a year I was leading the company's first data engineering team and building an internal platform that extracts data from over twenty different source systems into BigQuery for production clients.

This book comes from that work. I run thousands of tables across dozens of clients, and every pattern in this book is something I've either built, broken, or fixed in production. The domain model, the war stories, the opinions about full replace being the default – all of it comes from staring at pipelines at 3 AM and thinking "there has to be a better way to explain this."

I live in Santiago, Chile. I use Arch Linux as my personal OS, Obsidian for writing, and I think Dagster is the best orchestrator on the market – but you already knew that from the appendix.

alonsoburon.cl · github.com/alonsoburon

Battle-Tested Data Pipelines
The step ELT forgot – patterns for extraction, syntactic transformation, and loading

First edition, 2026.

Typeset in Libertinus Serif using Typst.
Diagrams by the author.

alonsoburon.cl

Every pipeline has a syntactic transformation problem.
Most just don't have a name for it.

ELT tells you to load raw and transform later. ETL tells you to transform before loading. Neither one tells you what to do about the timezone that silently shifts every row, the primary key that gets recycled, the hard delete that leaves no trace, or the schema that mutates overnight.

This book does.

- When to full replace, when to go incremental, and how to earn the complexity
- Extraction patterns for cursors, hard deletes, late-arriving data, and mutable windows
- Load strategies from append to merge – with the cost and read/write tradeoffs of each
- The syntactic layer: type casting, null handling, timezones, synthetic keys
- Operating at scale: health tables, SLA management, alerting, backfill, and recovery
- Decision flowcharts, SQL dialect reference, and tool recommendations

Written by an engineer running thousands of tables in production. Every pattern comes from something that broke, something that scaled, or something that took too long to figure out the first time.

Data Engineering / Software